What's a protocol?

INTRO

human protocols:

- "what's the time?"
- "I have a question"
- introductions

… specific messages sent

… specific actions taken when messages received, or other events

network protocols:

- machines rather than humans
- all communication activity in Internet governed by protocols

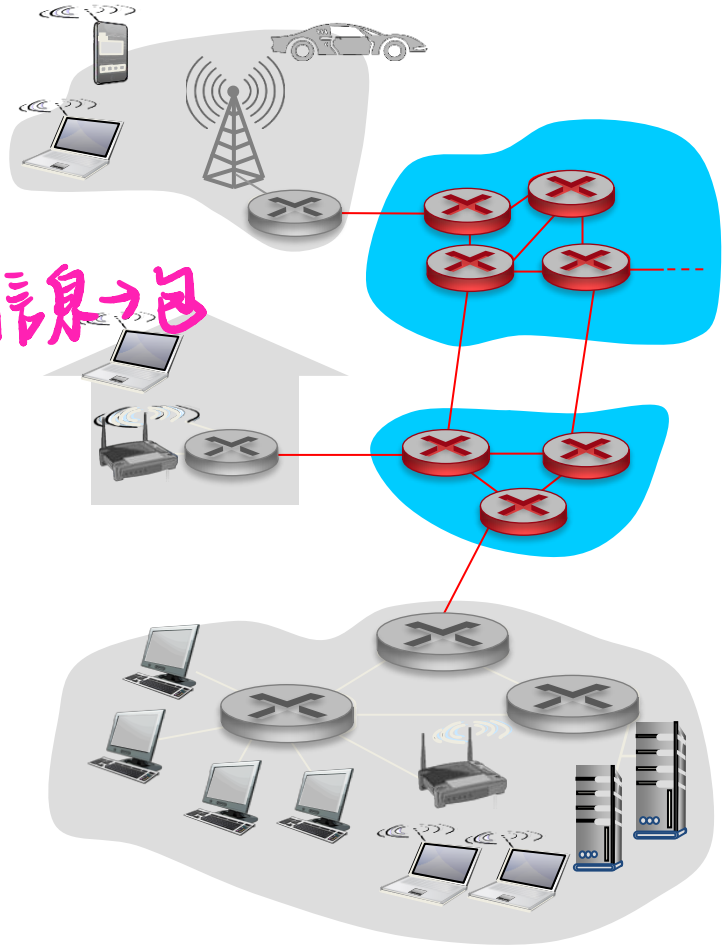
protocols define format, order of messages sent and received among network entities, and actions taken on message transmission, receipt

Network basics bits and bytes

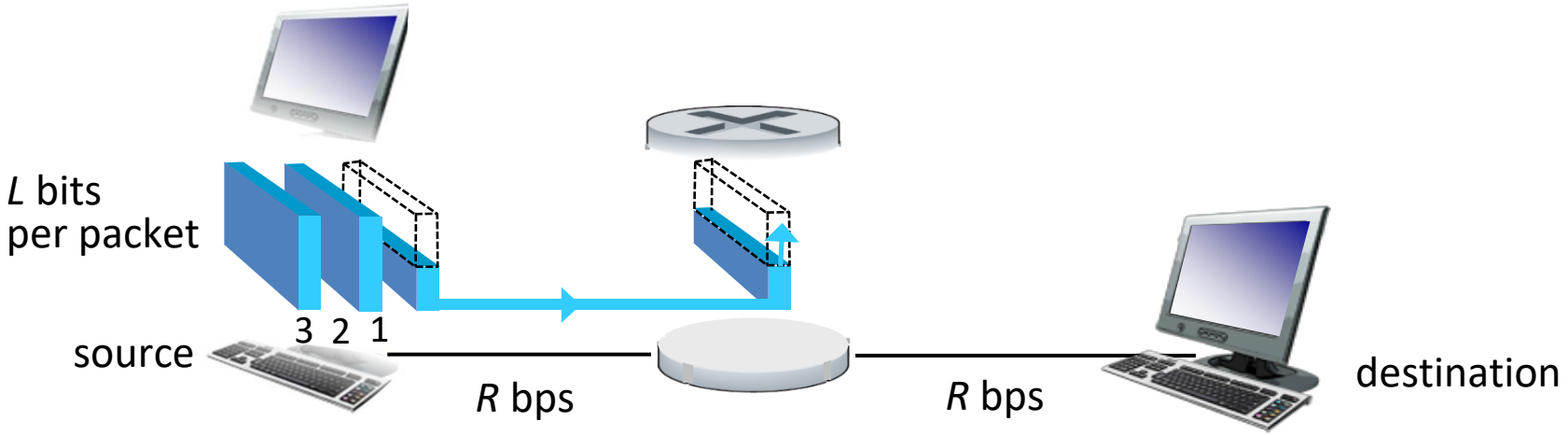
- A bit is a "binary digit" – a single 0 or 1.
- A byte is a group of eight bits – can be thought of as a number from 0 to 255 (or -128 to 127) or as two digits of "hexadecimal" (eg A0, FF, 10).
- Amounts of data are usually specified in bytes.
  - 1KB = 1 kilobyte = 1000 bytes = 8000bits
  - 1MB = 1 megabyte = 1 million bytes = 8 million bits
  - 1GB = 1 gigabyte = 1000 million bytes = 8000 million bits
- BUT speeds are usually in bits per second (not bytes)
  - 1b/s (or bps) = 1 bit per second
  - 1Kb/s = 1000 bits per second
  - 1Mb/s = 1 million bits per second
  - 1Gb/s = 1000 million bits per second

The network core

- mesh of interconnected routers
- packet-switching: hosts break application-layer messages into *packets*
  - forward packets from one router to the next, across links on path from source to destination
  - each packet transmitted at full link capacity



Packet-switching: store-and-forward



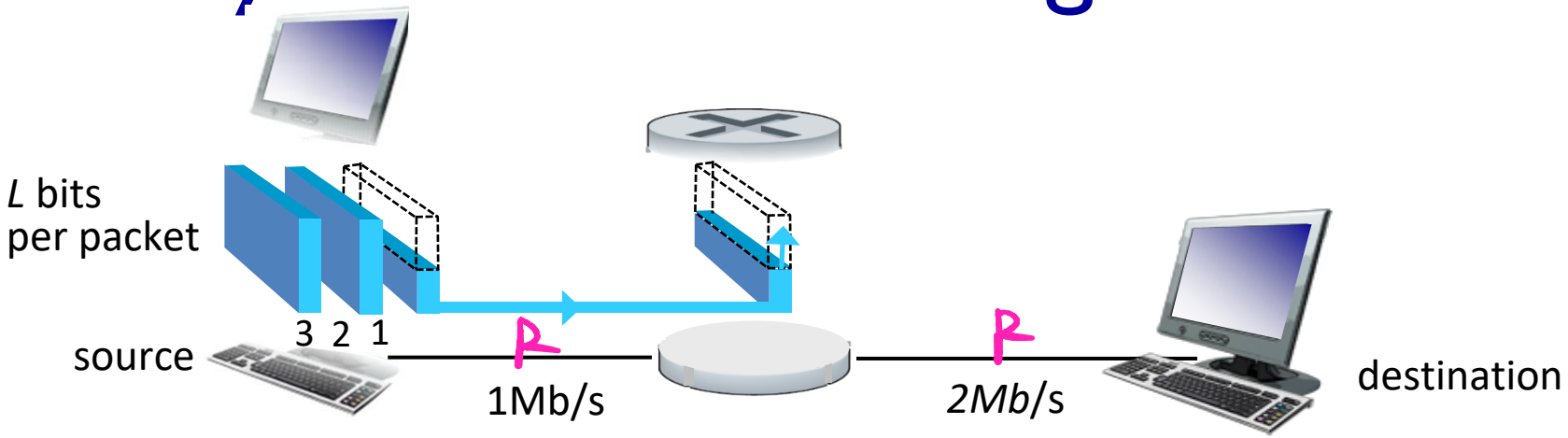
- takes *L*/*R* seconds to transmit (push out) *L*-bit packet into link at *R* bps
- *store and forward:* entire packet must arrive at router before it can be transmitted on next link
- end-end delay = 2*L*/*R* (assuming zero propagation delay)

one-hop numerical example:

- *L* = 7.5 Mbits
- *R* = 1.5 Mbps
- one-hop transmission delay = 5 sec

more on delay shortly …

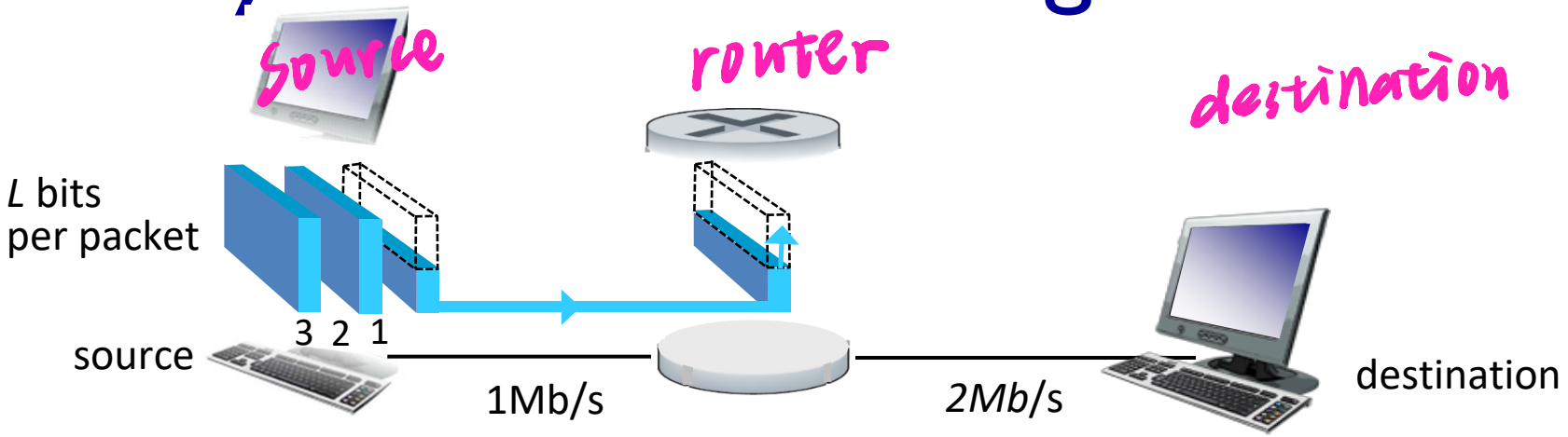
Test your understanding



- If we count only the "transmission" delay (the time taken to send packets) what is the delay from sending a 1500byte packet from source to destination?

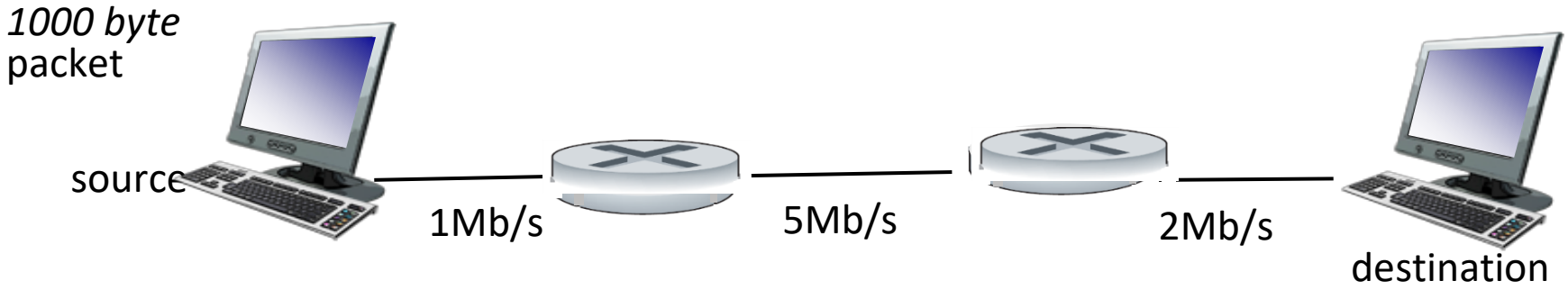
- Pause the lecture. Write down your answer.

Test your understanding



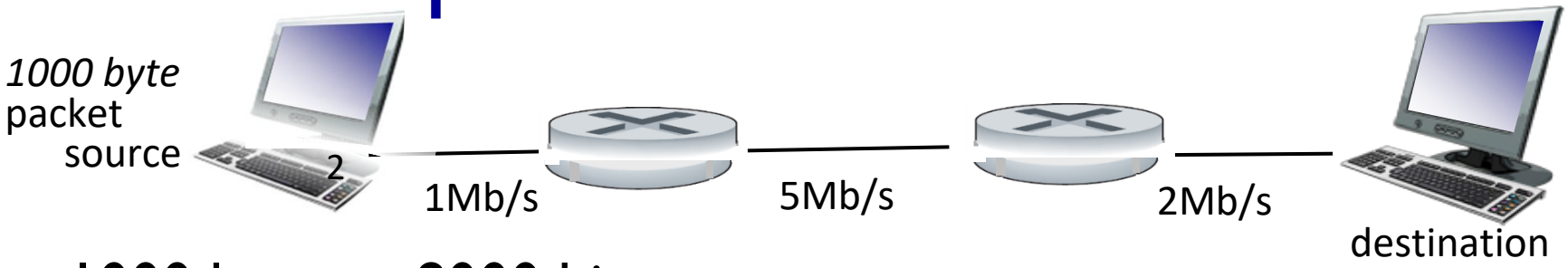
- 1,500 bytes = 1,500x8 bits = 12,000 bits
- 1Mb/s is 1000000bits/sec
- From source to router L/R = 12000/1000000 secs= 0.012 seconds or 12 ms
- From router to dest = 12000/2000000 seconds= 0.006 seconds or 6 ms
- Total end-to-end delay = 12+6ms = 18ms
- Did you get it right? If not, review the previous slides

Practice question



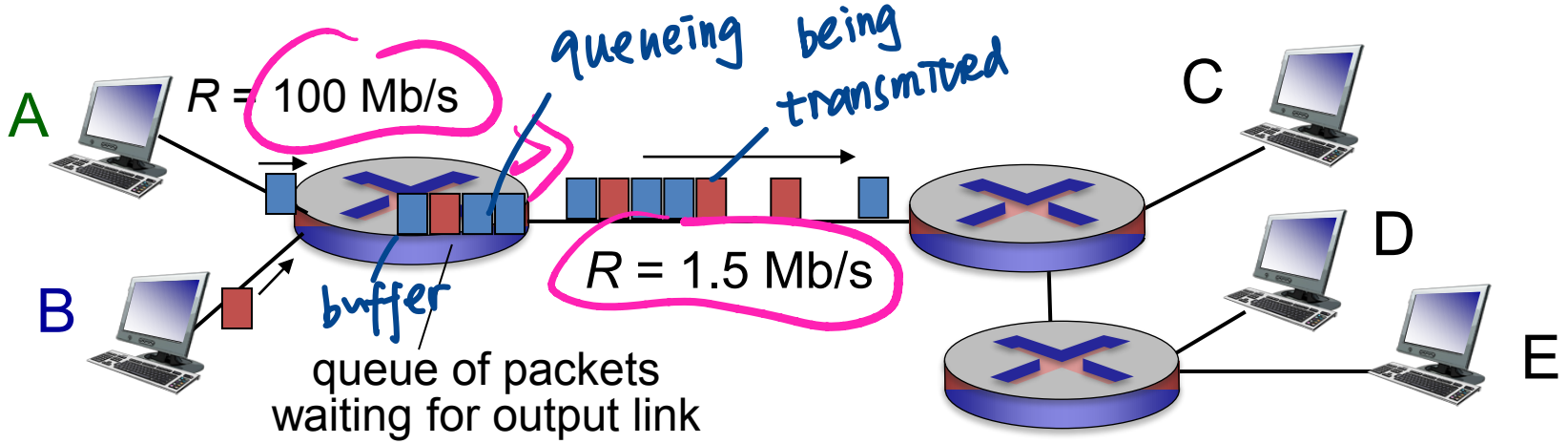
- A 1000 byte packet is sent from source to destination through 3 links 1Mb/s, 5Mb/s, 2Mb/s
- Counting only transmission delay:
  - a) What is the end-to-end delay?
  - b) What is the round-trip-time (RTT)?
  - c) How long will it be between starting sending until the 3rd packet reaches the destination (assume 2nd and 3rd packets transmit as soon as they possibly can)?

Practice question



- 1000 bytes = 8000 bits.
- 1Mb/s = 1000000 bits/second
- Delay to first router = 8000/1000000secs= 0.008sec = 8 ms
- Delay to first to second router = 1.6ms
- Delay second to third router = 4ms
- **a) End-to-end delay = 8+1.6+4ms = 13.6 ms**
- **b) RTT = 2x end-to-end = 27.2ms**
- Second packet starts transmission 8ms after first.
- Third packet starts transmission 8x2ms = 16 ms after first.
- **c) Third packet arrives after 13.6+8x2ms = 29.6ms.**

Packet Switching: queueing delay, loss



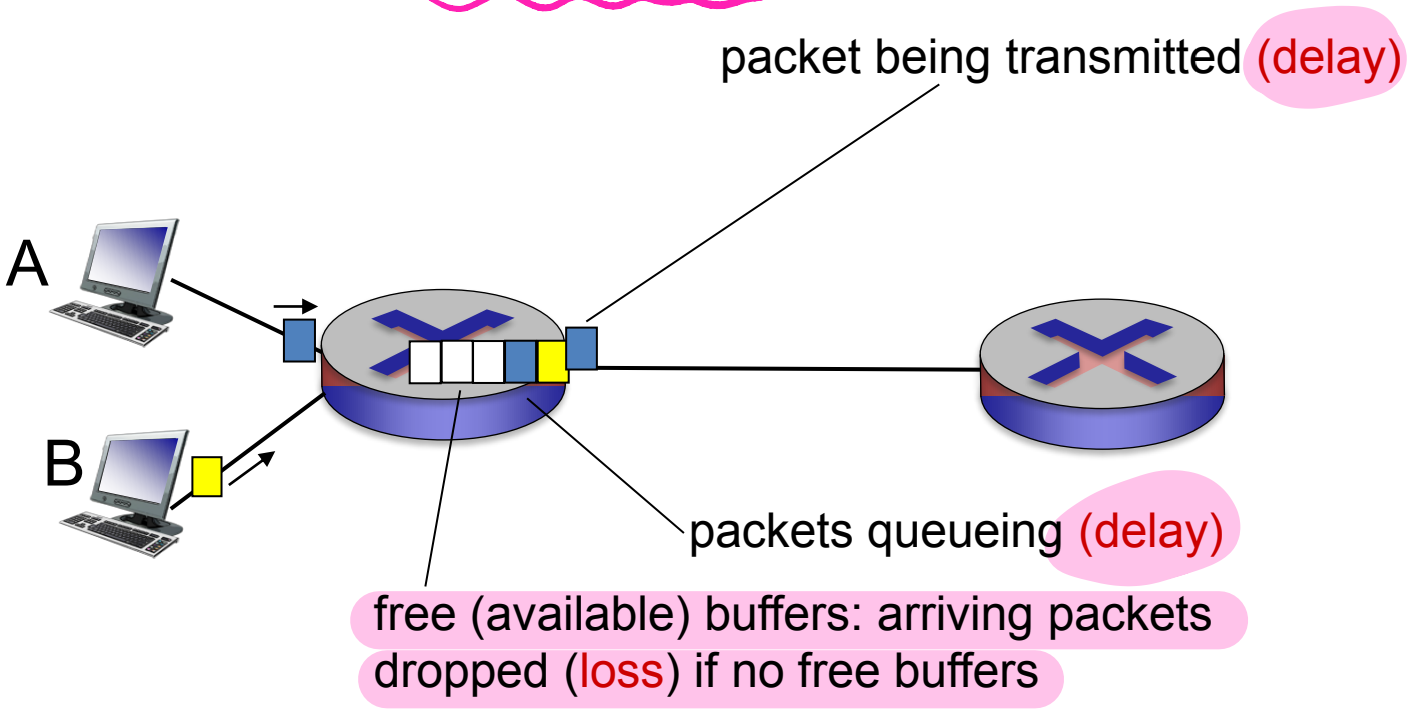
queuing and loss:

- if arrival rate (in bits) to link exceeds transmission rate of link for a period of time:
  - packets will queue, wait to be transmitted on link
  - packets can be dropped (lost) if memory (buffer) fills up

How do loss and delay occur?

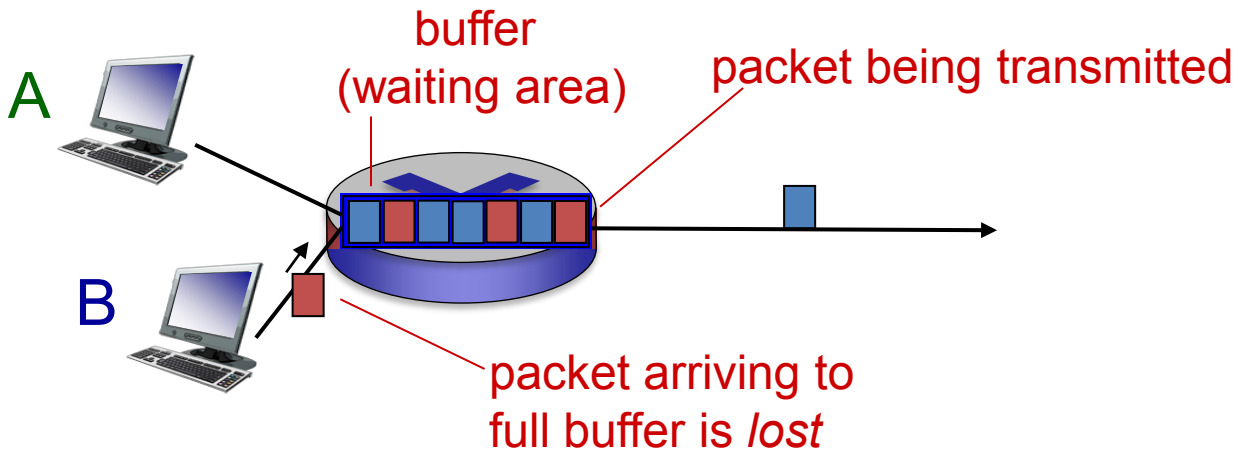
packets *queue* in router buffers

- packet arrival rate to link (temporarily) exceeds output link capacity
- packets queue, wait for turn



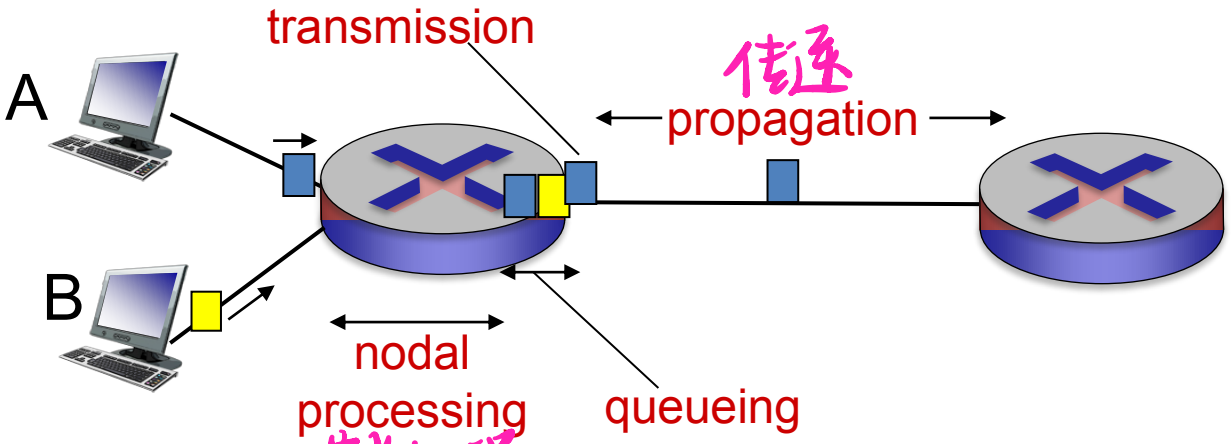
Packet loss

- queue (aka buffer) preceding link in buffer has finite capacity
- packet arriving to full queue dropped (aka lost)
- lost packet may be retransmitted by previous node, by source end system, or not at all



* Check out the Java applet for an interactive animation on queuing and loss

Four sources of packet delay

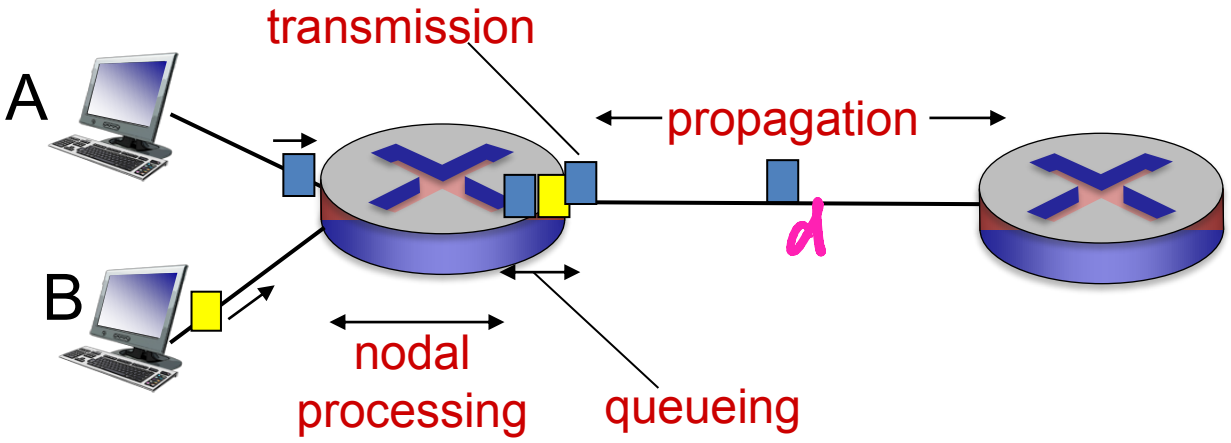


$$d_{nodal} = d_{proc} + d_{queue} + d_{trans} + d_{prop}$$

d_{proc}: nodal processing
- check bit errors
- determine output link
- typically < msec

d_{queue}: queueing delay
- time waiting at output link for transmission
- depends on congestion level of router

Four sources of packet delay



$$d_{nodal} = d_{proc} + d_{queue} + d_{trans} + d_{prop}$$

d_{trans}: transmission delay:

- L: packet length (bits)
- R: link *bandwidth* (bps)
- $d_{trans} = L/R$

d_{trans} and d_{prop} *very* different

d_{prop}: propagation delay:

- d: length of physical link
- s: propagation speed (~2×10^8 m/sec)
- $d_{prop} = d/s$

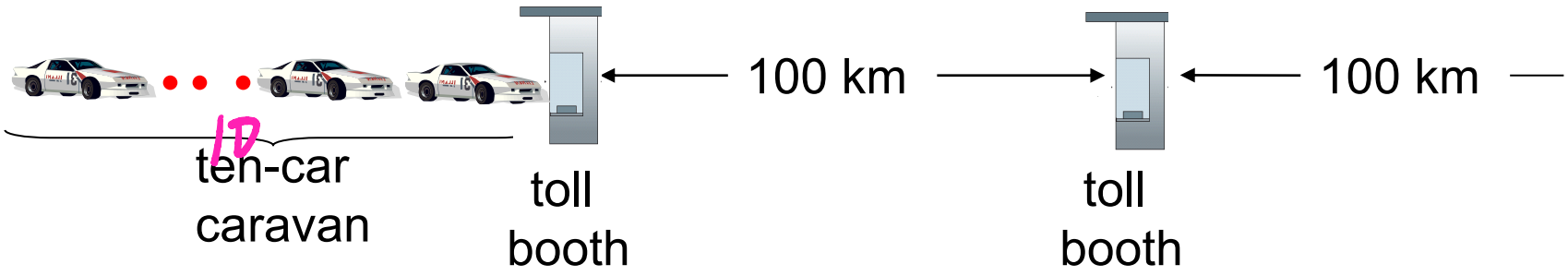
(fibre optic link)

fast

* Check out the online interactive exercises for more examples: http://gaia.cs.umass.edu/kurose_ross/interactive/

* Check out the Java applet for an interactive animation on trans vs. prop delay

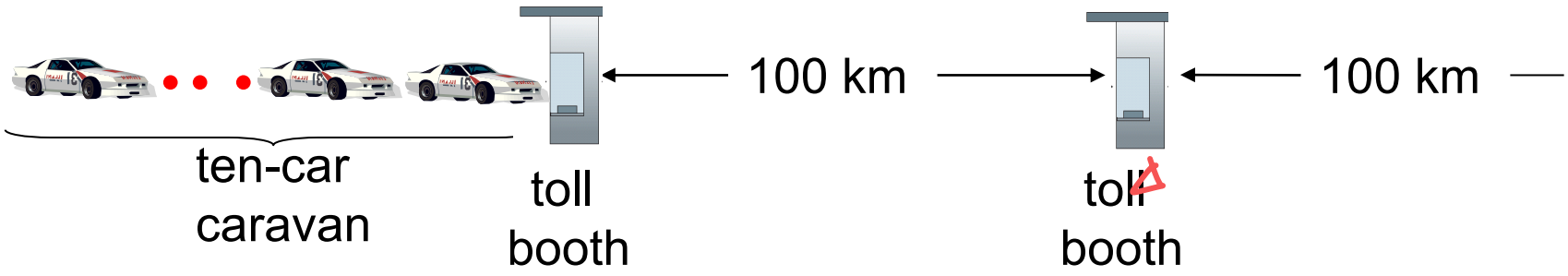
Caravan (group of cars) analogy



- cars "propagate" at 100 km/hr
- toll booth takes 12 sec to service car (bit transmission time)
- car ~ bit; caravan ~ packet
- *Q:* How long until caravan is lined up before 2nd toll booth?

- time to "push" entire caravan through toll booth onto highway = 12*10 = 120 sec
- time for last car to propagate from 1st to 2nd toll both: 100km/(100km/hr)= 1 hr
- *A:* 62 minutes

Caravan (group of cars) analogy (2)



- suppose cars now "propagate" at 1000 km/hr
- and suppose toll booth now takes one min to service a car
- Q: Will cars arrive to 2nd booth before all cars serviced at first booth?
  - A: Yes! after 7 min, first car arrives at second booth; three cars still at first booth

Example question

- A packet of 1500B arrives at a router:
  - The router takes 2ms to process the packet.
  - The router takes 1ms to work out which outbound link it should be sent to.
  - The packet waits behind four other packets each of which takes 0.5 ms to transmit.
  - The router sends the packet down a 1Mb/s link.
  - The packet travels at 200,000,000 m/s (2/3 speed of light) down a 1,000 km fibre optic link under the ocean.
- Calculate how much delay in each of the four types. What is the total delay?

Example question

- A packet of 1500B arrives at a router:
- Name the four sources of delay and how much delay each is. What is the total delay?
  - The router takes 2ms to process the packet.
  - The router takes 1ms to work out which outbound link it should be sent to.
  - **Total 3ms nodal processing delay.**
  - The packet waits behind four other packets each of which takes 0.5 ms to transmit.
  - **Total 2ms queuing delay** 0.5×4

Example question

- A packet of 1500kB arrives at a router:
- Name the four sources of delay and how much delay each is. What is the total delay?
  - The router sends the packet down a 1Mb/s link.
  - 1500 bytes = 1500x8bits= 12000 bits
  - Time =L/R = 12000/1000000 sec = 12ms
  - **Transmission delay 12ms**
  - The packet travels at 200,000,000 m/s (2/3 speed of light) down a 1,000 km fibre optic link under the ocean.
  - 1,000km= 1,000,000m
  - Time = 1,000,000/200,000,000 seconds = 0.005sec $= \frac{d}{s}$
  - **Propagation delay = 5ms**
- **Total delay = 3+2+12+5 ms= 22ms**

Test your understanding

- Sources of delay
  - Which of the four types of delay are the following.
- Options:
  - (A) Time a packet waits behind other packets

    (queueing delay)
  - (B) Time a router takes to write the bits from a packet into an outgoing optical fibre.

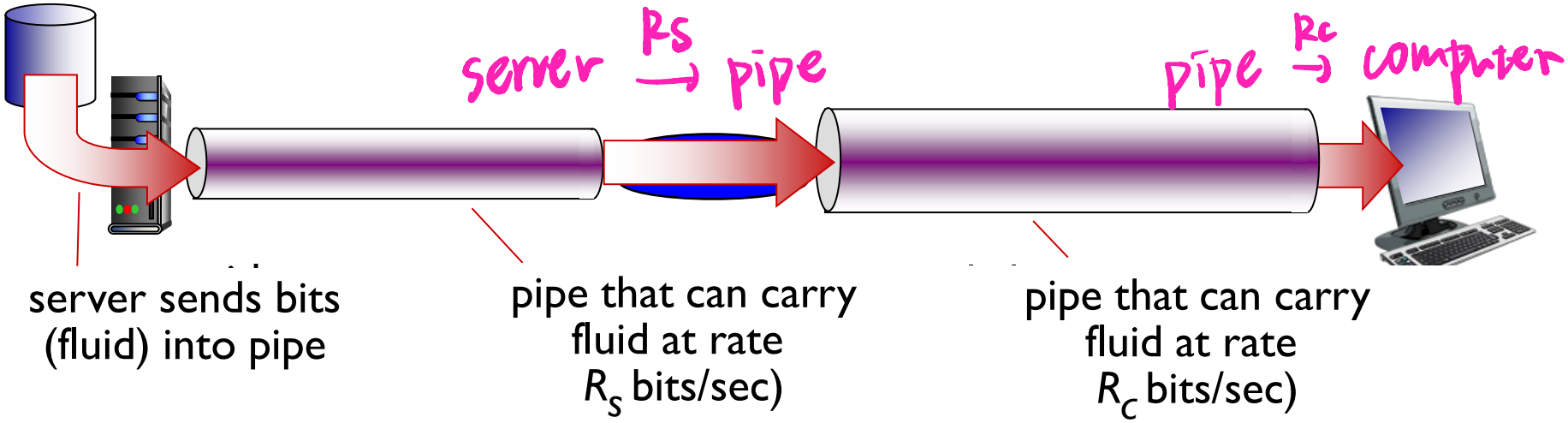
    (transmission delay)
  - (C) Time it takes packets to travel down an optical fibre to the next router.

    (propagation delay)
  - (D) Time it takes a router to determine which of its outputs to send a packet down.

    (nodal processing)
- If you did not get it right review section 1.4

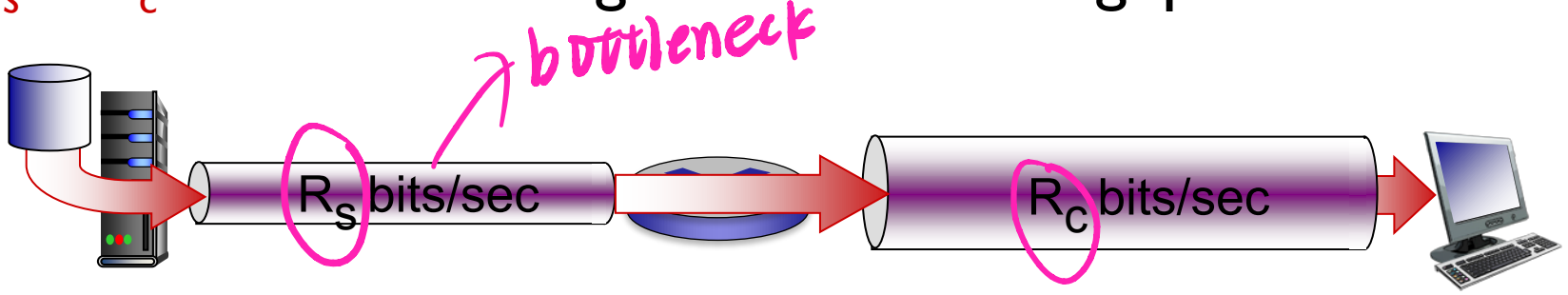
Throughput 吞吐量

- *throughput:* rate (bits/time unit) at which bits transferred between sender/receiver
  - *instantaneous:* rate at given point in time 瞬时
  - *average:* rate over longer period of time 平均

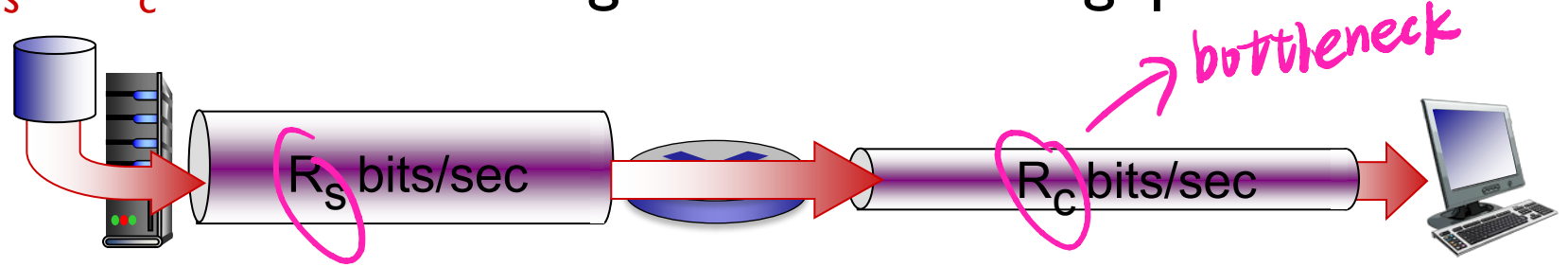


Throughput (more)

- $R_s < R_c$ What is average end-end throughput?



- $R_s > R_c$ What is average end-end throughput?

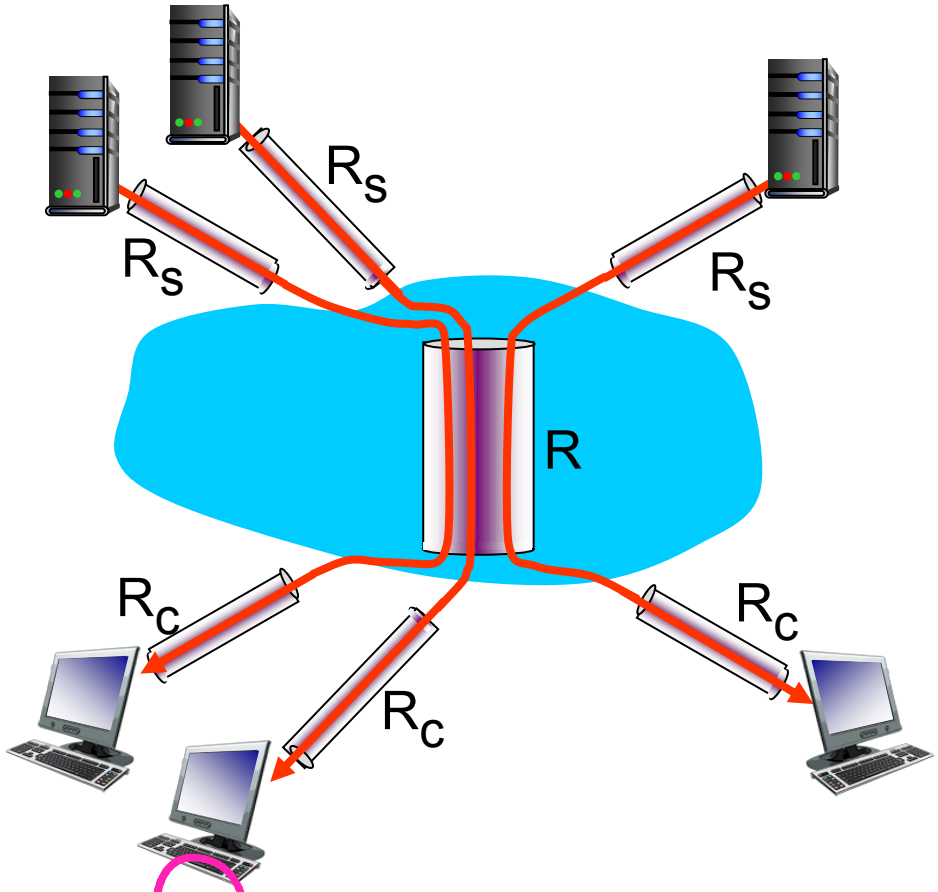


bottleneck link

link on end-end path that constrains end-end throughput

Throughput: Internet scenario

- per-connection end-end throughput: $min(R_c, R_s, R/10)$
- in practice: R_c or R_s is often bottleneck



* Check out the online interactive exercises for more examples: http://gaia.cs.umass.edu/kurose_ross/interactive/

Test your understanding

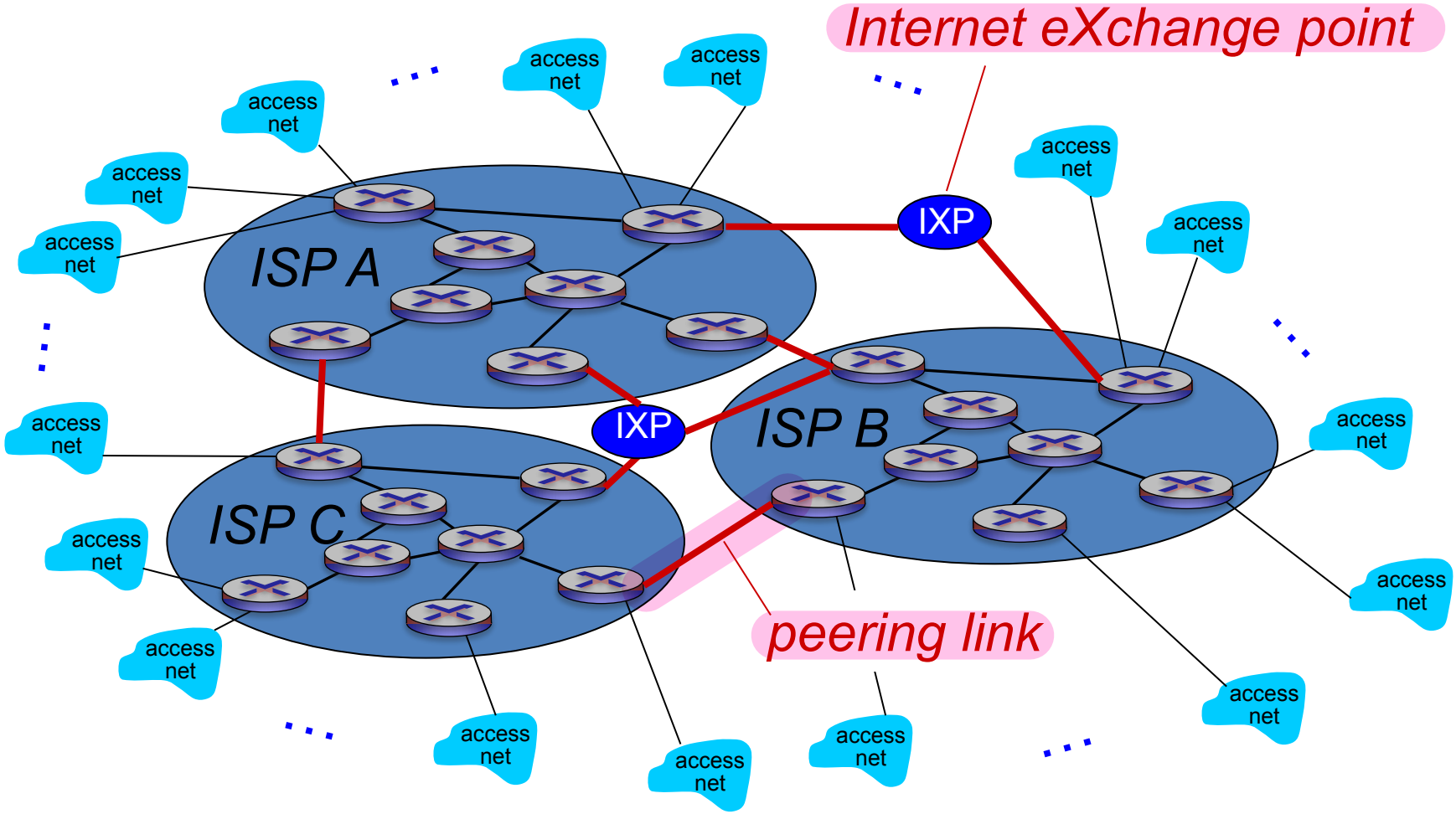
- R_s is 512kb/s R_c is 0.6Mb/s.
- Which is the bottleneck?
  - R_s=0.512Mb/s this is the bottleneck
- R_s is 1.2Mb/s R_c is 1024kb/s
- R (the central link) is 5.1Mb/s divided equally between 5 users.
- Which is the bottleneck?
- For R the central link each user gets (5.1/5) Mb/s each R= 1.02Mb/s or 1020kb/s
- Here R is the bottleneck.

Internet structure: network of networks

- End systems connect to Internet via access ISPs (Internet Service Providers)
  - residential, company and university ISPs
- Access ISPs in turn must be interconnected. 相互连接
  - so that any two hosts can send packets to each other
- Resulting network of networks is very complex
  - evolution was driven by economics and national policies
- Let's take a stepwise approach to describe current Internet structure

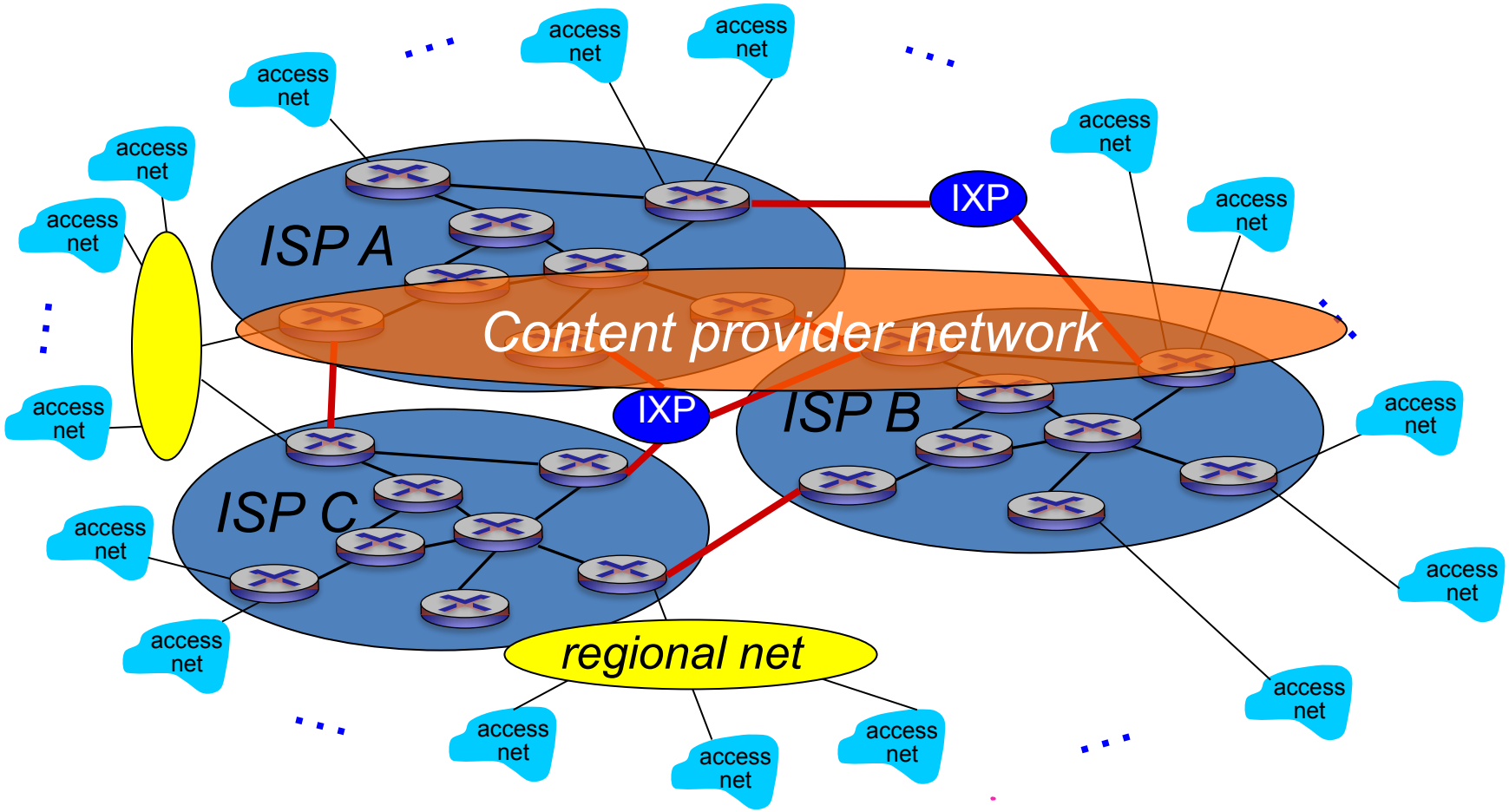
Internet structure: network of networks

But if one global ISP is viable business, there will be competitors …. which must be interconnected



Internet structure: network of networks

… and content provider networks (e.g., Google, Microsoft, Akamai) may run their own network, to bring services, content close to end users

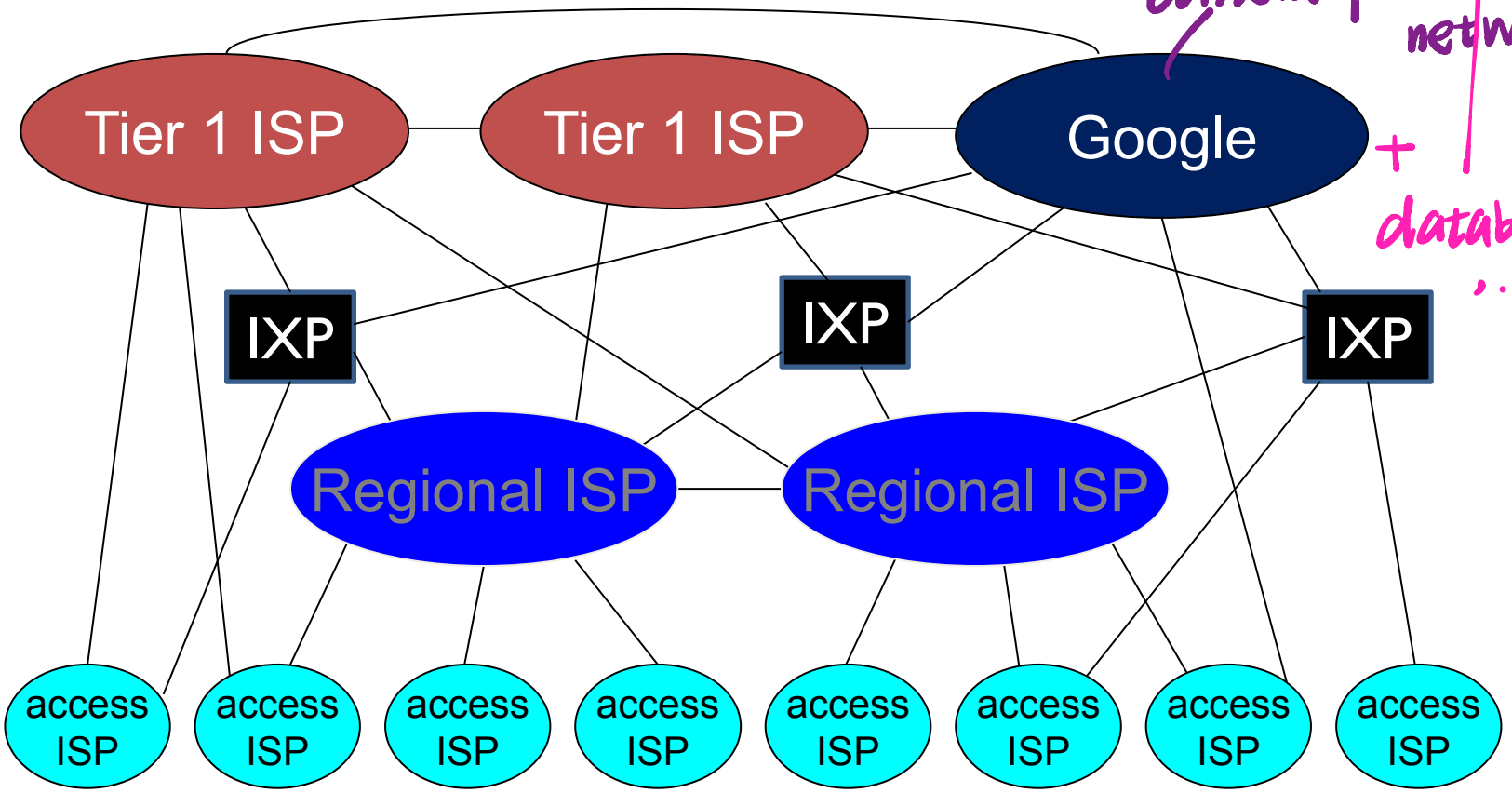


Internet structure: network of networks

provide broad things, not just internet

content provider network

+ database, ...



- at center: small # of well-connected large networks
  - "tier-1" commercial ISPs (e.g., Level 3, Sprint, AT&T, NTT), national & international coverage
  - content provider network (e.g., Google): private network that connects it data centers to Internet, often bypassing tier-1, regional ISPs

Test your understanding

- Which of the following are network core or network edge
- Options: access / regional ISP : core
  - (A) Tier I ISP (Core)
  - (B) Internet Exchange point (IXP) (Core)
  - (C) Your mobile phone (手机) (Edge)
  - (D) Your laptop (Edge)

- Did you get it all right? If not review network edge and network core material.

Practice question: Edge or core

- Where do you find the following: network core or network edge or both
- Options:
  - (A) WiFi access point
    - Network edge (allows end user devices access to network)
  - (B) Content distribution network
    - Core (but near the network edge)
  - (C) Switches
    - Both (necessary all over the network for layer 2 connections some switches are tiny cheap devices and some are huge)

Why layering?

dealing with complex systems:

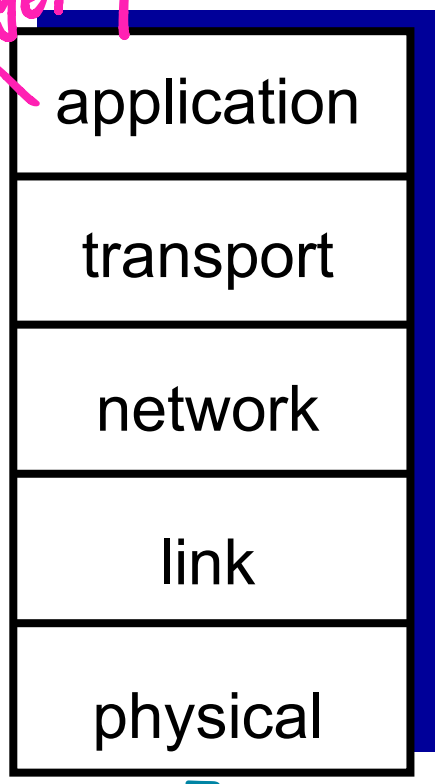
- explicit structure allows identification, relationship of complex system's pieces
  - layered *reference model* for discussion
- modularization eases maintenance, updating of system

模块化简化了系统的维护和更新

  - change of implementation of layer's service transparent to rest of system
  - e.g., change in gate procedure doesn't affect rest of system
- layering considered harmful?

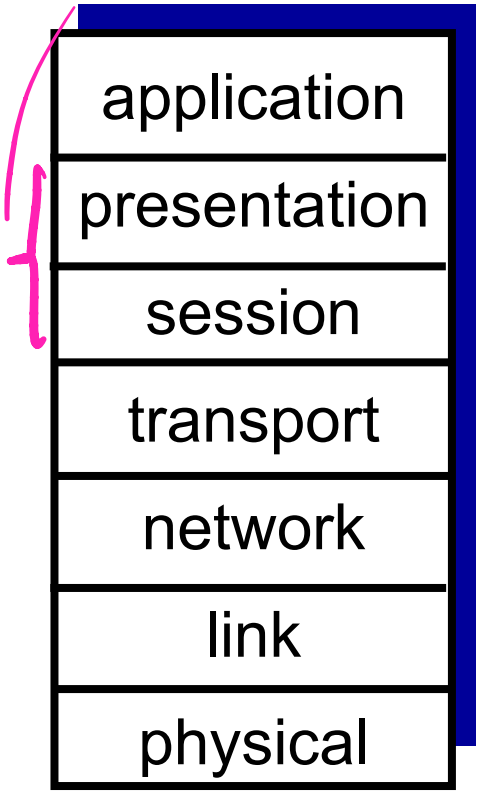
TCP/IP model (internet model)

- *application:* supporting network applications
  - FTP, SMTP, HTTP
- *transport:* process-process data transfer
  - TCP, UDP
- *network:* routing of datagrams from source to destination
  - IP, routing protocols
- *link:* data transfer between neighboring network elements
  - Ethernet, 802.111 (WiFi), PPP
- *physical:* bits "on the wire"



ISO/OSI reference model

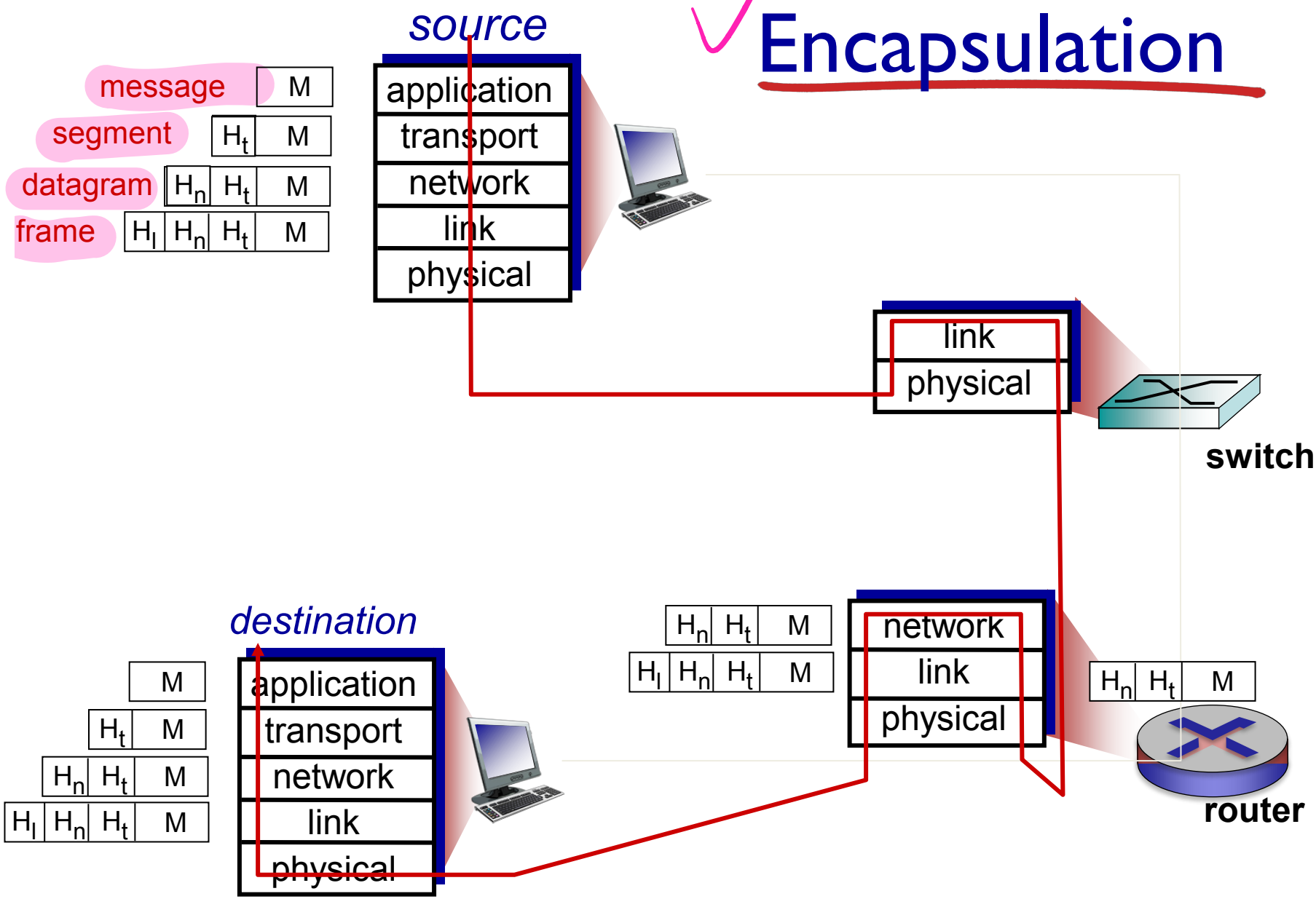
- ISO = International Standards Office
- OSI = Open Systems Interconnection
- *presentation:* allow applications to interpret meaning of data, e.g., encryption, compression, machine-specific conventions
- *session:* synchronization, checkpointing, recovery of data exchange
- Internet stack "missing" these layers!
  - these services, *if needed*, must be implemented in application
  - needed?



Layering and headers

- Most layers of the TCP/IP model are associated with a particular type of "header" or sometimes a "header and trailer".
- What is a header?
  - Information separate from the data being sent that says things about that data.
  - Which computer is it being sent to?
  - Which program on that computer must receive it?
  - How long is this data?
- For example layer 3 (network) has a "network address" which identifies the host that should receive the data.
- Layer 4 (transport) has a port that identifies which program should receive it.
- At each lower layer a new header is added incorporating the headers underneath. A layer 2 packet has a layer 2 header but includes headers from layer 3 and 4.

Encapsulation



Practice question

- M is the message being sent
- H_l,H_n,H_t headers for link, network, transport layers.
- L_l,L_n,L_t are the lengths of these headers in bytes.
- L_m is the length of the message in bytes
- A) How many bytes are present at the transport layer?
  - Message + H_t only present =L_m+L_t bytes
- B) How many bytes are present at the link layer?
  - Message + H_t + H_n + H_l = L_m+L_t +L_n+L_l bytes
- C) The user only cares about the bytes in the message. What is the efficiency if we define:
  - Efficiency= bytes layer 7 receives /bytes sent at layer two
  - Efficiency = $L_m/(L_m+L_t+L_n+L_l)$

Test your understanding

- Name two advantages of layering?
  1. Modularisation eases maintenance and updates (change one part of the system leave others the same)
  2. Helps with complexity: break a complex design task into several smaller ones
- Which headers are included in a layer two packet?
  - Data link header, network header, transport header (layer 2,3,4 headers)
- If you got it wrong review section 1.5

TCP/IP layers

- Layer 7 – Application layer – this is the data for programs you use on your computer
  - HTTP (www) data, SMTP (sent emails), FTP (file transfer) and specific formats for games, torrent etc.
- Layer 6 – Presentation layer
  - Related to character sets and presentation of data (unused in TCP/IP or real Internet)
- Layer 5 – Session layer
  - Related to whole lifetime of connection – is the connection real time (unused in TCP/IP or real Internet)
- Layer 4 – Transport layer – this is for the end-to-end connection between machines.
  - Information related to reliability
  - Information related to which program on machine sent/receives data.

address: port number

驱证传送的信息是否可靠

go to the right destination in order / reliably

TCP/IP layers

- Layer 3 – Network (Internet) layer – this is to get the data the whole journey from its start computer to its end computer (Internet, between networks)
  - Address of computer on internet (IP address)
  - Checksum to see if data is corrupted
- Layer 2 – Data link layer – this is to get the data to nearby computers (on same local network)
  - Media Access Control (MAC address) specific to individual computer (see later lectures)
- Layer 1 – Physical layer (how bits 1s and 0s are actually transmitted)
  - Think of cables in the ground or radio waves in the air

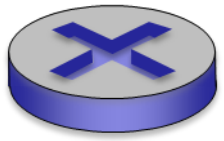
What layer? 什么层

- Answer quickly using mentimeter (or chat if mentimeter is not working) the number of the layer. If you don't know take a guess as to which fits best.
- Responsible for reliable in order traffic delivery.
  - Transport (layer 4) – [TCP]
- Delivers traffic to "neighbouring" routers/hosts.
  - Datalink (layer 2)
- Router.
  - Network (layer 3)

What layer? 什么层

- Answer quickly using mentimeter (or chat if mentimeter is not working) the number of the layer. If you don't know take a guess as to which fits best.
- Transmission Control Protocol.
  - Transport (layer 4)
- Delivery of traffic across the whole network.
  - Network (layer 3)
- Delivery of traffic between a laptop and a WiFi router.
  - Datalink (layer 2)
- A repeater (device to boost a signal).
  - Physical (layer 1)

Devices for layers



- Router
  - This is a layer 3 device – it reads a layer 3 address and works out which direction a packet should go.
  - It is typically more complex and adaptive than a switch.

- Switch
  - This is a layer 2 device – it reads a layer 2 address and works out which nearby computer should get a message.
  - Typically simpler than a router.

- Repeater
  - This is a layer 1 device – it strengthens or reconstructs a corrupted signal and carries on sending it.

Test your understanding

- Layers quiz
  - Which layer are the following – give the name and the number:
- Options:
  - (A) An optical fibre
    (Layer 1 physical)
  - (B) A web browser
    (Layer 7 application)
  - (C) A router
    (Layer 3 network)
  - (D) Layer which makes sure packets are in the right order
    (Layer 4 transport)
  - (E) Layer which gets packets to the right machine anywhere on the whole internet.
    (Layer 3 network)
- If you did not get the right answers review section 1.5

TCP

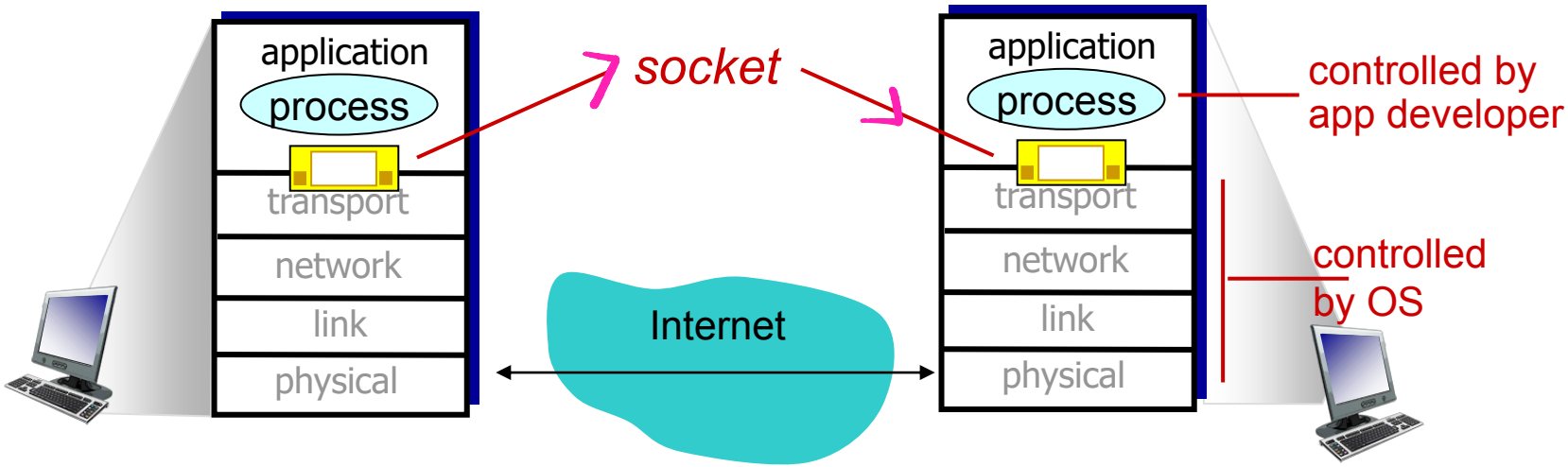
global traveller

Application Layer (very briefly)

- This is covered in many other modules you will take.
- This is the layer you (mostly) work with as a programmer.
- At the application layer the "network" is abstracted away and you access a stream of data that arrives at a "socket".
- It is up to you to define the format of data your application writes and receives.
- Different applications have different formats:
  - HTTP (Hyptertext transfer protocol) world-wide web
  - FTP (File transfer protocol) moving data
  - SMTP (Send Mail transfer protocol) sending email
  - IMAP (Internet message access protocol) receiving email

Sockets

- process sends/receives messages to/from its socket
- socket analogous to door
  - sending process shoves message out door
  - sending process relies on transport infrastructure on other side of door to deliver message to socket at receiving process



Addressing processes

- to receive messages, process must have *identifier*
- host device has unique 32-bit IP address
- *Q:* does IP address of host on which process runs suffice for identifying the process?
  - *A:* no, *many* processes can be running on same host

运行进程的主机IP地址是否足以识别该进程？

- *identifier* includes both IP address and port numbers associated with process on host.
- example port numbers:
  - HTTP server: 80
  - mail server: 25
- to send HTTP message to gaia.cs.umass.edu web server:
  - IP address: 128.119.245.12
  - port number: 80
- more shortly…

4 different addresses in TCP/IP

- Physical address Layer 2
  - Also known as the link address, is the address of a node as defined by its LAN or WAN
- Logical address Layer 3 (32-bit , IPv4 128-bit IPv6)
  - Logical (IP) addresses are for universal communications that are independent of underlying physical networks
- Port address Layer 4 (16-bit)
  - Port addresses differentiate different processes
- Application-specific address Layer 7
  - Some applications have user-friendly addresses that are designed for that specific application, such as email address, URL.

App-layer protocol defines

- types of messages exchanged,
  - e.g., request, response
- message syntax:
  - what fields in messages & how fields are delineated 勾画轮廓
- message semantics 语义
  - meaning of information in fields
- rules for when and how processes send & respond to messages

open protocols:

- defined in RFCs (request for comments)
- allows for interoperability 互操作性
- e.g., HTTP (web), SMTP (email)

proprietary protocols: 私有协议

- e.g., Skype

Session and Presentation Layers

- Theoretical only – not implemented in the Internet
- Session Layer (layer 5):
  - Takes care of a connection between two hosts for the lifetime of that connection
  - Authentication + authorisation (who is the user, what can they do)
  - In working Internet this is at the application layer.
- Presentation Layer (layer 6):
  - Translates data for an application.
  - For example takes care of detail of character set used to encode string of characters.
  - In working Internet this is at the application layer.

Test your understanding

- In ISO/OSI model which layer was planned to be responsible for:
- Options:
  - (A) Authentication and authorisation (Session Layer – layer 5)
  - (B) Sending and receiving email (Application layer – layer 7)
  - (C) Encoding character sets (Presentation Layer – layer 6)
  - (D) Converting bits to a signal for WiFi (Physical Layer – layer 1)
- If you had problems review start of lecture.

Transport Layer: outline

3.1 transport-layer services

3.2 multiplexing and demultiplexing

3.3 connectionless transport: UDP

3.4 principles of reliable data transfer

3.5 connection-oriented transport: TCP

- segment structure
- reliable data transfer
- flow control
- connection management

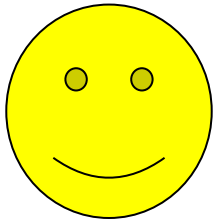
3.6 principles of congestion control

3.7 TCP congestion control

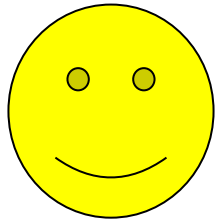
Transport layer True/False

- A) Every device on the network needs a transport layer.

- B) A device must have a transport layer to run networked applications over TCP/IP



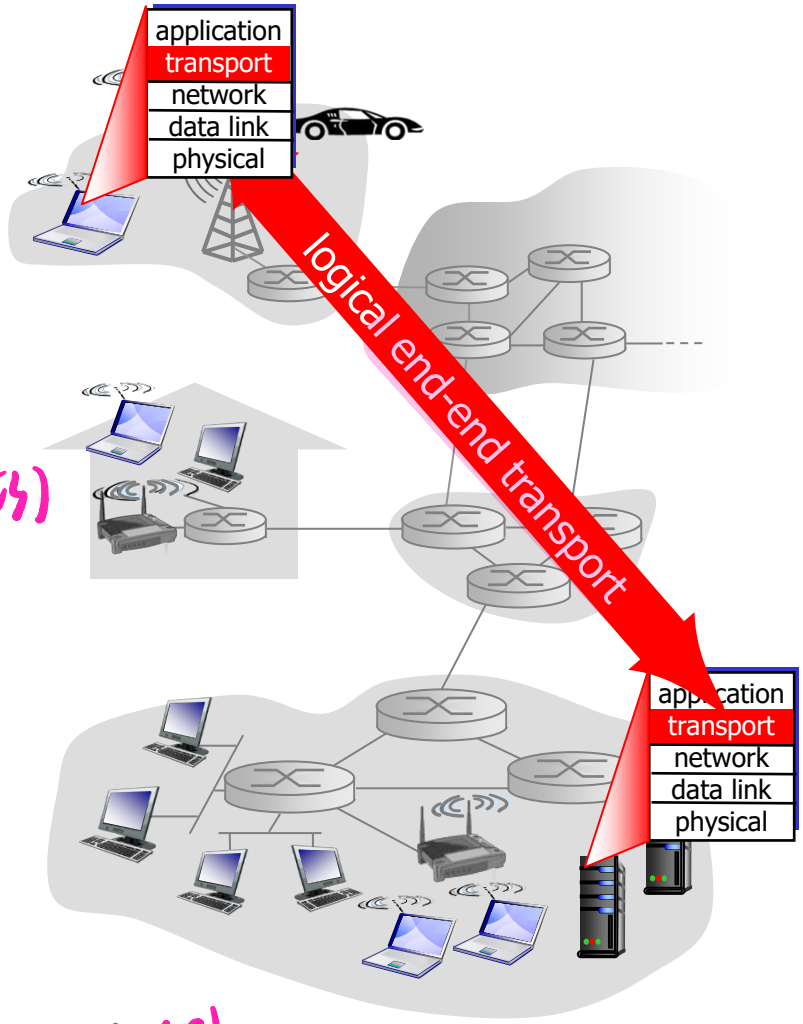
- C) If it is needed the Transport layer provides reliable in order delivery of packets.



rdt

Transport services and protocols

- provide *logical communication* between app processes running on different hosts
- transport protocols run in end systems
  - send side: breaks app messages into *segments*, passes to network layer
  - rcv side: reassembles segments into messages, passes to app layer
- more than one transport protocol available to apps
  - Internet: TCP and UDP



Transport vs. network layer

- *network layer:* logical communication between hosts
- *transport layer:* logical communication between processes
  - relies on, enhances, network layer services

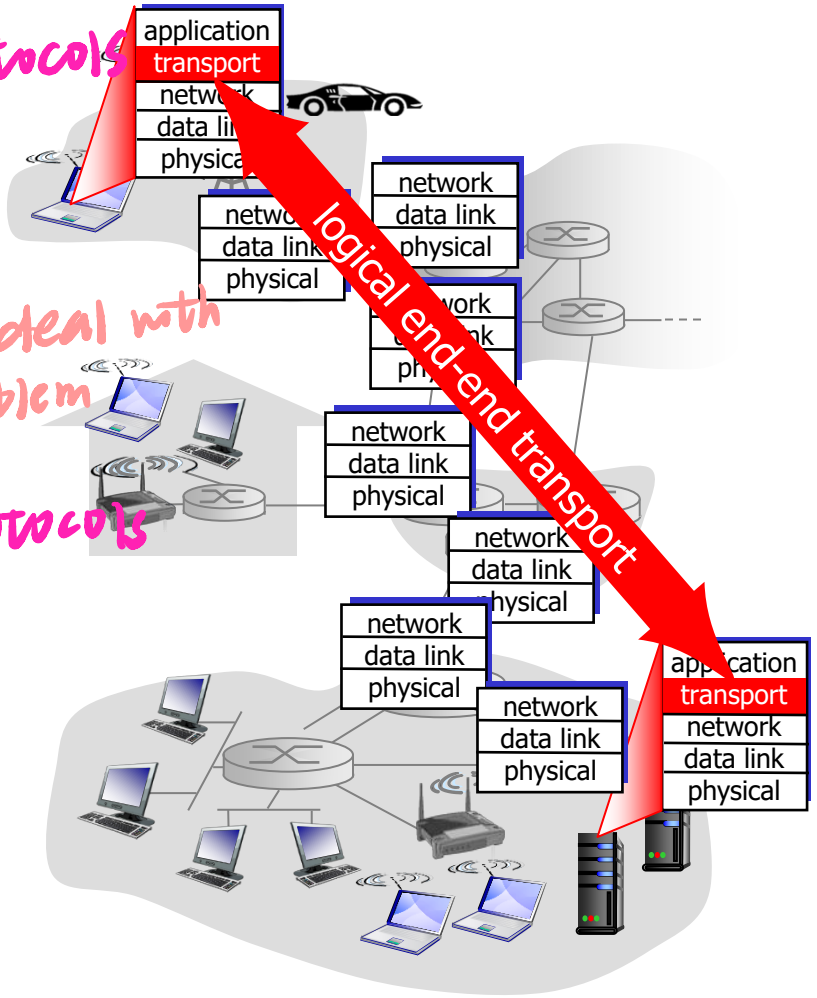
household analogy:

12 kids in Ann's house sending letters to 12 children in Bill's house:

- hosts = houses
- processes = children
- app messages = letters in envelopes
- transport protocol = (Ann and Bill who give letter to correct child)
- network-layer protocol = (postal service)

Internet transport-layer protocols

- reliable, in-order delivery (TCP)
  - congestion control
  - flow control
  - connection setup
- unreliable, unordered delivery: UDP
  - no-frills extension of "best-effort" IP
- services not available:
  - delay guarantees
  - bandwidth guarantees



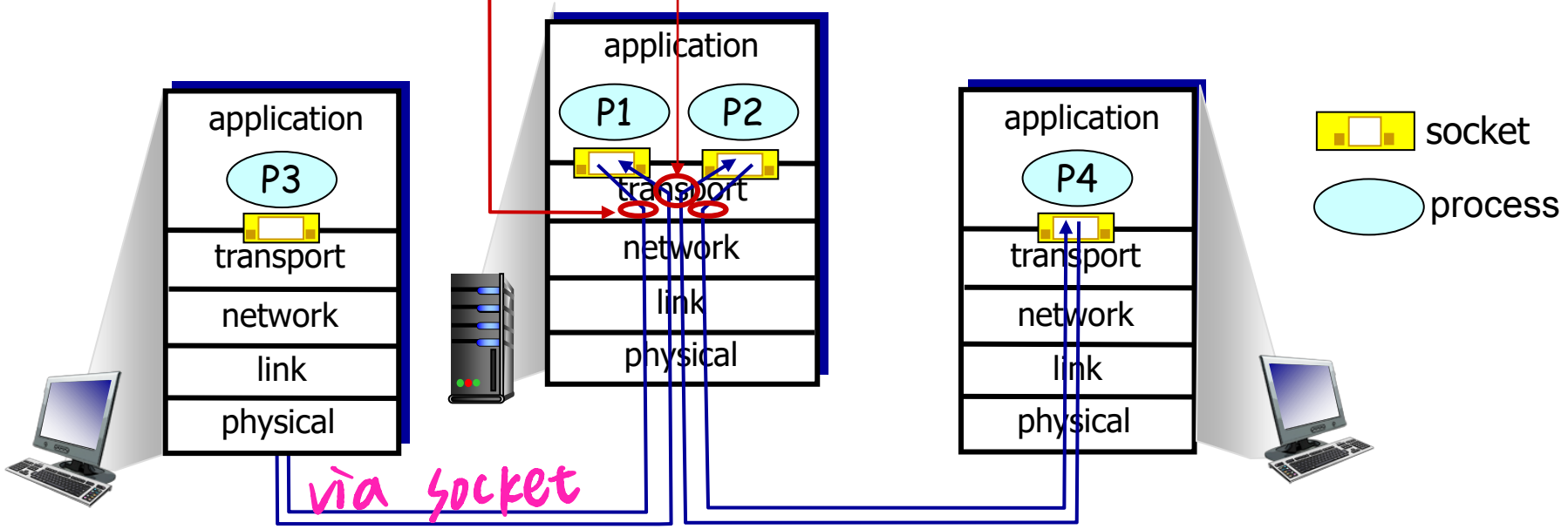
Multiplexing/demultiplexing

- Multiplexing (Mux):
  - Combining several streams of data into a single stream.
  - Example – your phone is browsing the web, refreshing your email, connecting to wechat at the same time. All these connections are sent over the same link.
- Demultiplexing (Demux)
  - The opposite process. A stream of data is separated out into its individual components.
  - The stream of packets the phone received is split up and sent to the appropriate program for web, email, wechat.

Multiplexing/demultiplexing

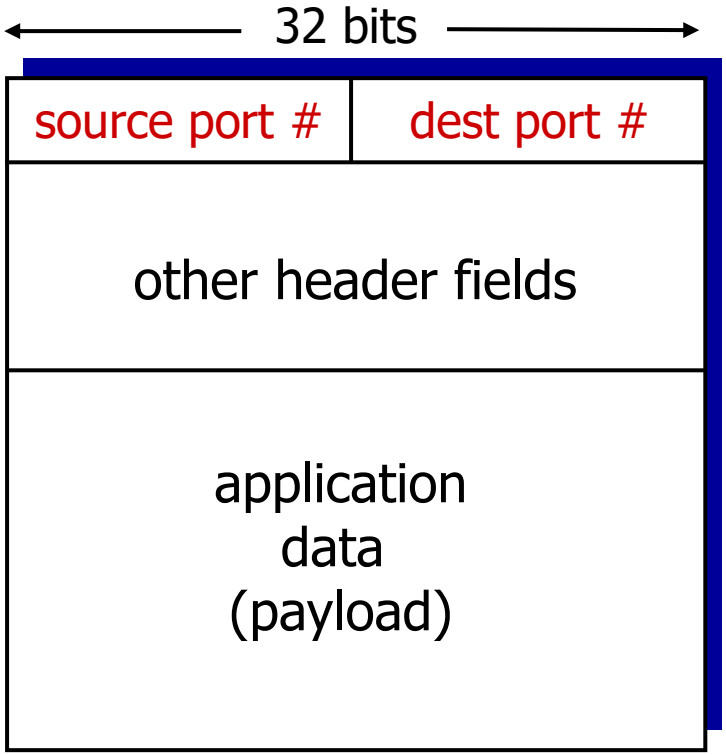
multiplexing at sender:
handle data from multiple sockets, add transport header (later used for demultiplexing)

demultiplexing at receiver:
use header info to deliver received segments to correct socket



How demultiplexing works

- host receives IP datagrams
  - each datagram has source IP address, destination IP address
  - each datagram carries one transport-layer segment
  - each segment has source, destination port number
- host uses *IP addresses & port numbers* to direct segment to appropriate socket



TCP/UDP segment format

Test your understanding

- Multiplexing (mux) or demultiplexing (demux)
- Options:
  - (A) Combining several traffic sources to one. (Multiplexing)
  - (B) Performed at the receiver. (Demultiplexing)
  - (C) Taking one stream of traffic and breaking it up into several to be sent to different processes. (Demultiplexing).
- If you cannot understand the answers review the start of section 3.2

connectionless
connection oriented

Connectionless demultiplexing

- *recall:* created socket has host-local port #:

```
DatagramSocket mySocket1
= new DatagramSocket(12534);
```

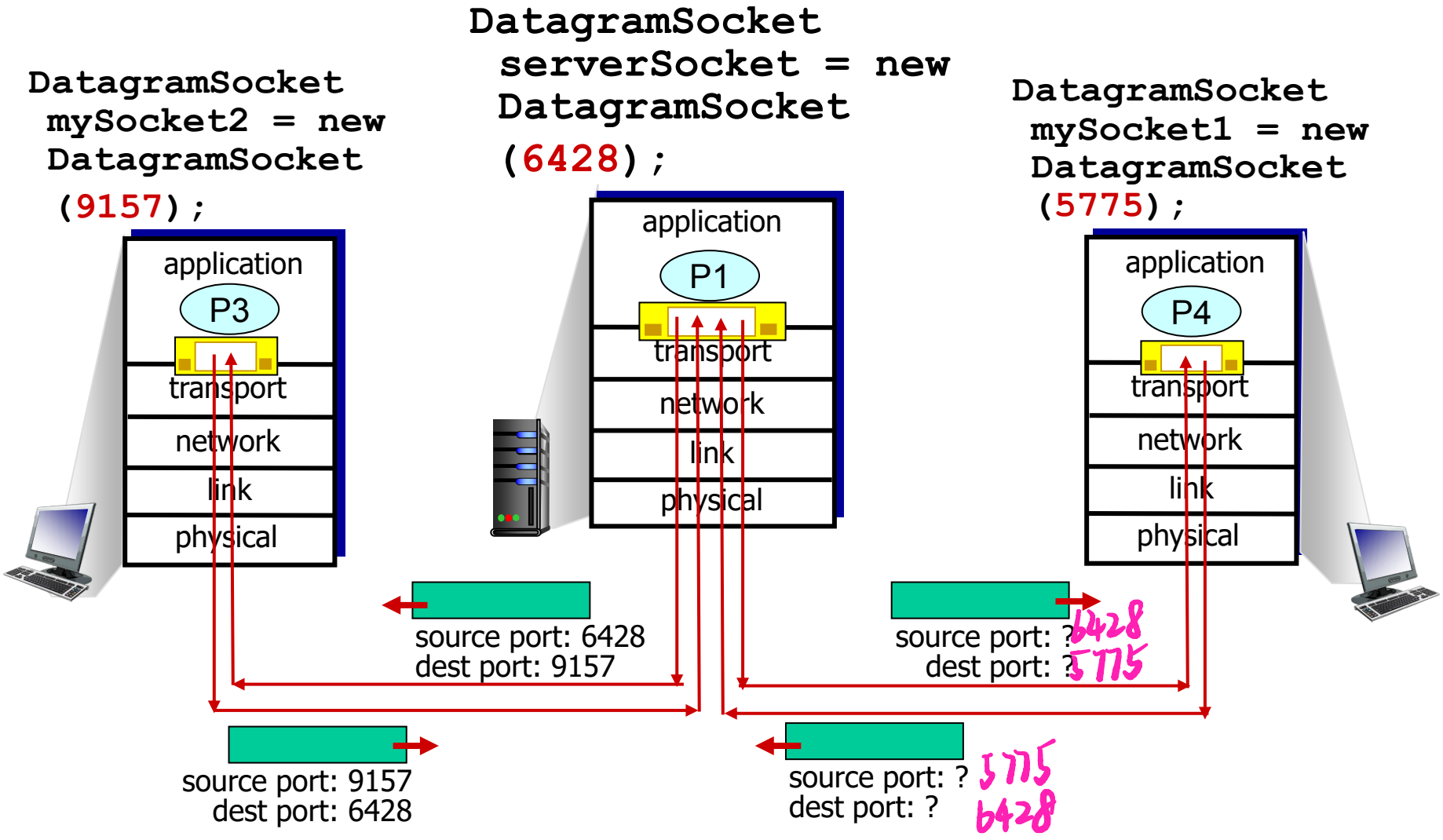
port number

- *recall:* when creating datagram to send into UDP socket, must specify
  - destination IP address
  - destination port #

- when host receives UDP segment:
  - checks destination port # in segment
  - directs UDP segment to socket with that port #

➡ IP datagrams with *same dest. port #,* but different source IP addresses and/or source port numbers will be directed to *same socket* at dest

Connectionless demux: example



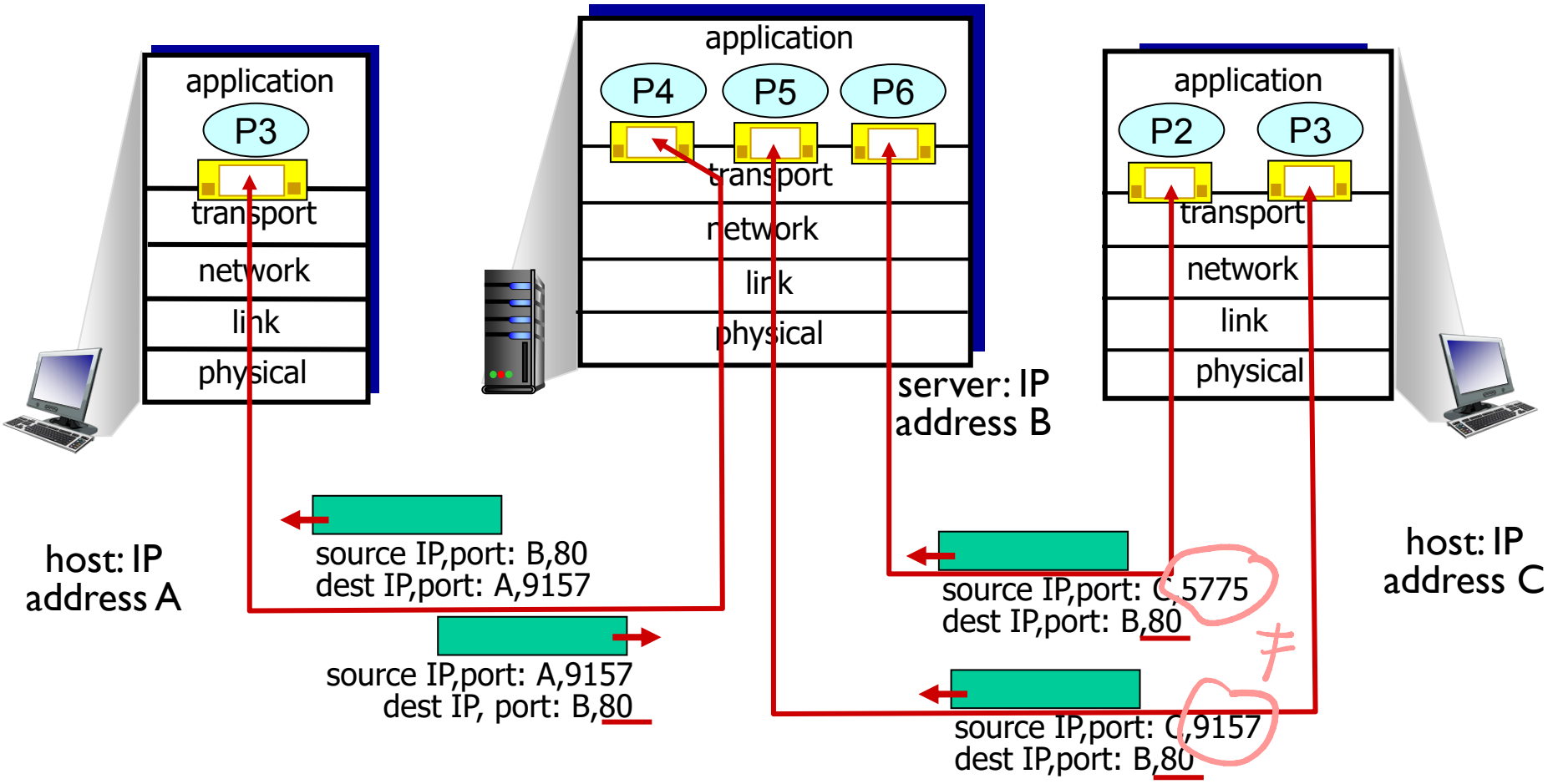
Connection-oriented demux

clear

- TCP socket identified by 4-tuple:
  - source IP address
  - source port number
  - dest IP address
  - dest port number
- demux: receiver uses all four values to direct segment to appropriate socket

- server host may support many simultaneous TCP sockets:
  - each socket identified by its own 4-tuple
- web servers have different sockets for each connecting client
  - non-persistent HTTP will have different socket for each request

Connection-oriented demux: example



three segments, all destined to IP address: B, dest port: 80 are demultiplexed to *different* sockets

Typical port numbers for applications

- Port 80: Standard HTTP (Hypertext Transfer Protocol – browse the web
- Port 22: ssh (secure shell) log in from remote computer
- Port 25: SMTP (simple mail transfer protocol) – send mail via this host
- Port 143: IMAP (Internet Message Access Protocol) – read your email
- Port 443: HTTPS (secure HTTP) browse the web securely
- These are just conventions, you could change them if you so wished on your own host.

Test your understanding

- True or false?
- Options:
  - (A) Multiplexing is performed when traffic is received.
  - False (multiplexing is combining)
  - (B) In a connection-oriented demux only one process can receive data ~~set~~ sent to a given port.
  - False (one port can connect to many processes).
  - (C) In a connection-oriented demux we need 4 pieces of information to identify the process to receive a packet.
  - True (source+dest port plus source + dest IP address)
  - (D) Connectionless demux is more efficient.
  - False (it depends on circumstances which is efficient)
- If you did not get the answers right review section 3.2

UDP: User Datagram Protocol [RFC 768]

- Simplest usable Internet transport protocol
- "best effort" service, UDP segments may be:
  - lost
  - delivered out-of-order to app
- *connectionless:*
  - no "handshaking" between UDP sender, receiver (can send immediately without asking first)
  - each UDP packet handled independently of others

- UDP use:
  - streaming multimedia apps (loss tolerant, rate sensitive)
  - DNS
  - SNMP
- reliable transfer over UDP:
  - add reliability at application layer
  - application-specific error recovery!

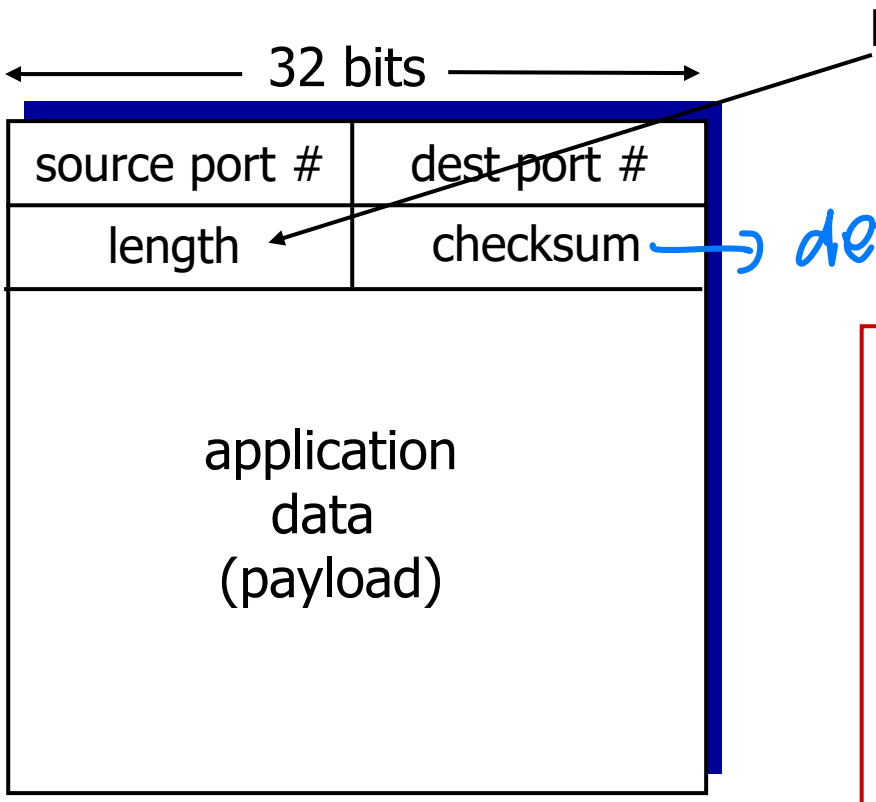
Give advantages of UDP over TCP

- Simple to implement , no timers, ACK, etc.
  - I just want to get something working.
- Efficient : No "overhead" (short header length)
  - Very good if I am sending short packets
- No setting up of a connection connectionless
  - I can send my data immediately – very fast
  - (Later we will see TCP handshake)
- Sometimes we don't want retransmission
  - In a live transmission might be not useful to resend data.
  - Broadcast of data to many people

& can choose rate of sending

UDP: segment header

header: 8 bytes.
4 x 16 bits



UDP segment format

why is there a UDP?

- no connection establishment (which can add delay)
- simple: no connection state at sender, receiver
- small header size
- no congestion control: UDP can blast away as fast as desired

Practice exam 15/16 Q3c

c) Answer the following questions related to User Datagram Protocol (UDP)

i) List and briefly describe each header field in UDP.

Source port: Port identifying the application socket sending the data

Dst port: Port identifying the application socket to receive the data

Checksum: A sum over the header and data used to detect if the packet has corrupt.

Length: Length of packet including header at layer 4

UDP checksum

Goal: detect "errors" (e.g., flipped bits – 0 becomes 1 or 1 becomes 0) in transmitted segment

sender:

- treat segment contents, including header fields, as sequence of 16-bit integers
- checksum: addition (one's complement sum) of segment contents
- sender puts checksum value into UDP checksum field

checksum

receiver:

- compute checksum of received segment
- check if computed checksum equals checksum field value:
  - NO - error detected
  - YES - no error detected. *But maybe errors nonetheless?* More later ….

Test your understanding

- UDP questions:
  - (A) When it is examined by the transport layer does a UDP packet have layer 2 and 3 headers?

    No – the network layer removes those headers before sending the packet up. The application layer could not add them.
  - (B) What is the purpose of the UDP checksum?

    Helps us tell if the packet is corrupted
  - (C) What are the four things in a UDP header apart from data?

    Checksum, packet length, source port, dest port
  - (D) Is UDP connection oriented or connectionless?

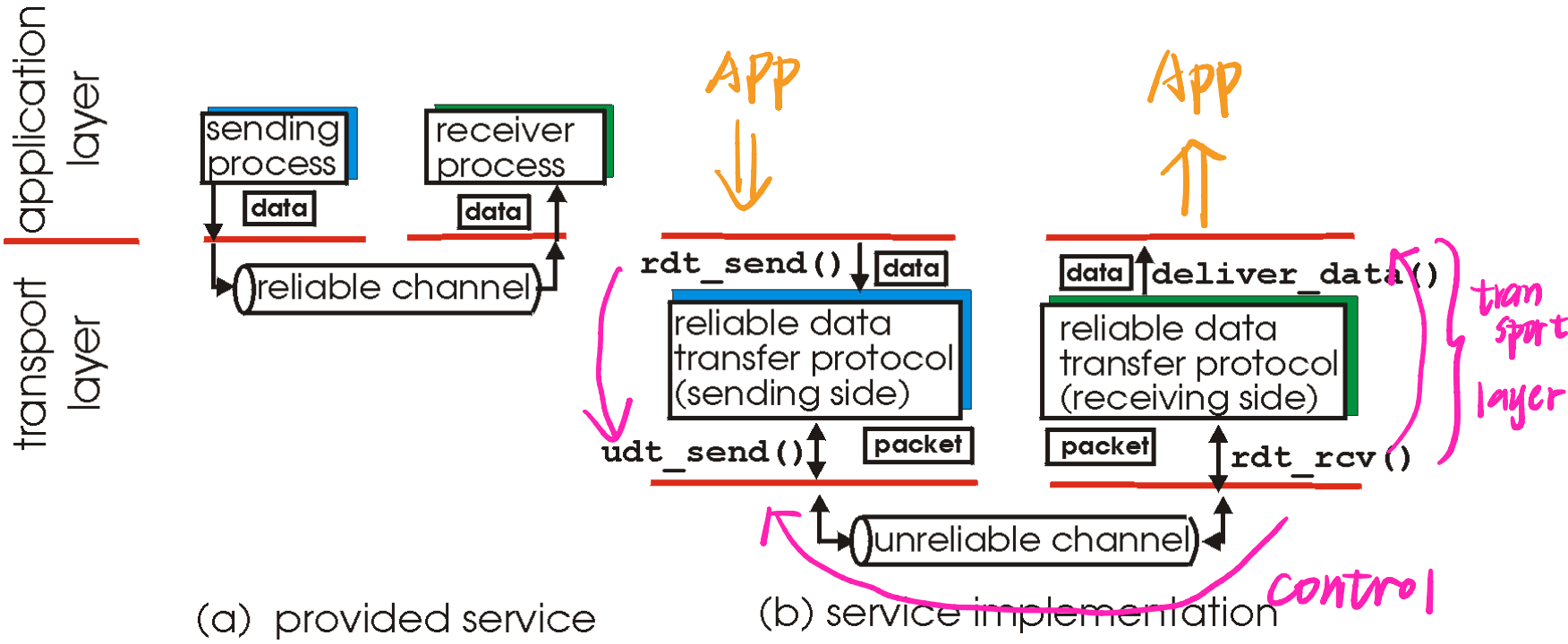
    Connectionless.
- If you had problems with these questions review section 3.3

What have we learned?

- Brief look at Application Layer (layer seven) protocols which programmers use.
- Transport Layer introduction (layer four).
  - Deliver data to application sockets themselves.
  - Responsible for "end-to-end" connection.
- User Datagram Protocol (transport layer):
  - Simple, connectionless.
  - Packets may be lost and reordered.
  - Useful for simple situations or real-time.

Principles of reliable data transfer

- important in application, transport, link layers
  - top-10 list of important networking topics!



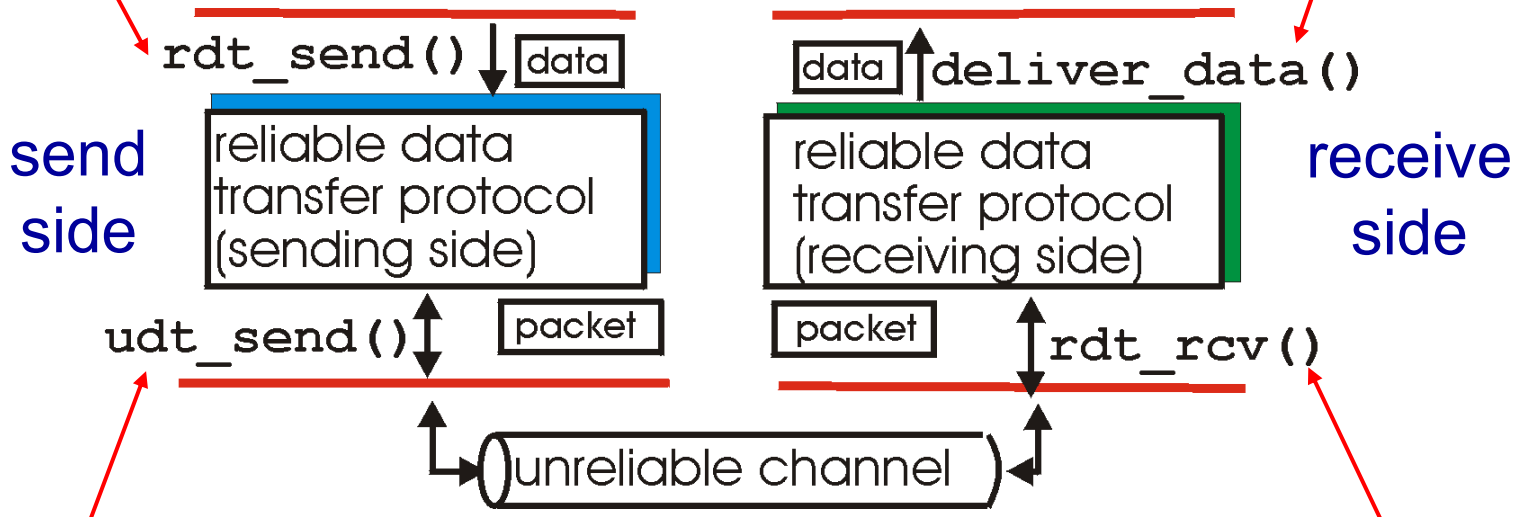
- characteristics of unreliable channel will determine complexity of reliable data transfer protocol (rdt)

not in reality

Reliable data transfer: getting started

`rdt_send()`: called from above, (e.g., by app.). Passed data to deliver to receiver upper layer

`deliver_data()`: called by **rdt** to deliver data to upper

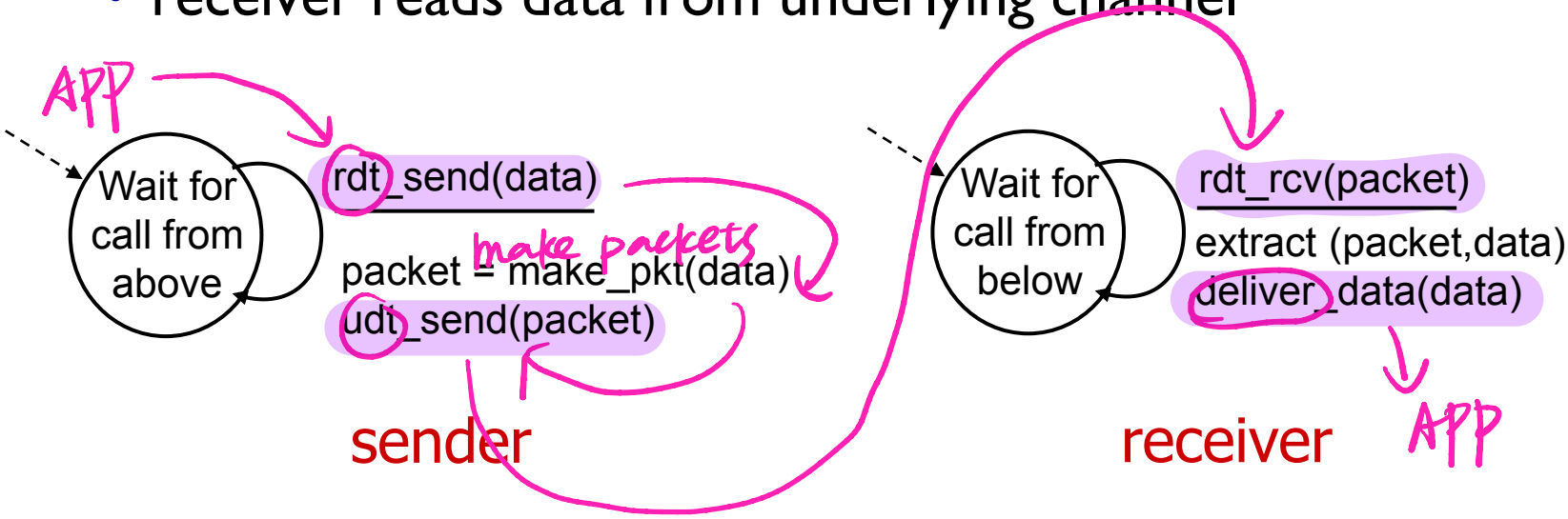


`udt_send()`: called by rdt, to transfer packet over unreliable channel to receiver

`rdt_rcv()`: called when packet arrives on rcv-side of channel

rdt1.0: reliable transfer over a reliable channel

- underlying channel perfectly reliable
  - no bit errors
  - no loss of packets
- separate FSMs for sender, receiver:
  - sender sends data into underlying channel
  - receiver reads data from underlying channel



rdt2.0: channel with bit errors

- We drop the assumption that the channel has no errors.
- Now the channel could have bit errors.
- We need to be able to detect and recover from errors in transmission.
- Consider this question:
  - How do "humans" recover from errors in conversation?

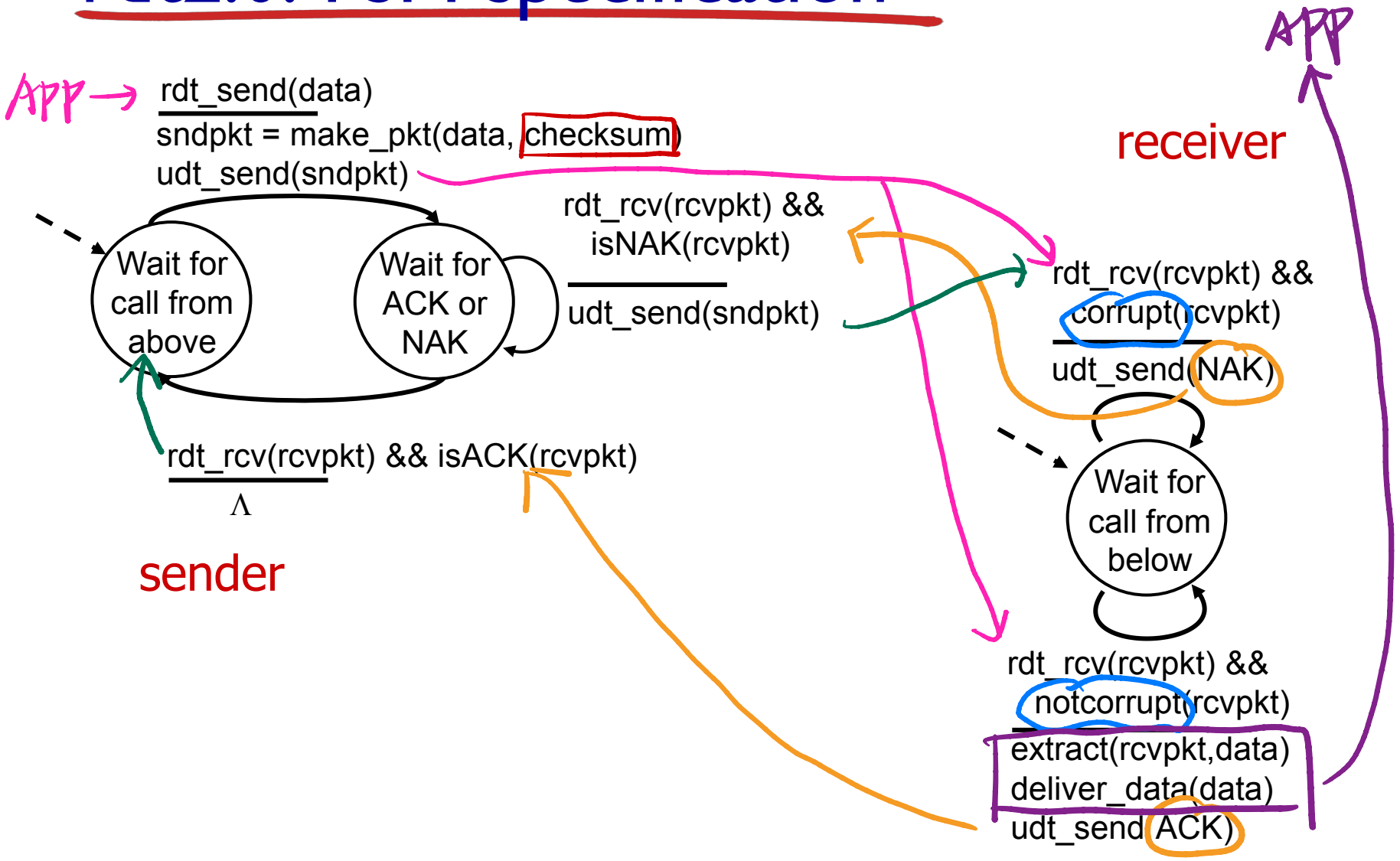
D | 0 SEND
1 | 0 RECEIVE

Checksum

rdt2.0: channel with bit errors

- underlying channel may flip bits in packet
  - checksum to detect bit errors
- *the* question: how to recover from errors:
  - *acknowledgements (ACKs):* receiver explicitly tells sender that pkt received OK
  - *negative acknowledgements (NAKs):* receiver explicitly tells sender that pkt had errors
  - sender retransmits pkt on receipt of NAK
- new mechanisms in `rdt2.0` (beyond `rdt1.0`):
  - error detection
  - feedback: control msgs (ACK,NAK) from receiver to sender

rdt2.0: FSM specification



rdt2.0 has a fatal flaw!

what happens if ACK/NAK corrupted?

- sender doesn't know what happened at receiver!
- can't just retransmit: possible duplicate

handling duplicates:

- sender retransmits current pkt if ACK/NAK corrupted
- sender adds *sequence number* (seq) to each pkt
- receiver discards (doesn't deliver up) duplicate pkt

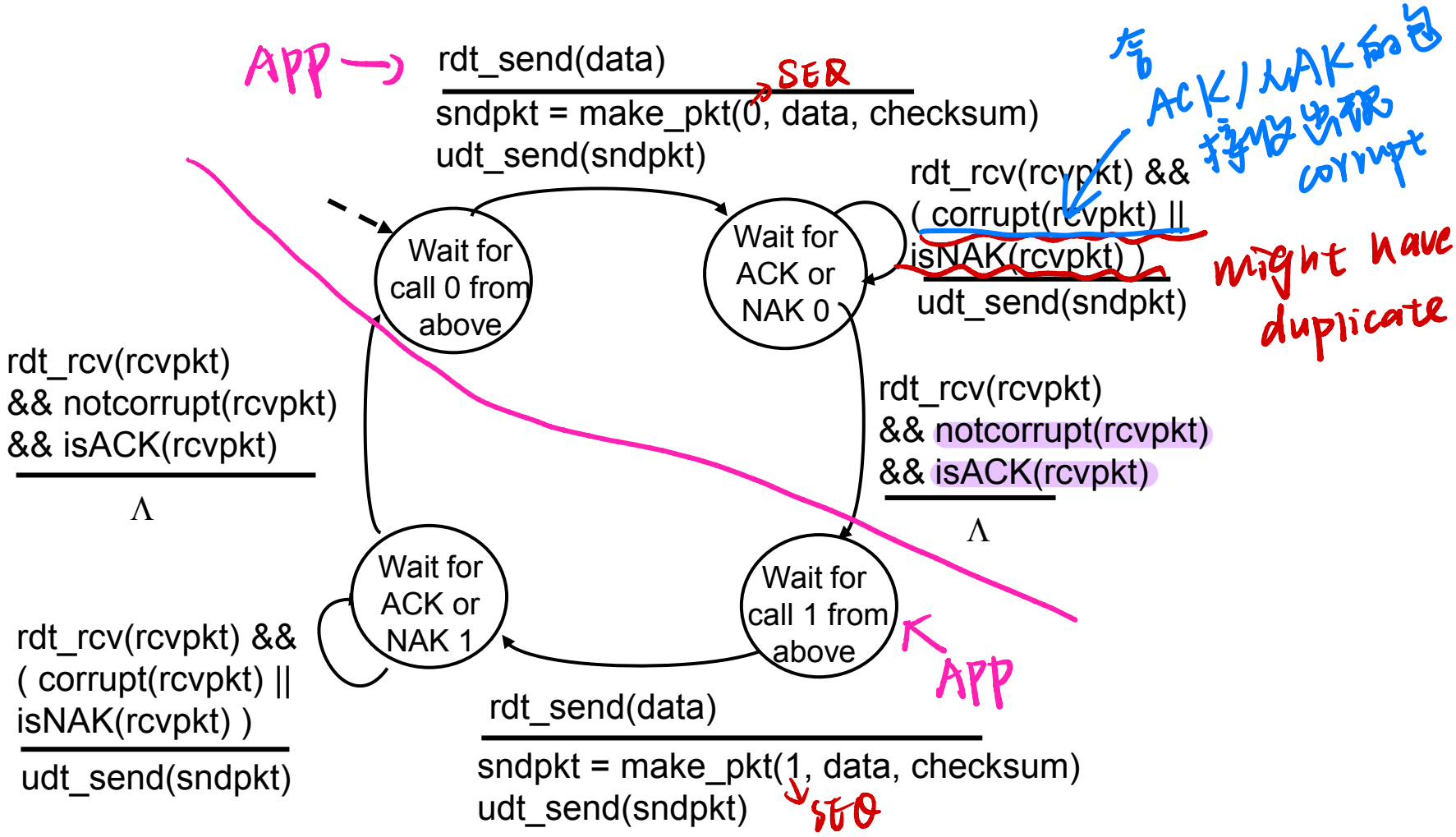
stop and wait

sender sends one packet, then waits for receiver response

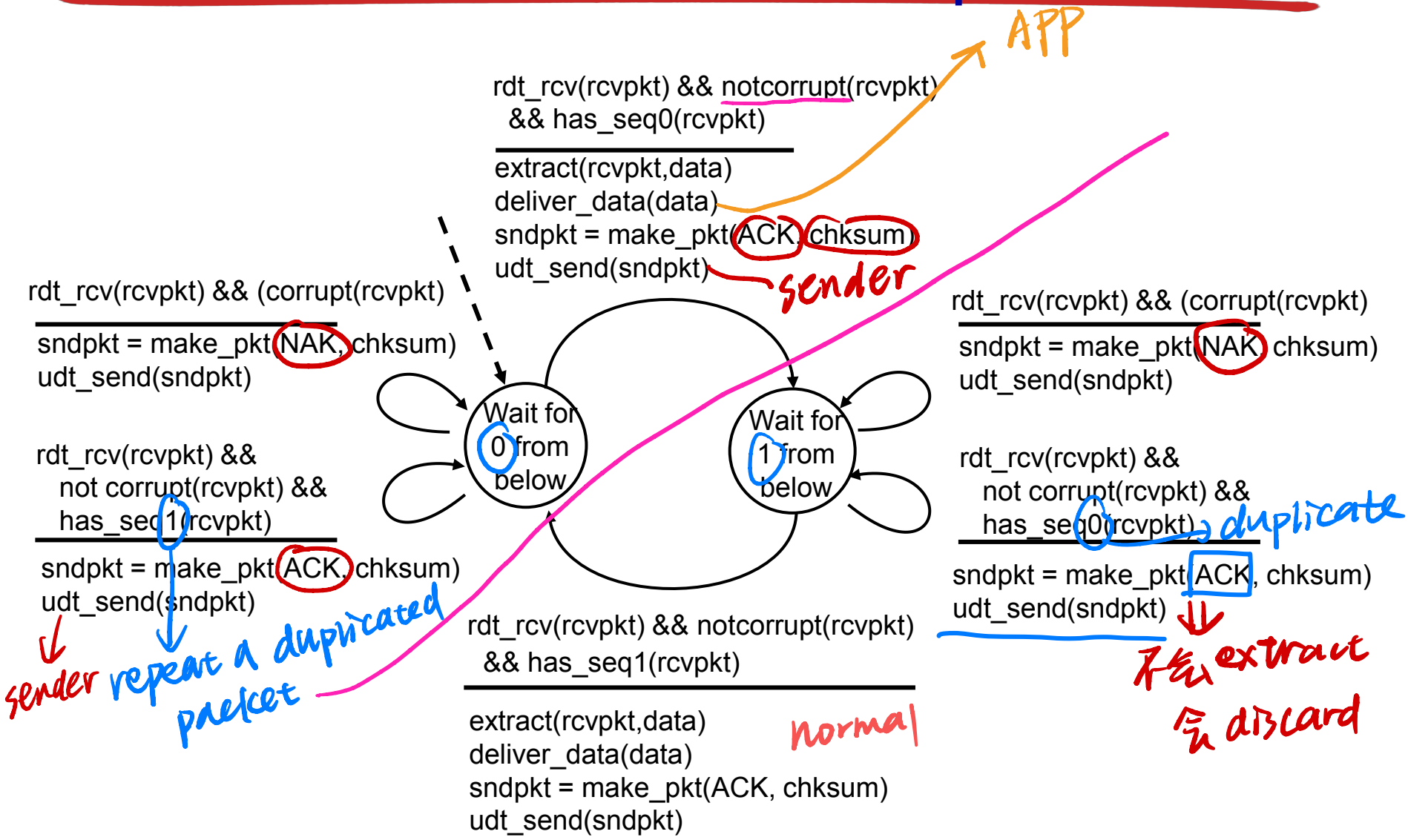
Test your understanding

- In rdt2.0:
  - A) What does the receiver send if it gets a corrupt packet?
    - Sends a NACK and stays in same state.
  - B) If the sender is in "Wait for call from above" and receives a NACK what does it do?
    - Nothing (this should never happen)
  - C) If the sender is in "Wait for ACK or NACK" and receives a NACK what does it do?
    - Sends another packet and stays in this state.
  - If you got it wrong review rdt2.0

rdt2.1: sender, handles corrupt ACK/NAKs



rdt2.1: receiver, handles corrupt ACK/NAKs



rdt2.1: discussion

sender:

- seq # added to pkt
- two seq. #'s (0,1) will suffice. Why?
- must check if received ACK/NAK corrupted
- twice as many states
  - state must "remember" whether "expected" pkt should have seq # of 0 or 1

receiver:

- must check if received packet is duplicate
  - state indicates whether 0 or 1 is expected pkt seq #
- note: receiver can *not* know if its last ACK/NAK received OK at sender

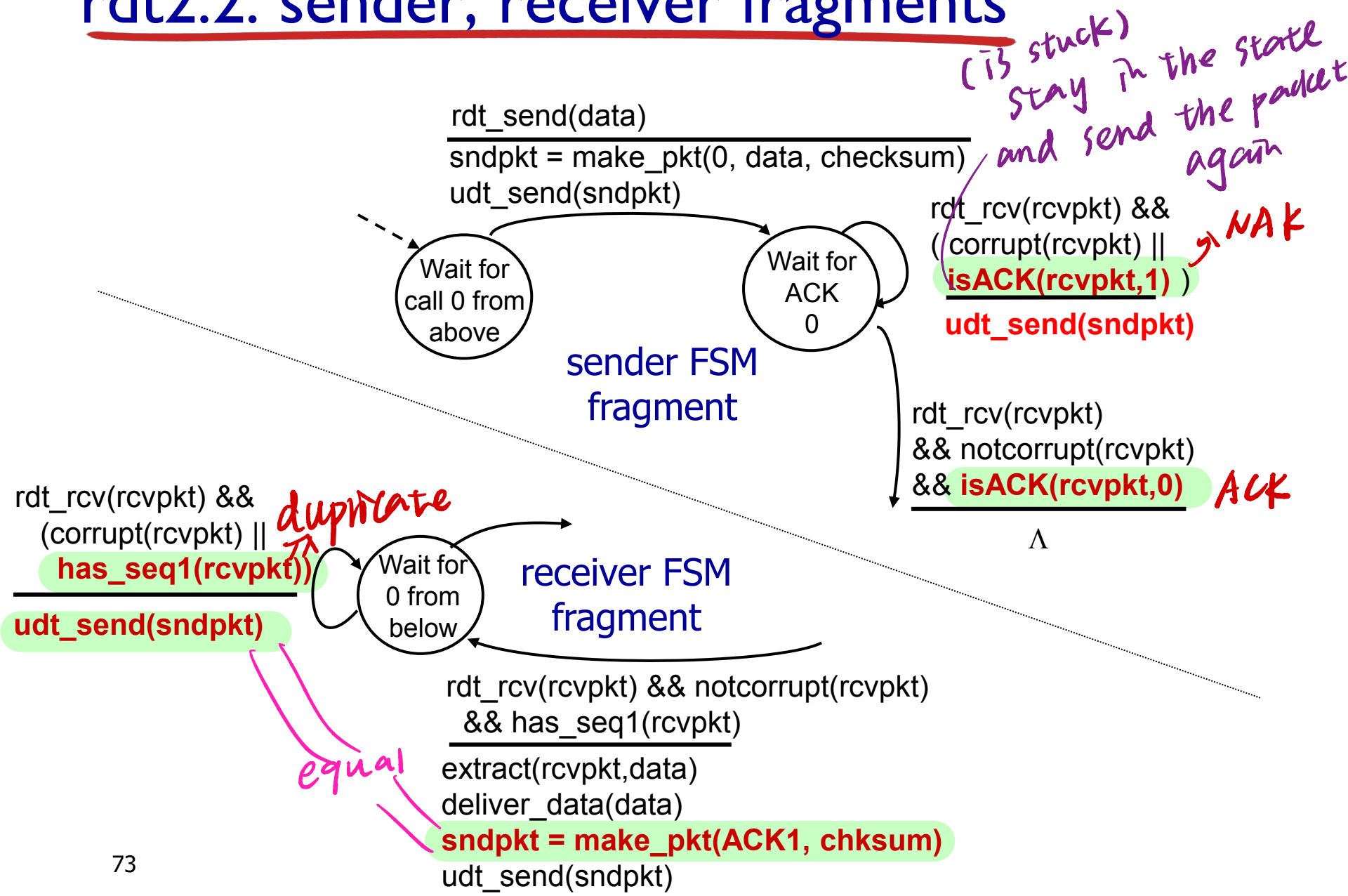
rdt2.2: a NAK-free protocol

- same functionality as rdt2.1, using ACKs only
- instead of NAK, receiver sends ACK for last pkt received OK
  - receiver must *explicitly* include seq # of pkt being ACKed
- duplicate ACK at sender results in same action as NAK: *retransmit current pkt*

ACK SEQ

NAK → ACK with wrong SEQ

rdt2.2: sender, receiver fragments



Test your understanding

- In rdt2.2:
  - A) What does the sender do if it is in "Wait for ACK 1" and it receives ACK 0?
    - This is either an ACK for an old packet (delay ACK) or a NACK. Send packet again and stay in same state. (NACK)
  - B) What does the receiver do if it is in "Wait for 1 from below" and it receives a packet with SEQ 0?
    - This is old packet. Stay in same state and resend ACK
  - C) What will cause rdt2.2 to send a NACK?
    - It can never send a NACK – if it wants to indicate NACK it sends an ACK with the "wrong" number.

rdt3.0: channels with errors *and* loss

timeout

new assumption:

underlying channel can also lose packets (data, ACKs)

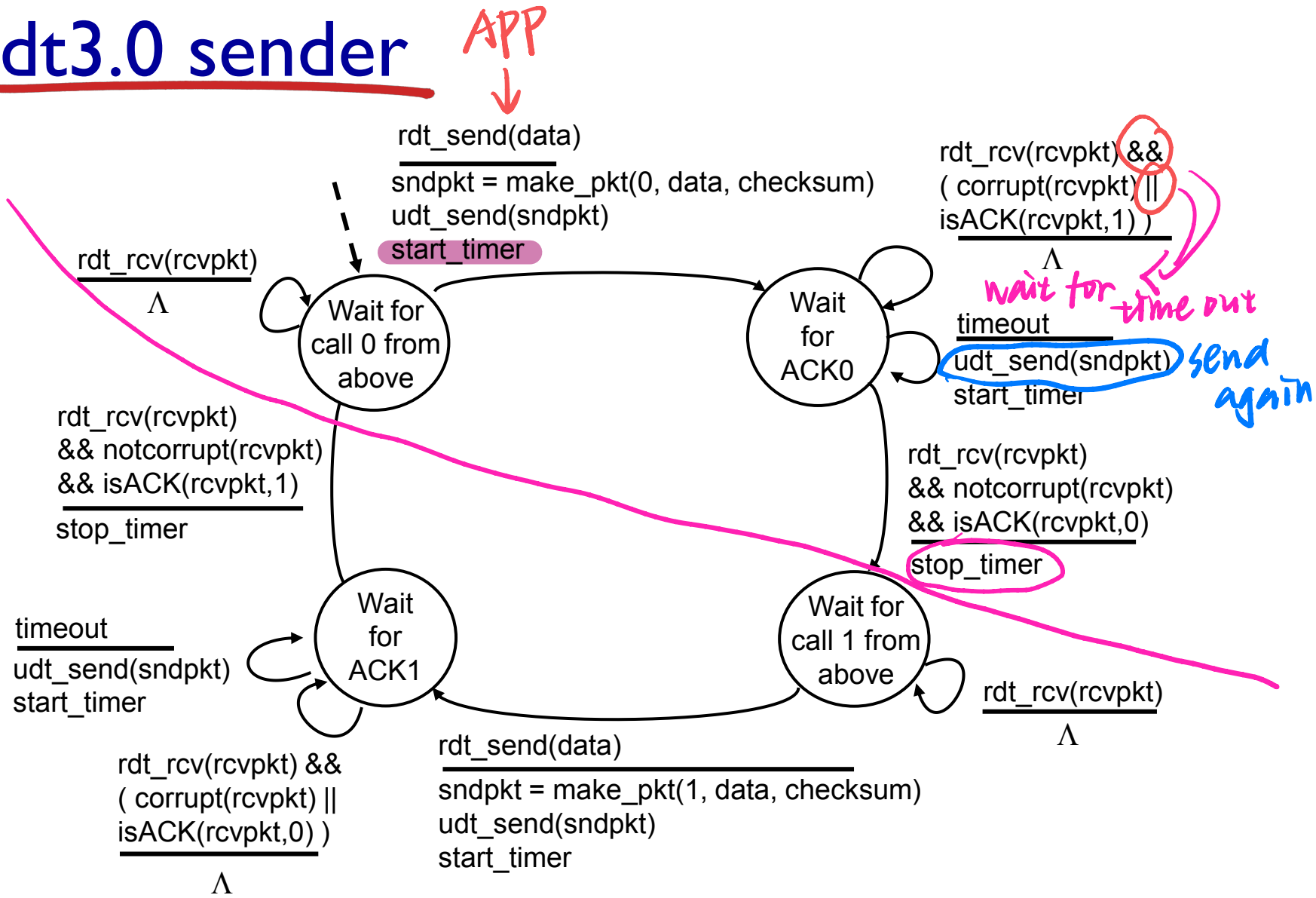
- checksum, seq. #, ACKs, retransmissions will be of help … but not enough

approach: sender waits "reasonable" amount of time for ACK

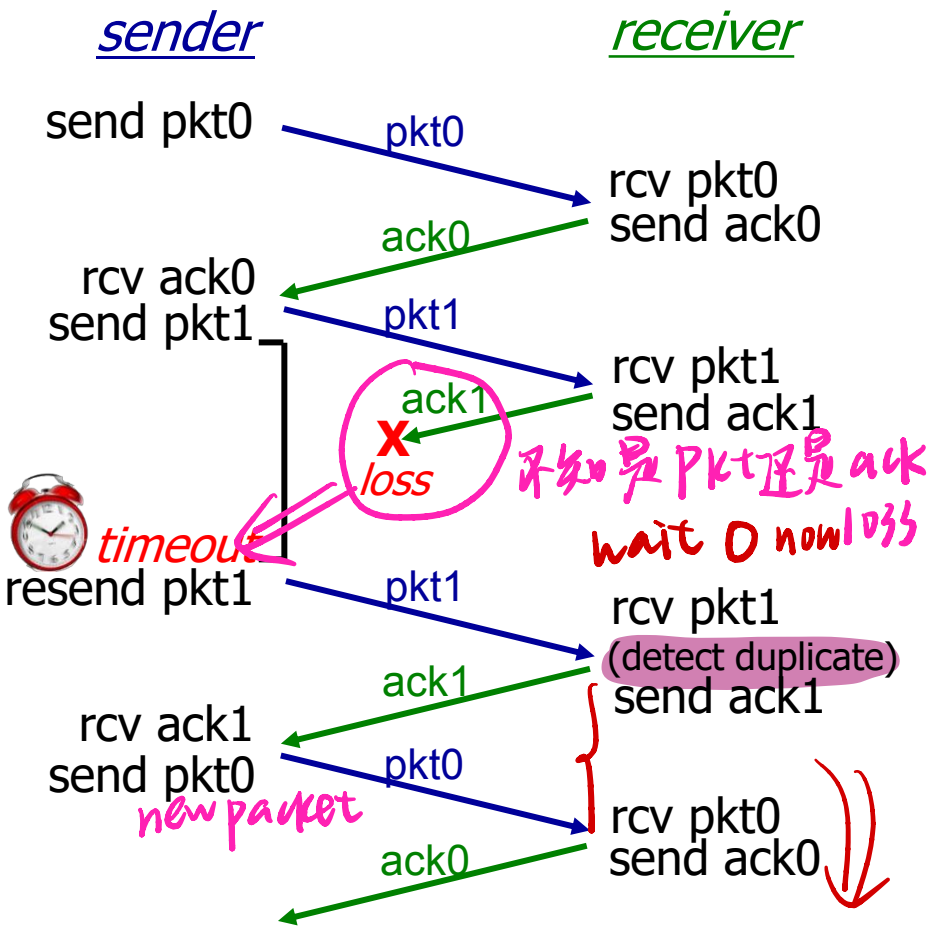
- retransmits if no ACK received in this time
- if pkt (or ACK) just delayed (not lost):
  - retransmission will be duplicate, but seq. #'s already handles this
  - receiver must specify seq # of pkt being ACKed
- requires countdown timer

计时器

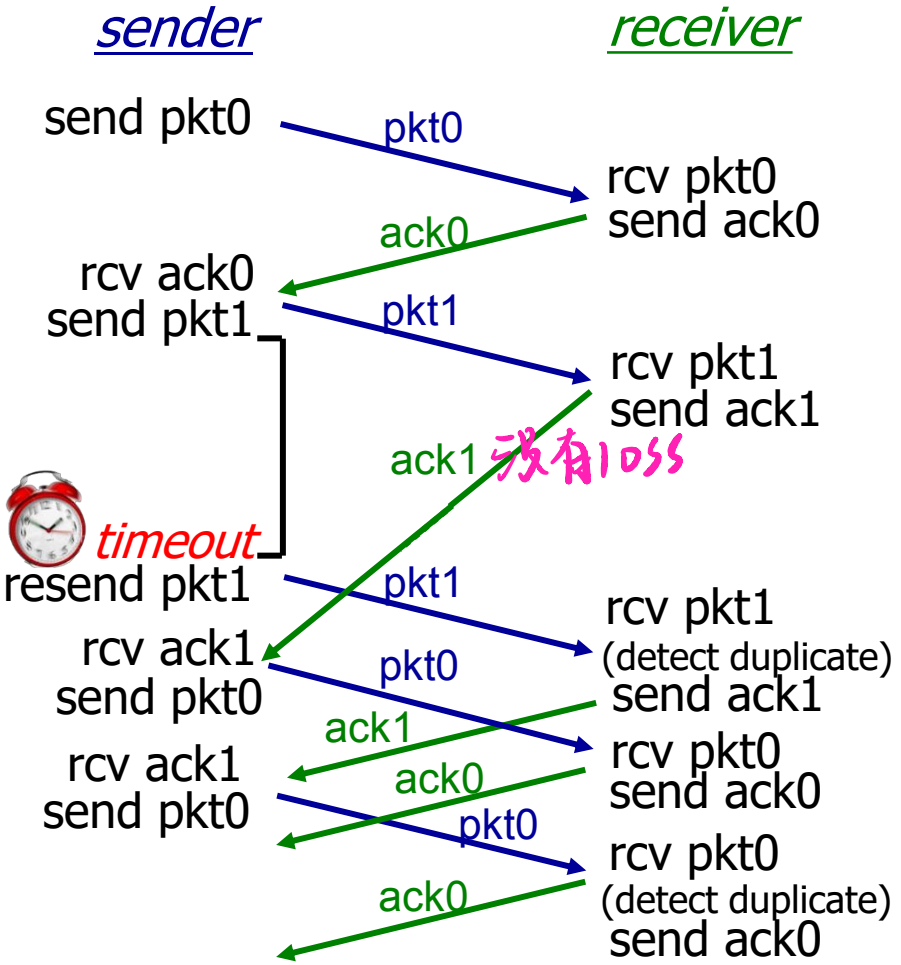
rdt3.0 sender



rdt3.0 in action



(c) ACK loss



(d) premature timeout/ delayed ACK

Give an advantage of rdt 3.0 over UDP

- Resends lost packets: deal with loss
  - If a packet is dropped it is resent
- Resends corrupted packets: deal with error
  - If a packet is corrupted it is resent
- We will see in later lectures TCP also:
  - Puts packets in order
  - Slows sending if receiver is "full"
  - Slows sending if congestion occurs

Test your understanding

- In rdt3.0:
  - A) Name the two types of error rdt3.0 can deal with?
    - Packet is corrupted. Packet did not arrive at all.
  - B) How many sequence numbers does rdt3.0 use?
    - Two (0 and 1)
  - C) How does rdt3.0 receiver indicate a packet is received but was corrupted?
    - Does not use NACK. Uses ACK with the "wrong" number – 0 if it was waiting for 1 and vice versa.

Practice exam 18/19 A Q1 c

iv) Consider a reliable data transfer (rdt) protocol over a channel with bit errors (packets are corrupted but not lost). Explain the procedure to detect and recover from errors. (5 marks)

detect

- Extract the checksum from the packet header.
- Calculate a new checksum from the packet and header data received.
- If these do not match there is an error.
- An error is reported with a NACK (or an ACK with the "wrong" number in rdt2.2, rdt3.0)

What have we learned?

- We are starting on our journey to create "reliable data transfer" over an unreliable network.
- Key concepts:
  - Sequence number – measures which packet we are currently sending
  - ACK (acknowledgement) – numbered, says that a specific packet has definitely been arrived
  - Timeout – a time to wait before we think a packet is lost

Recap of rdt 1.0 2.0, 2.1, 2.2, 3.0

- rdt 1.0 assumed no losses or errors
- rdt 2.0 introduced ACK (ACKnowledgment) and NACK to say a packet had been received or an error occurred.
- rdt 2.1 introduced the concept of the SEQuence number to deal with errors in ACKs. has corrupted
- rdt 2.2 introduced the repeated ACK – ACK with repeated sequence number means same as NACK.
- rdt 3.0 introduced timeout in case packet lost completely not just corrupted.

Test your understanding

- Which version of rdt?
  - Assumes no losses or corruption
    - rdt 1.0 – assumes a reliable channel
  - Works when packets can be corrupted but not lost.
    - rdt 2.2 – introduces ACK and sequence number to deal with corruption

    +rdt 2.1
  - Works with corruption and packets being lost or delayed.
    - rdt 3.0 – introduces timeout to cope with loss
  - If you got it wrong review rdt.

Performance of rdt3.0

- rdt3.0 is correct, but performance is very bad
- e.g.: 1 Gbps link, 15 ms delay, 8000 bit packet
- Define round-trip-time as time to propagate there and back -- (2 x end-to-end delay)

$$D_{trans} = \frac{L}{R} = \frac{8000\ bits}{10^9\ bits/sec} = 8\ microsecs$$

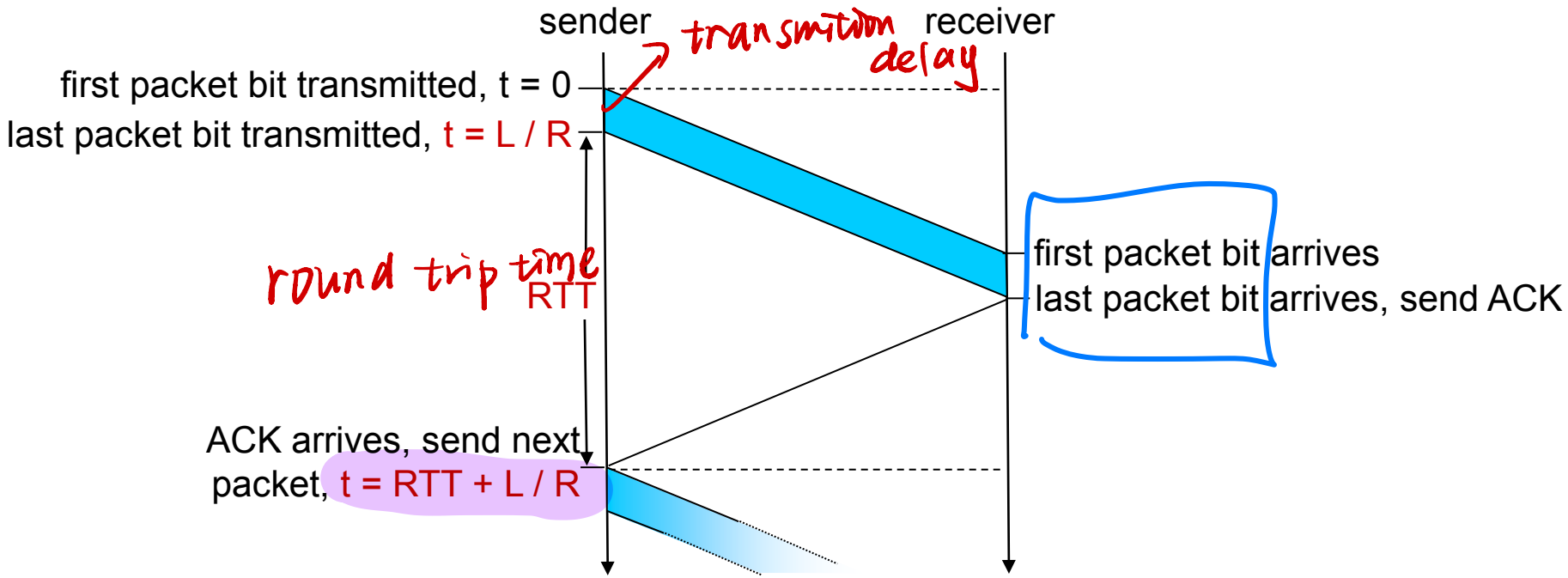
  - U_{sender}: *utilization* – fraction of time sender busy sending

$$U_{sender} = \frac{L/R}{RTT + L/R} = \frac{.008}{30.008} = 0.00027$$

    - if RTT=30 msec, 1KB pkt every 30 msec: 33kB/sec throughput over 1 Gbps link

- Badly designed protocol limts how we use resource.

rdt3.0: stop-and-wait operation



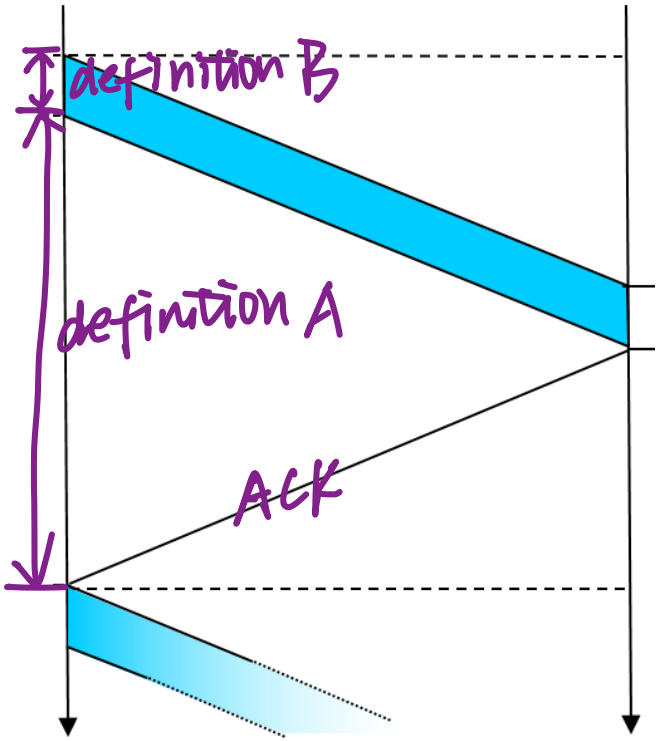
$$U_{sender} = \frac{L/R}{RTT + L/R} = \frac{.008}{30.008} = 0.00027$$

总花费时间

$$\text{throughput} = \frac{\text{总 packet}}{\text{总时间}}$$

Link utilisation

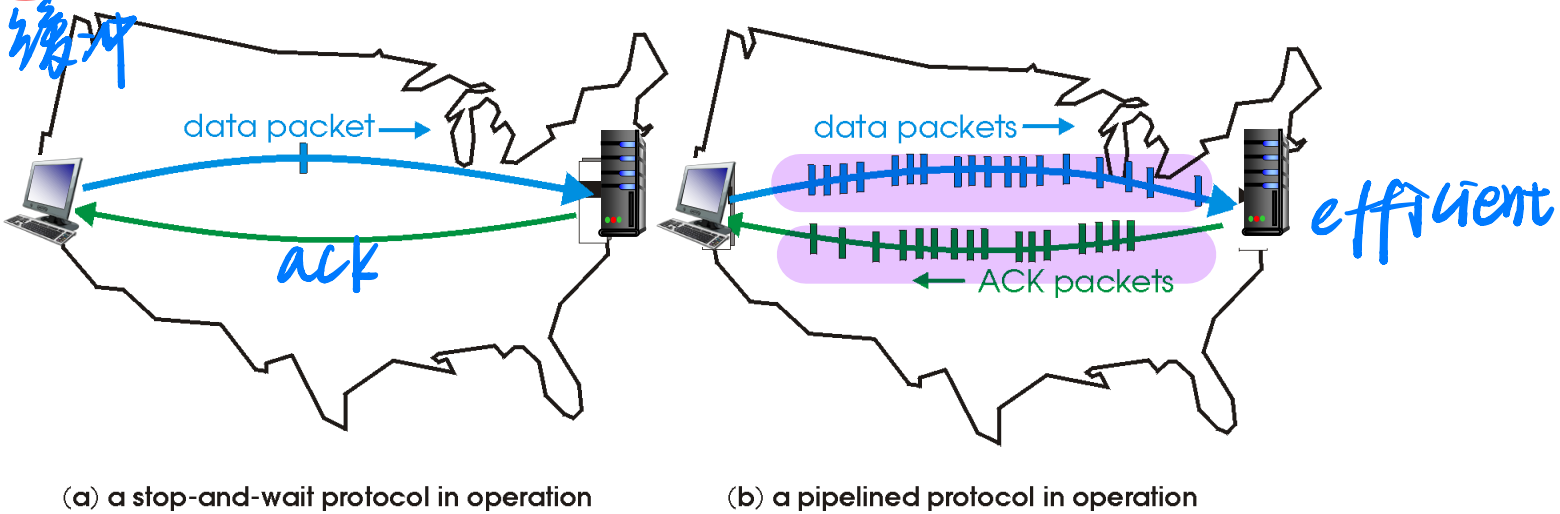
- Define link utilisation:
  - A. The time it takes between the sender finishing sending and the sender getting an ACK from the receiver.
  - B. The time it takes to send a packet.
  - C. The proportion of the time that the link is used for sending data.
  - D. The proportion of the time that the link is empty.



Pipelined protocols

pipelining: sender allows multiple, "in-flight", yet-to-be-acknowledged packets (packets with no ACK as yet)

- range of sequence numbers must be increased
- buffering at sender and/or receiver

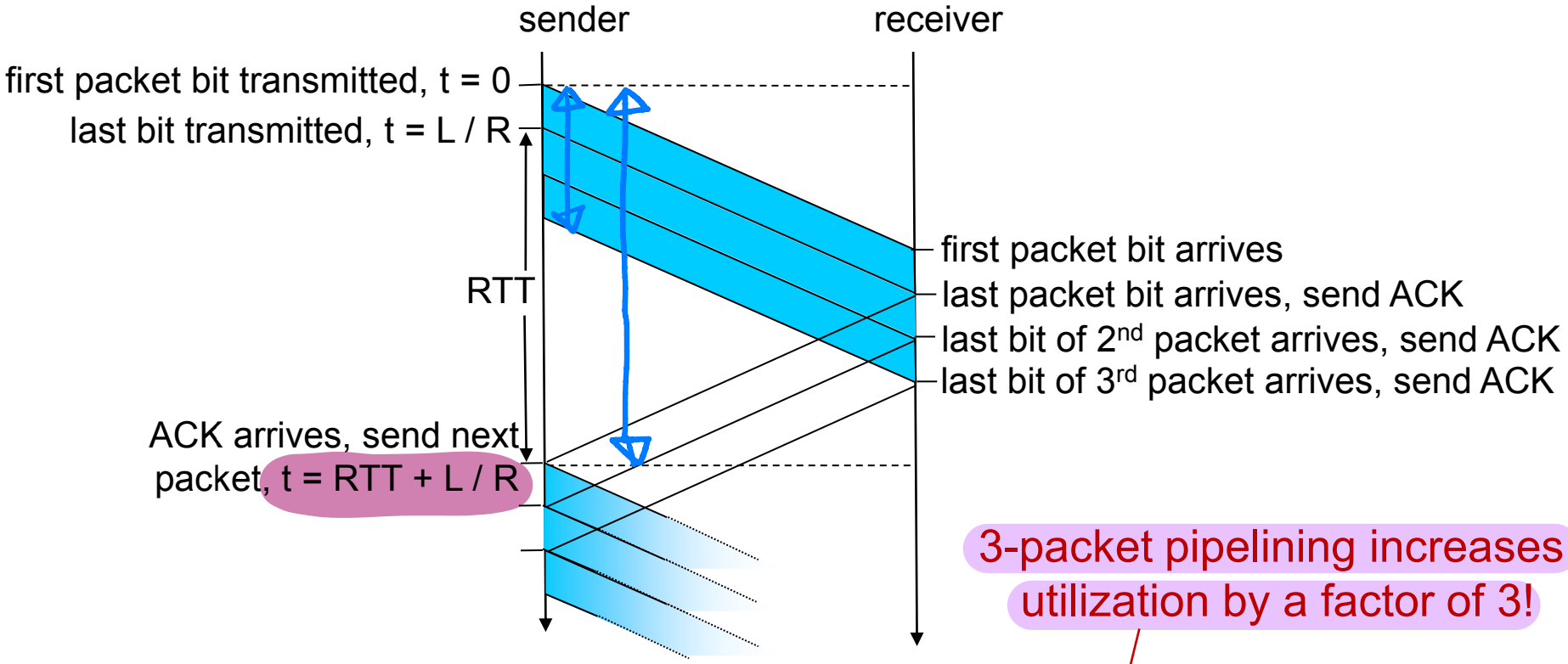


(a) a stop-and-wait protocol in operation

(b) a pipelined protocol in operation

- two generic forms of pipelined protocols: *go-Back-N*, *selective repeat*

Pipelining: increased utilization



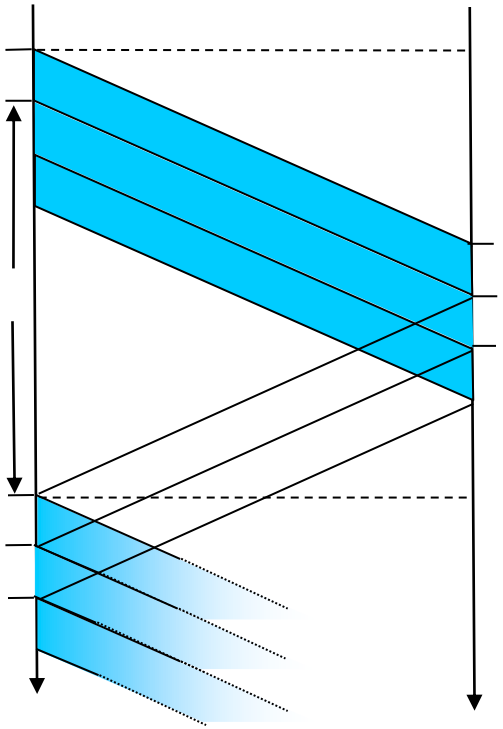
3-packet pipelining increases utilization by a factor of 3!

$$U_{sender} = \frac{3L / R}{RTT + L / R} = \frac{.0024}{30.008} = 0.00081$$

Pipelining to increase utilisation

Instead of sending packets one after another packets are sent in batches of N. The link is not full and no losses are experienced. What is effect on utilisation

A. Increases by a factor of N
B. Stays the same
C. Becomes three times as much
D. Decreases to 1/N



Test your understanding

- 1250 byte packet. 100 Mb/s link.
- End to end delay = 5ms

L= 1250x8 bits = 10,000 bits

Transmission delay = L/R = 10,000/100,000,000 = 0.0001 s = 0.1ms

- What is the utilisation if we use stop and wait.
- U = (L/R)/ (RTT+L/R) = 0.1/(10 + 0.1) ~ 0.01*
- What is the utilisation if we use stop and wait.
- U= 25* (L/R)/ (RTT+L/R) ~0.25
  If you did't get it review slides 8-11

Pipelined protocols: overview

Go-back-N:

- sender can have up to N unacked packets in pipeline
- receiver only sends *cumulative ACK*
  - doesn't ACK packet if there's a gap
  - Doesn't buffer out of order
- sender has timer for oldest unacked packet
  - when timer expires, retransmit *all* unacked packets

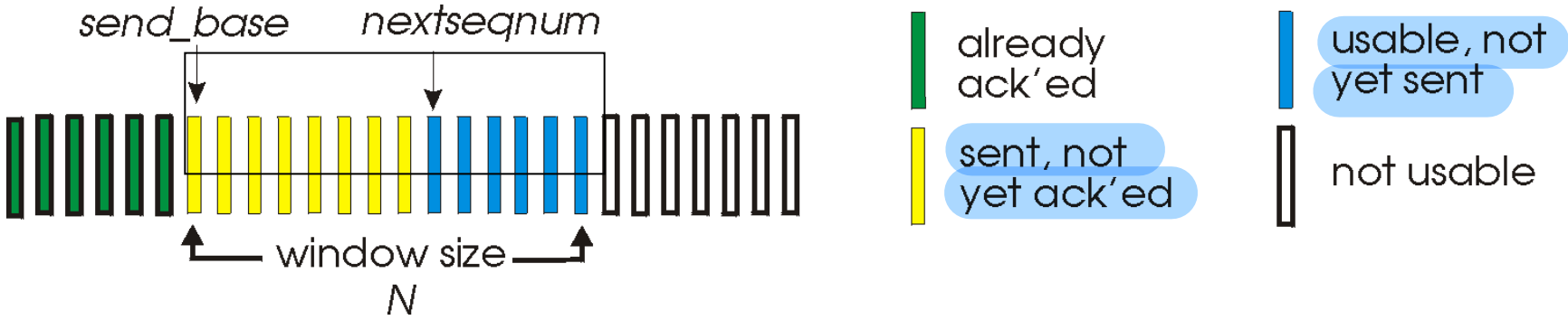
Selective Repeat:

- sender can have up to N packets which it has not seen ACK for in "pipeline"
- Receiver sends *individual ACK* for each packet
- sender maintains timer for each packet with no ACK
  - when timer expires, retransmit only that unacked packet

more complicated
more efficient

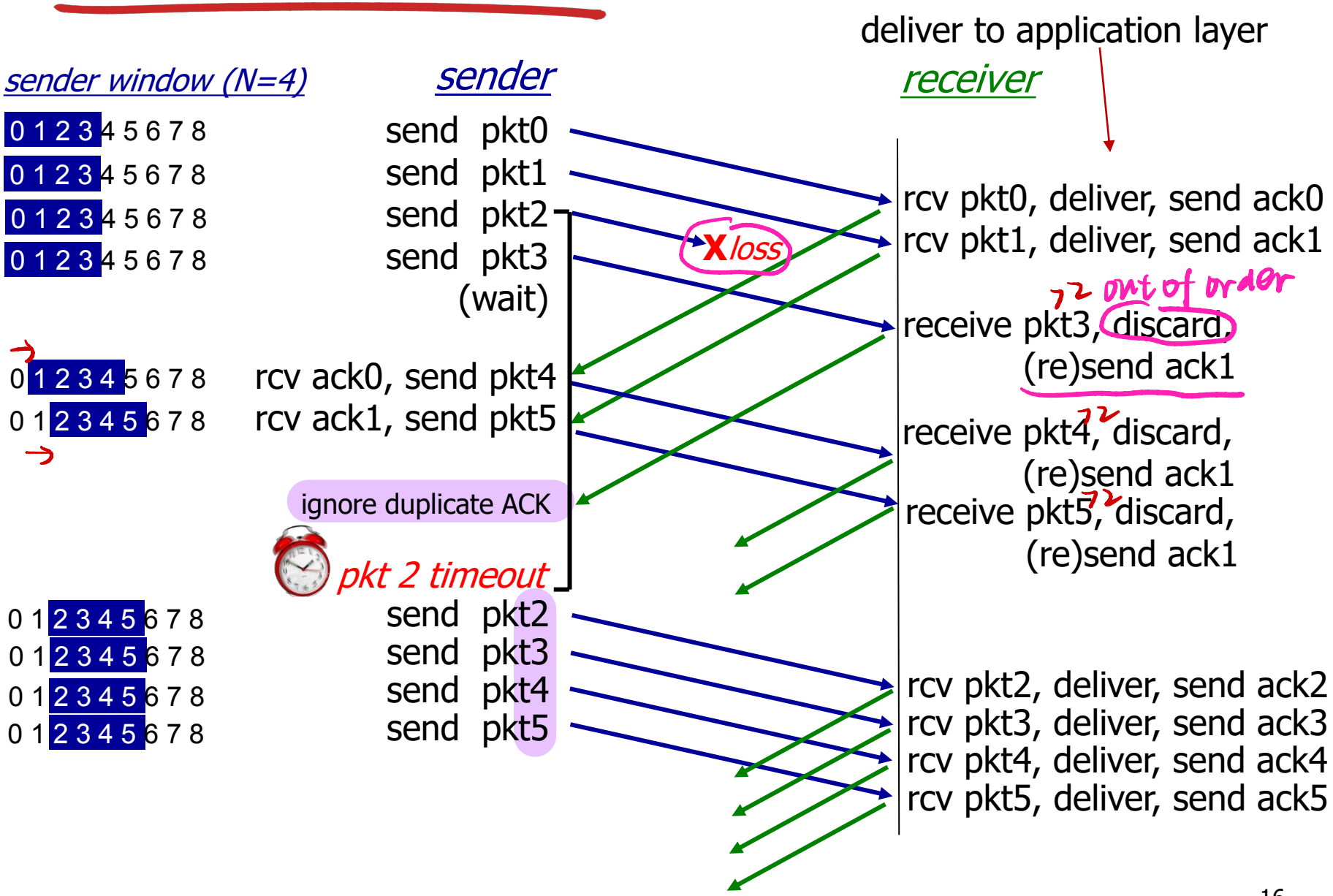
Go-Back-N: sender

- Now we need more sequence numbers k-bit sequence number in packet header
- "window" of up to N, consecutive packets with no ACK



- ACK(n): ACKs all packets up to, including sequence # n - *"cumulative ACK"*
  - may receive duplicate ACKs (see receiver)
- timer for oldest in-flight pkt
- *timeout(n)*: retransmit packet n and all higher seq # pkts in window

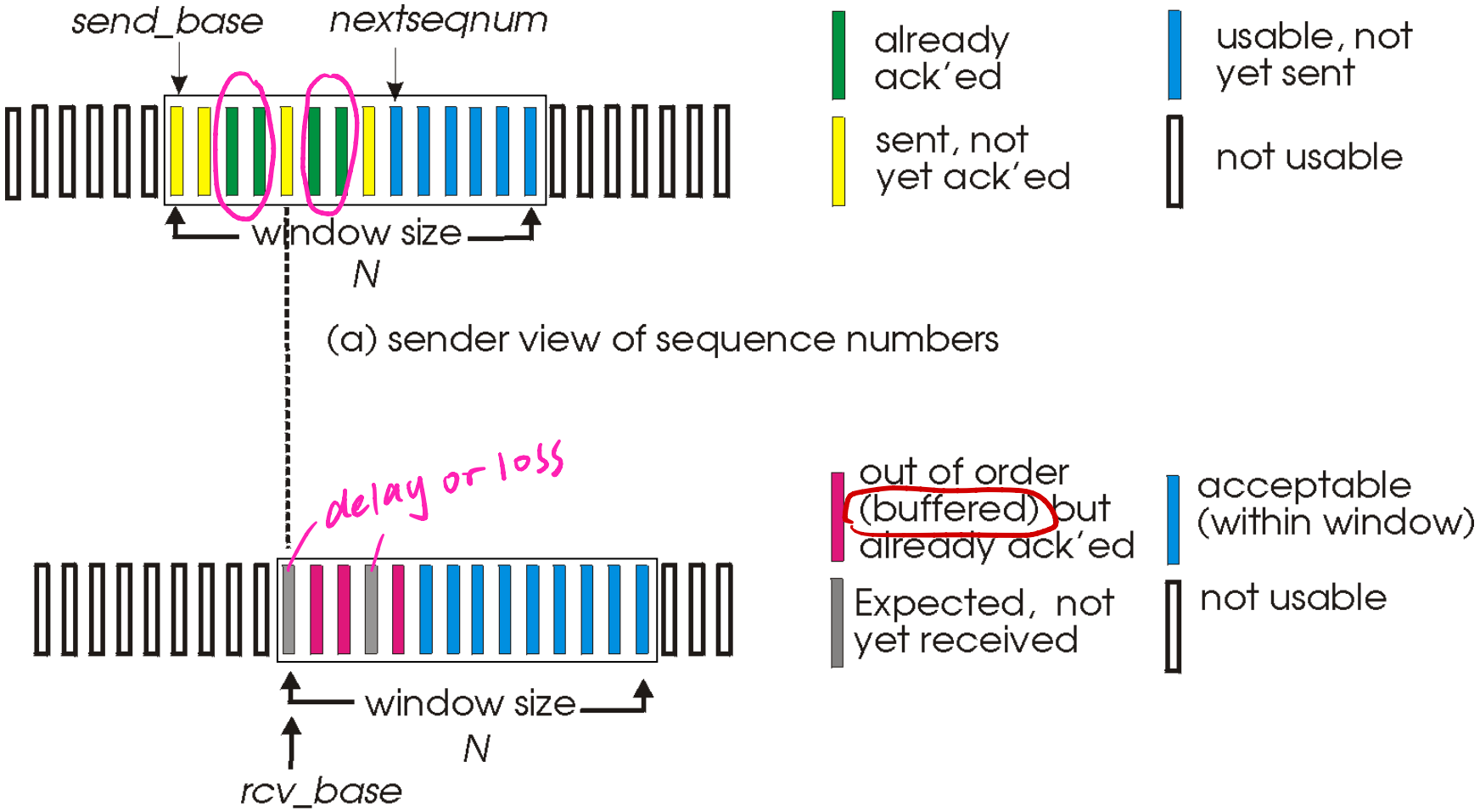
GBN in action



Selective repeat

- receiver *individually* acknowledges all correctly received packets
  - buffers packets, as needed, for eventual in-order delivery to upper layer
- sender only resends packets for which ACK not received
  - sender timer for each unACKed packet
- sender window
  - *N* consecutive seq #'s
  - limits seq #s of sent, unACKed packets

Selective repeat: sender, receiver windows



Selective repeat

sender

data from above:

- if next available seq # in window, send pkt

timeout(n):

- resend pkt n, restart timer

ACK(n) in [sendbase,sendbase+N-1]:

- mark pkt n as received
- if n smallest unACKed pkt, advance window base to next unACKed seq #

move the window along one

receiver

pkt n in [rcvbase, rcvbase+N-1]

- send ACK(n)
- out-of-order: buffer
- in-order: deliver (also deliver buffered, in-order pkts), advance window to next not-yet-received pkt

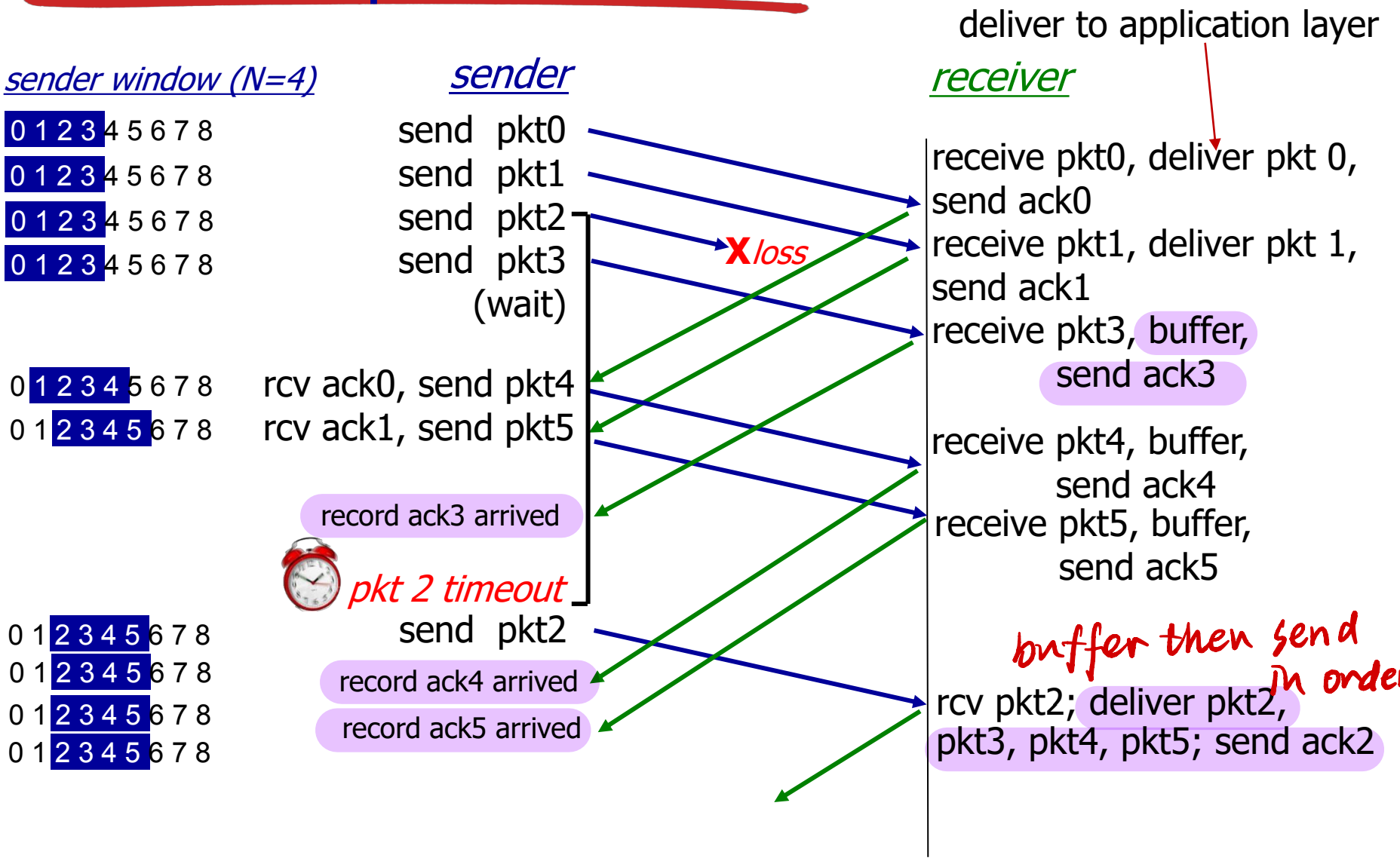
pkt n in [rcvbase-N,rcvbase-1]

- ACK(n)

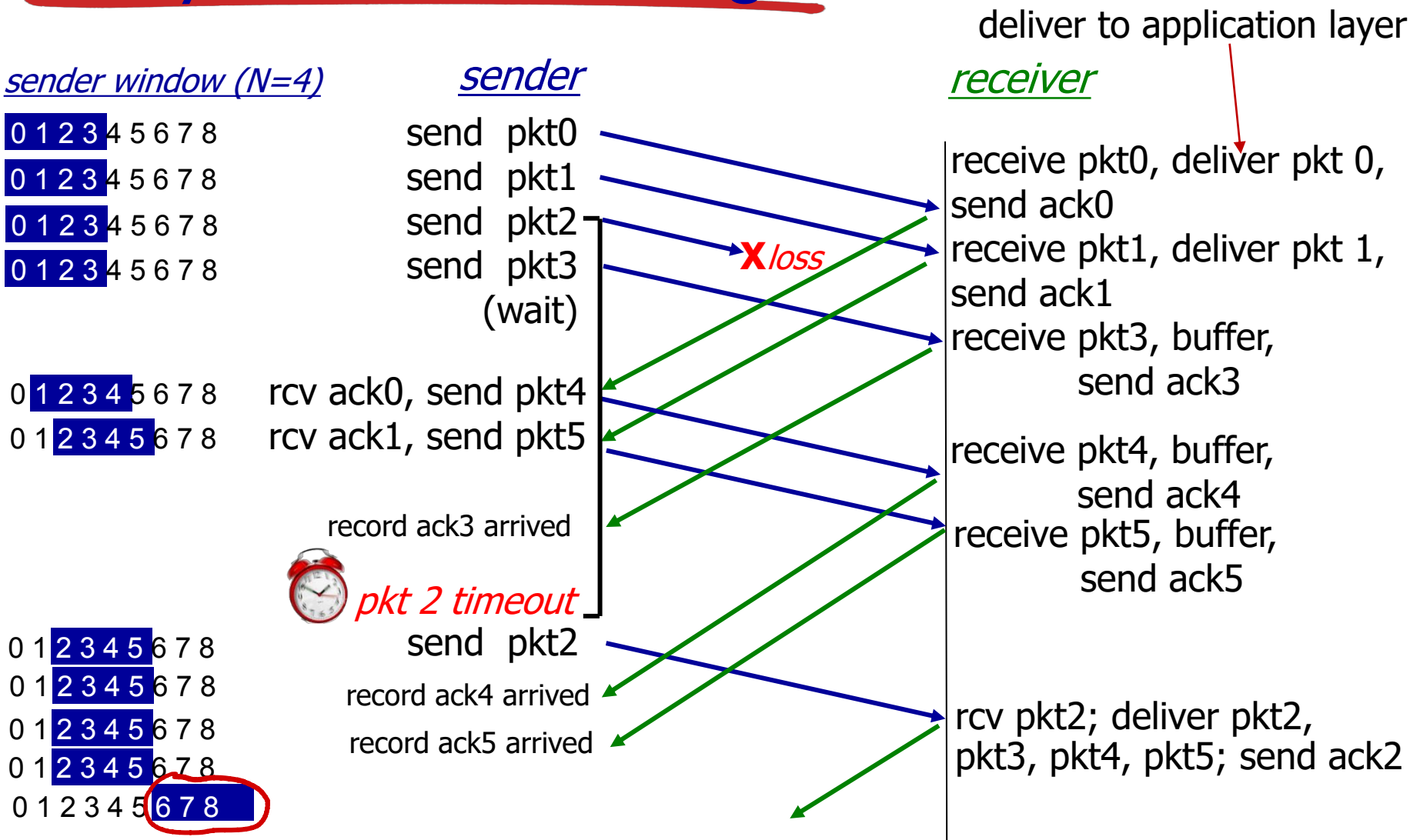
otherwise:

- ignore

Selective repeat in action



Test your understanding



The window advances to 6 7 8 since ACKs 1-5 arrived

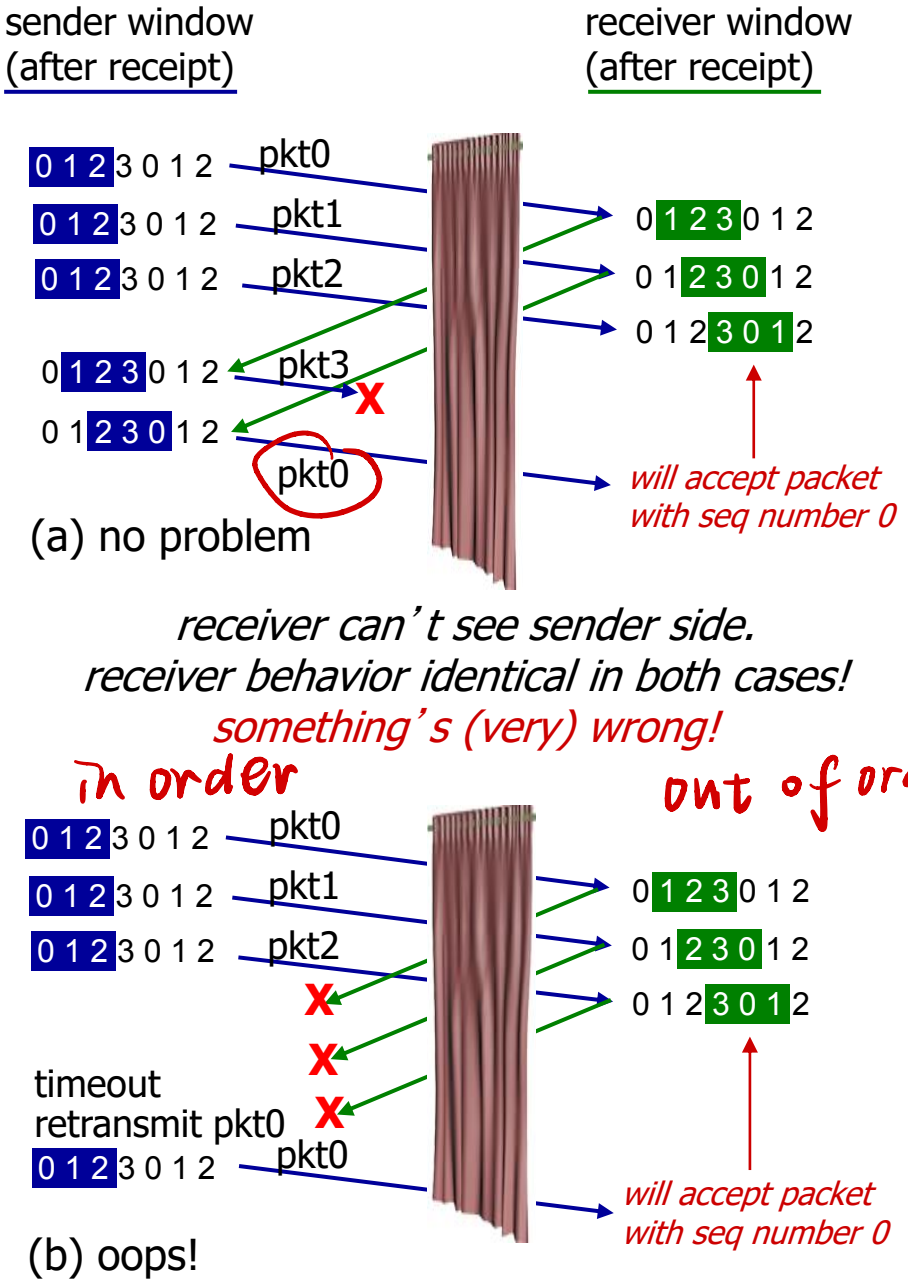
If you got it wrong review selective repeat

Selective repeat: dilemma

example:

- seq #'s: 0, 1, 2, 3
- window size=3
- receiver sees no difference in two scenarios!
- duplicate data accepted as new in (b)

Q: what relationship between seq # size and window size to avoid problem in (b)?



Advantages each scheme

both secure
both in order
both accurately locate error and retransmit

- Go Back N
  - Simple to implement
  - Only a single timer
  - One ACK can acknowledge many packets

without buffer. less memory size.

- Selective repeat → less transmission more memory
  - Minimises resending of packets (better use of network)
  - Individual ACKs give more information to sender

Pipelined protocols: overview

Question: Which of the following statement is correct :

a) Go-Back-N is more bandwidth efficient because it would retransmit an entire window even if a single packet is lost.

b) Go-Back-N uses cumulative acknowledgements which can reduce the traffic.

c) Selective Repeat is more bandwidth efficient compared to Go-Back-N because it resend only the defective or missing packets and not the entire windows.

d) In Go-Back-N, if a sent packet is found damaged, then all the packets are retransmitted till the last packet.

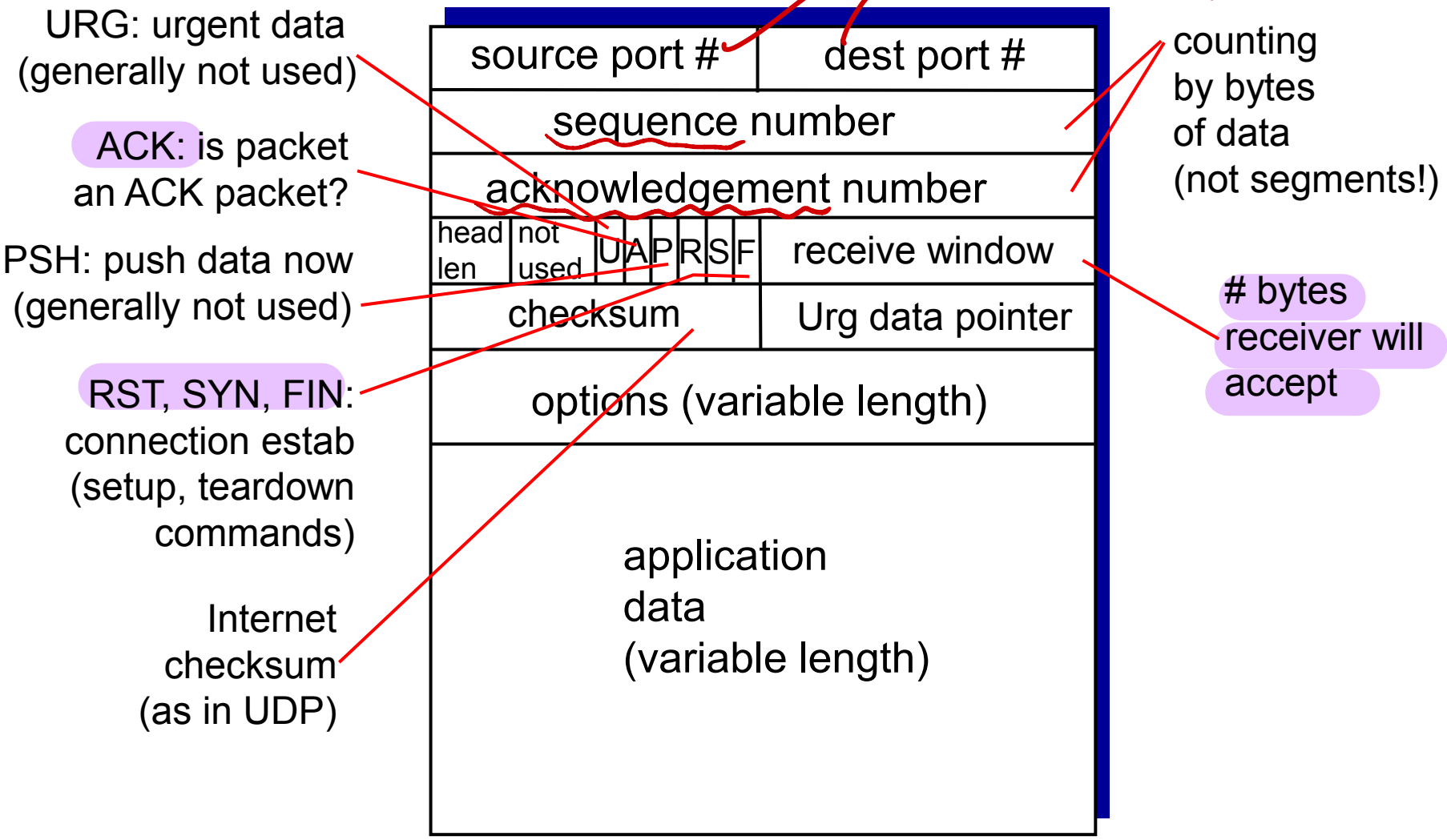
TCP: Overview RFCs: 793,1122,1323, 2018, 2581

- point-to-point:
  - one sender, one receiver
- reliable, in-order *byte steam:*
  - no "message boundaries"
- pipelined:
  - TCP congestion and flow control set window size
- full duplex data:
  - bi-directional data flow in same connection
  - MSS: maximum segment size
- connection-oriented: "polite"
  - handshaking (exchange of control msgs) inits sender, receiver state before data exchange (initialize)
- flow controlled:
  - sender will not overwhelm receiver

won't be too fast

TCP header

used to identify source and destination application



TCP seq. numbers, ACKs

累加：原100.现send 10 bits → 110

sequence numbers:

- byte stream "number" of first byte in segment's data

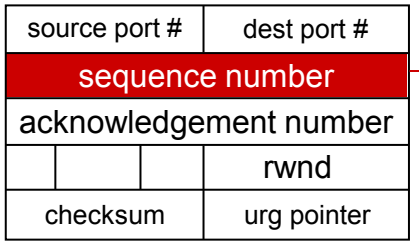
acknowledgements:

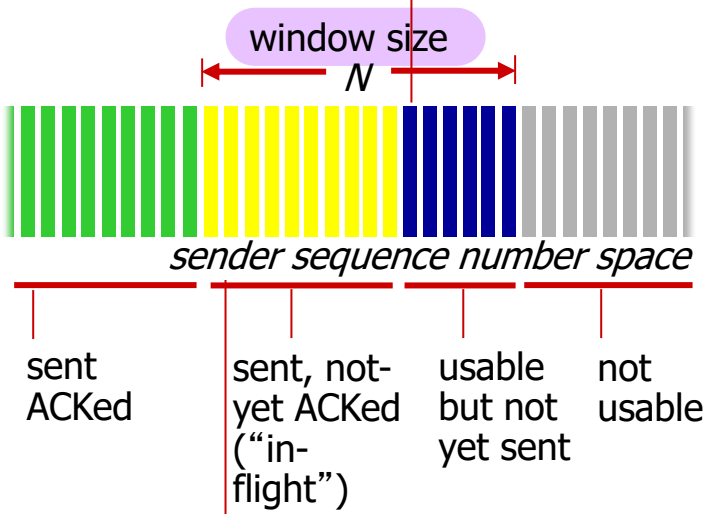
- seq # of **next byte expected from other side** (cumulative ACK)

Q: how receiver handles out-of-order segments

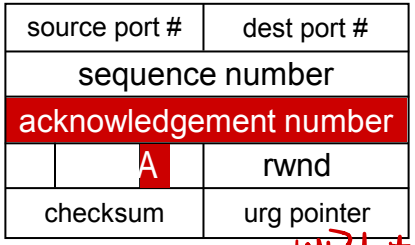
- A: TCP spec does not say, - up to those writing TCP code as long as data is delivered in order to application layer

outgoing segment from sender



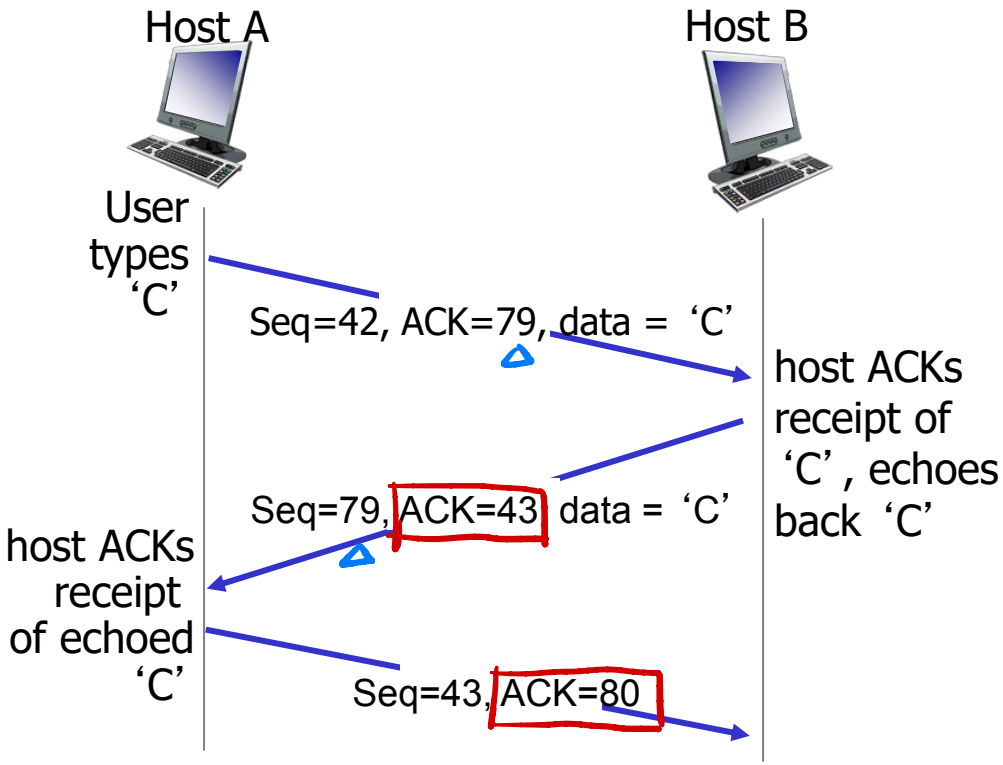


incoming segment to sender



→ the sequence number we wish to see next

TCP seq. numbers, ACKs



simple shell application scenario

TCP round trip time, timeout

Q: how to set TCP timeout value?

- longer than RTT
  - but RTT varies
- *too short:* early timeout, unnecessary retransmissions
- *too long:* slow reaction to segment loss

Q: how to estimate RTT?

- **SampleRTT:** measured time from segment transmission until ACK receipt
  - ignore retransmissions
- **SampleRTT** will vary, want estimated RTT "smoother"
  - average several *recent* measurements, not just current **SampleRTT**

- i) Describe briefly the purpose of this algorithm in TCP?

(4 marks)

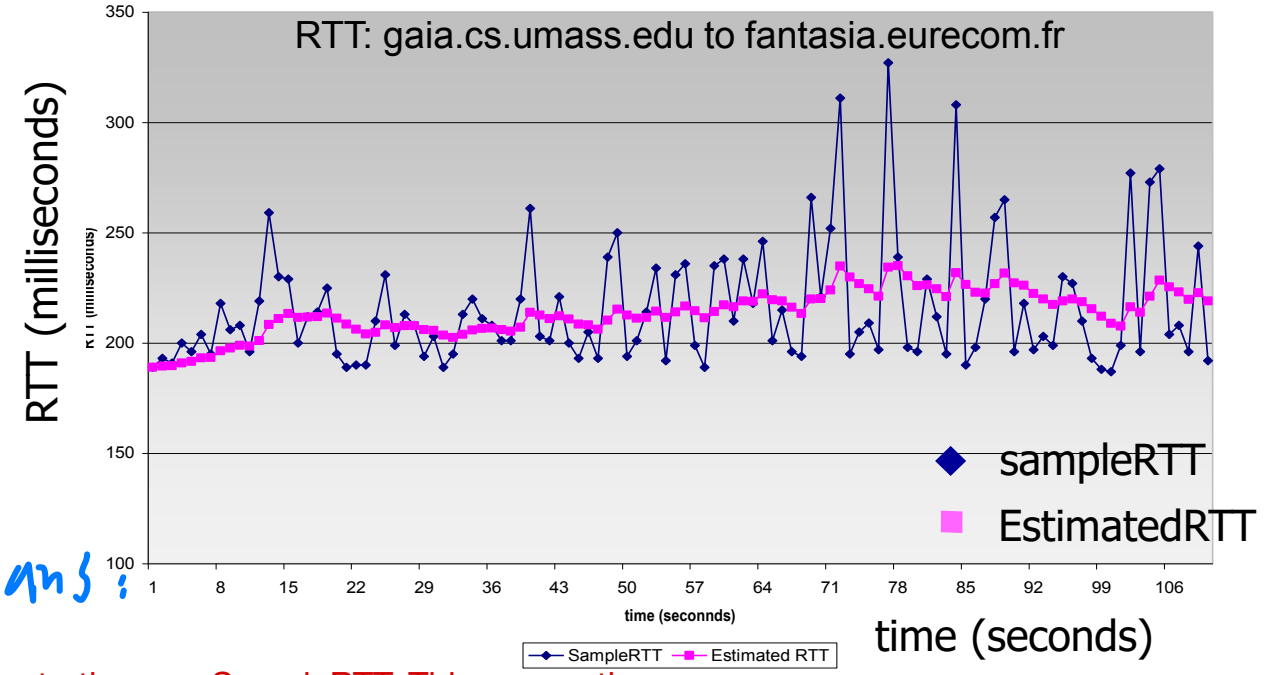
- Estimates a smoothed value of RTT used to determine the value of timeout (how long to wait before assuming packet is lost). Uses estimates of RTT to get a smoothed RTT estimate.

TCP round trip time, timeout

EstimatedRTT = (1- α)*EstimatedRTT + α*SampleRTT

- exponential weighted moving average
- influence of past sample decreases exponentially fast
- typical value: α = 0.125

EstimatedRTT_new = 0.875 · EstimatedRTT + 0.125 · SampleRTT



α↑ means:

More weight is given to the new SampleRTT. This means the estimate is smoothed less but reacts to changes more quickly.

TCP round trip time, timeout

- Jacobsen/Karel's Algorithm
- timeout interval: **`EstimatedRTT`** plus "safety margin"
  - large variation in **`EstimatedRTT`** -> larger safety margin
- estimate SampleRTT deviation from EstimatedRTT:

```
DevRTT = (1-β)*DevRTT +
         β*|SampleRTT-EstimatedRTT|
```

(typically, β = 0.25)

Where |x| means the "absolute" value – x made positive.

```
TimeoutInterval = EstimatedRTT + 4*DevRTT
```

estimated RTT

"safety margin"

* Check out the online interactive exercises for more examples: http://gaia.cs.umass.edu/kurose_ross/interactive/

Test your understanding

- What is the problem if a TCP timeout is set too short?
  - Packets are retransmitted needlessly (they were delayed not lost) wasting bandwidth
- What is the problem if a TCP timeout is set too long?
  - Packets take a long time to retransmit meaning your connection is slow.
- Estimated RTT is 12ms α = 0.125, Sample RTT is 8ms. Calculate a new Estimated RTT
  - Estimated RTT= (1- α) 12 + α 8 ms = (0.875).12 + 0.125 . 8 ms = 10.5+1ms = 11.5 ms

  If you did not get it right revise this section

TCP sender events (simplified)

data received from application:

- create segment with seq #
- seq # is byte-stream number of first data byte in segment
- start timer if not already running
  - think of timer as for oldest unacked segment
  - expiration interval: `TimeOutInterval`

increment by the bytes we send

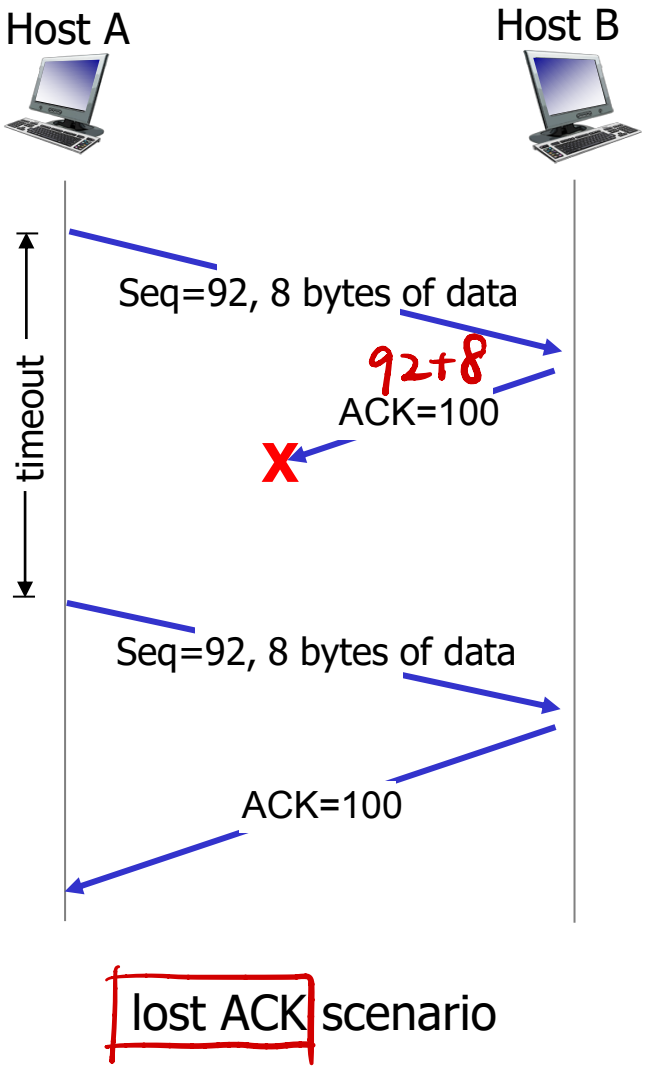
timeout:

- retransmit segment that caused timeout
- restart timer

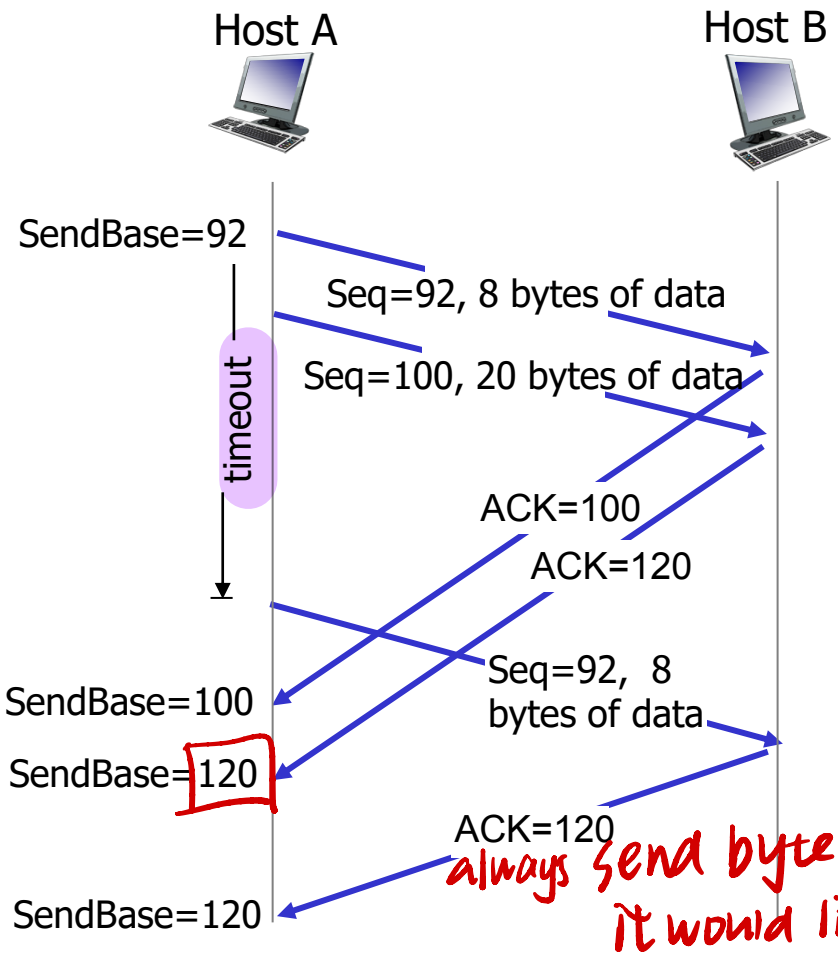
ack recieved:

- if ack acknowledges previously unacked segments
  - update what is known to be ACKed
  - start timer if there are still unacked segments

TCP: retransmission scenarios



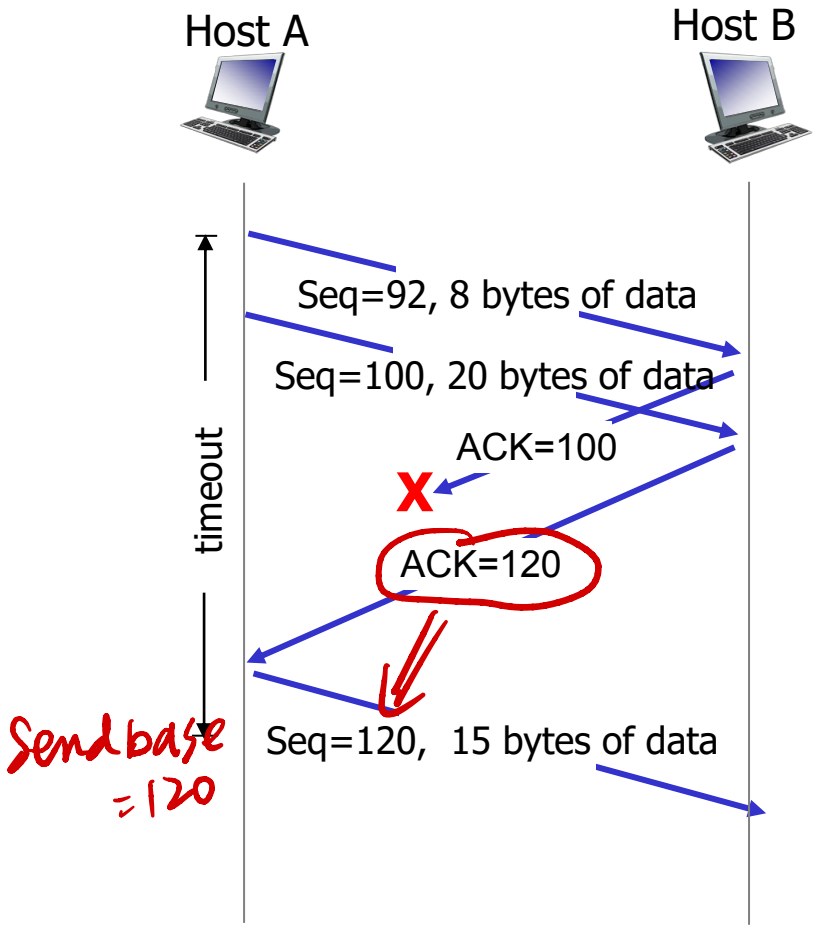
lost ACK scenario



premature timeout

always send bytes it would like to see next

TCP: retransmission scenarios



cumulative ACK

TCP fast retransmit

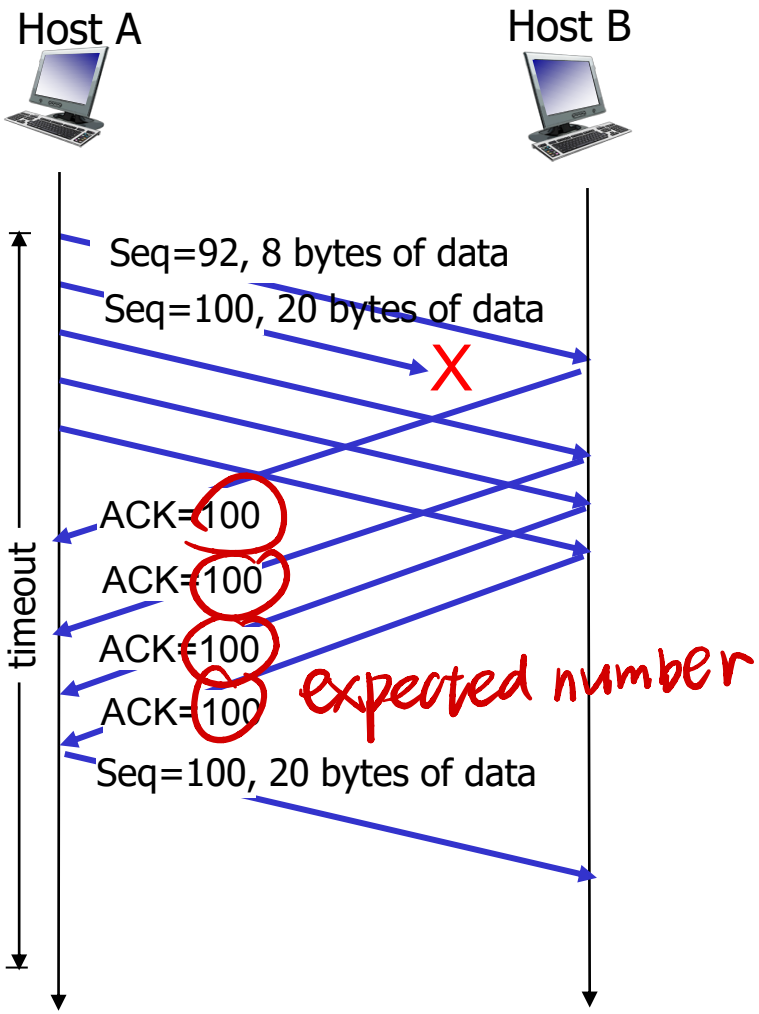
- time-out period often relatively long:
  - long delay before resending lost packet
- detect lost segments via duplicate ACKs.
  - sender often sends many segments back-to-back
  - if segment is lost, there will likely be many duplicate ACKs.

TCP fast retransmit

if sender receives 3 ACKs for same data ("triple duplicate ACKs"), resend unacked segment with smallest seq #

- likely that unacked segment lost, so don't wait for timeout

TCP fast retransmit



fast retransmit after sender receipt of triple duplicate ACK

Test your understanding

- Simple TCP sender has a window size of 5. It has 10 packets to send. It sends packets 1,2,3,4,5.
- Receives ACK for 1, 2, 3, 4, 5
  - Send packets 6,7,8,9,10 (everything got there fine).
- Receives ACK for 1, 3. it also tells packet 2 got there
- Send packet 6, 7, 8. Wait for timeout for packet 4. (ACK 3 tells us packets 2 and 3 must have arrived – if packet 2 had not arrived ACK 1 would be generated.)
- Receives ACK for 1 four times.
  - Send packet 6. Send packet 2 (Triple duplicate). Packet 2 is lost, corrupted or delayed (hence receiver sends duplicate when it receives packets 3, 4, 5)

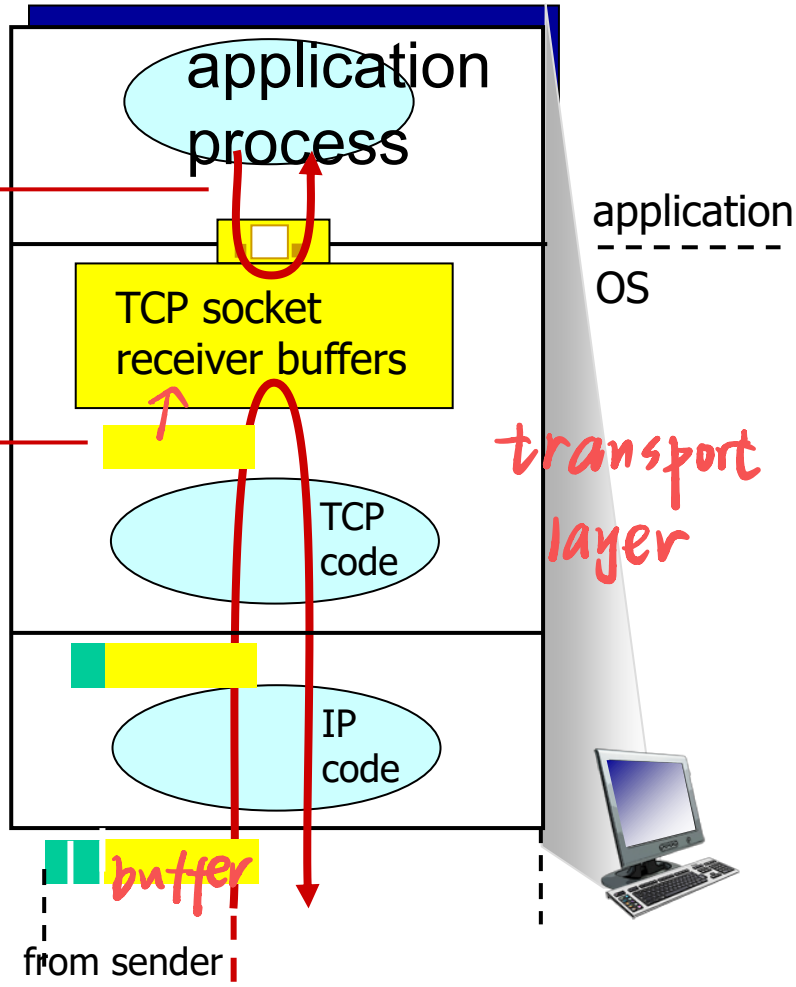
If you did not get the right answer review the last section.

TCP flow control

The sender is more powerful than the receiver (send data too quickly)

application may remove data from TCP socket buffers ....

... slower than TCP receiver is delivering (sender is sending)



receiver protocol stack

flow control

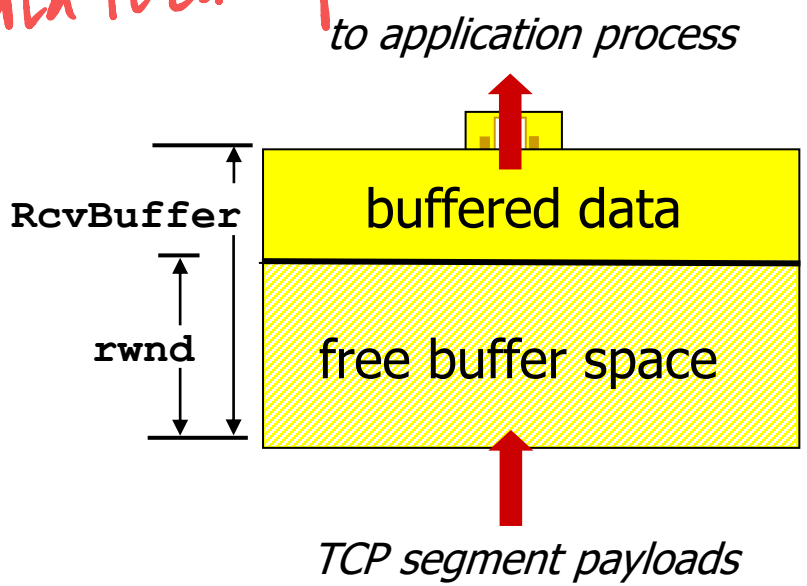
receiver controls sender, so sender will not overflow receiver buffer by transmitting too much, too fast

Flow control protocol is required to prevent data items being discarded at consumer when over pushing by the producer. It involves preventing senders from overrunning the capacity of the receivers (2 marks). Receiver controls the Send window size, and adjust its own Receive window, usually opening or closing to control the data flow (1 mark).

TCP flow control

- receiver "advertises" free buffer space by including **`rwnd`** value in TCP header of receiver-to-sender segments
  - **`RcvBuffer`** size set via socket options (typical default is 4096 bytes)
  - many operating systems autoadjust **`RcvBuffer`**
- sender limits amount of unacked ("in-flight") data to receiver's **`rwnd`** value
- guarantees receive buffer will not overflow

tell how much data it can cope



receiver-side buffering

`rwnd` = receive window

TCP flow control

Which statements about flow control are true:

- A) The receive window (rwnd) is sent over the network to the sender.

  Yes – it is in the TCP header.

- B) The receive window measures how much congestion is on the network

  No, it measures the receive buffer

- C) Flow control reduces network congestion

  Not deliberately, cannot see network congestion

- D) Sender sees rwnd=6000bytes. It has 4 UNACK'd packets of 1500bytes. It can send 1 more.

  No – 6000 bytes already sent, there is no more room.

MSS and MTU

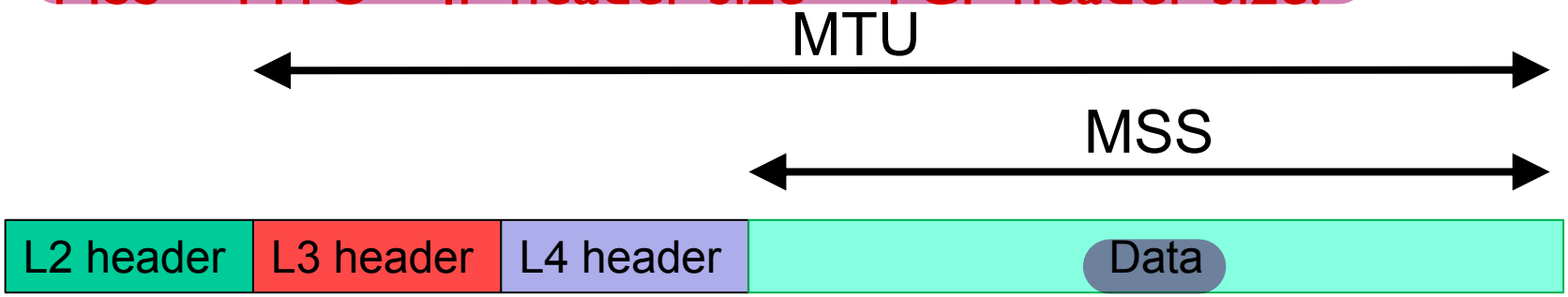
maximum transfer unit

Note: To L2 the L3 and L4 header are "just data". To L3, the L4 header is "just data".

maximum segment size

- MSS (mentioned in Nagle algorithm) is a parameter specifying the largest amount of data in a single IP datagram that should be sent by a remote host.
- MTU is a parameter specifying the largest amount of data that a communication protocol or system can pass onwards. For example, standards (e.g. Ethernet) can fix the size of an MTU, or systems (such as point-to-point serial links) may set MTU at connect time.
- MSS size is set according to MTU:

MSS = MTU – IP header size – TCP header size.



Nagle's algorithm

- A problem can occur when an application generates data very slowly.
- Consider, ssh that generates data only when a user types.
- TCP will send the data as it arrives at the send buffer if there is space left in the send buffer.
- This means (for ssh) one packet sent every time user hits key.
- Overhead of this is huge (TCP header + IP header + frame header to send one byte).
- Cure is known as "Nagle's algorithm".

Nagle's Algorithm

1. The sending TCP sends the first piece of data it receives – no matter no small or large
2. Sending TCP accumulates data in the buffer and waits until one of the following before sending the segment:
   - The receiving TCP sends an acknowledgement
   - Data has accumulated to fill a maximum size segment
3. Repeat step 2

- Note: Sometimes Nagle's algorithm should be switched off – e.g. when fast interaction is vital and you want small packet sizes to be sent.

Silly Window Syndrome

- Silly Window Syndrome occurs when the TCP system is forced to send very small packets. Named because window size is "silly".
- This can happen in two separate ways:

1. Sender produces data very slowly.
   - Same problem as Nagle's algorithm.
2. Receiver processes data very slowly.
   - Single byte or small number removed from full receive buffer.
   - Sender is informed of opportunity to send small number of bytes and immediately sends filling buffer.
   - Process repeats.
   - Cure – receiver does not advertise windows that would cause sender to send small amounts of data.

it won't wait until there is a large window.

Check your understanding

- What is the reason for flow control?

  Stop the sender sending too much for the receiver

- Who sends rwnd?

  The receiver sends it – it is the "receiver window". It says how much buffer space the receiver has.

- What problem might be caused by a very very small rwnd? (How might this happen?)

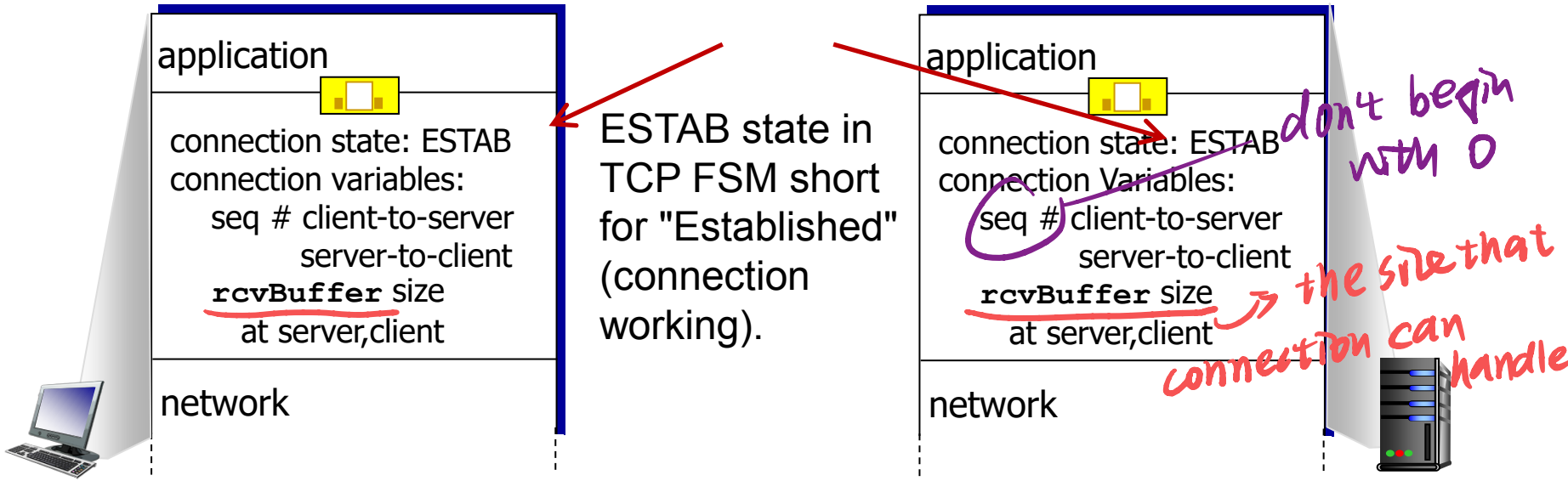
  Sender might send small (inefficient) packets. This might happen if receiver is very slow – processing one or two bytes at a time.

If you are puzzled review the previous section.

Connection Management

before exchanging data, sender/receiver "handshake":

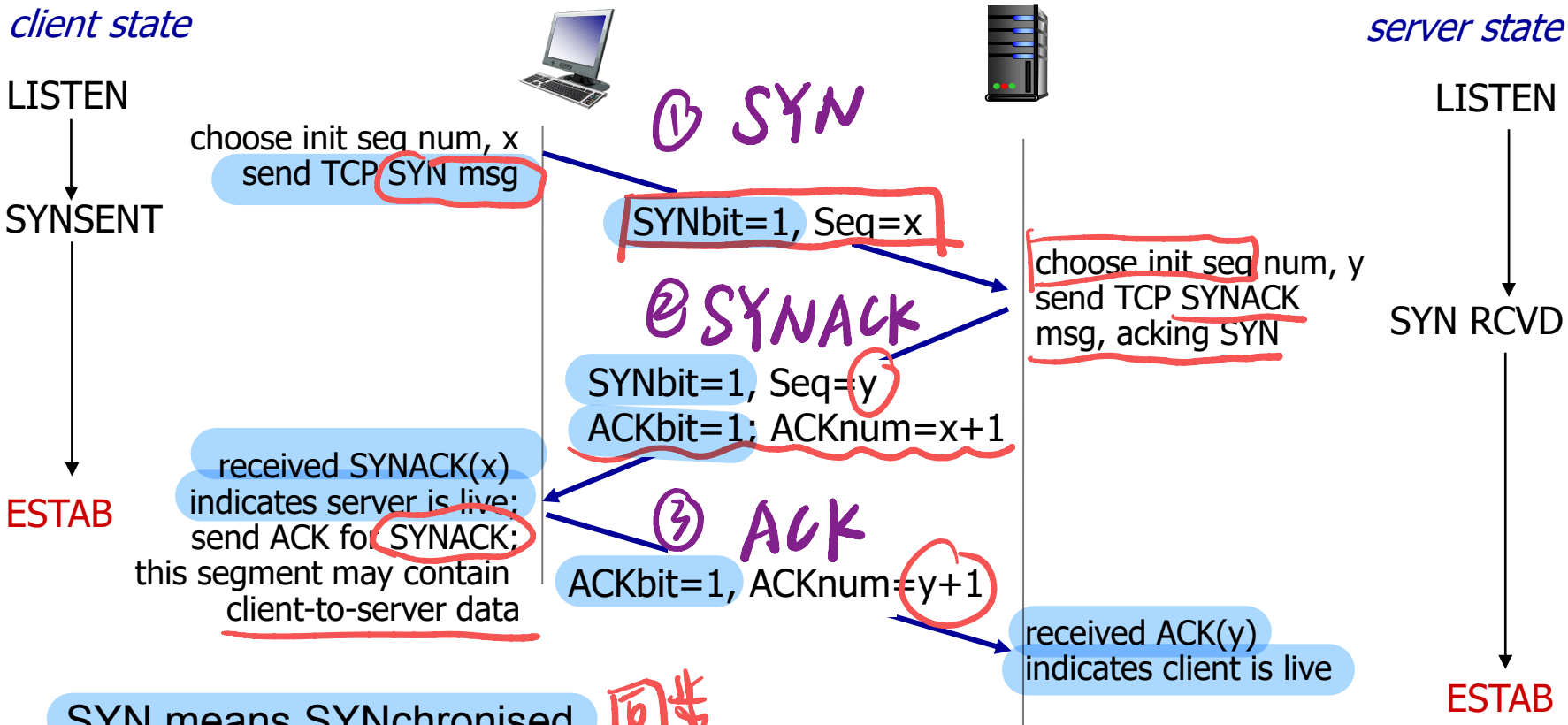
- agree to establish connection (each knowing the other willing to establish connection)
- agree on connection parameters




```
Socket clientSocket =
  newSocket("hostname","port
   number");
```

```
Socket connectionSocket =
   welcomeSocket.accept();
```

TCP 3-way handshake



SYN means SYNchronised 同步

SYNbit = bit in TCP header saying this is SYN packet

ACKbit = bit in TCP header saying this is ACK packet

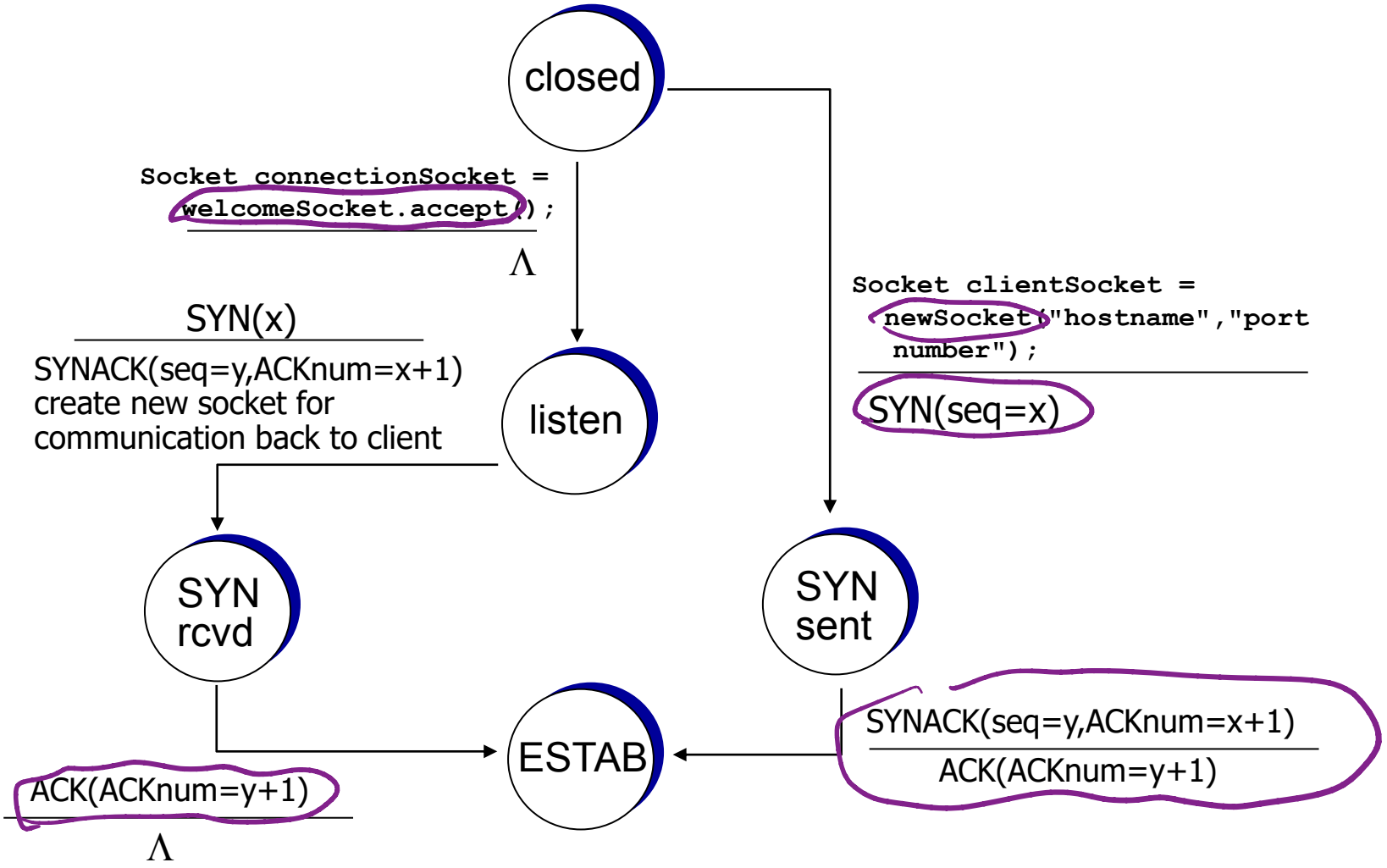
2017-2018 Q2d

- d) In TCP/IP describe (briefly) the handshake mechanism to set up a connection between a client and a server (you do not need to describe the state of the TCP FSM). The client is starting the connection.

[4 marks]

A:
TCP starts a connection with a three way handshake
The client sends a SYN packet
The server replies with a SYN ACK packet
The client replies with an ACK packet

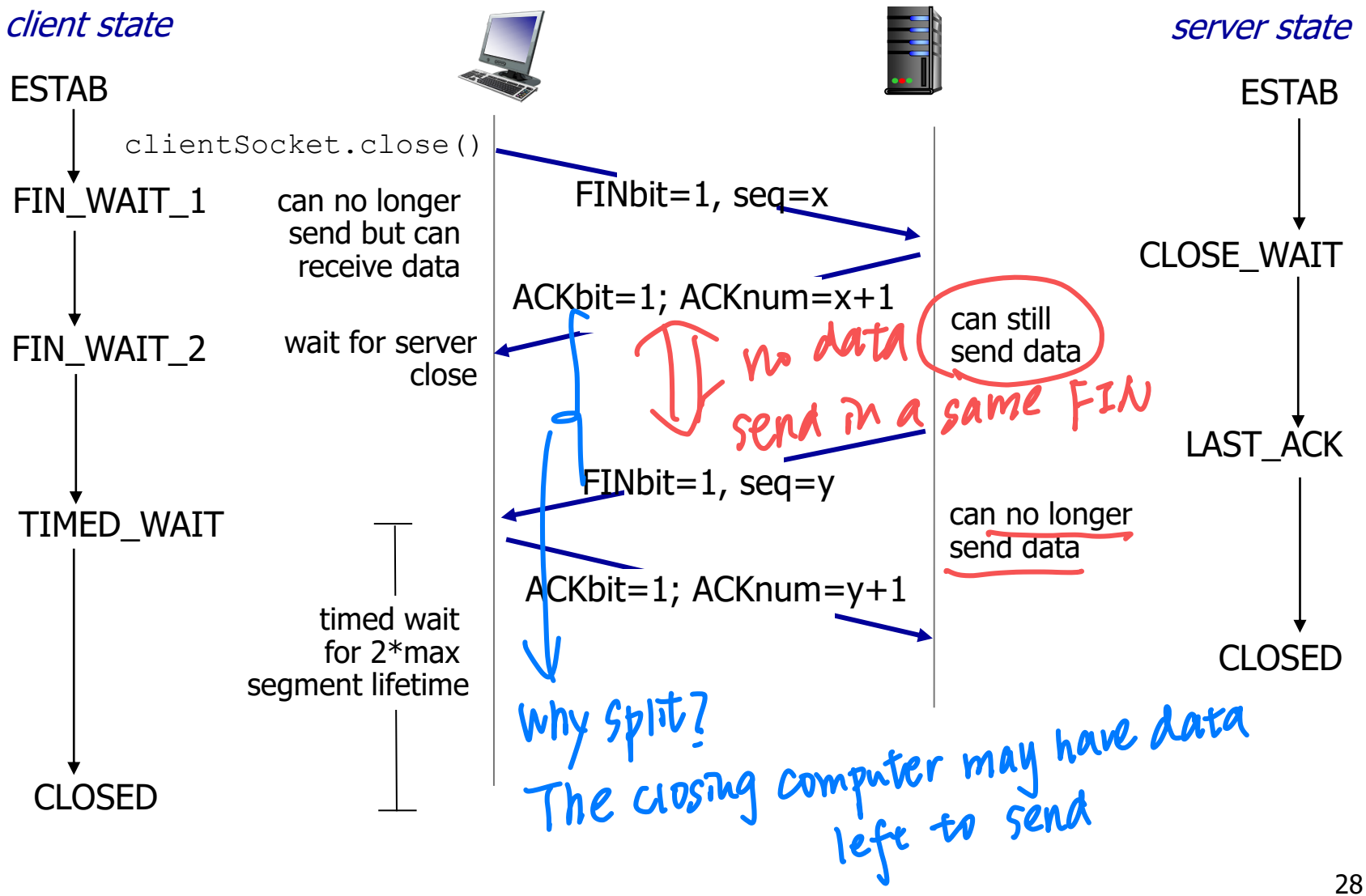
TCP 3-way handshake: Finite State Machine



TCP: closing a connection

- client, server each close their side of connection
  - send TCP segment with FIN bit = 1
- respond to received FIN with ACK
  - on receiving FIN, ACK can be combined with own FIN
- simultaneous FIN exchanges can be handled

TCP: closing a connection



Test your understanding

1) SYN – ACK -- Response to SYN, will accept a new connection

2) FIN – I want to close the connection

3) FIN – ACK – I agree to close the connection

- What are the flag combinations for the three packets in the three way handshake?
- SYN SYN—ACK ACK
- If you had problems review the previous section

Principles of congestion control

congestion:

- informally: "too many sources sending too much data too fast for *network* to handle"
- different from flow control!
- how does this look?
  - lost packets (buffer overflow at routers)
  - long delays (queueing in router buffers)
- a top-10 problem!

Congestion control prevents too much data from being injected into the network, thereby causing switches or links to become overloaded (2 marks).

Causes and costs of congestion

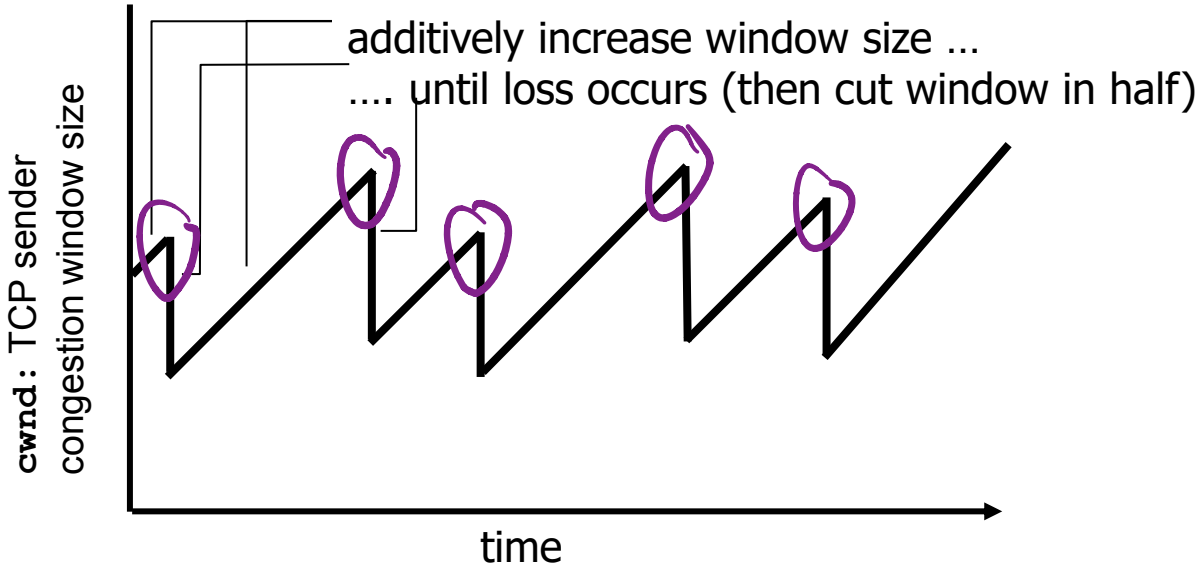
- Too much traffic enters router – buffer fills up, this increases delay (and hence reduces throughput).
- Much too much traffic enters router – buffer overfills and causes loss. Packet needs to be retransmitted. wasteful
- If packet is lost after several "hops" then many resources are wasted. (e.g. Packet travels from A to B to C to D then lost at D – it has taken up space at A, B and C unnecessarily).
- Useful concept: goodput – this is the rate at which data reaches the application layer. Different from throughput because of: lower than throughput
  - loss
  - retransmission
  - corrupted packets

TCP congestion control: additive increase multiplicative decrease

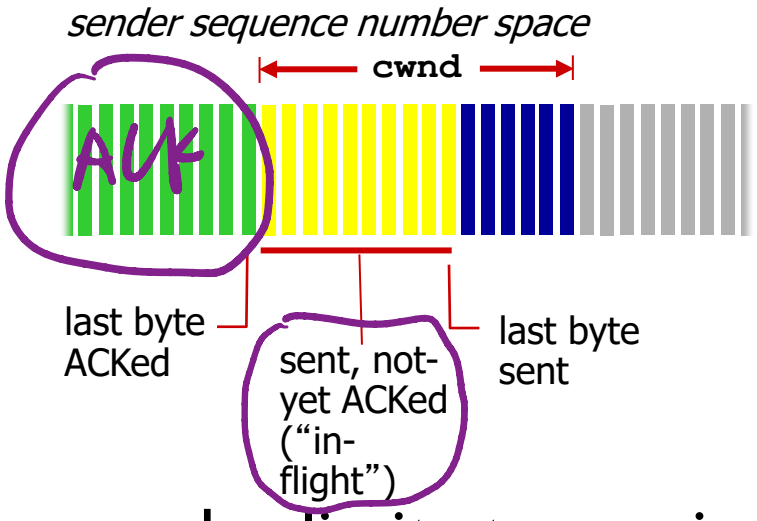
- *approach:* sender increases transmission rate (window size), probing for usable bandwidth, until loss occurs
  - Set cwnd – congestion window to initial value
  - *additive increase:* increase **cwnd** by 1 MSS every RTT until loss detected
  - *multiplicative decrease:* cut **cwnd** in half after loss

until the network cannot handle

AIMD saw tooth behavior: probing for bandwidth



TCP Congestion Control: details



- sender limits transmission:

$$\texttt{LastByteSent} - \texttt{LastByteAcked} \leq \texttt{cwnd}$$

free → cwnd ↑

- **cwnd** is dynamic, function of perceived network congestion

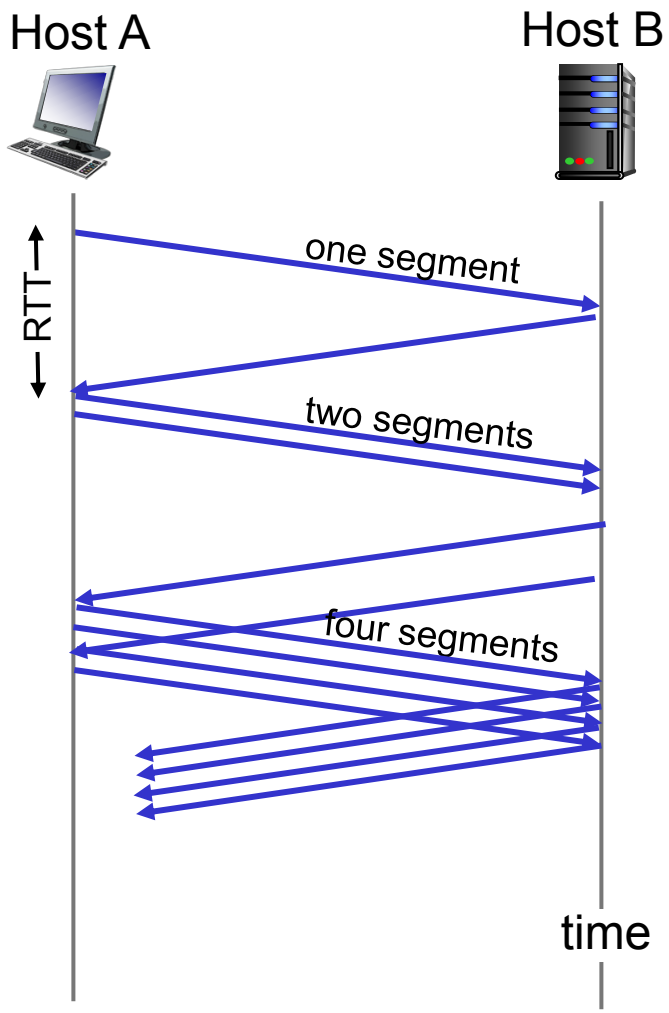
TCP sending rate:

- *roughly:* send cwnd bytes, wait RTT for ACKS, then send more bytes

$$\text{rate} \approx \frac{\text{cwnd}}{\text{RTT}} \text{ bytes/sec}$$

TCP Slow Start

- when connection begins, increase rate exponentially until first loss event:
  - initially **cwnd** = 1 MSS
  - double **cwnd** every RTT
  - done by incrementing **cwnd** for every ACK received
- *summary:* initial rate is slow but ramps up exponentially fast



Test your understanding

- What are the purposes of windows rwnd and cwnd?

  rwnd = flow control

  (not too fast for receiver)

  cwnd = congestion control

  (not too fast for network)

- In additive increase multiplicative decrease what triggers TCP to reduce cwnd?

  loss (or timeout) of a packet

- What does a receiver send to indicate is buffer is getting full and sender must slow down?

  Smaller rwnd

- If you had problem review previous section.

TCP: detecting, reacting to loss

- loss indicated by timeout: can't wait forever
  - **cwnd** set to 1 MSS;
  - window then grows exponentially (as in slow start) to threshold, then grows linearly
- loss indicated by 3 duplicate ACKs: TCP RENO
  - dup ACKs indicate network capable of delivering some segments (still can receive, network is still functioning)
  - **cwnd** is cut in half window then grows linearly (slow down)
- TCP Tahoe always sets **cwnd** to 1 (timeout or 3 duplicate acks)

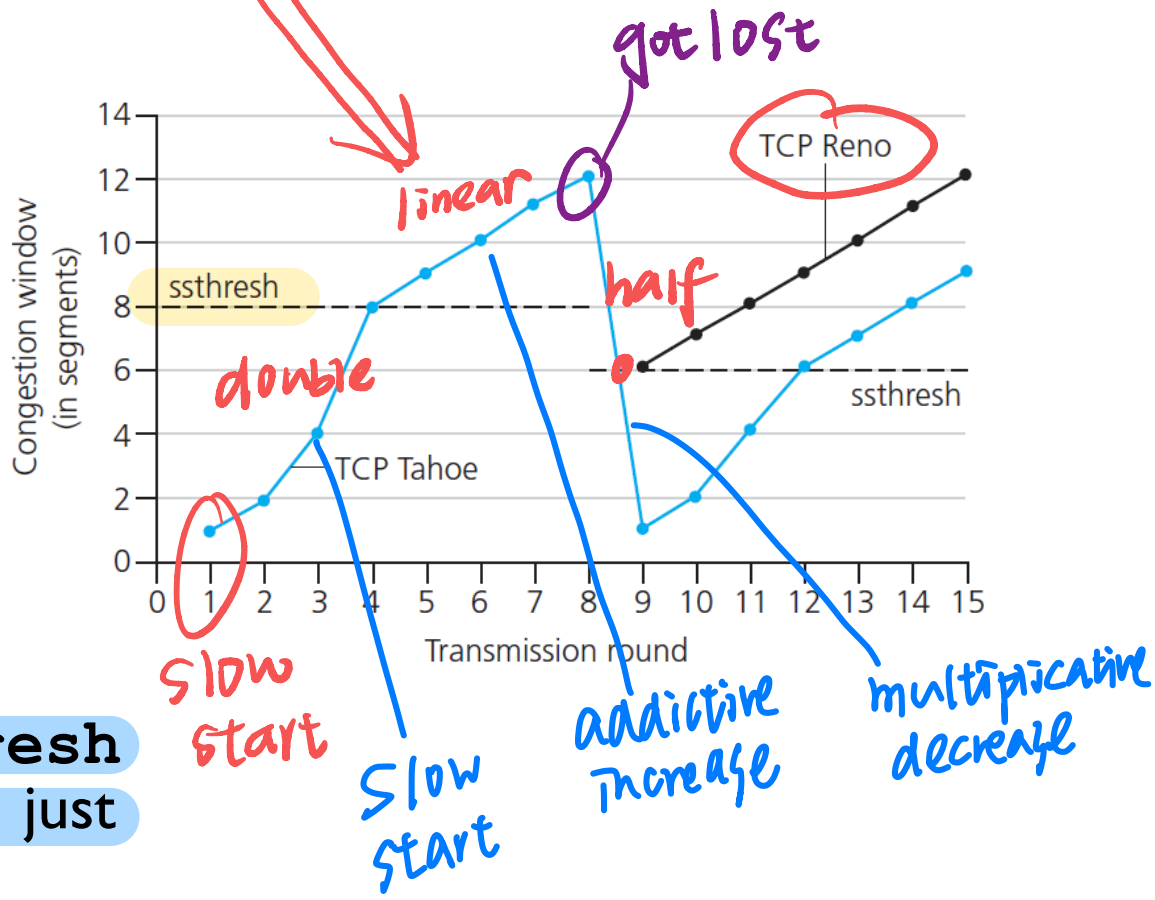
TCP: switching from slow start to Congestion Avoidance

Q: when should the exponential increase switch to linear?

A: when **cwnd** gets to 1/2 of its value before timeout.

Implementation:

- variable **ssthresh**
- on loss event, **ssthresh** is set to 1/2 of **cwnd** just before loss event



* Check out the online interactive exercises for more examples: http://gaia.cs.umass.edu/kurose_ross/interactive/

TCP congestion control

Which statements about congestion control are true:

rwnd ✓

A) The congestion window (cwnd) is sent over the network to the sender. ✗

No, it is calculated by the sender.

B) AIMD means that cwnd increases only slowly when there is no loss

Yes – it increases slowly decreases quickly.

C) Congestion control does not work well due to loss

Loss is fundamental to most congestion control

D) Congestion control reduces network congestion

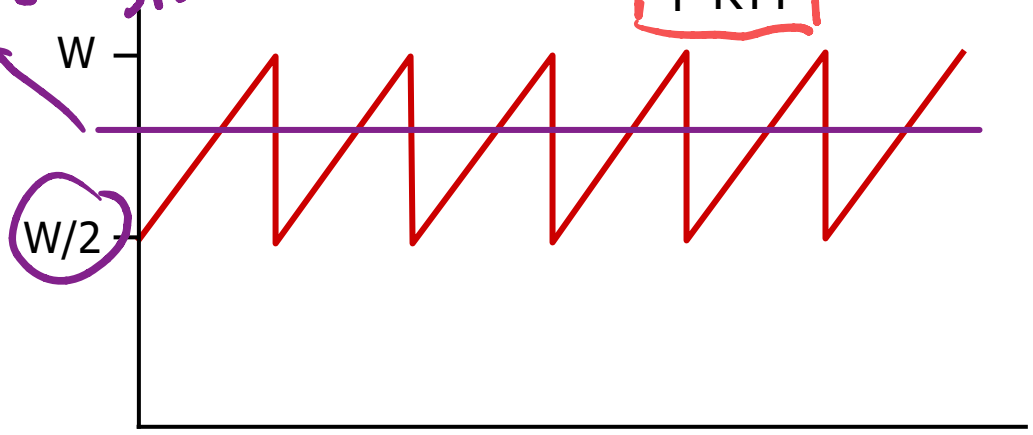
Yes – this is its very aim.

TCP throughput

- avg. TCP throughput as function of window size, RTT?
  - ignore slow start, assume always data to send
- W: window size (measured in bytes) where loss occurs
  - avg. window size (# in-flight bytes) is ¾ W
  - avg. throughput is 3/4W per RTT

average window size

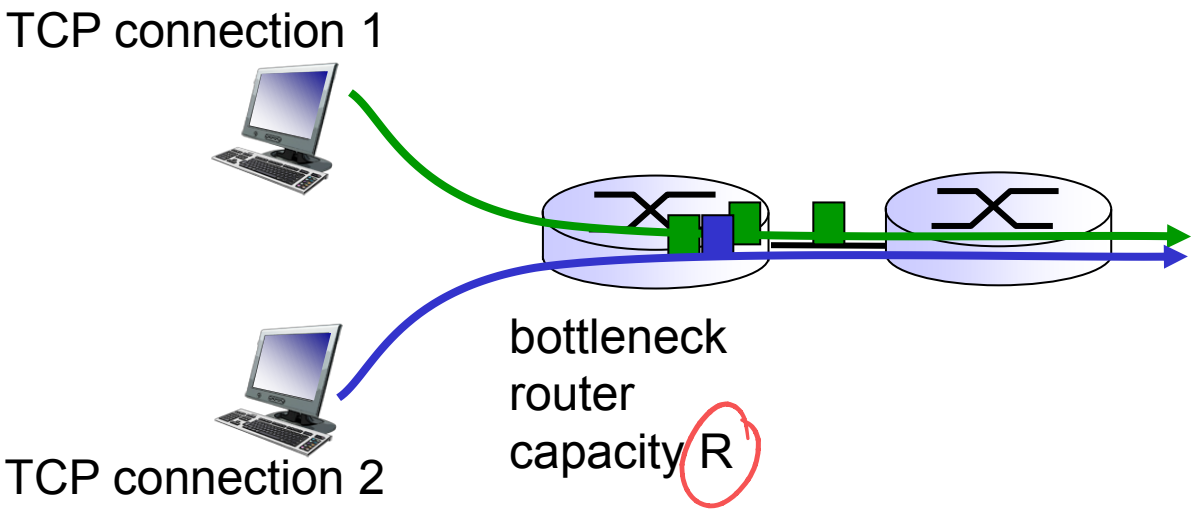
$$\text{avg TCP thruput} = \frac{3}{4}\frac{W}{RTT} \text{ bytes/sec}$$



predictable system

TCP Fairness

fairness goal: if K TCP sessions share same bottleneck link of bandwidth R, each should have average rate of R/K



Fairness (more)

Fairness and UDP

- multimedia apps often do not use TCP
  - do not want rate throttled by congestion control
- instead use UDP:
  - send audio/video at constant rate, tolerate packet loss

Fairness, parallel TCP connections

- application can open multiple parallel connections between two hosts
- web browsers do this
- e.g., link of rate R with 9 existing connections:
  - new app asks for 1 TCP, gets rate R/10
  - new app asks for 11 TCPs, gets just over R/2

Test your understanding

- The receiver will only send a very small rwnd that way it can never get too much traffic.

  This will be inefficient. The sender can only have a small window (like rdt 3.0)

- The sender will only reduce window size very slowly in case of congestion.

  We may lead to lots of lost packets.

  We may no longer respect fairness.

- We can be very quick if we send another packet if we have only one duplicate ACK (not triple)

  We will sends lots of unnecessary packets

- If you are confused review the previous section.

TCP mechanisms

Name these three TCP mechanisms

- When I see four Acknowledgement packets with the same number I retransmit a packet.
- Fast retransmit
- When cwnd is small I double cwnd every RTT where no loss is experienced.
- Slow start
- When I think there is no congestion I increase the window size slowly, when I think there is congestion I decrease the window size rapidly
- AIMD (Additive Increase Multiplicative Decrease)

Network Layer: outline

4.1 Overview of Network layer
- data plane
- control plane

4.2 What's inside a router

4.3 IP: Internet Protocol
- datagram format
- fragmentation
- IPv4 addressing
- network address translation
- IPv6

4.4 Generalized Forward and SDN
- match
- action
- OpenFlow examples of match-plus-action in action

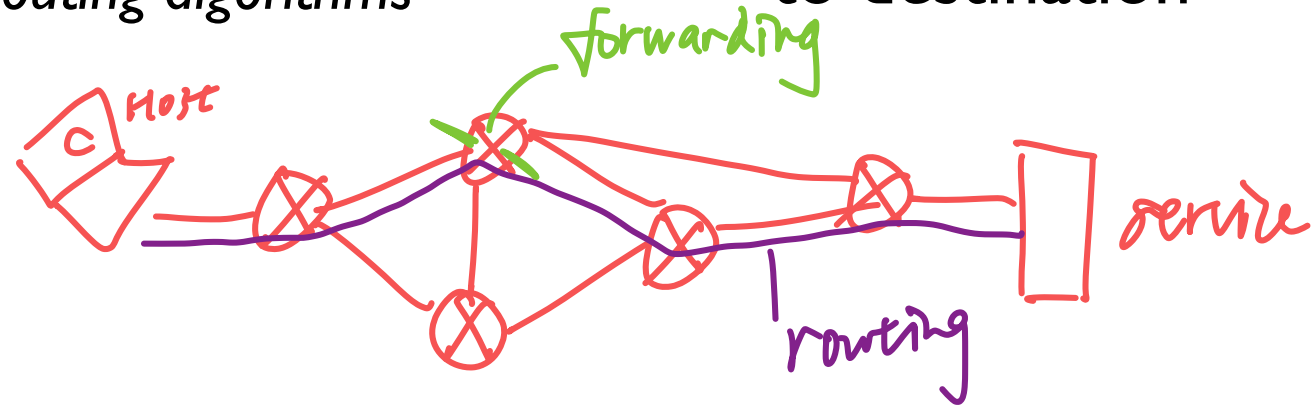
Two key network-layer functions

network-layer functions:

- *forwarding:* move packets from router's input to appropriate router output
- *routing:* determine route taken by packets from source to destination
  - *routing algorithms*

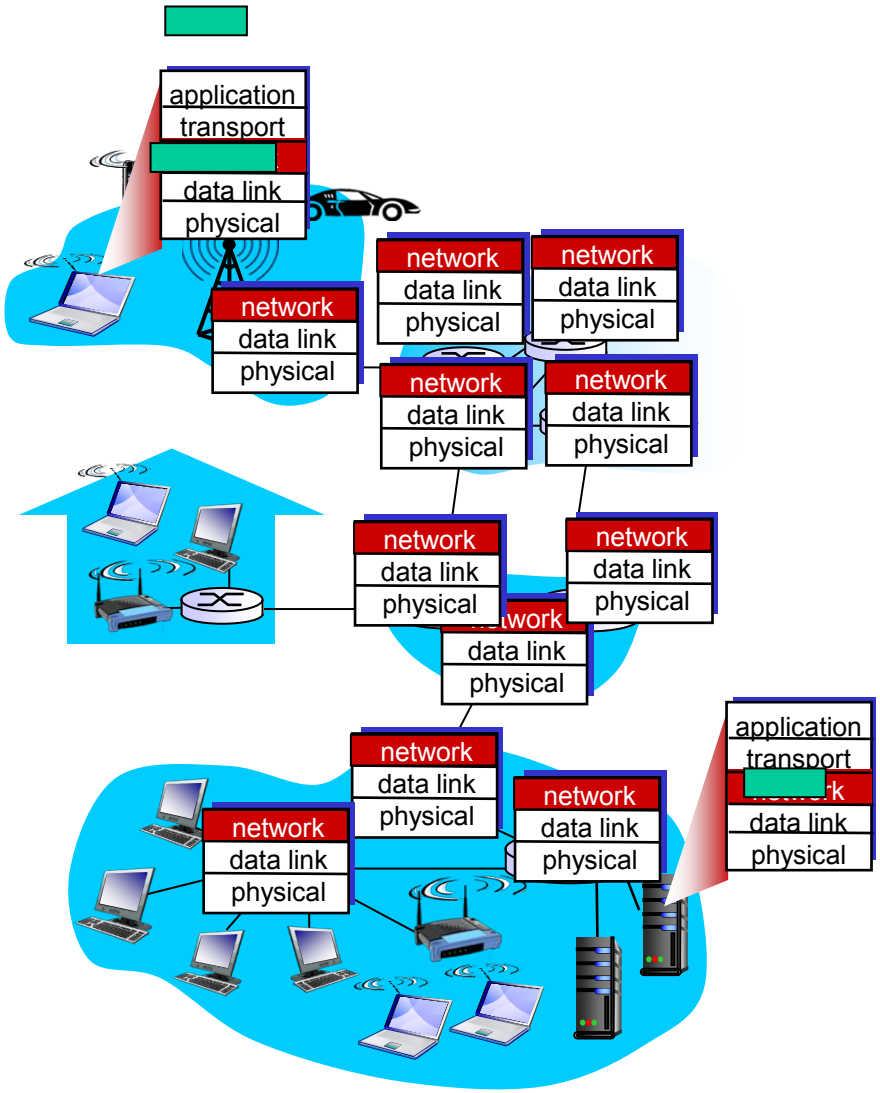
analogy: taking a trip

- *forwarding:* process of getting through one road junction
- *routing:* process of planning trip from source to destination



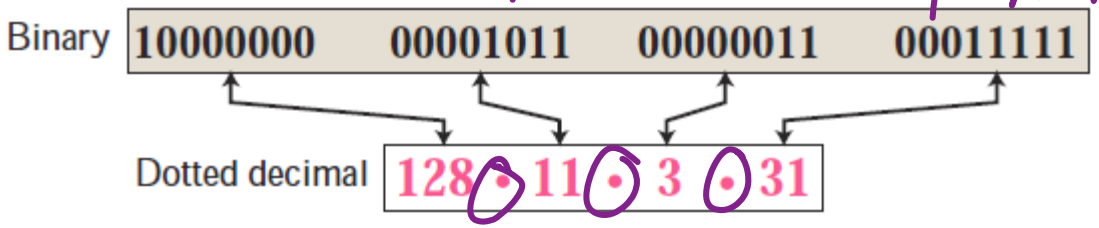
Network layer

- transport segment from sending to receiving host
- on sending side encapsulates segments into datagrams
- on receiving side, delivers segments to transport layer
- network layer protocols in *every* host, router
- router examines header fields in all IP datagrams passing through it



IPv4 address notation

- There are three common notations to show an IPv4 address:
  - binary notation
  - dotted-decimal notation (most commonly used)
  - hexadecimal notation



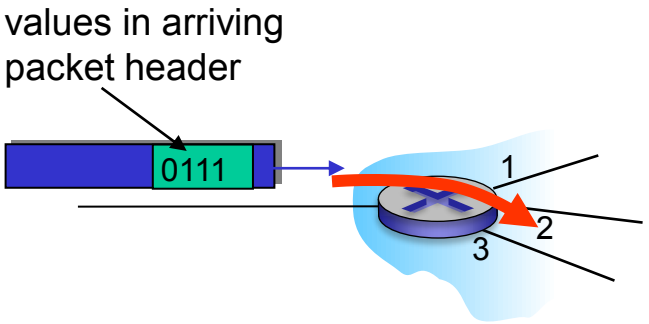
Exercise: convert the following IPv4 addresses from binary to dotted-decimal notation.

a. 10000001 00001011 00001011 11101111
b. 11000001 10000011 00011011 11111111
c. 11100111 11011011 10001011 01101111
d. 11111001 10011011 11111011 00001111

Network layer: data plane, control plane

Data plane

- local, per-router function
- determines how datagram arriving on router input port is forwarded to router output port within router
- forwarding function



Control plane

- network-wide logic
- determines how datagram is routed among routers along end-end path from source host to destination host
- two control-plane approaches:
  - *traditional routing algorithms:* implemented in routers
  - *software-defined networking (SDN):* implemented in (remote) servers

Data plane or control plane

Q: Finding the correct route for a packet

Control plane

Q: Forwarding a packet using Longest Prefix Match

Data plane – takes place locally using forwarding table

Q: An SDN controller

Control plane (it is in the name)

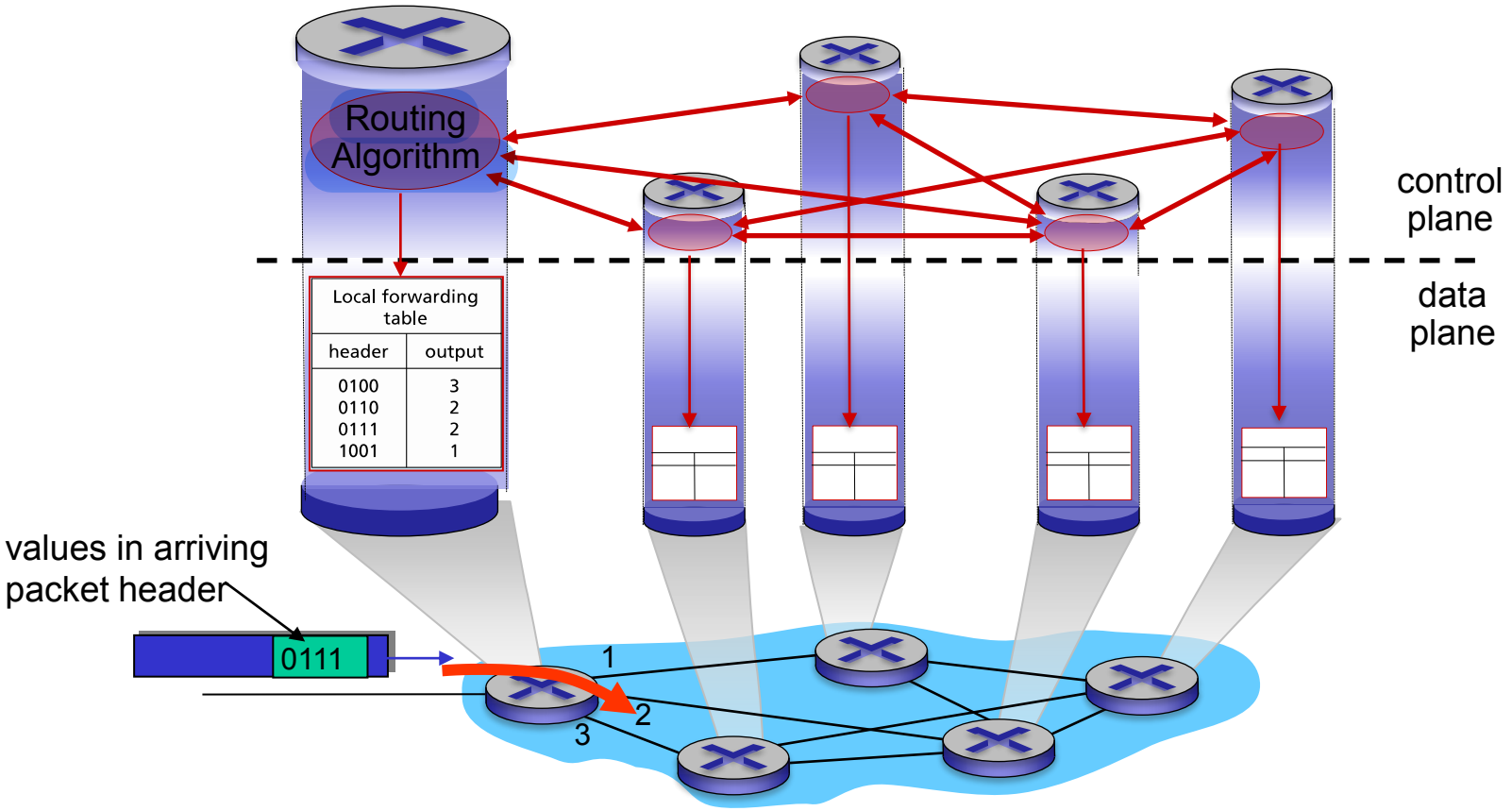
Q: A traditional router

Both (a router has forwarding and routing)

a router

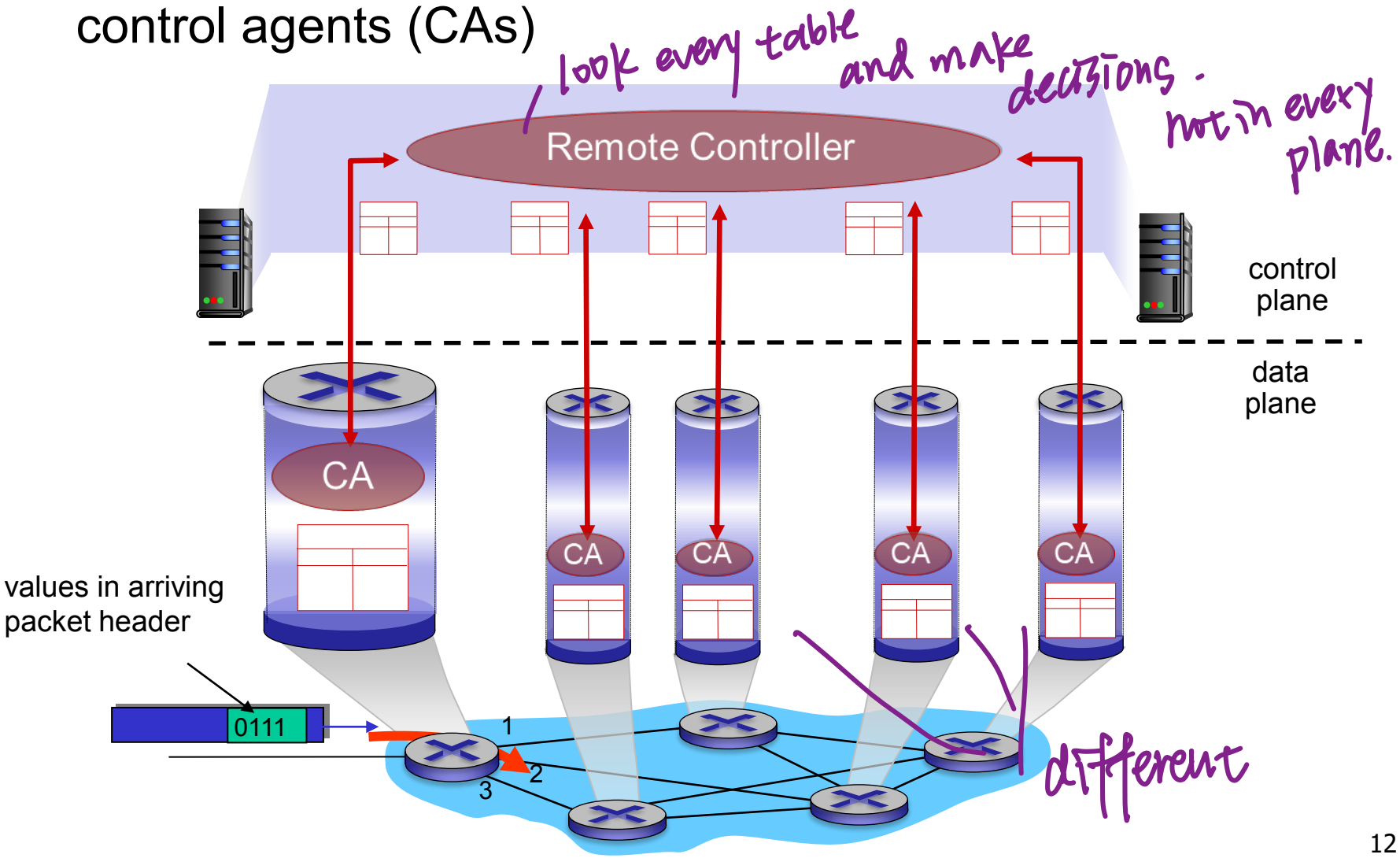
Per-router control plane

Individual routing algorithm components *in each and every router* interact in the control plane



Logically centralized control plane

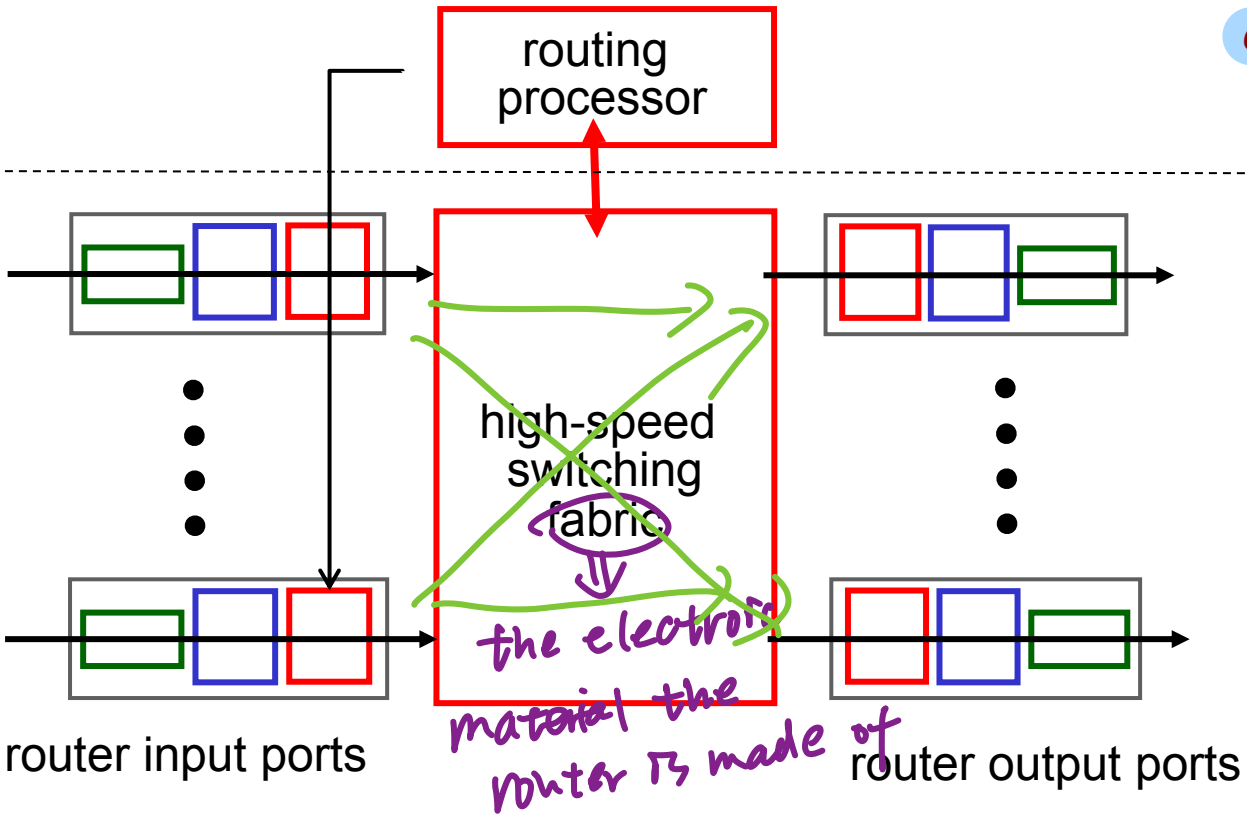
A distinct (typically remote) controller interacts with local control agents (CAs)



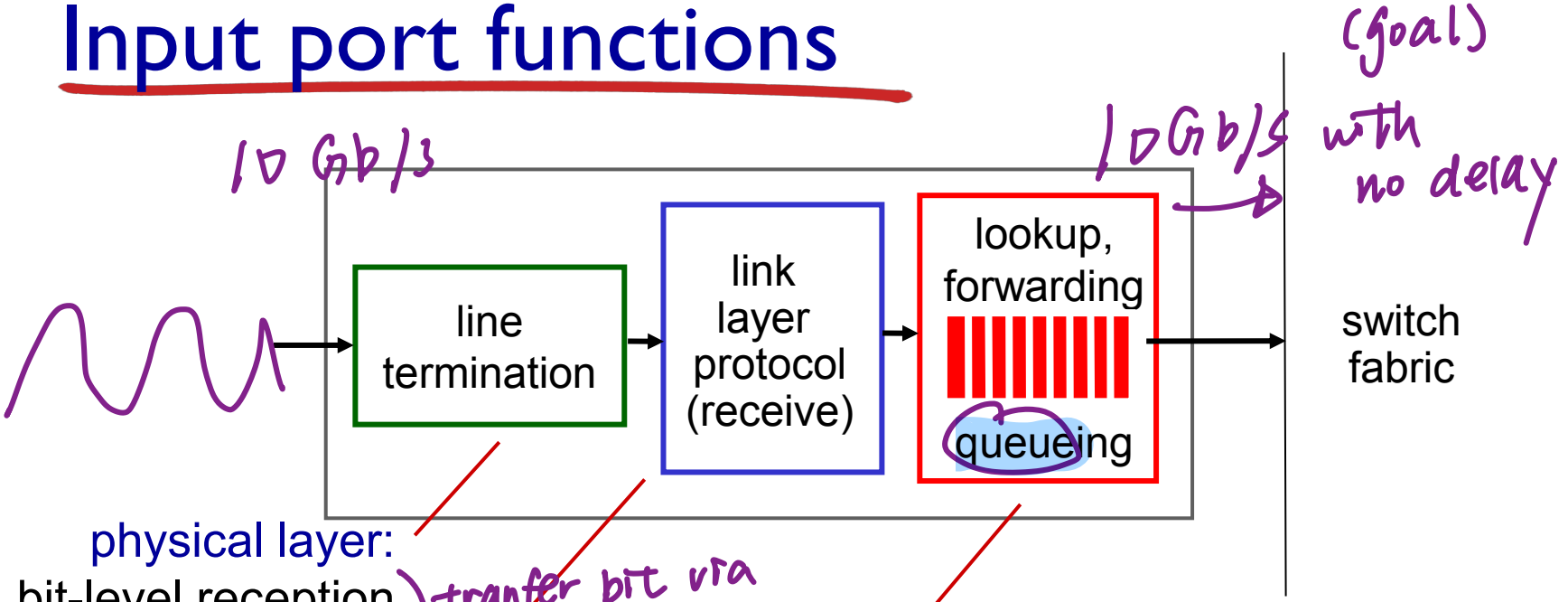
Router architecture overview

普通的

- high-level view of generic router architecture:



Input port functions



physical layer:
bit-level reception

data link layer:
e.g., Ethernet
see chapter 5
of course text

tranfer bit via a certain format

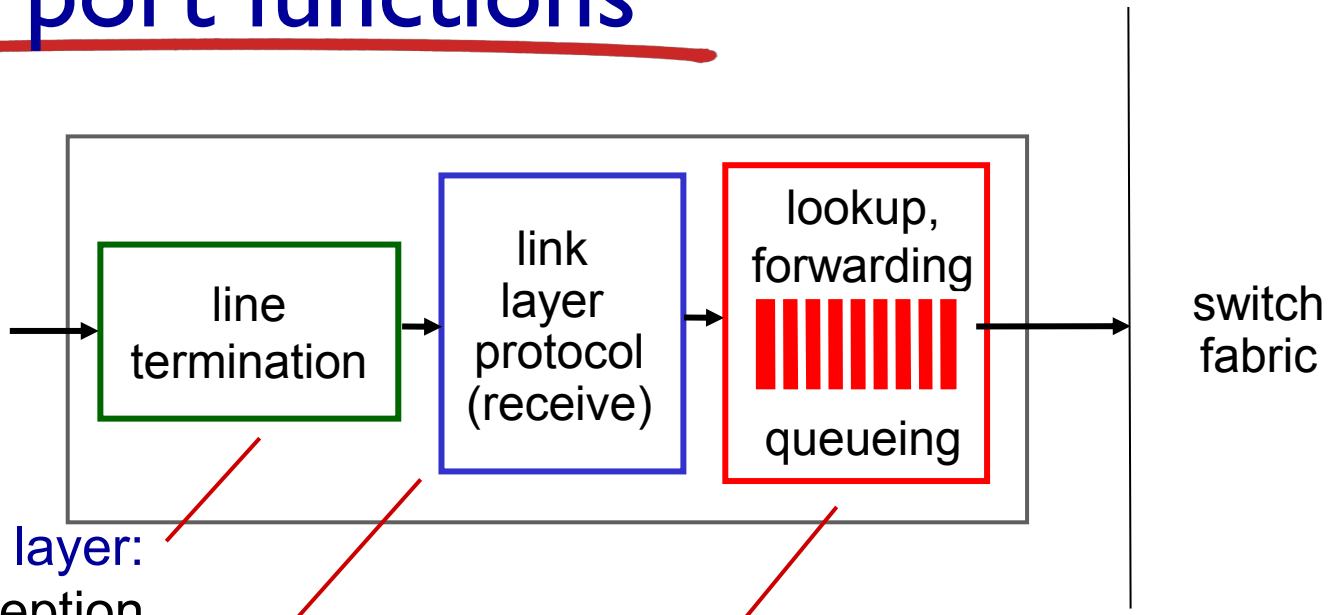
decentralized switching:

look address → to port x

- using header field values, lookup output port using forwarding table in input port memory (*"match plus action"*)
- goal: complete input port processing at 'line speed'
- queuing: if datagrams arrive faster than forwarding rate into switch fabric

no delays

Input port functions



physical layer:
bit-level reception

data link layer:
e.g., Ethernet
see chapter 5
of course text

decentralized switching:

- using header field values, lookup output port using forwarding table in input port memory *("match plus action")*
- *destination-based forwarding:* forward based only on destination IP address (traditional)
- *generalized forwarding:* forward based on any set of header field values

Longest prefix matching

longest prefix matching

when looking for forwarding table entry for given destination address, use *longest* address prefix that matches destination address.

前缀

Destination Address Range	Link interface
11001000 00010111 00010*** *********	0
11001000 00010111 00011000 *********	1 longest
11001000 00010111 00011*** *********	2
otherwise	3

examples:

DA: 11001000 00010111 00010110 10100001 which interface? 0

DA: 11001000 00010111 00011000 10101010 which interface? 1 (longest)

How does this look in exam paper?

200.23.120.0/21	**A**
200.23.120.0/24	**B**
200.23.24.0/21	**C**
0.0.0.0/0	**D**

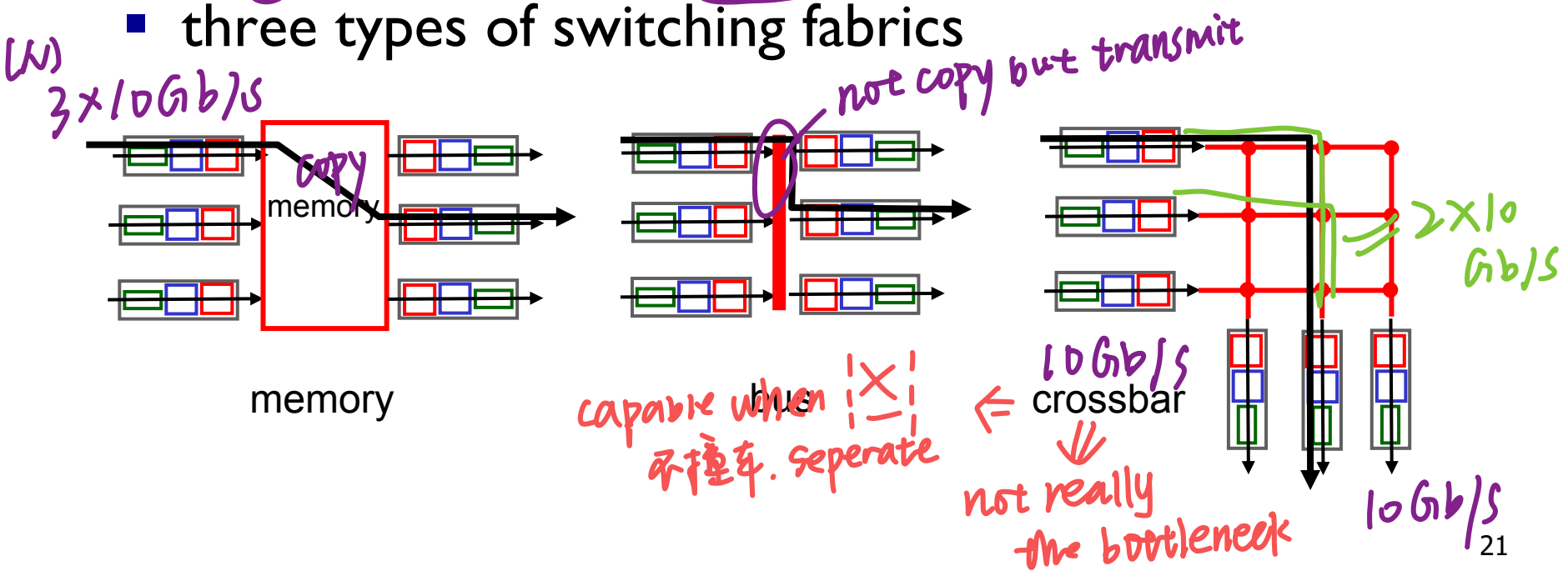
In decimal number

```
Packet1: 200.23.120.170  B
Packet2: 200.23.124.170  A
Packet3: 200.23.56.170   D
```

Convert to Binary → show prefix matching

Switching fabrics

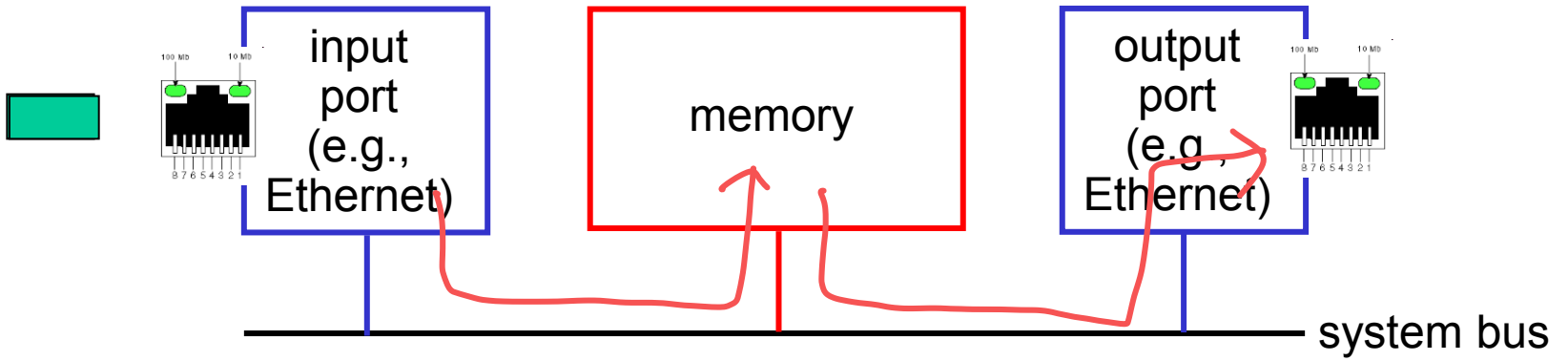
- transfer packet from input buffer to appropriate output buffer
- switching rate: rate at which packets can be transfer from inputs to outputs
  - often measured as multiple of input/output line rate
  - N inputs: switching rate N times line rate desirable
- three types of switching fabrics



Switching via memory

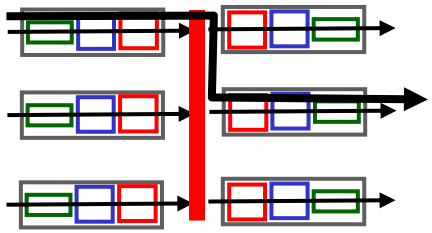
first generation routers:

- traditional computers with switching under direct control of CPU
- packet copied to system's memory
- speed limited by memory bandwidth (2 bus crossings per datagram)



Switching via a bus

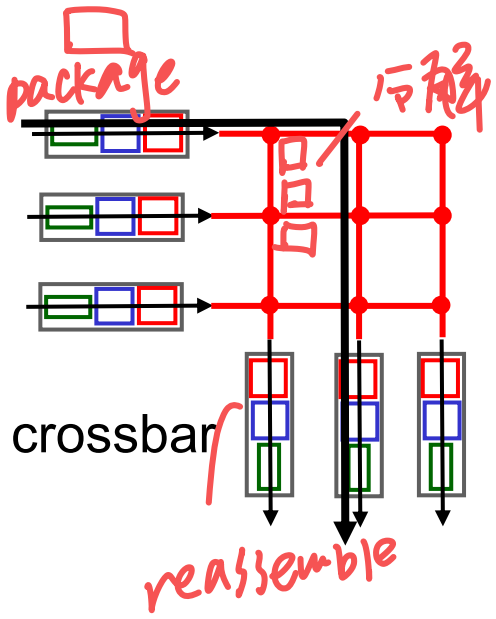
- datagram from input port memory to output port memory via a shared bus
- *bus contention:* switching speed limited by bus bandwidth
- 32 Gbps bus, Cisco 5600: sufficient speed for access and enterprise routers



bus

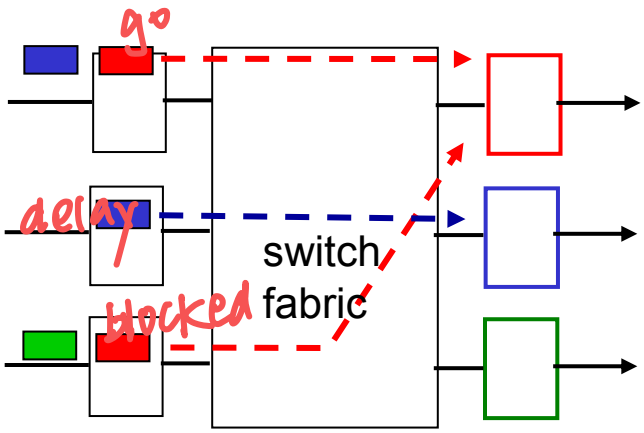
Switching via interconnection network

- overcome bus bandwidth limitations
- banyan networks, crossbar, other interconnection nets initially developed to connect processors in multiprocessor
- advanced design: fragmenting datagram into fixed length cells, switch cells through the fabric.
- Cisco 12000: switches 60 Gbps through the interconnection network

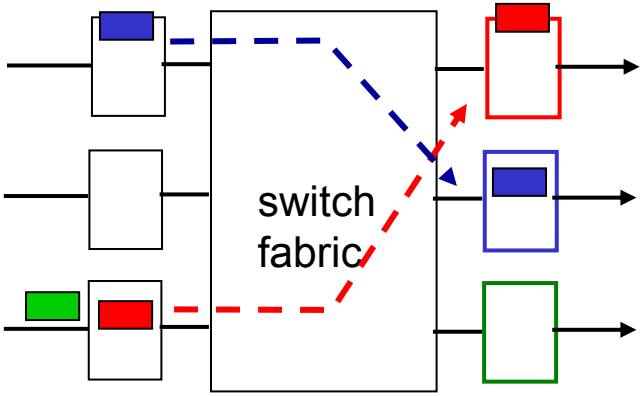


Input port queuing

- fabric slower than input ports combined -> queueing may occur at input queues
  - *queueing delay and loss due to input buffer overflow!*
- Head-of-the-Line (HOL) blocking: queued datagram at front of queue prevents others in queue from moving forward



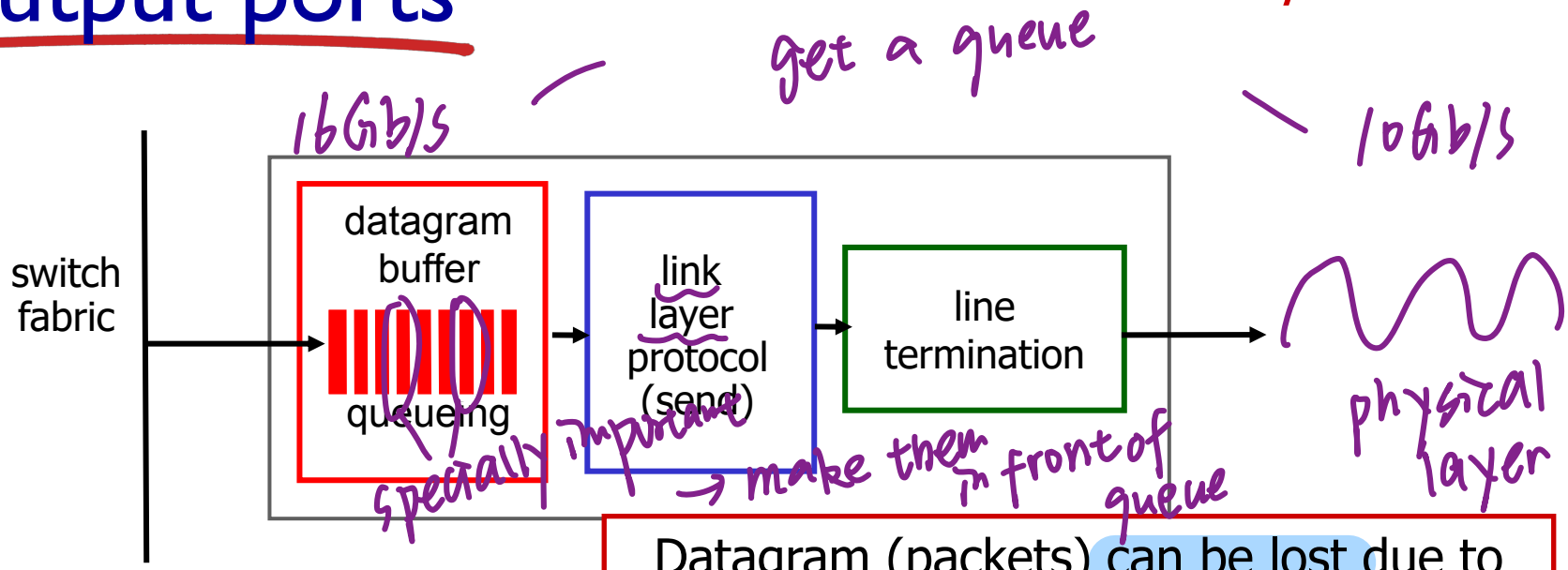
output port contention:
only one red datagram can be transferred.
lower red packet is blocked



one packet time later:
green packet experiences HOL blocking

Output ports

This slide is HUGELY important!



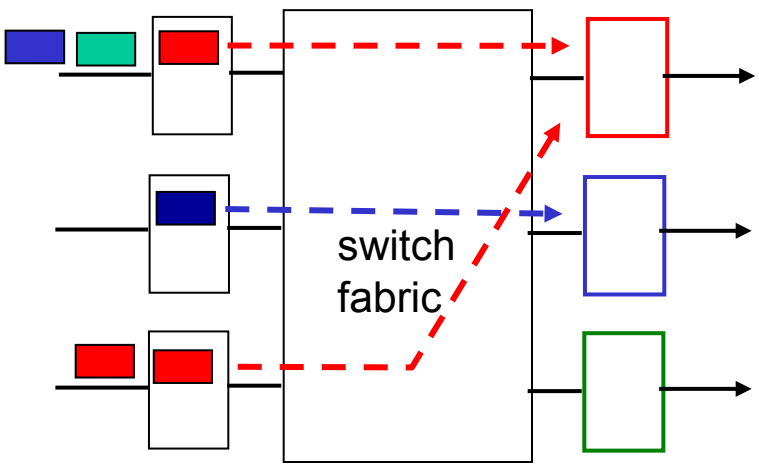
Datagram (packets) can be lost due to congestion, lack of buffers

- *buffering* required when datagrams arrive from fabric faster than the transmission rate
- *scheduling discipline* chooses among queued datagrams for transmission

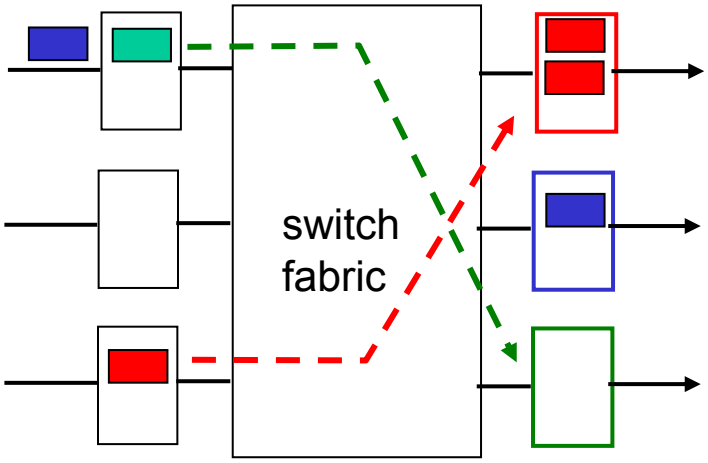
Priority scheduling – who gets best performance, network neutrality

controversial

Output port queueing



at *t*, packets more from input to output



one packet time later

- buffering when arrival rate via switch exceeds output line speed
- *queueing (delay) and loss due to output port buffer overflow!*

How much buffering?

- RFC 3439 rule of thumb: average buffering equal to "typical" RTT (say 250 msec) times link capacity C
  - e.g., C = 10 Gpbs link: 2.5 Gbit buffer
- recent recommendation: with *N* flows, buffering equal to

round trip time

$$\frac{RTT \cdot C}{\sqrt{N}}$$

- Why? Why not simply have very large buffers so no loss?

Scheduling mechanisms

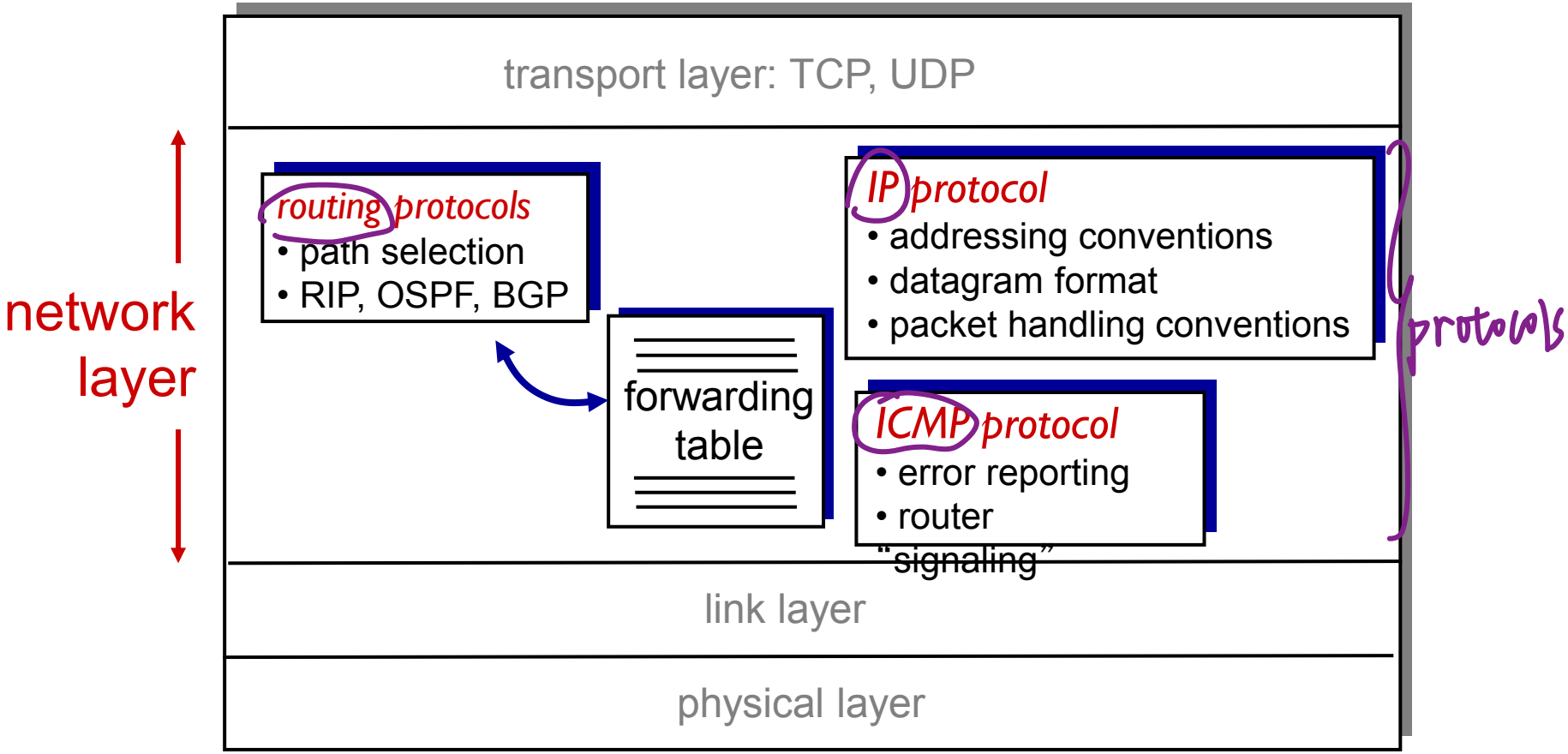
- *scheduling:* choose next packet to send on link.
- FIFO scheduling – first in first out
  - Like an orderly queue of people, no pushing in.
  - If queue is full last packets are dropped.
- Priority scheduling
  - Some packets are more important
  - Example: You need live video packets now, email could wait.
- Round robin scheduling
  - If your queue is from several inputs ports treat them fairly
  - Pick a packet from input port 1, then 2, then 3, then 4 → back
  - Port which is sending lots of traffic doesn't block others.
  - Weighted Fair Queue (like this but give some queues a little more priority -- give a little more traffic to port 1)

port 1 twice then port 2 once

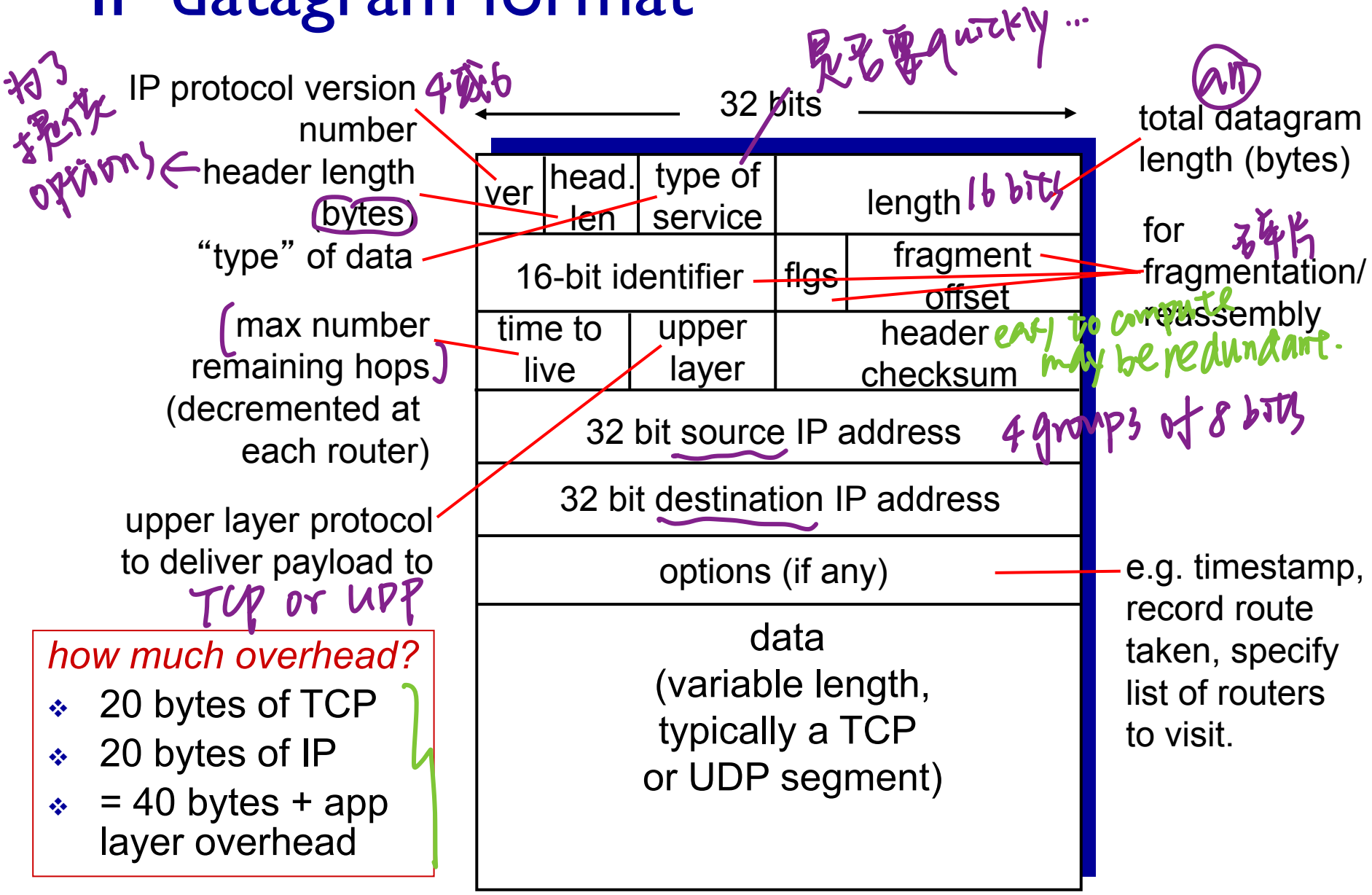
1 2 2 3 2 4 2

The Internet network layer

host, router network layer functions:



IP datagram format

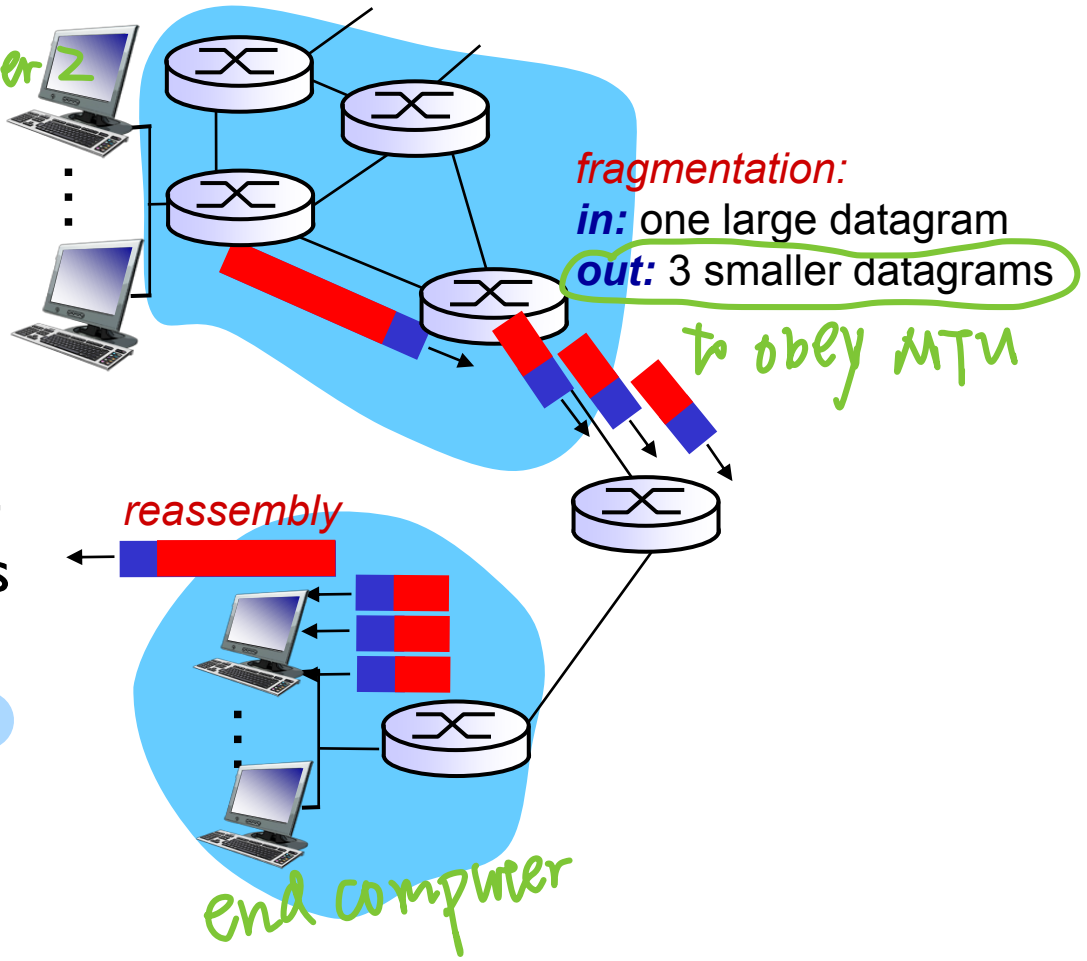


how much overhead?

- 20 bytes of TCP
- 20 bytes of IP
- = 40 bytes + app layer overhead

IP fragmentation, reassembly

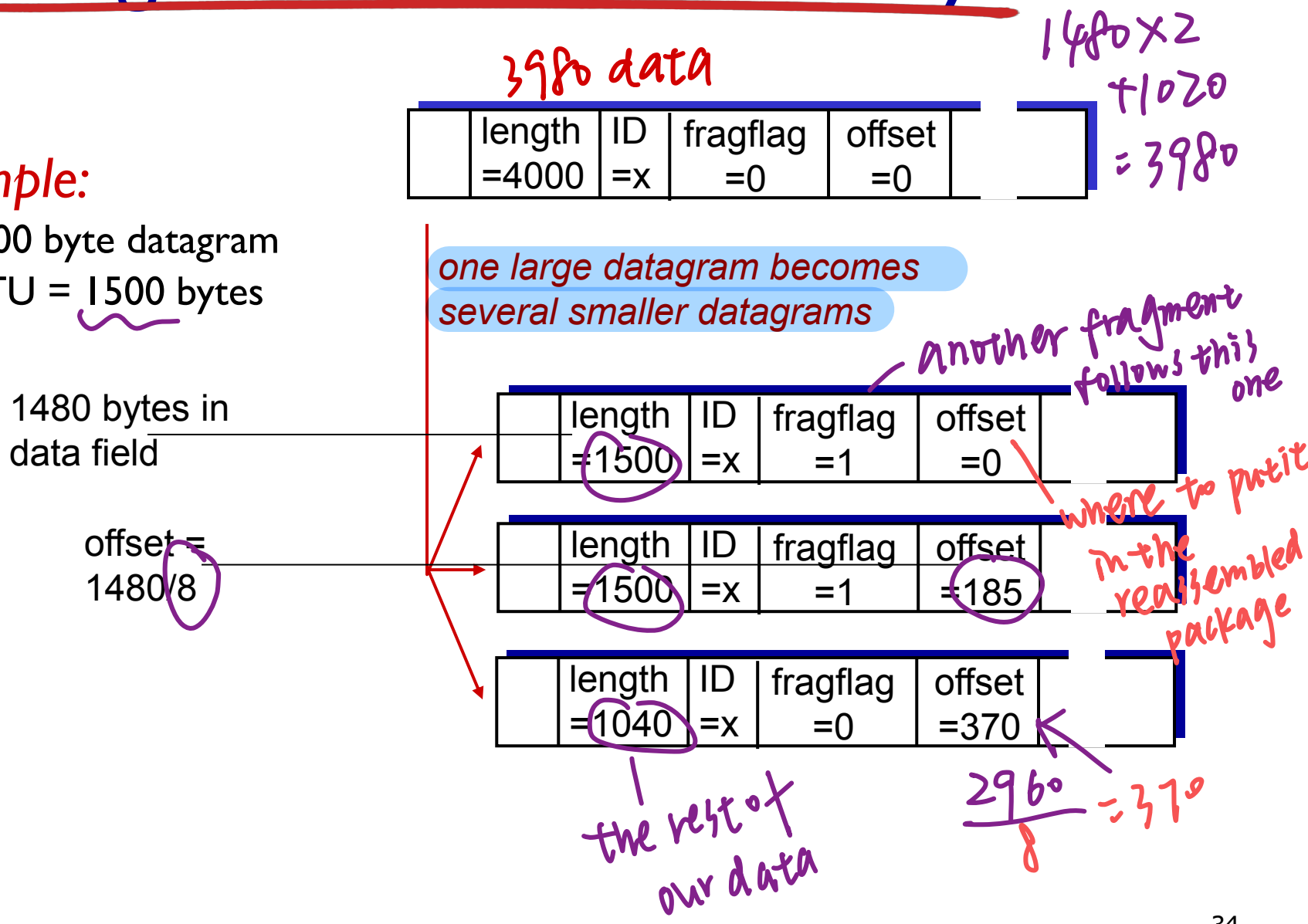
- network links have MTU (max.transfer size) - largest possible link-level frame
  - different link types, different MTUs
- large IP datagram divided ("fragmented") within net
  - one datagram becomes several datagrams
  - "reassembled" only at final destination
  - IP header bits used to identify, order related fragments



IP fragmentation, reassembly

example:

- 4000 byte datagram
- MTU = 1500 bytes



IP packet fragmentation Q 2

- A datagram has 2600 bytes. (This is the size at layer 3 including the layer 3 header).
- It is sent over a network with MTU 1300 bytes.
- It is split into 3 fragments. For the third fragment give the length, offset and fragflag

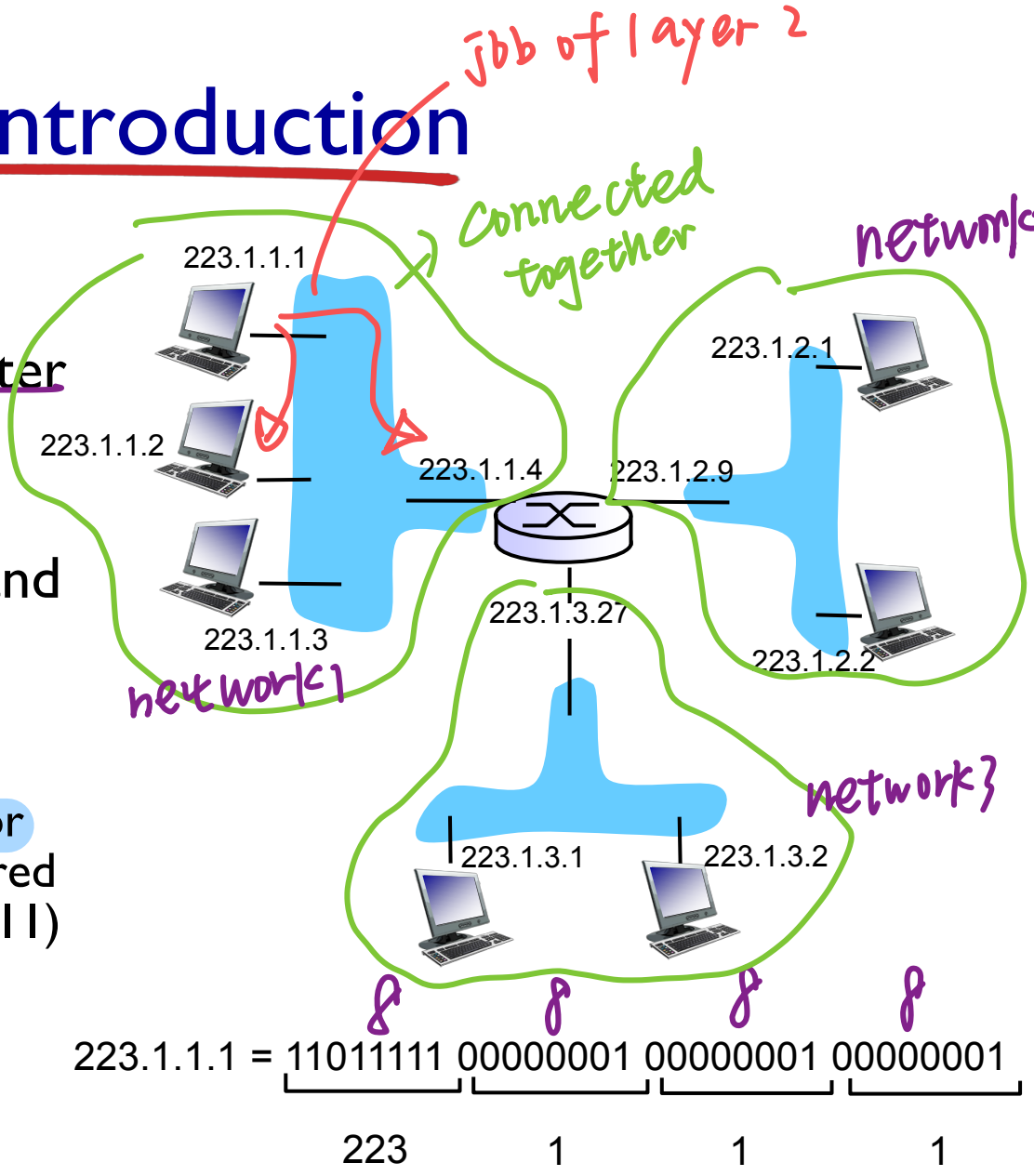
A:
1st fragment packet length 1300 bytes = 20 bytes (IP) + 1280 bytes (data), Offset=0, Fraglag=1
2nd fragment packet length 1300 bytes = 20 bytes (IP) + 1280 bytes (data)
A: 3rd fragment carries length 40 bytes = 20 bytes (IP) + 2600 – 2580 bytes (data)
A: Offset = (1280*2)/8 = 320 (This is because the size of payload is multiple of 8 bytes.
A: Fragflag = 0 (no following fragments)

What have we learned?

- Network layer moves data from "source" all the way to the "destination". global traveller
- Control plane makes decisions about routes taken by packets it spans a network.
- Data plane implements the decisions and gets the data from place to place. It is local to a router.
- Longest prefix matching on a router is used to pick where to send data.
- Queuing at routers can have huge effects.
- Scheduling polices can benefit some traffic over other traffic.
- Fragmentation and reassembly can be used to split and rejoin IP packets.

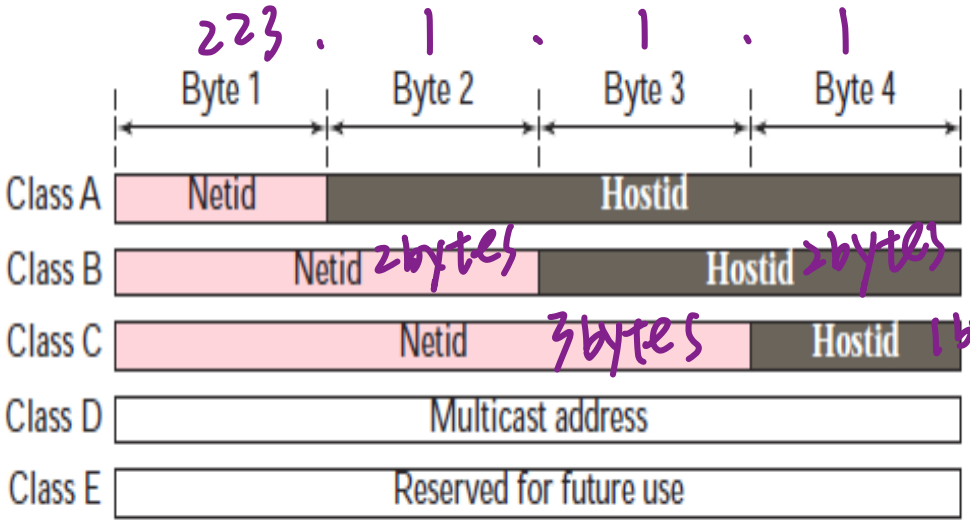
IP addressing: introduction

- *IP address:* 32-bit identifier for host, router *interface*
- *interface:* connection between host/router and physical link
  - routers typically have multiple interfaces
  - host typically has one or two interfaces (e.g., wired Ethernet, wireless 802.11)
- *IP addresses associated with each interface*



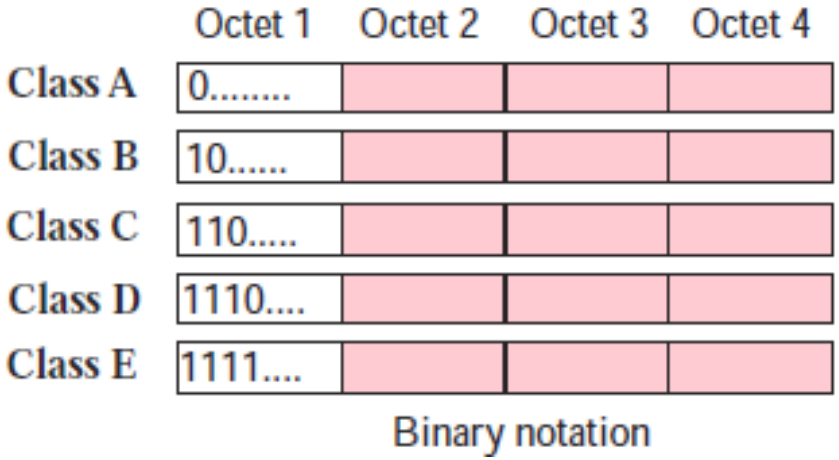
Classful IP addressing

historic, not the main way now

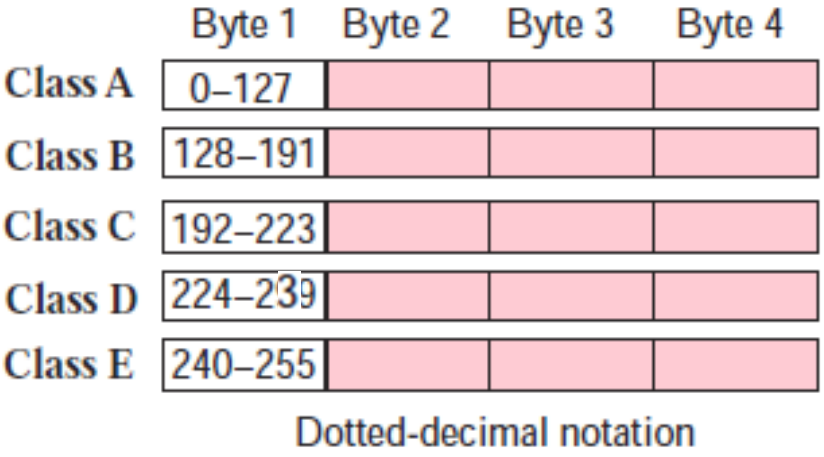


- In classful addressing, the IP address space is divided into five classes: A, B, C, D, E.
- Starting number, n (first byte), shows whether Class A, B or C
  - **Class A**: n < 128 (up to 16m hosts)
  - **Class B**: 128 ≤ n < 192 (up to 65K hosts)
  - **Class C**: 192 ≤ n < 224 (up to 254 hosts)

223.1.1 | 1
network | host



Binary notation

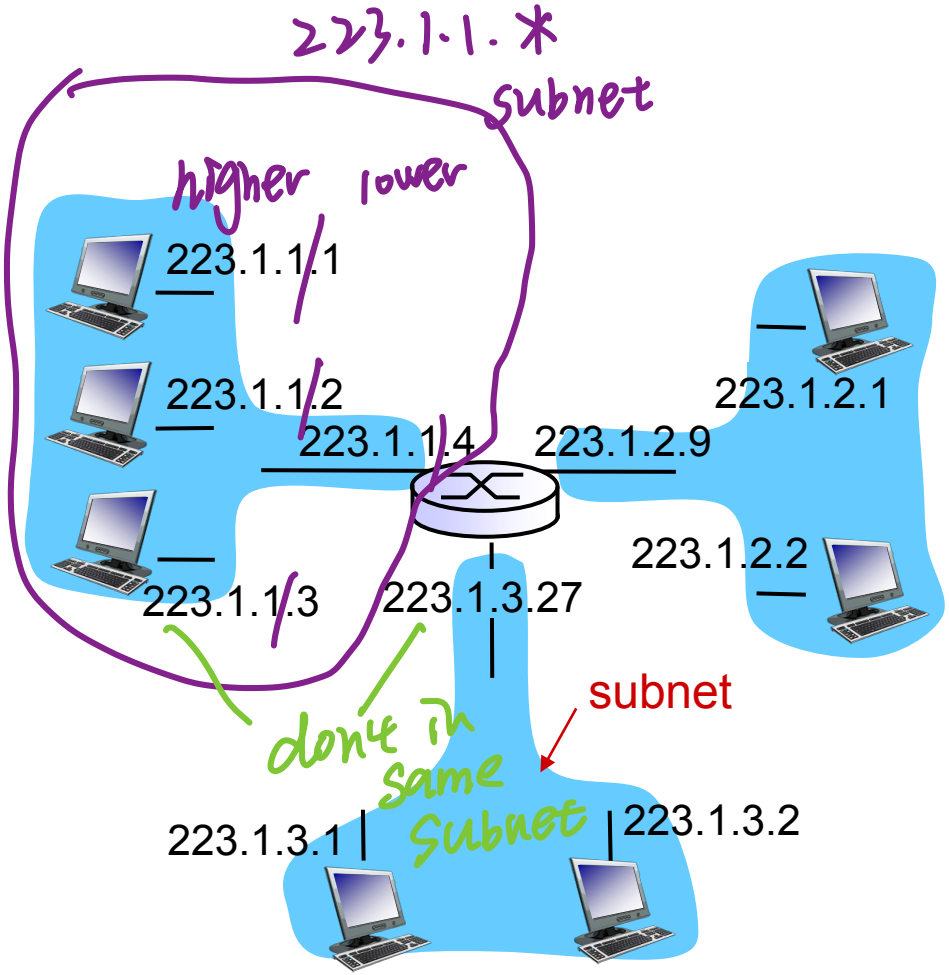


Dotted-decimal notation

Subnets

- IP address:
  - subnet part - high order bits
  - host part - low order bits
- *what's a subnet ?*
  - device interfaces with same subnet part of IP address
  - can physically reach each other *without intervening router*

=going through

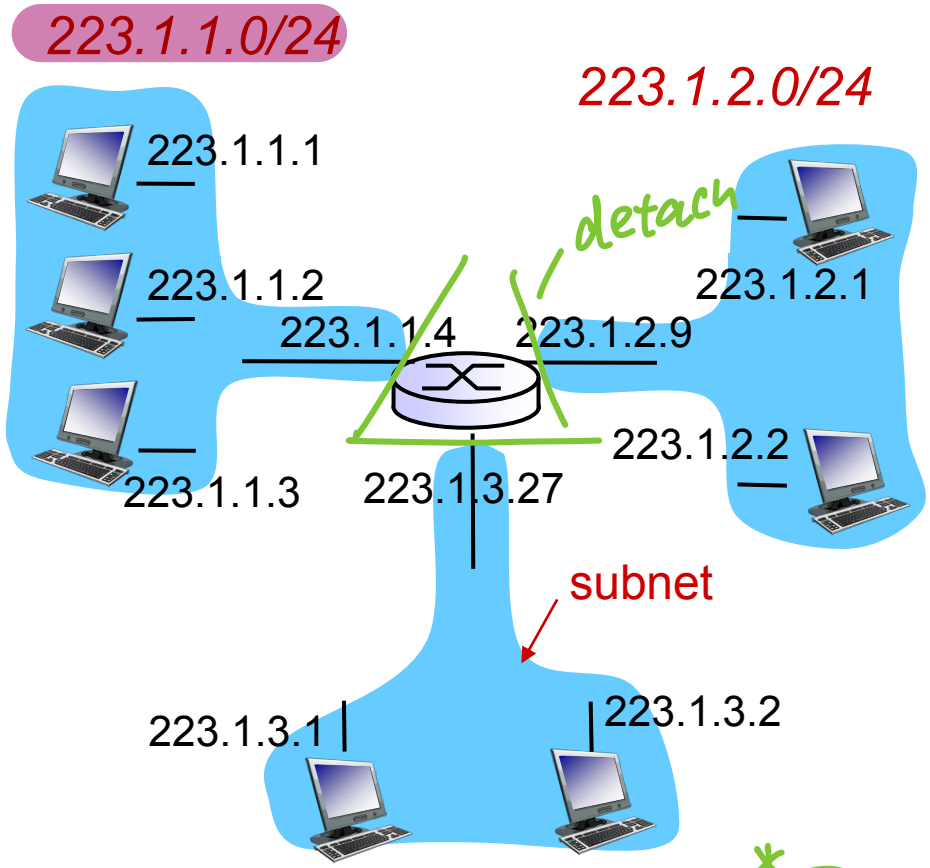


network consisting of 3 subnets

Subnets

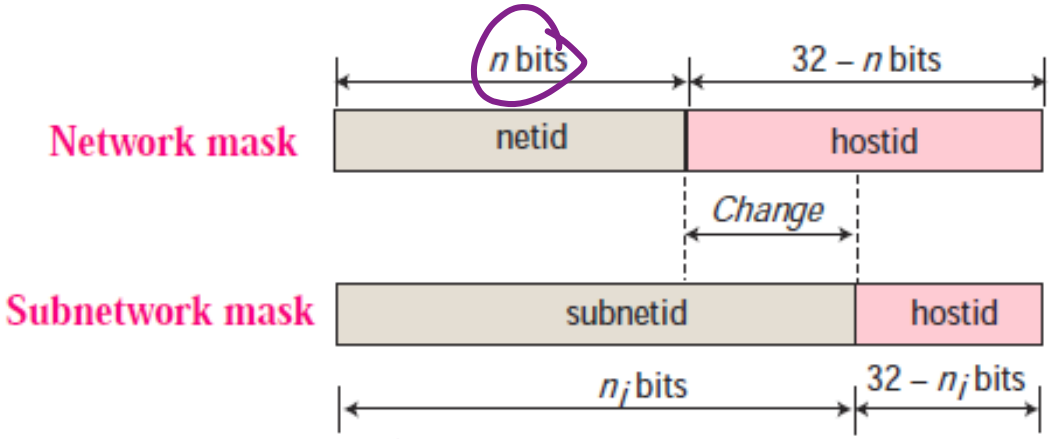
recipe

- to determine the subnets, detach each interface from its host or router, creating islands of isolated networks
- each isolated network is called a *subnet*
- *subnet mask (or slash notation) number of bits taken to identify network*
  - */8 size of old class A*
  - */16 size of class B*
  - */24 size of class C*



Network mask and subnetwork mask

- Subnetting increases length of **netid** and decreases length of **hostid**



- To divide a network to **s** number of subnetworks, each of equal numbers of hosts, the **subnetid** for each subnetwork can be calculated as

$$n_{sub} = n + \log_2 s$$

n - length of netid, n_{sub} - length of each subnetid, ***s*** – number of subnets

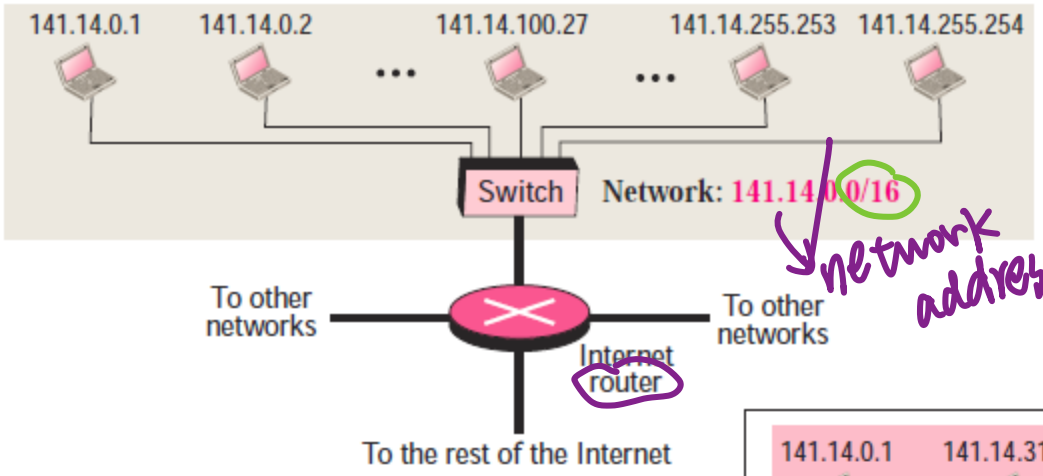
Three-Level Addressing: Subnetting

- The idea of splitting a block to smaller blocks is referred to as subnetting.
- In subnetting, a network is divided into several smaller subnetworks (subnets) with each subnetwork having its own subnetwork address.

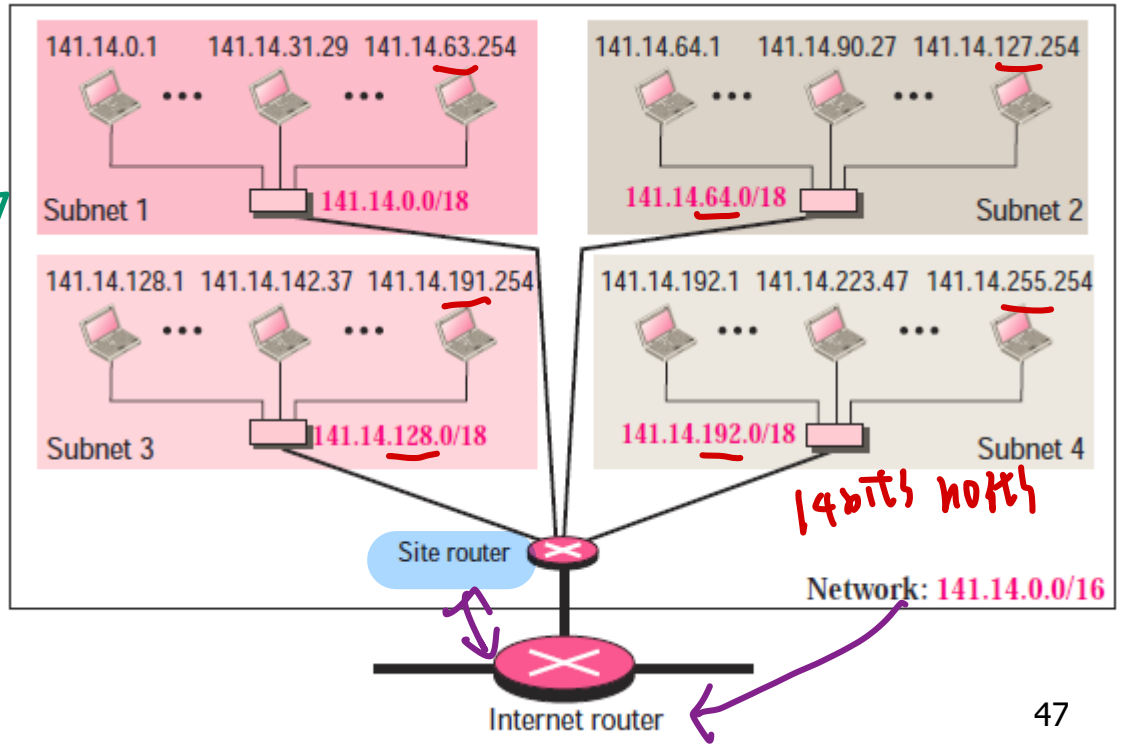
Why subnetting ?

- an organization that was granted a large block of IP addresses (a long time ago this would be class A, class B etc)
- wants to divide this into smaller blocks of addresses that are individual networks.
- Or perhaps organization wants to sell some of its IP addresses off.

A subnetting example



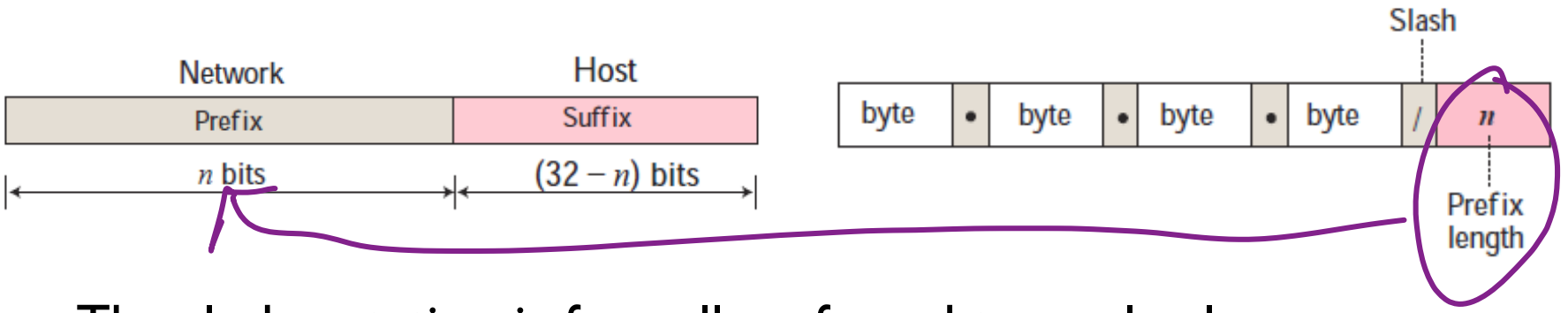
- A network using class B addresses before subnetting
- /16 to show the length of the netid (class B)



- a private site router is used to divide the network into four subnetworks.
- after subnetting, each subnetwork can now have almost 2^{14} hosts.
- /16 and /18 show the length of the netid and subnetids.

Classless addressing

- In classless addressing, variable-length blocks are assigned that belong to no class.
- In this architecture, the entire address space (2^{32} addresses) is divided into blocks of different sizes.



- The slash notation is formally referred to as classless interdomain routing or CIDR (Classless InterDomain Routing) notation.

Subnets and / notation

- Example 129.66.25.5/22

Network part (22 bits – because it is /22)

Host part 10 bits
(32 bits – 22 bits)

10000001.01000010.00011001.00000101

Network address is the address with all the host bits set to zero – address like any other but represents the network.

10000001.01000010.00011000.00000000

Network address: 129.66.24.0/22

The network address is also the first usable IP address in the block

Broadcast address (sends to everyone) is the address with all the host bits set to one.

10000001.01000010.00011011.11111111

Broadcast address: 129.66.27.255/22

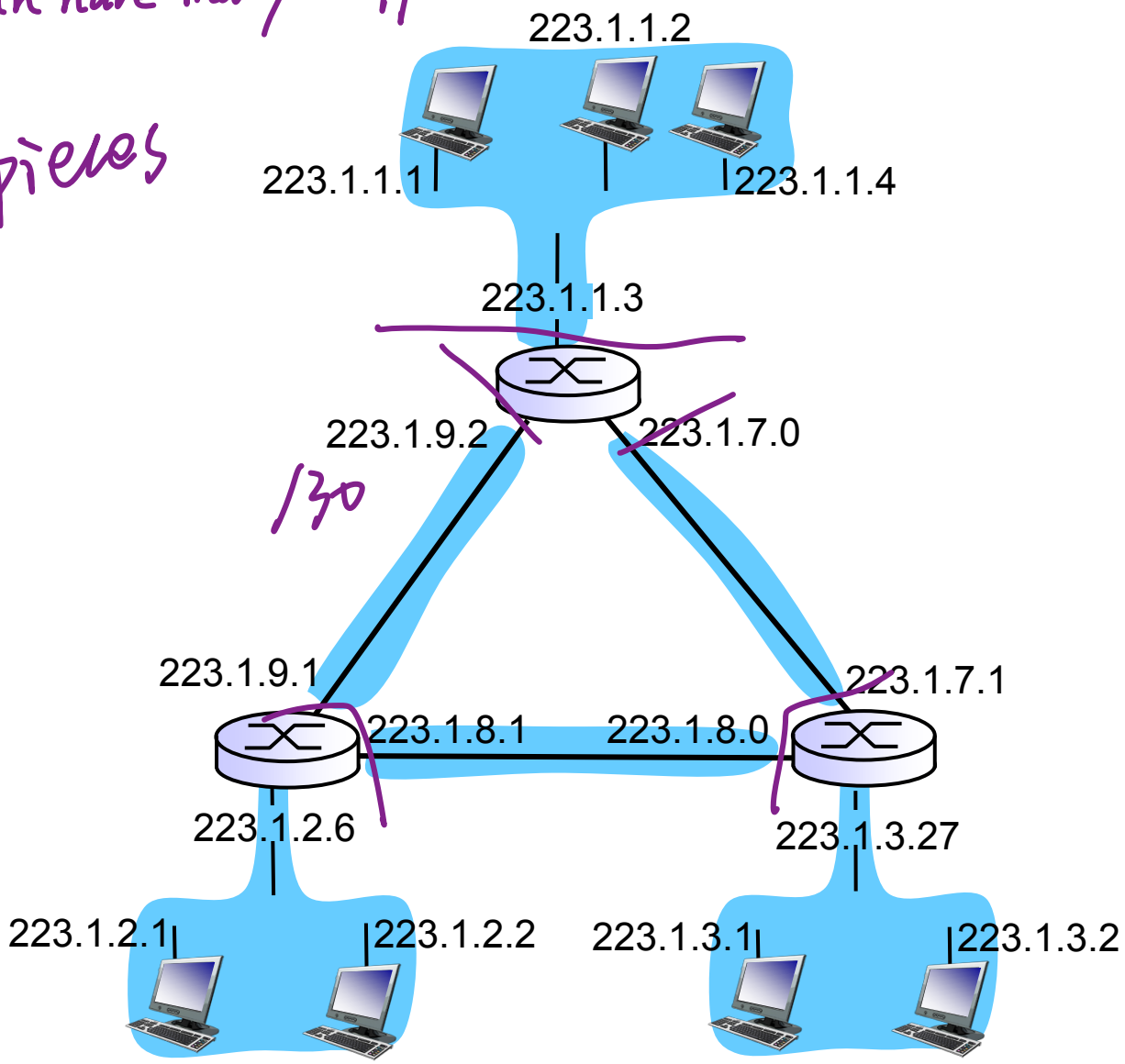
Splitting IP address by slash notation

- For a /n address, the first n bits are the network address and the last 32-n bits are for the host.
- If the host parts are all 1s then this is the broadcast address – sent to all hosts on the network.
- Host has m = 32-n bits (e.g. /20 has m=12)
- Room for $2^m - 1$ hosts (e.g. /20 has 4095)
- Example:
  - 140.120.84.24/20
  - Network address is 140.120.80.0/20
  - Broadcast address is 140.120.95.255/20
  - (95 is 01011111 255 is 11111111)
- /31 only has 1 host address
- /30 is smallest subnet we can have – 3 hosts
- /30 commonly used to connect just two routers (which must be on same subnet).

Subnets

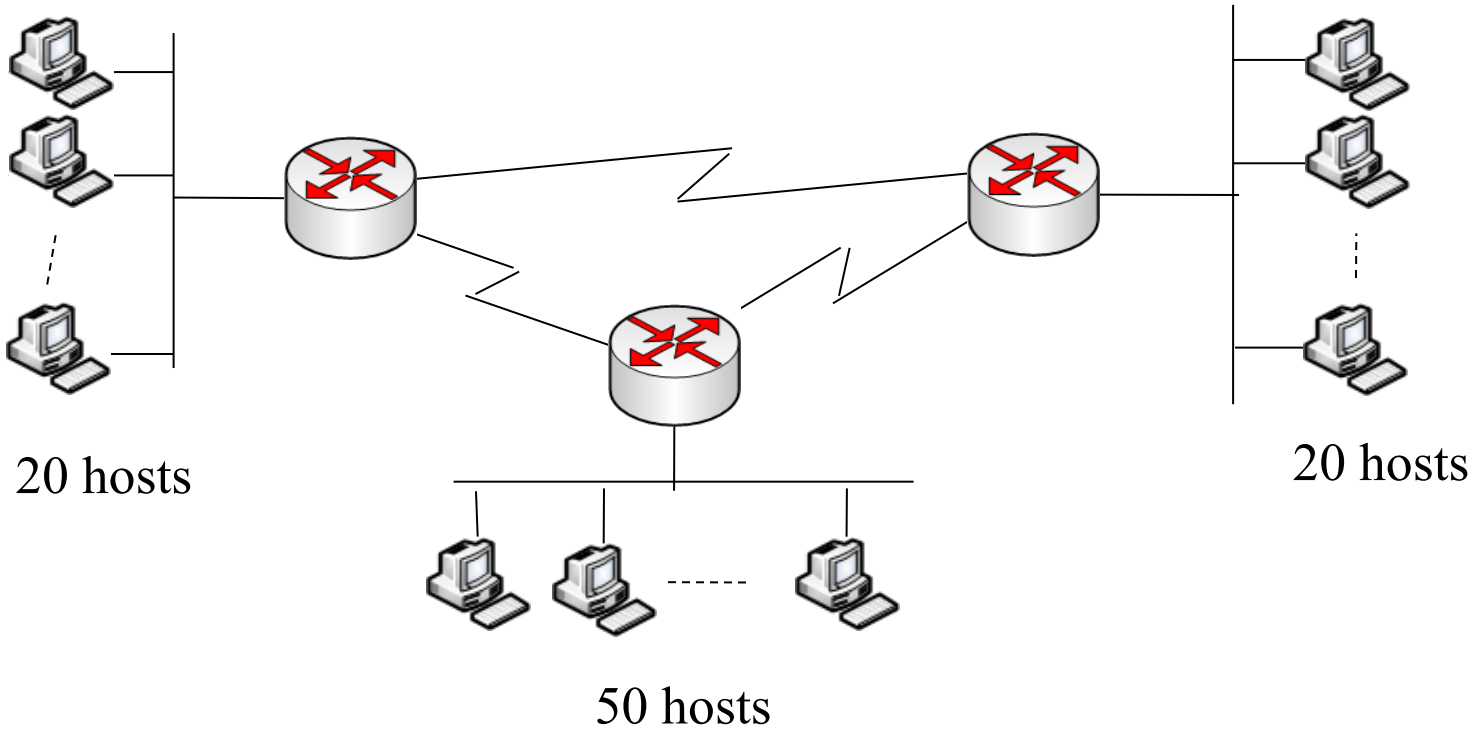
can have many different routers

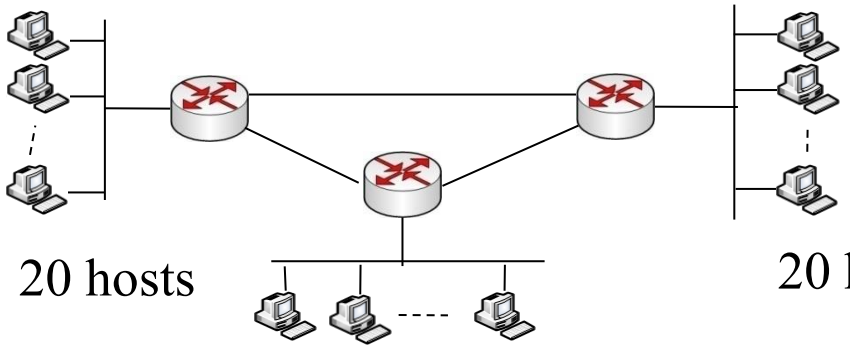
how many? 6 pieces



VLSM (variable length subnet mask)

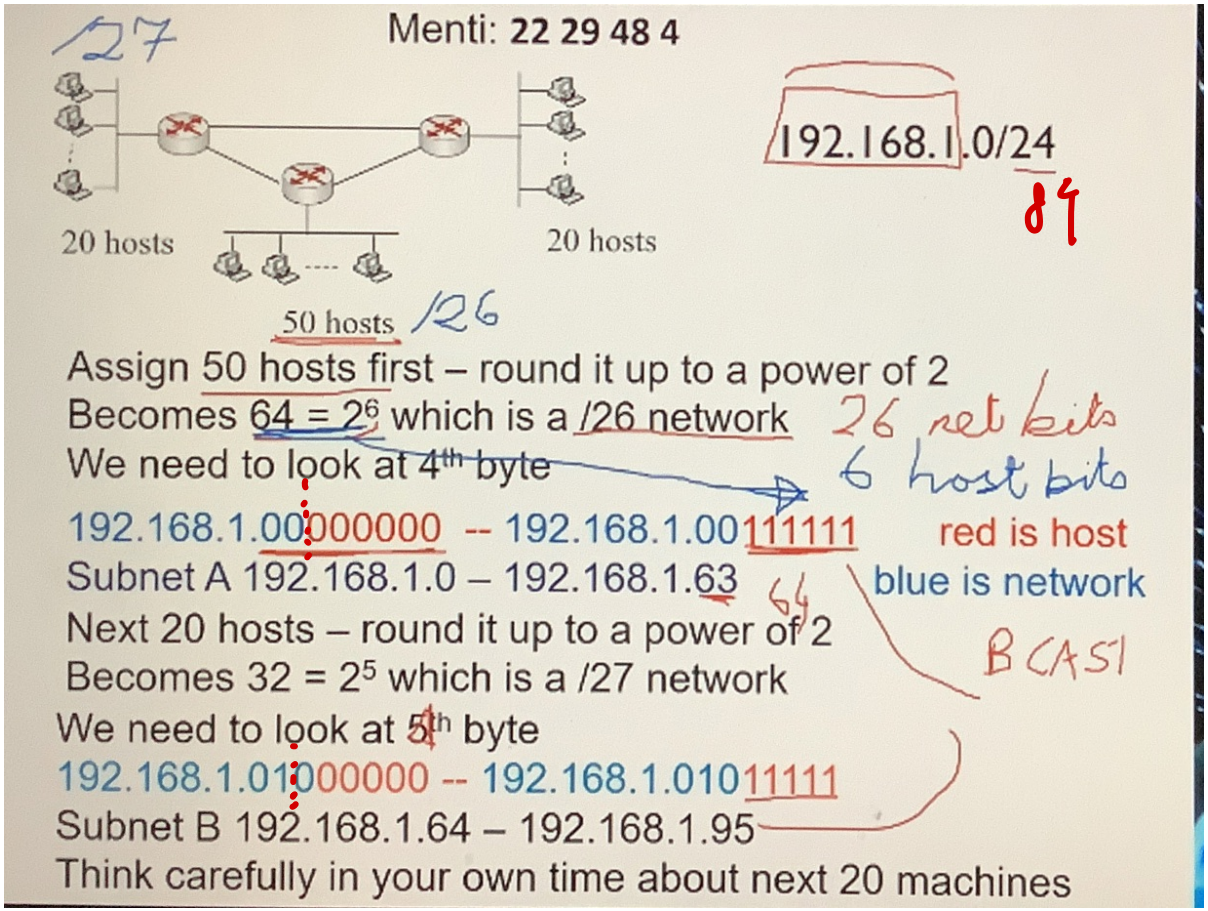
Subnet 192.168.1.0/24 to address this network by using the most efficient addressing possible.





192.168.1.0/24

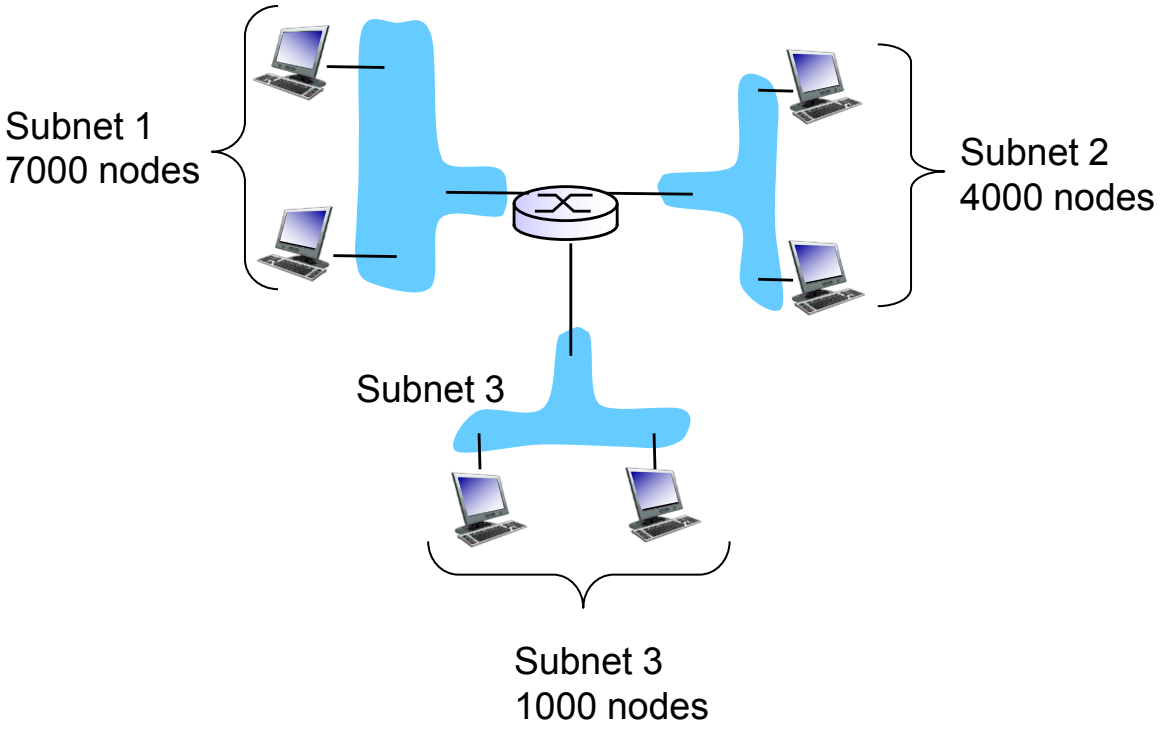
network address range?



IP Subnet

Question: Given the IP address of 192.0.0.0/18 which is enough for 16382 nodes, you are required to divide the IP address into 3 subnets, from largest subnet to the smallest.

What is Subnet 1 IP address range?



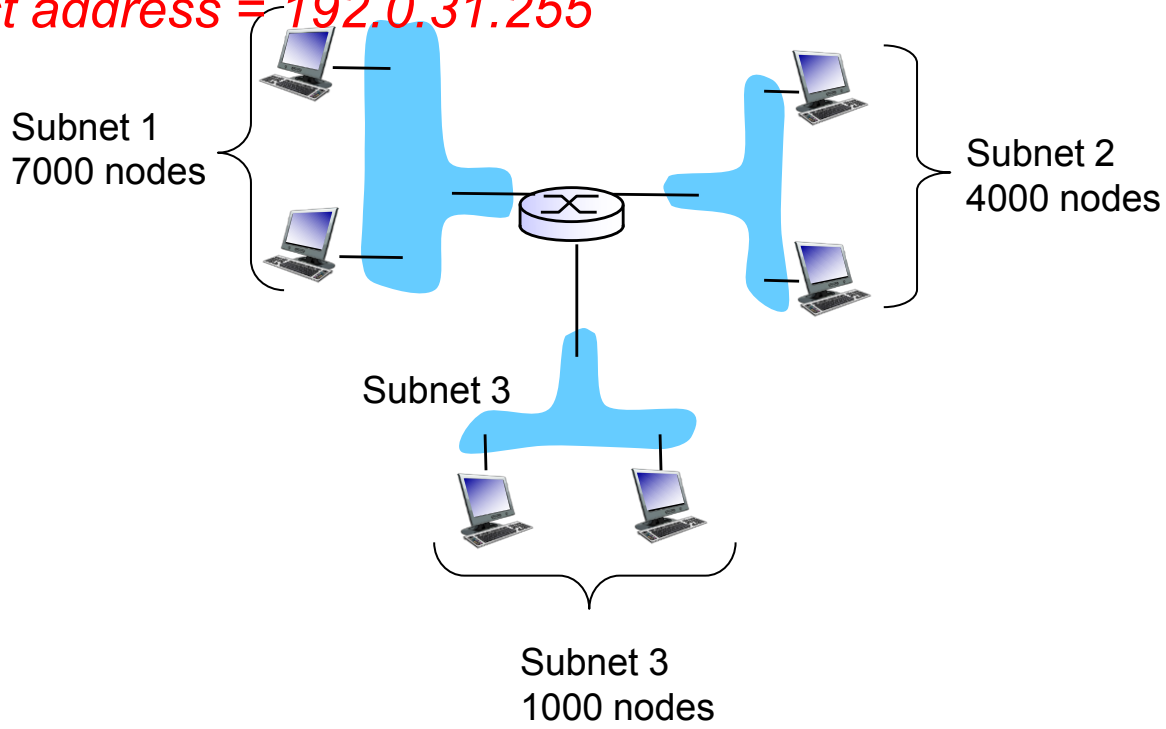
IP Subnet

Answer: We start from 192.0.0.0 with a prefix /19 that creates 8192 addresses enough for all Subnet 1 (7000 nodes), hence the answer are:

(i) IP range of Subnet 1 = 192.0.0.0 – 192.0.31.255

Usable IP range of Subnet 1 = 192.0.0.1 – 192.0.31.254

Broadcast address = 192.0.31.255



– do: assign a new IP address to a new computer that wants to join the network.

DHCP: Dynamic Host Configuration Protocol

Q: How does a *host* get IP address?

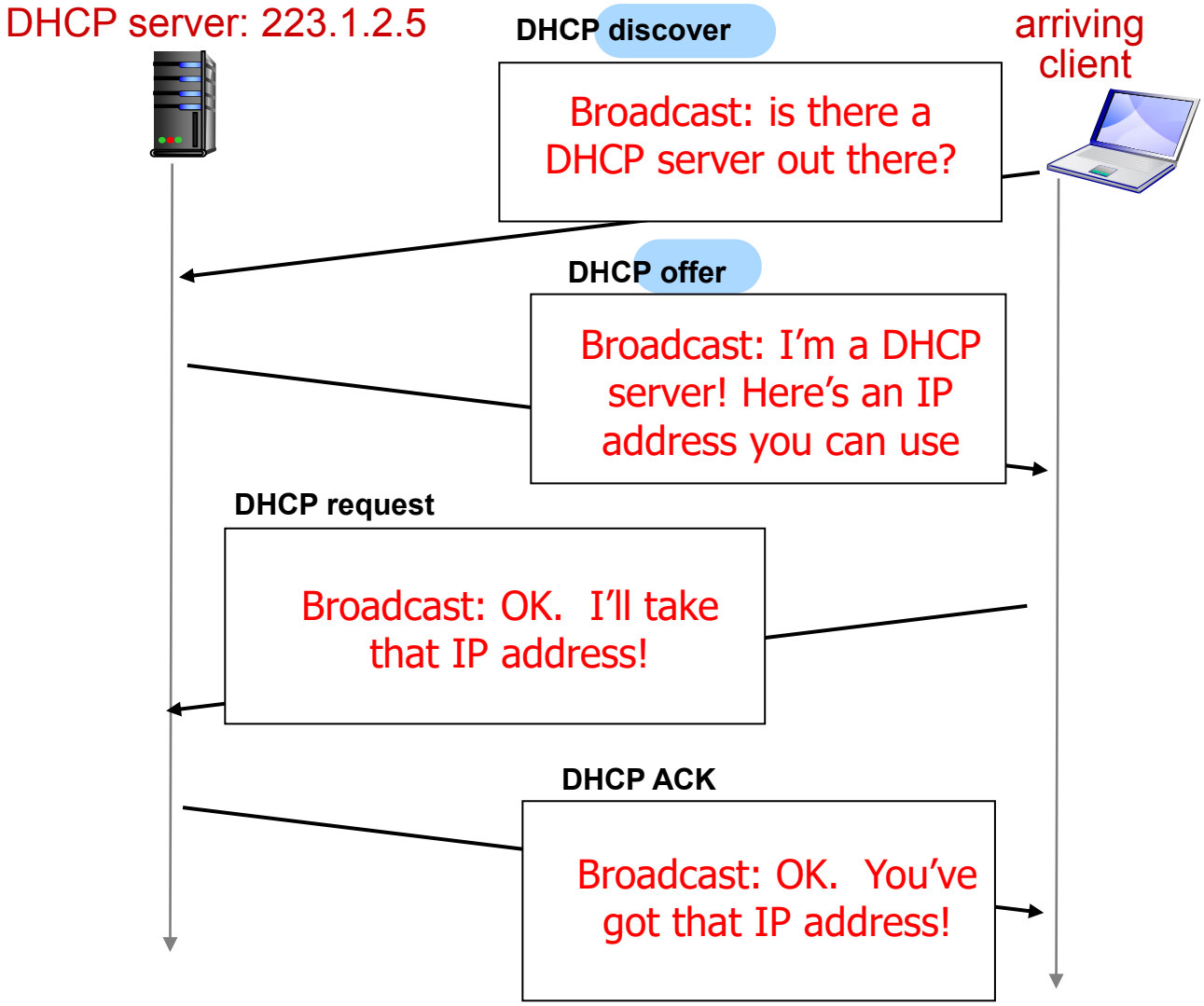
goal: allow host to *dynamically* obtain its IP address from network server when it joins network

- can renew its lease on address in use
- allows reuse of addresses (only hold address while connected/"on")
- support for mobile users who want to join network (more shortly)

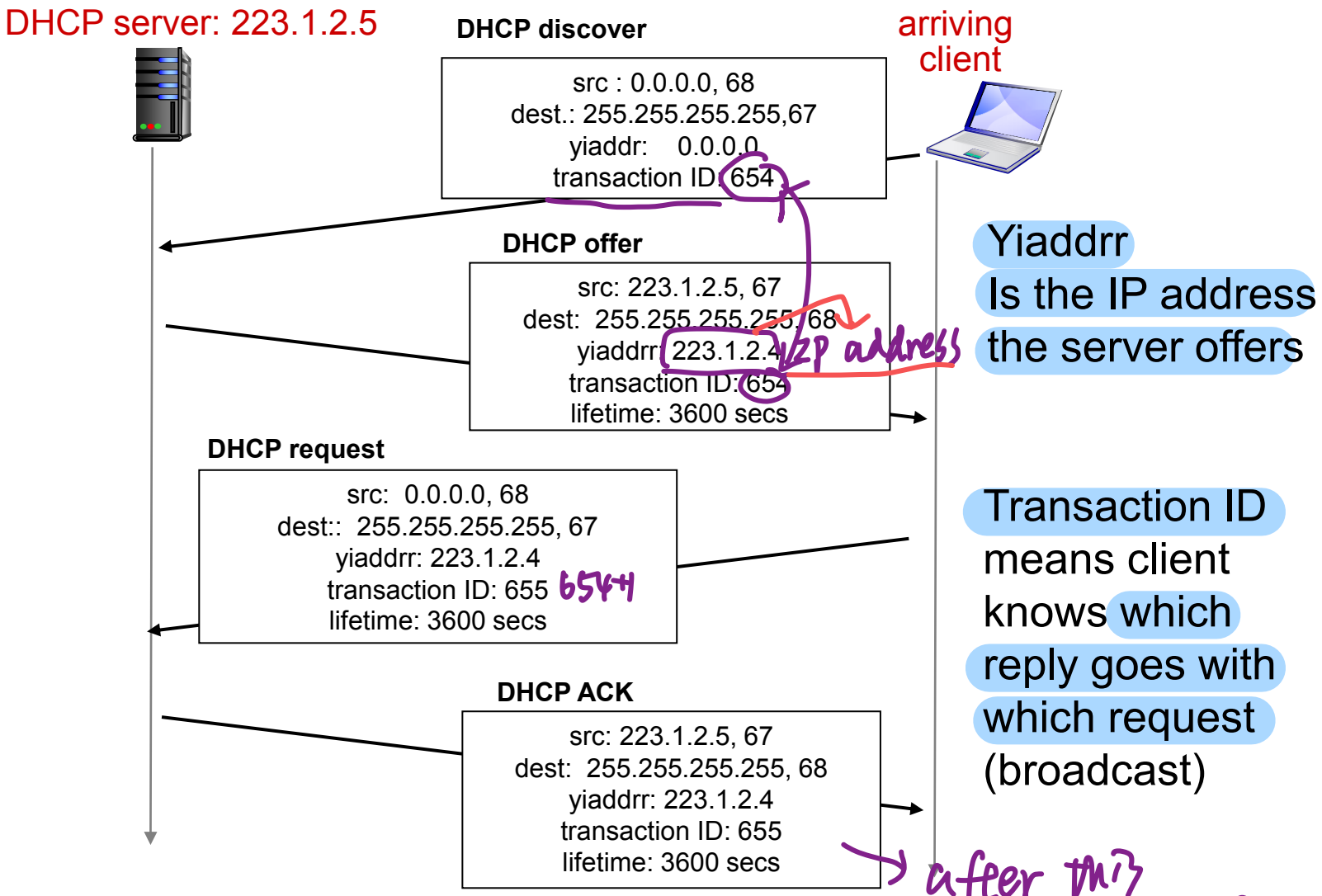
DHCP overview:

- host broadcasts "DHCP discover" msg [optional]
- DHCP server responds with "DHCP offer" msg [optional]
- host requests IP address: "DHCP request" msg
- DHCP server sends address: "DHCP ack" msg

DHCP client-server scenario



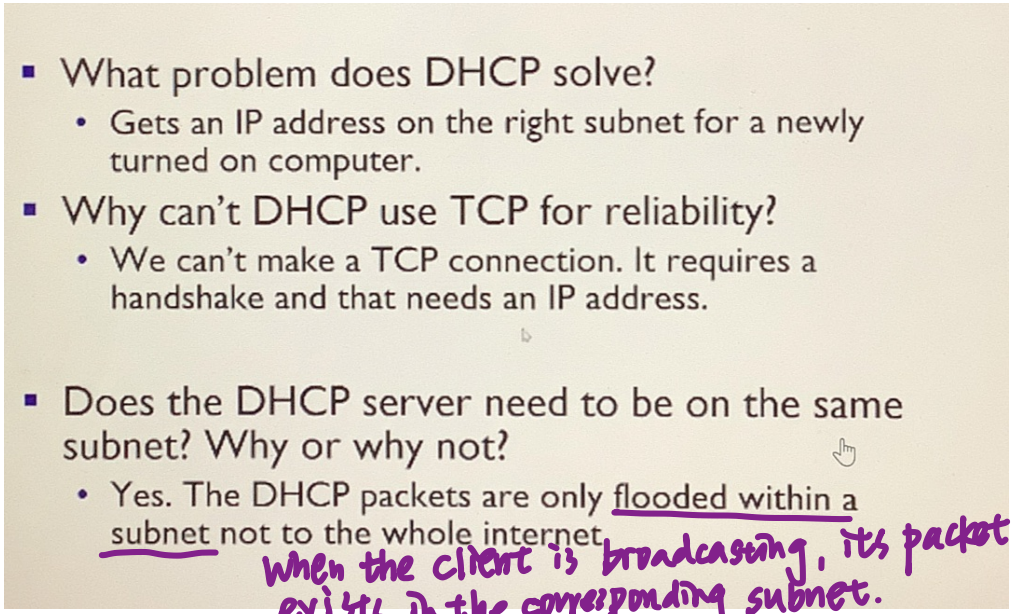
DHCP client-server scenario



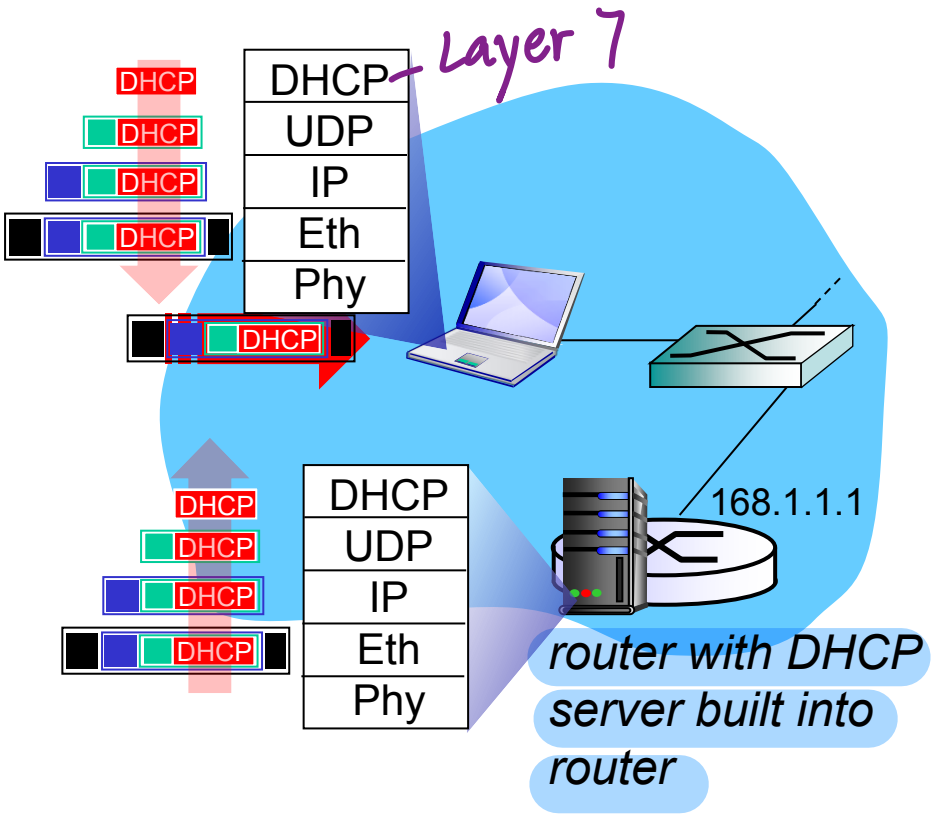
DHCP: more than IP addresses

DHCP can return more than just allocated IP address on subnet:

- address of first-hop router for client
- name and IP address of DNS sever
- network mask (indicating network versus host portion of address)

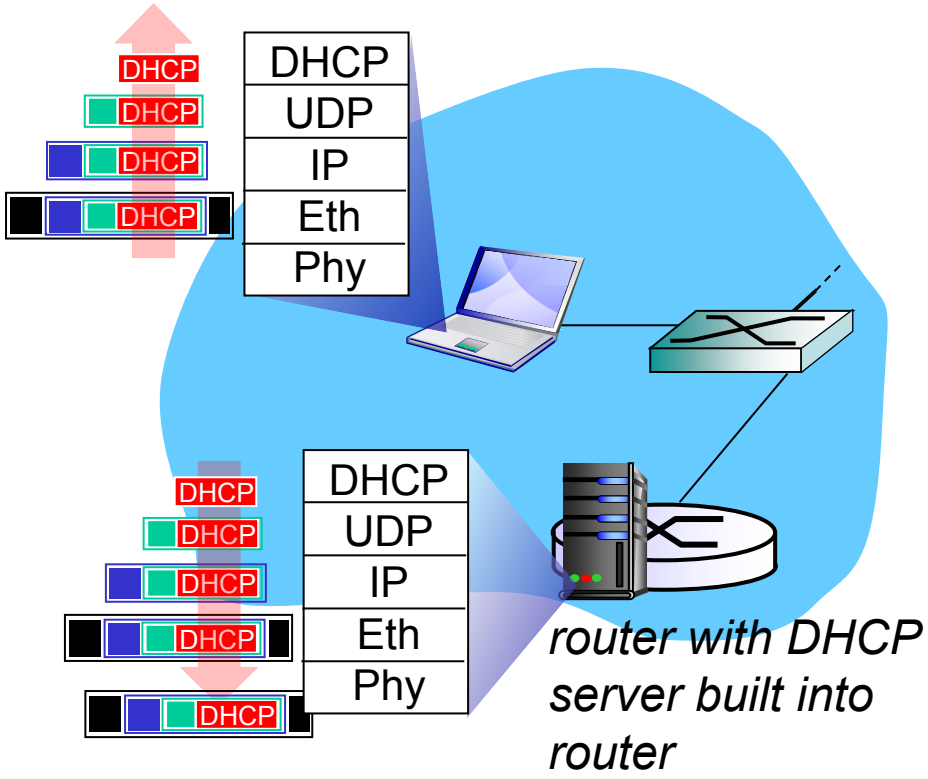


DHCP: example



- connecting laptop needs its IP address, addr of first-hop router, addr of DNS server: use DHCP
- DHCP request encapsulated in UDP, encapsulated in IP, encapsulated in 802.1 Ethernet
- Ethernet frame broadcast (dest: FFFFFFFFFFFF) on LAN, received at router running DHCP server
- Ethernet demuxed to IP demuxed, UDP demuxed to DHCP

DHCP: example



- DCP server formulates DHCP ACK containing client's IP address, IP address of first-hop router for client, name & IP address of DNS server
- encapsulation of DHCP server, frame forwarded to client, demuxing up to DHCP at client
- client now knows its IP address, name and IP address of DNS server, IP address of its first-hop router

Dynamic Host Configuration Protocol Q1

- Q: Why networked devices don't comes with IP address pre-assigned?

A:
A host that always has the same IP address can only connect to one network.

DHCP true or false

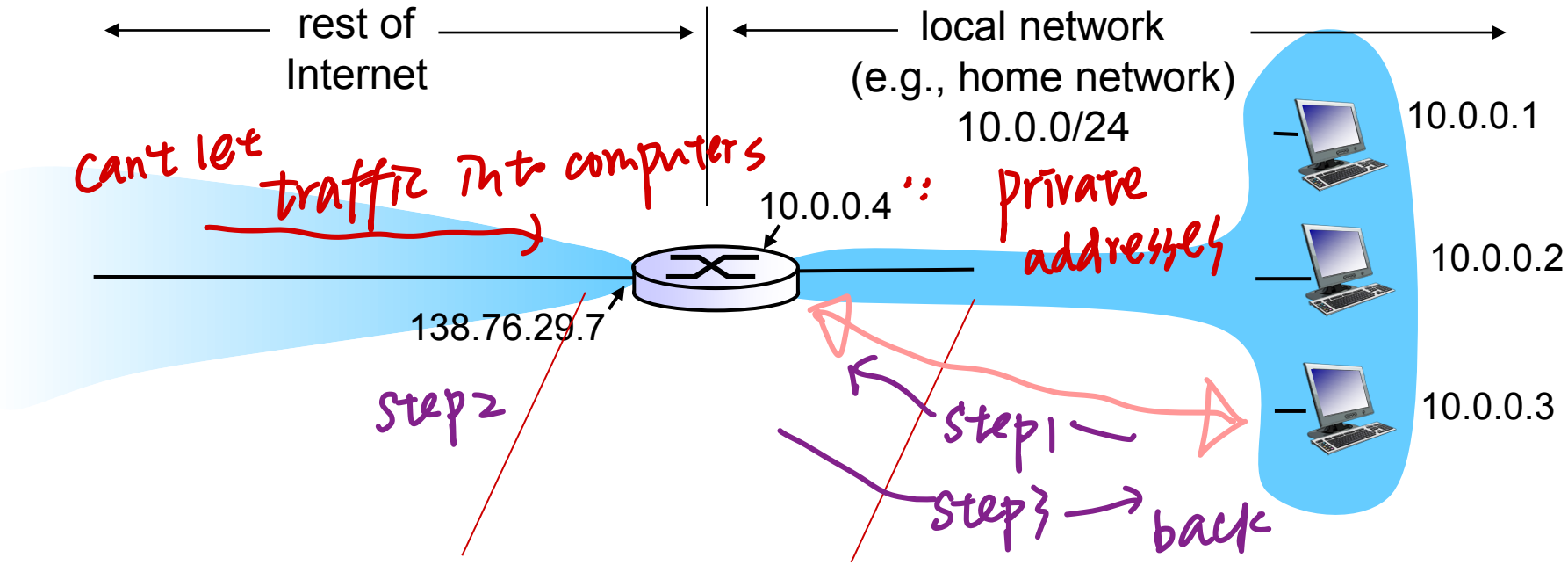
A. The server must broadcast as the client has not got an IP address
TRUE

B. The connection should use TCP in case of lost packets
FALSE – TCP could not set up a connection if the client has no IP (how does it handshake).

C. The DCHP server can be anywhere in the world, the network layer can take data to anywhere.
FALSE – Broadcast only reaches the subnet

D. DHCP is used by the client to obtain an Ethernet (MAC) address.
FALSE – the client has a MAC it needs an IP

NAT: network address translation



all datagrams *leaving* local network have *same* single source NAT IP address: 138.76.29.7, different source port numbers

datagrams with source or destination in this network have 10.0.0/24 address for source, destination (as usual)

NAT: network address translation

motivation: local network uses just one IP address as far as outside world is concerned:

- range of addresses not needed from ISP: just one IP address for all devices
- can change addresses of devices in local network without notifying outside world
- can change ISP without changing addresses of devices in local network
- devices inside local net not explicitly addressable, visible by outside world (a security plus)

telling

|| can't tell those devices exist

more secure. hacker can't get access to them

NAT: network address translation

implementation: NAT router must:

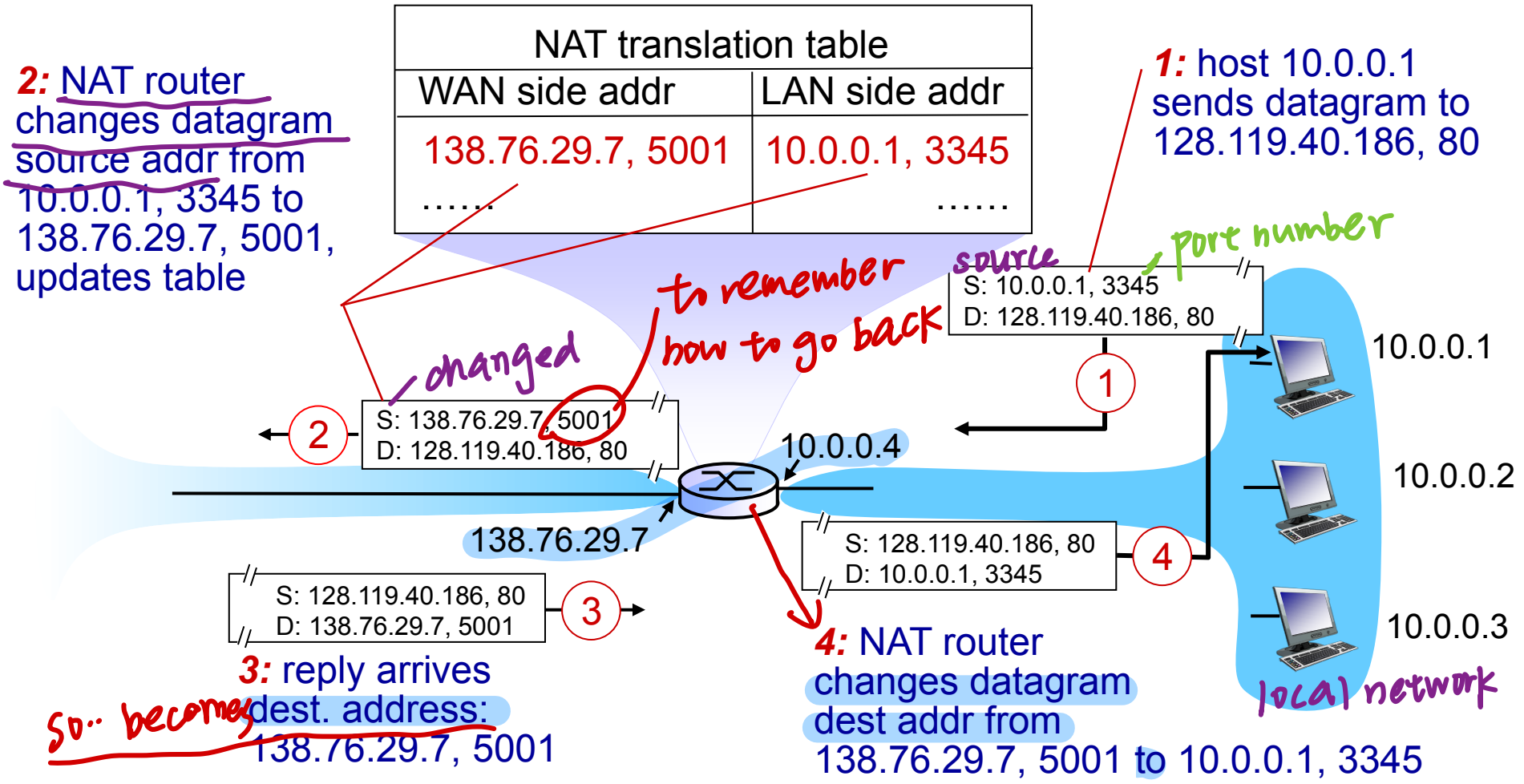
- *outgoing datagrams: replace* (source IP address, port #) of every outgoing datagram to (NAT IP address, new port #)
  . . . remote clients/servers will respond using (NAT IP address, new port #) as destination addr

Integration. know how to translate back

- *remember (in NAT translation table)* every (source IP address, port #) to (NAT IP address, new port #) translation pair

- *incoming datagrams: replace* (NAT IP address, new port #) in dest fields of every incoming datagram with corresponding (source IP address, port #) stored in NAT table

NAT: network address translation



* Check out the online interactive exercises for more examples: http://gaia.cs.umass.edu/kurose_ross/interactive/

Test your understanding

- A TCP packet from 10.0.0.5:5443 (private address) is going to 12.34.56.7:443 using NAT. The NAT box has IP address 34.56.67.78. It chooses port 5000 for the mapping.
- A) In the NAT table what are the WAN and LAN side mappings for this packet?

  WAN 34.56.67.78:5000

  LAN 10.0.0.5:5543
- B) The NAT box will change some parts of the packet as it leaves the network. What are the src and dst IP and port after these changes.

  Src 34.56.67.78:5000

  Dst 12.34.56.7:443
- C) The TCP packet gets a reply. The NAT box will change some parts of the reply packet as it enters the network. What are the src and dst IP and port after these changes?

  Src 12.34.56.7:443

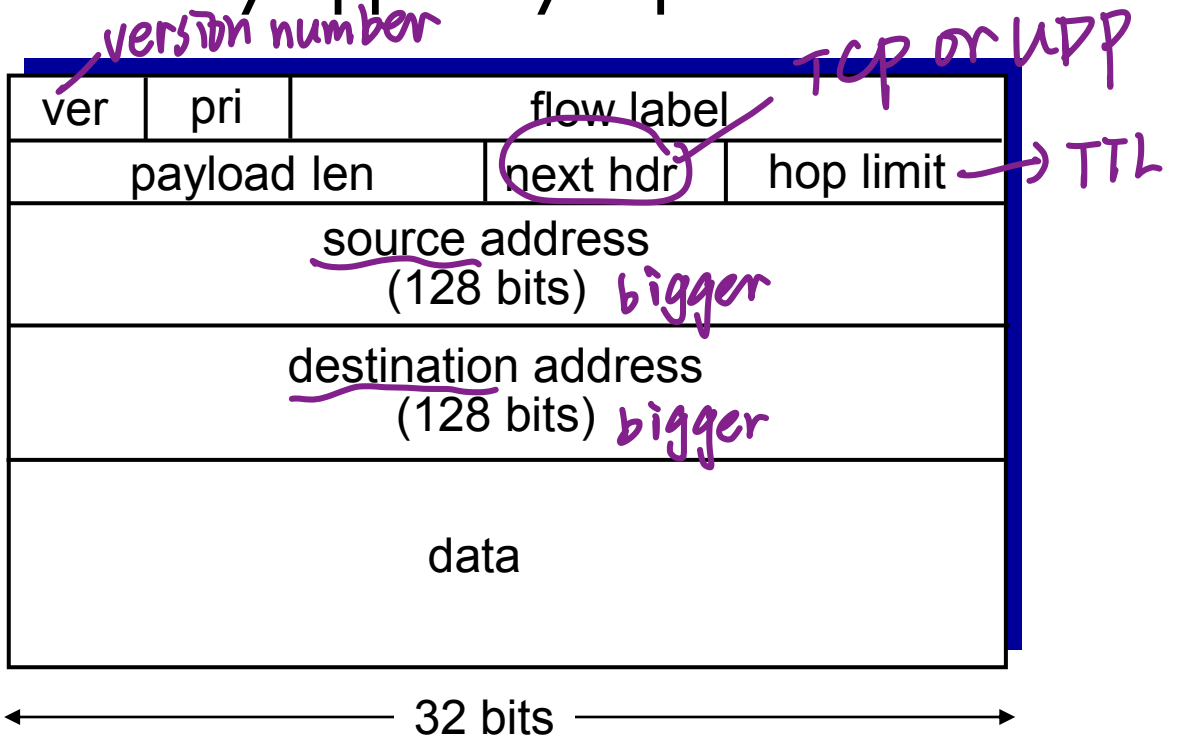
  Dst 10.0.0.5:5543

IPv6 datagram format

priority: identify priority among datagrams in flow

flow Label: identify datagrams in same "flow."
(concept of"flow" not well defined).

next header: identify upper layer protocol for data



Other changes from IPv4

no checksum — already have on layer 4

- *checksum:* removed entirely to reduce processing time at each hop
- *options:* allowed, but outside of header, indicated by "Next Header" field
- *ICMPv6:* new version of ICMP (see later lecture)
  - additional message types, e.g. "Packet Too Big"
  - multicast group management functions

IPv6

Mentimeter Question: List some differences between IPv4 and IPv6?

IPv4	IPv6
IPv4 has a 32-bit address length	IPv6 has a 128-bit address length
It can generate 4.29×10^9 address space	Address space of IPv6 is quite large it can produce 3.4×10^{38} address space
Address representation of IPv4 is in decimal	Address Representation of IPv6 is in hexadecimal
In IPv4 checksum field is available	In IPv6 checksum field is not available
IPv4 has a header of 20-60 bytes	IPv6 has header of 40 bytes fixed
IPv4 consist of 4 fields which are separated by dot (.)	IPv6 consist of 8 fields, which are separated by colon (:)
Example of IPv4: 66.94.29.13	Example of IPv6: 2001:0000:3238:DFE1:0063:0000:0000:FEFB

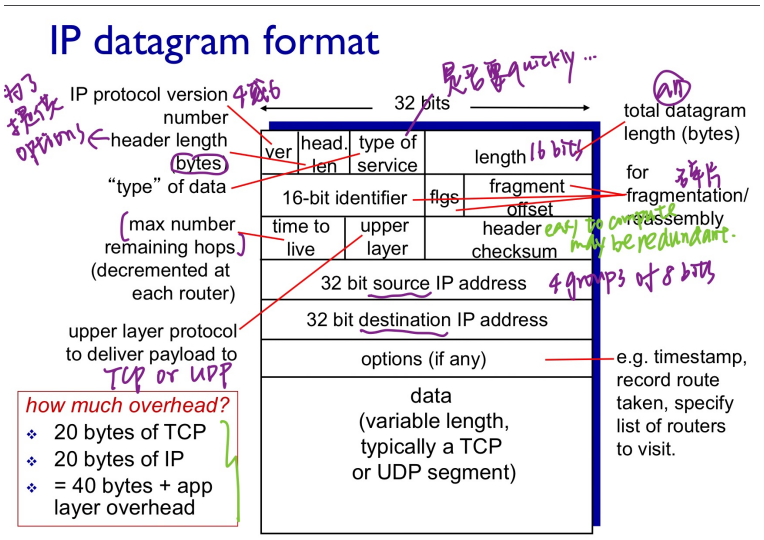
IPv6: motivation

Question: Which of the following is correct about the IPv6?

a) 32-bit address space soon to be completely allocated.
b) IPv6 header format requires less bandwidth than IPv4
c) IPv6 datagram allows checksum
d) Changes in header format to help speeding up processing/forwarding
e) IPv6 has many overlays to handle Internet growth
f) Changes in header format to facilitate QoS
g) IPv6 datagram allows fragmentation

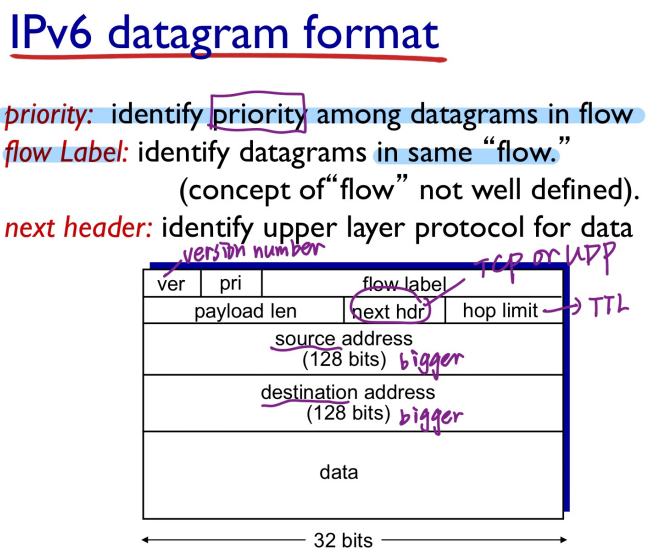
2018/2019 A Q3b

b) Internet Protocol version 6:



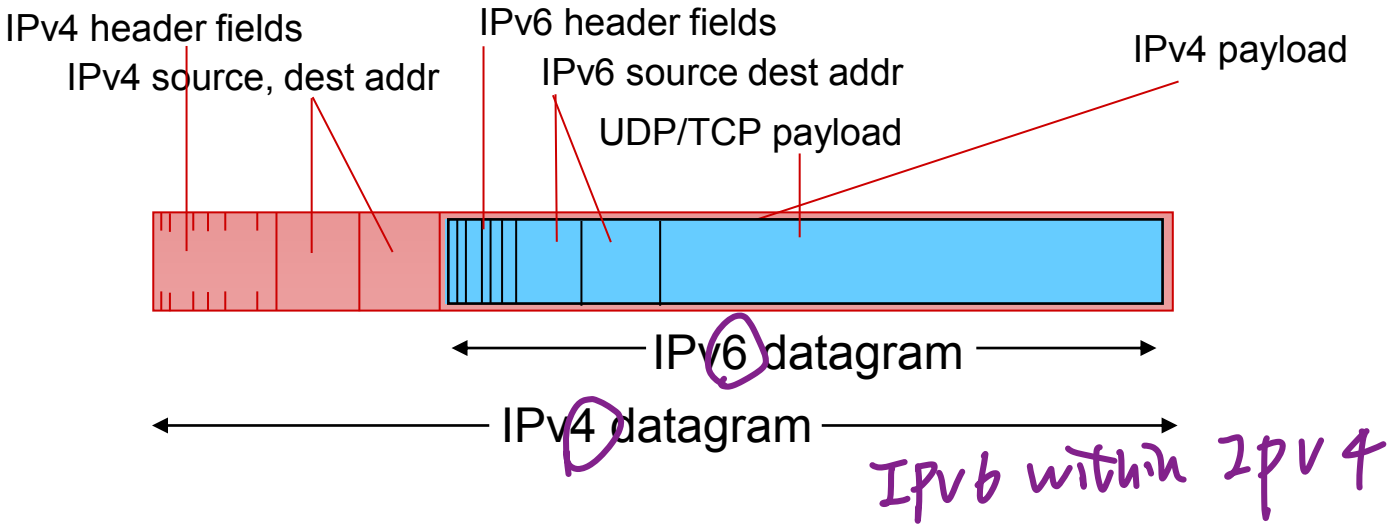
i) Name 4 fields present (not including flags) in the IPv4 header that have no equivalent in IPv6.

Diffserv Codepoint (DSCP), ECN, Fragment offset, header checksum, header length, protocol, (half mark for TTL or identification)



Transition from IPv4 to IPv6

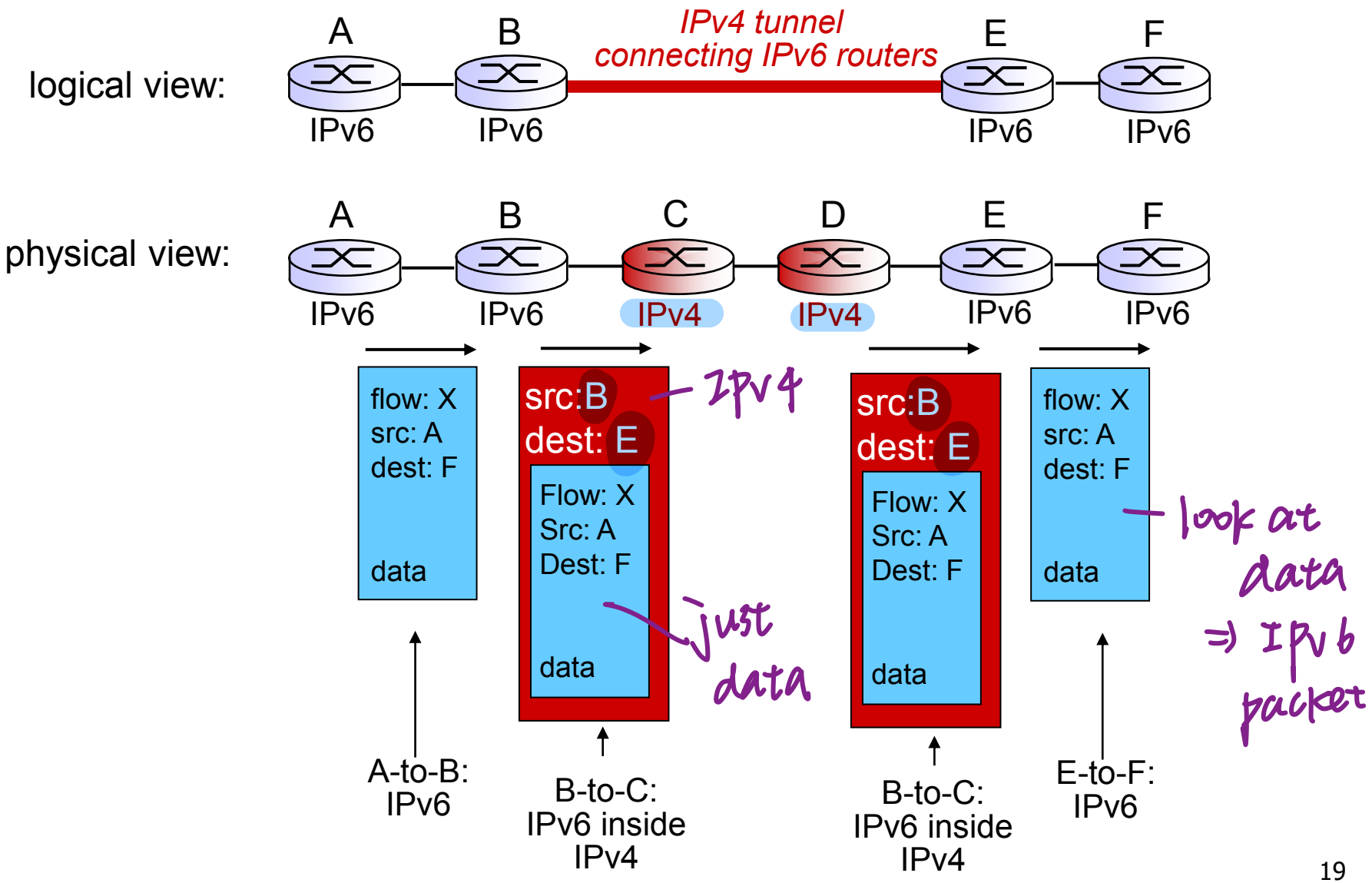
- not all routers can be upgraded simultaneously
  - no "flag days" upgrade things every time
  - how will network operate with mixed IPv4 and IPv6 routers?
- *tunneling:* IPv6 datagram carried as *payload* in IPv4 datagram among IPv4 routers



Tunneling

ii) Briefly explain tunnelling within the context of IPv4 to IPv6 transition.

Tunelling allows IPv6 packets to move over IPv4 network. IPv6 packet is copied as the payload in the IPv4 packet



What is Software Defined Networking?

advantages / comparison

- Traditional routing methods:
  - Data Plane is longest-prefix match forwarding using a forwarding table. input → output
  - The control plane calculates the forwarding table for each router (we will see how later).
  - The forwarding table can forward packets by their IP address and nothing else.
- SDN allows more flexibility in the control and data plane.
  - Program your own control algorithms in language you know (Java, python etc).
  - Forwarding can use any part of the packet header.

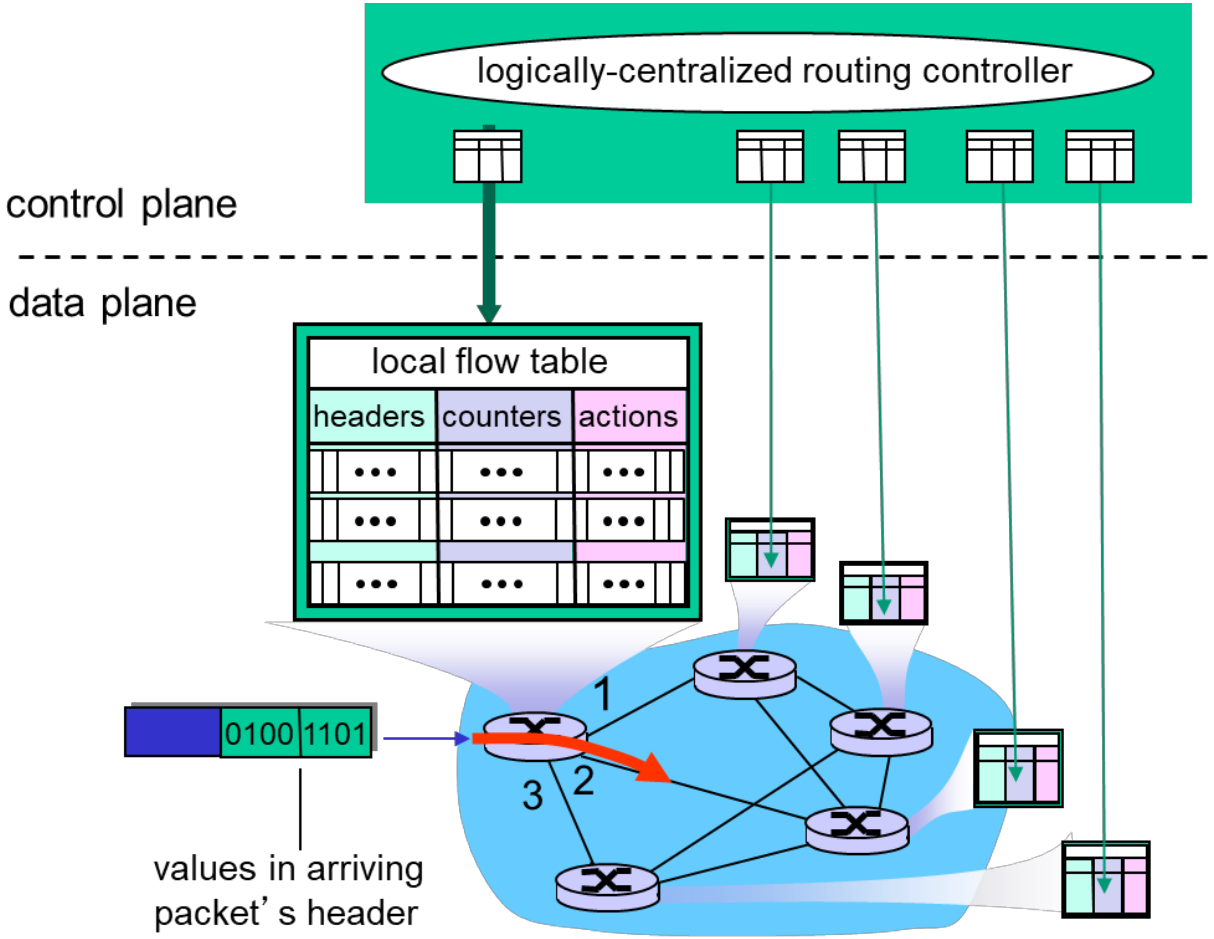
clevere way

How does SDN work?

- Data plane:
  - A set of "match-action" rules send by a controller can do many different things to packets. (Forward them, change data etc)
  - Much more flexible (route video packets differently? route private data separately? Drop suspicious data!)
- Control plane (later lectures):
  - Not a distributed system, but a single centralised control point.
  - Programmable, not fixed – you can program the controller in a high-level language that you know.
  - Create your own algorithms and test them on the network without spending a million dollars to create a new hardware router.

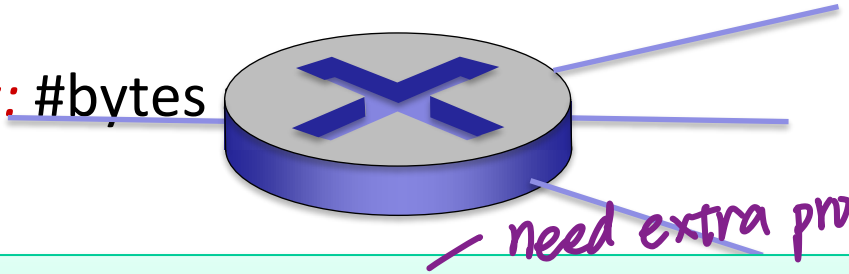
Generalized Forwarding and SDN

Each router contains a *flow table* that is computed and distributed by a *logically centralized* routing controller



OpenFlow data plane abstraction

- OpenFlow is a specific SDN protocol
- *flow*: defined by header fields
- generalized forwarding: simple packet-handling rules
  - *Pattern:* match values in packet header fields
  - *Actions: for matched packet:* drop, forward, modify, matched packet or send matched packet to controller
  - *Priority*: disambiguate (tell difference between) overlapping patterns
  - *Counters:* #bytes



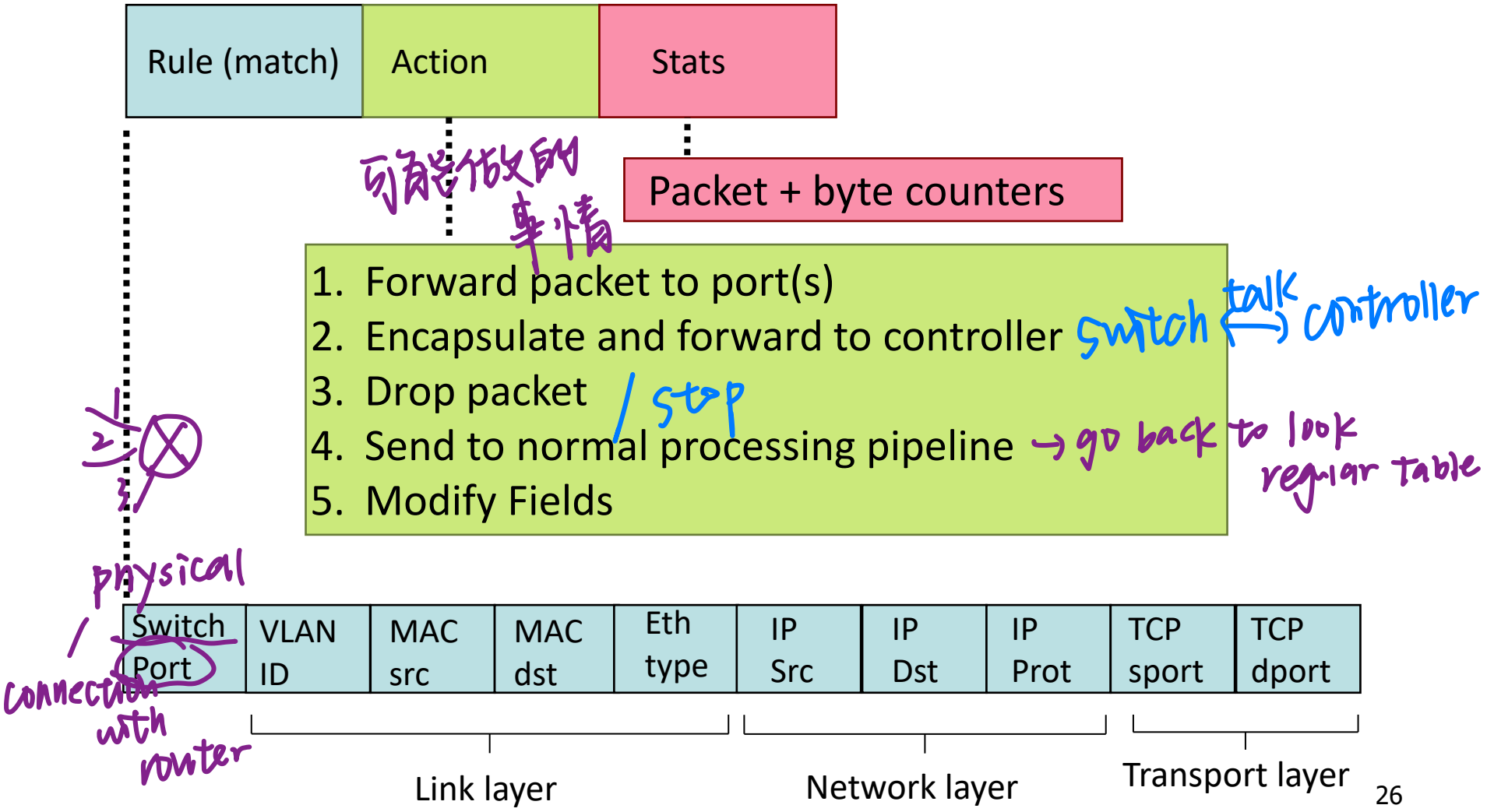
need extra protection

* : wildcard

1. src=1.2.*.*, dest=3.4.5.* → drop
2. src = *.*.*.*, dest=3.4.*.* → forward(2)
3. src=10.1.2.3, dest=*.*.*.* → send to controller

special address

OpenFlow: Flow Table Entries



Examples Test your understanding

Destination-based forwarding:

Switch Port	MAC src	MAC dst	Eth type	VLAN ID	IP Src	IP Dst	IP Prot	TCP sport	TCP dport	Action
*	*	*	*	*	*	51.6.0.8	*	*	*	port6

IP datagrams destined to IP address 51.6.0.8 should be forwarded to router output port 6

Firewall:

Switch Port	MAC src	MAC dst	Eth type	VLAN ID	IP Src	IP Dst	IP Prot	TCP sport	TCP dport	Forward
*	*	*	*	*	*	*	*	*	22	drop

do not forward (block) all datagrams destined to TCP port 22

/drop

Firewall:

Switch Port	MAC src	MAC dst	Eth type	VLAN ID	IP Src	IP Dst	IP Prot	TCP sport	TCP dport	Forward
*	*	*	*	*	128.119.1.1	*	*	*	*	drop

do not forward (block) all datagrams sent by host 128.119.1.1

/drop

OpenFlow abstraction

flexible

- *match+action:* different kinds of devices become one

- Router
  - *match:* longest destination IP prefix
  - *action:* forward out a link
- Switch
  - *match:* destination MAC address
  - *action:* forward or flood

send every possible link out of switch

- Firewall
  - *match*: IP addresses and TCP/UDP port numbers
  - *action:* permit or deny
- NAT
  - *match:* IP address and port
  - *action:* rewrite address and port

Network Control Plane: outline

5.1 introduction

5.2 routing protocols

- link state
- distance vector

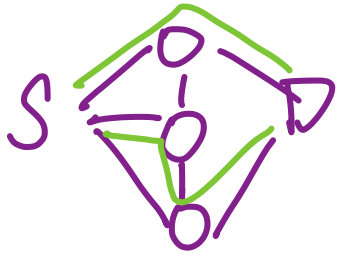
5.3 intra-AS routing in the Internet: OSPF

5.4 routing among the ISPs: BGP

5.5 The SDN control plane

5.6 ICMP: The Internet Control Message Protocol

Routing protocols



Routing protocol goal: determine "good" paths (equivalently, routes), from sending hosts to receiving host, through network of routers

- path: sequence of routers packets will traverse in going from given initial source host to given final destination host
- "good": least "cost", "fastest", "least congested"
- routing: a "top-10" networking challenge!

Routing algorithm classification

Q: global or decentralized information?

global:

- all routers have complete topology, link cost info
- "link state" algorithms

decentralized:

- router knows physically-connected neighbors, link costs to neighbors
- iterative process of computation, exchange of info with neighbors
- "distance vector" algorithms

Q: static or dynamic?

static:

- routes change slowly over time

dynamic:

- routes change more quickly
  - periodic update
  - in response to link cost changes

A link-state routing algorithm

Dijkstra's algorithm

- net topology, link costs known to all nodes
  - accomplished via "link state broadcast"
  - all nodes have same info
- computes least cost paths from one node ("source") to all other nodes
  - gives *forwarding table* for that node
- iterative: after k iterations, know least cost path to k dest.'s

notation:

- c(x,y): link cost from node x to y; = ∞ if not direct neighbors
- D(v): current value of cost of path from source to dest. v
- p(v): predecessor node along path from source to v
- N': set of nodes whose least cost path definitively known

Dijsktra's algorithm

```
1  Initialization:
2    N' = {u}
3    for all nodes v
4      if v adjacent to u
5          then D(v) = c(u,v)
6      else D(v) = ∞
7
8  Loop
9     find w not in N' such that D(w) is a minimum
10    add w to N'
11    update D(v) for all v adjacent to w and not in N' :
12       D(v) = min( D(v), D(w) + c(w,v) )
13    /* new cost to v is either old cost to v or known
14     shortest path cost to w plus cost from w to v */
15  until all nodes in N'
```

current value

Shortest path

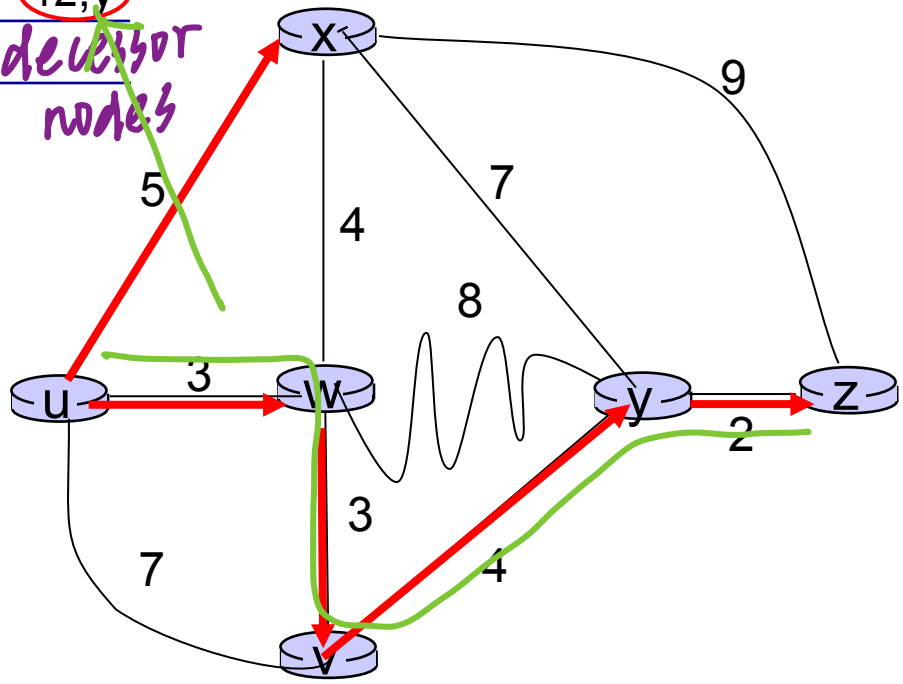
Iteration

Dijkstra's algorithm: example

Step	N'	D(v) p(v)	D(w) p(w)	D(x) p(x)	D(y) p(y)	D(z) p(z)
0	u	7,u	3,u	5,u	∞	∞
1	uw	6,w		5,u	11,w	∞
2	uwx	6,w			11,w	14,x
3	uwxv				10,v	14,x
4	uwxvy					12,y
5	uwxvyz					

notes:

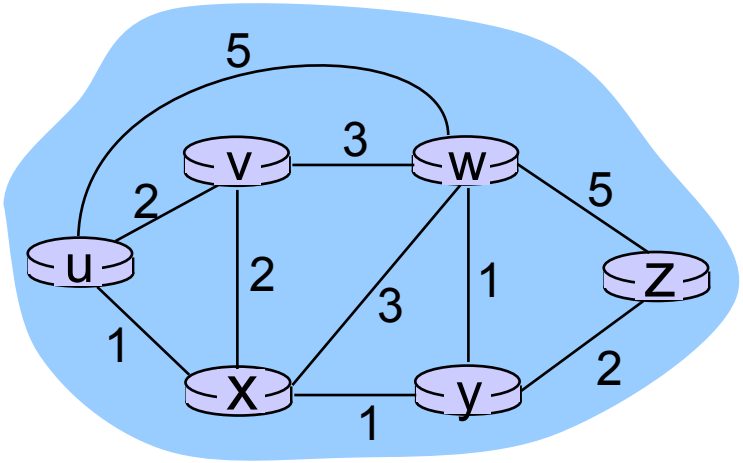
- construct shortest path tree by tracing predecessor nodes
- ties can exist (can be broken arbitrarily)



Dijkstra's algorithm: another example

Shortest distance

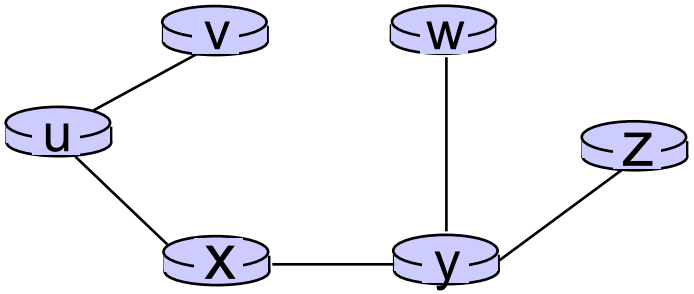
Step	N'	D(v),p(v)	D(w),p(w)	D(x),p(x)	D(y),p(y)	D(z),p(z)
0	u	2,u	5,u	1,u	∞	∞
1	ux	2,u	4,x		2,x	∞
2	uxy	2,u	3,y			4,y
3	uxyv		3,y			4,y
4	uxyvw					4,y
5	uxyvwz					

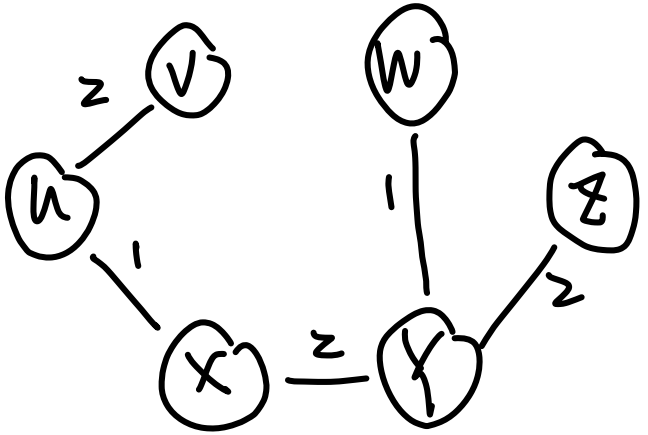


* Check out the online interactive exercises for more examples: http://gaia.cs.umass.edu/kurose_ross/interactive/

Dijkstra's algorithm: example (2)

resulting shortest-path tree from u:





resulting forwarding table in u:

destination	link
v	(u,v)
x	(u,x)
y	(u,x)
w	(u,x)
z	(u,x)

Distance vector algorithm

Bellman-Ford equation (dynamic programming)

let

$d_x(y)$:= cost of least-cost path from x to y

then

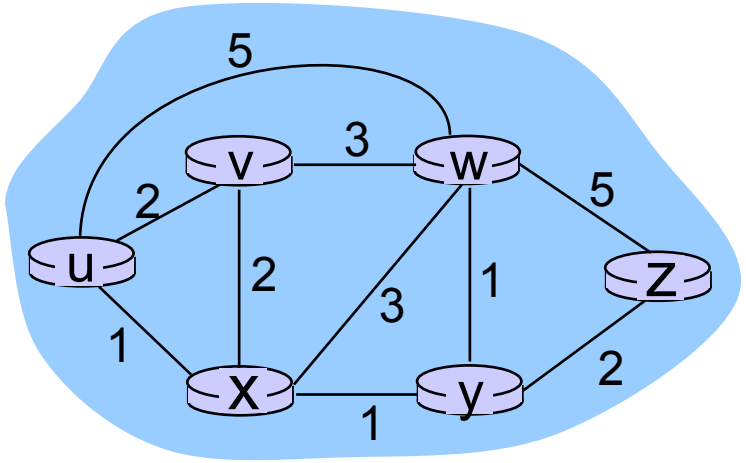
$$d_x(y) = \min_v \{c(x,v) + d_v(y)\}$$

$d_v(y)$: cost from neighbor v to destination y

$c(x,v)$: cost to neighbor v

$\min_v$: *min* taken over all neighbors v of x

Bellman-Ford example



clearly, $d_v(z) = 5$, $d_x(z) = 3$, $d_w(z) = 3$

B-F equation says:

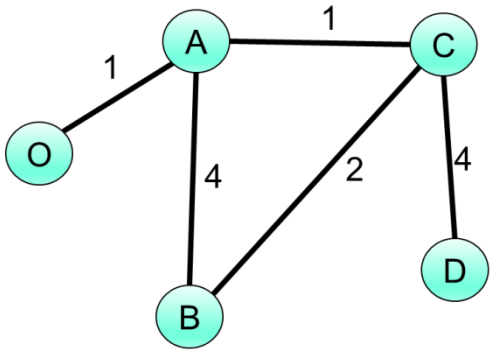
$$d_u(z) = \min \{ c(u,v) + d_v(z), \; c(u,x) + d_x(z), \; c(u,w) + d_w(z) \}$$

$$= \min \{2 + 5, \; 1 + 3, \; 5 + 3\} = 4$$

minimum

node achieving minimum is next hop in shortest path, used in forwarding table

Bellman-Ford algorithm



- Consider Bellman-Ford algorithm as executed on Nodes A and C. Each node knows the distance to its neighbours. They will have initial distance vector tables as shown.

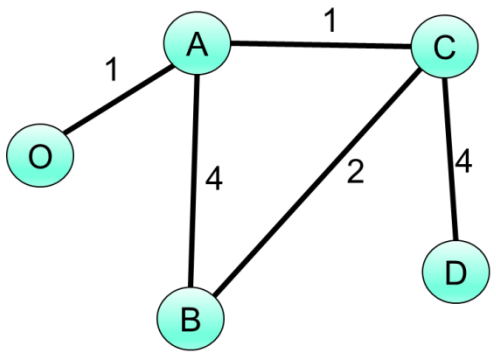
NODE A

Dest	Cost	(route)
O	1	O
C	1	C
B	4	B

NODE C

Dest	Cost	(route)
B	2	B
D	4	D
A	1	A

Bellman-Ford algorithm



- Node A sends its table to node C

Dest	Cost
O	1
C	1
B	4

NODE A table

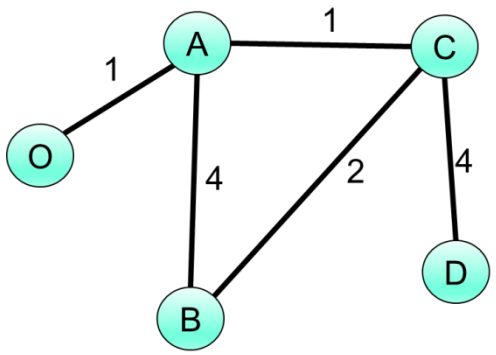
NODE C old table

Dest	Cost	(route)
B	2	B
D	4	D
A	1	A

NODE C new table

Dest	Cost	(route)
B	2	B
D	4	D
A	1	A
O	2	A

Bellman-Ford algorithm



- Node C sends its table to node A (because it made update)

NODE C table

Dest	Cost
B	2
D	4
A	1
O	2

NODE A old table

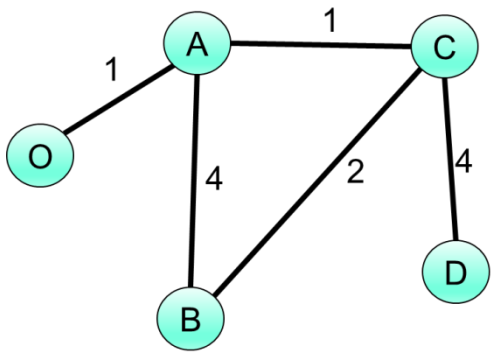
Dest	Cost	(route)
O	1	O
C	1	C
B	4	B

$\min(4, 2+1) = 3$

NODE A new table

Dest	Cost	(route)
O	1	O
C	1	C
B	3	C
D new	5	C

Bellman-Ford algorithm



- Node A made an update so sends its table again but node C does not need more updates. All nodes will keep their tables until the network changes.

NODE A final table

Dest	Cost	(route)
O	1	O
C	1	C
B	3	C
D	5	C

NODE C final table

Dest	Cost	(route)
B	2	B
D	4	D
A	1	A
O	2	A

Distance vector algorithm

key idea:

- from time-to-time, each node sends its own distance vector estimate to neighbors
- when x receives new DV estimate from neighbor, it updates its own DV using B-F equation:

$$D_x(y) \leftarrow \min_v\{c(x,v) + D_v(y)\} \quad \text{for each node } y \in N$$

- under minor, natural conditions, the estimate $D_x(y)$ *converge to the actual least cost* $d_x(y)$

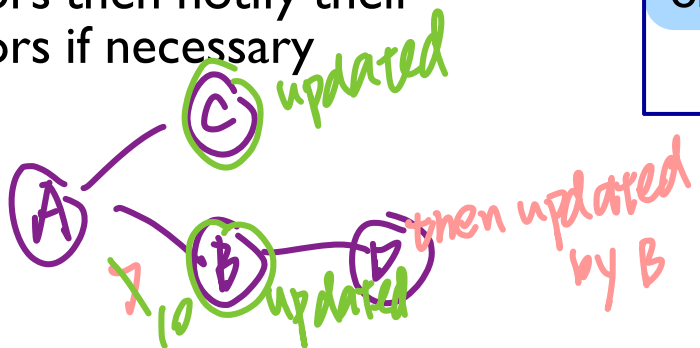
Distance vector algorithm

iterative, asynchronous:
each local iteration caused by:

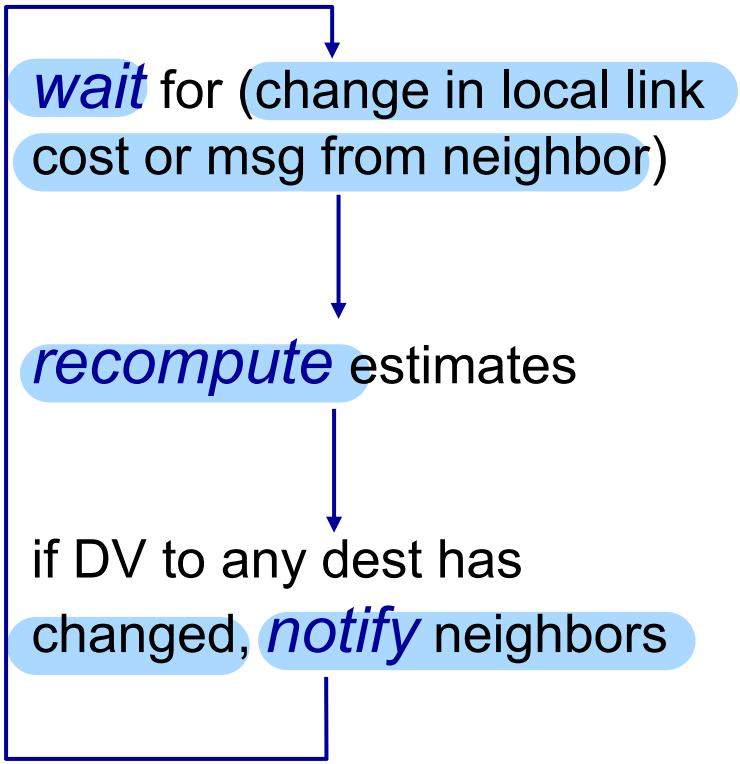
- local link cost change
- DV update message from neighbor

distributed:

- each node notifies neighbors *only* when its DV changes
  - neighbors then notify their neighbors if necessary



each node:



$D_x(y) = \min\{c(x,y) + D_y(y), c(x,z) + D_z(y)\}$
$= \min\{2+0 , 7+1\} = 2$

$D_x(z) = \min\{c(x,y) + D_y(z), c(x,z) + D_z(z)\}$
$= \min\{2+1 , 7+0\} = 3$

node x table

from \ cost to	x	y	z
x	0	2	7
y	∞	∞	∞
z	∞	∞	∞

from \ cost to	x	y	z
x	0	2	3
y	2	0	1
z	7	1	0

from \ cost to	x	y	z
x	0	2	3
y	2	0	1
z	3	1	0

node y table

from \ cost to	x	y	z
x	∞	∞	∞
y	2	0	1
z	∞	∞	∞

from \ cost to	x	y	z
x	0	2	7
y	2	0	1
z	7	1	0

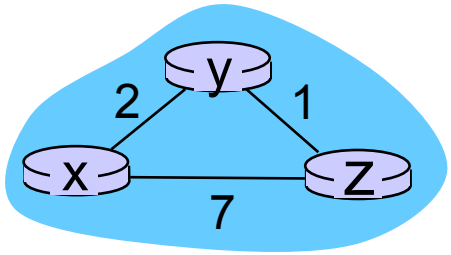
from \ cost to	x	y	z
x	0	2	3
y	2	0	1
z	3	1	0

node z table

from \ cost to	x	y	z
x	∞	∞	∞
y	∞	∞	∞
z	7	1	0

from \ cost to	x	y	z
x	0	2	7
y	2	0	1
z	3	1	0

from \ cost to	x	y	z
x	0	2	3
y	2	0	1
z	3	1	0

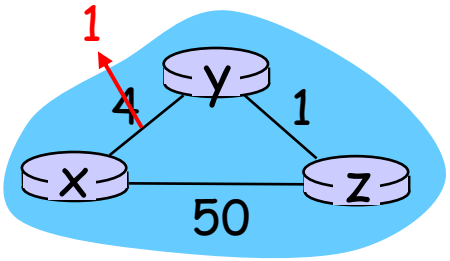


time

Distance vector: link cost changes

link cost changes:

- node detects local link cost change
- updates routing info, recalculates distance vector
- if DV changes, notify neighbors



"good news travels fast"

t_0 : y detects link-cost change, updates its DV, informs its neighbors.

t_1 : z receives update from y, updates its table, computes new least cost to x, sends its neighbors its DV.

t_2 : y receives z's update, updates its distance table. y's least costs do *not* change, so y does *not* send a message to z.

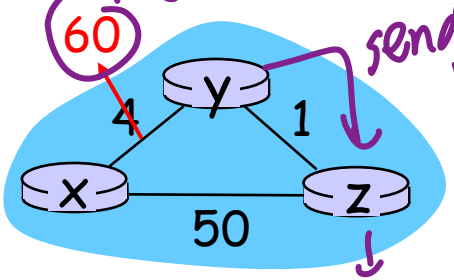
* Check out the online interactive exercises for more examples: http://gaia.cs.umass.edu/kurose_ross/interactive/

Distance vector: link cost changes

link cost changes:

- node detects local link cost change
- *bad news travels slow* - "count to infinity" problem!
- 44 iterations before algorithm stabilizes: see text

well higher



send message

slow to receive
still travel to x
via y

poisoned reverse:

- If Z routes through Y to get to X:
  - Z tells Y its (Z's) distance to X is infinite (so Y won't route to X via Z)
- will this completely solve count to infinity problem? no

Test your understanding

- Which statements are true for distance vector algorithm:

A. Routers compute the best path from information passed from their neighbours — TRUE
B. The least cost route/path is computed by adding distance vectors from router to router. — TRUE
C. Routers get a common view of the entire network topology — FALSE
D. Routers broadcast copies of their routing tables to neighouring routers. — TRUE

Internet approach to scalable routing

aggregate routers into regions known as "autonomous systems" (AS) (a.k.a. "domains") – big areas of internet

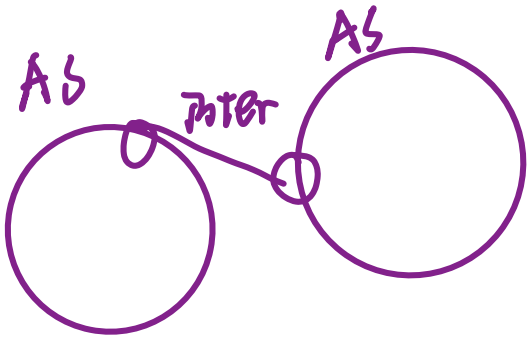
think of a large university or company or ISP.

intra-AS routing

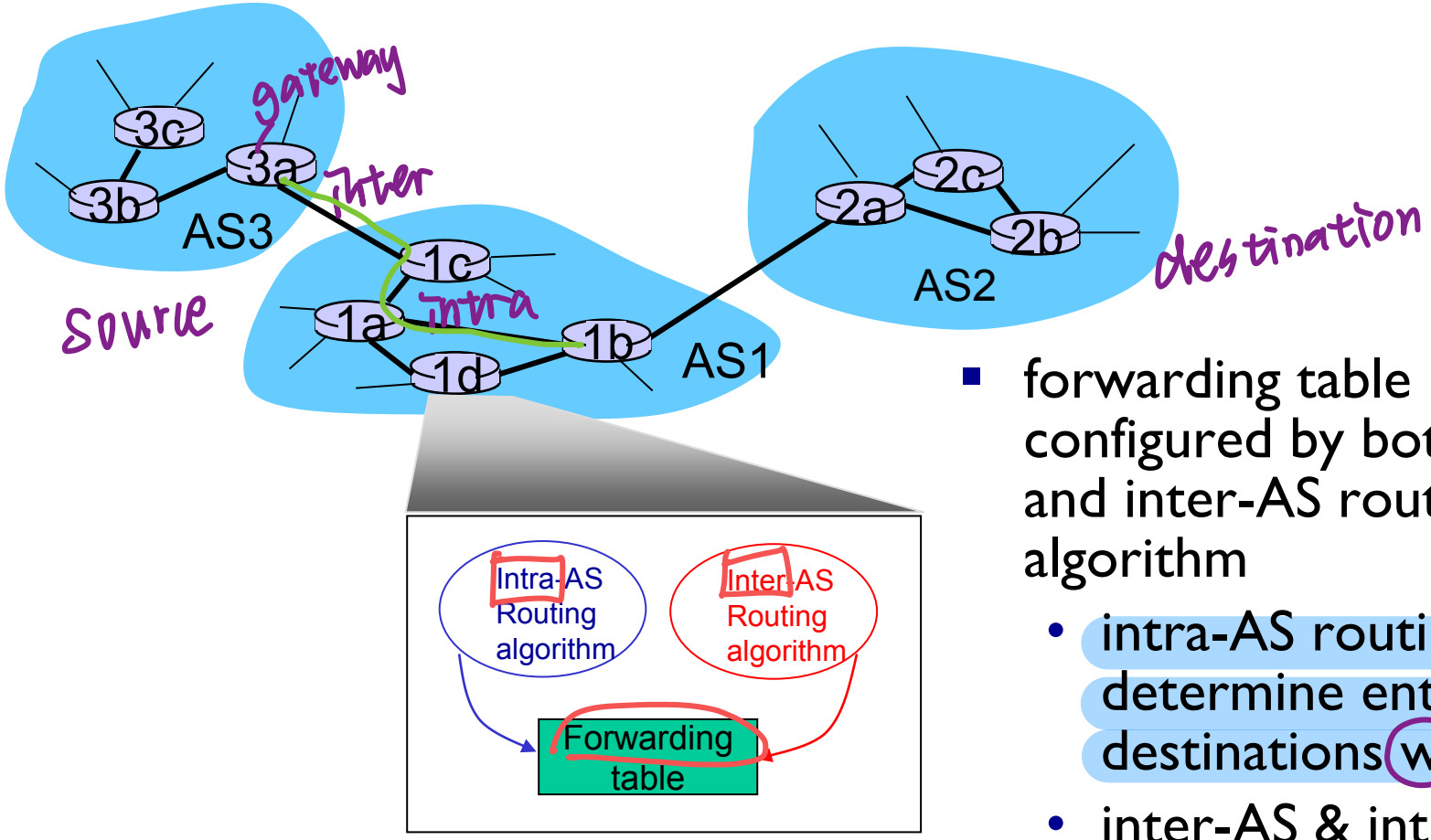
- routing among hosts, routers in same AS ("network")
- all routers in AS must run *same* intra-domain protocol
- routers in *different* AS can run *different* intra-domain routing protocol
- gateway router: at "edge" of its own AS, has link(s) to router(s) in other AS'es

inter-AS routing

- routing among AS'es
- gateways perform inter-domain routing (as well as intra-domain routing)

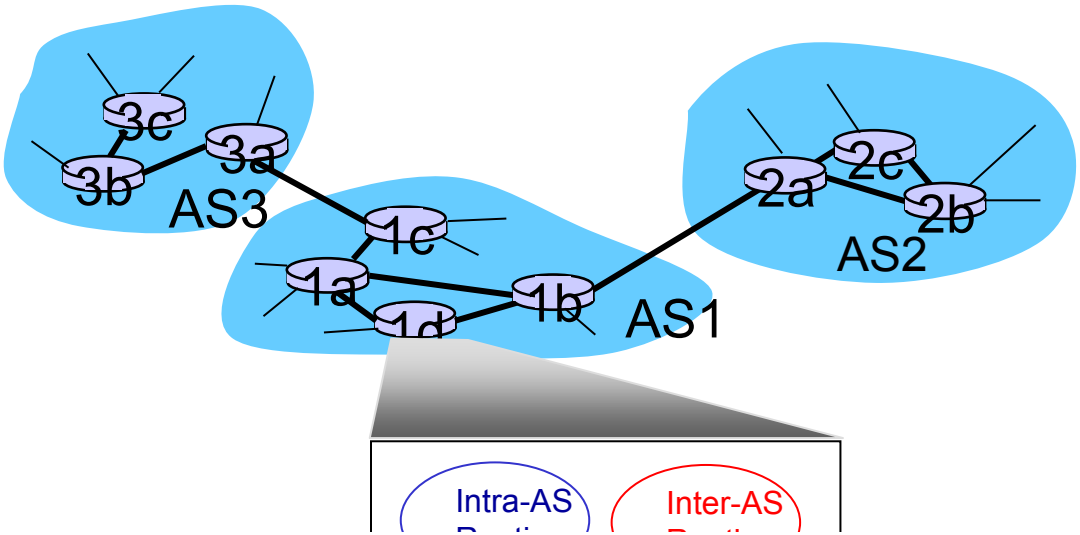


Interconnected ASes



- forwarding table configured by both intra- and inter-AS routing algorithm
  - intra-AS routing determine entries for destinations within AS
  - inter-AS & intra-AS determine entries for external destinations

Interconnected ASes



- Forwarding table configured by both intra- and inter-AS routing algorithm
  - intra-AS routing determine entries for destinations within AS
  - inter-AS & intra-AS determine entries for external destinations

Mentimeter Question: Based on your understanding of Interconnected ASes. Discuss how a hierarchical organization of the Internet has made it possible to scale to millions of users.

Answer:
Routers are organized into autonomous systems (ASs). Within an AS, all routers run the same intra-AS routing protocol. The problem of scale is solved since an router in an AS need only know about routers within its AS and the subnets that attach to the AS. To route across ASes, the inter-AS protocol is based on the AS graph and does not take individual routers into account.

Inter-AS tasks

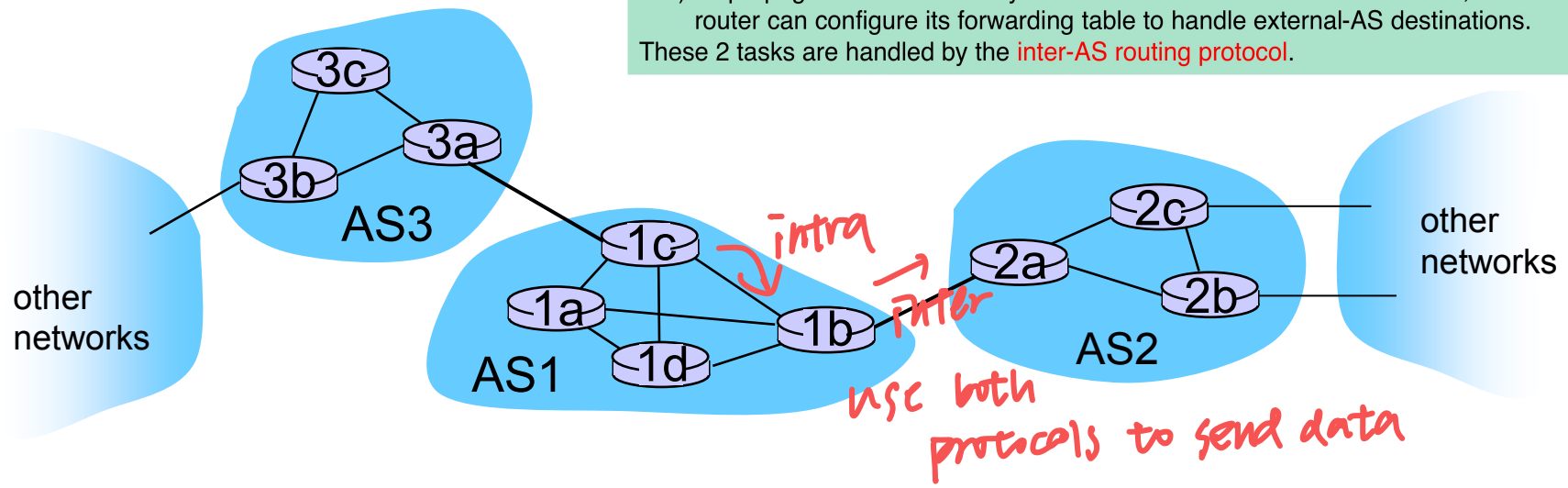
- suppose router in AS1 receives datagram destined outside of AS1:
  - router should forward packet to gateway router, but which one?

AS1 must:

1. learn which dests are reachable through AS2, which through AS3
2. propagate this reachability info to all routers in AS1

job of inter-AS routing!

b) To propagate this reachability information to all the routers within AS1, so that each router can configure its forwarding table to handle external-AS destinations.
These 2 tasks are handled by the inter-AS routing protocol.



Intra-AS Routing

- also known as *interior gateway protocols (IGP)*
- most common intra-AS routing protocols:
  - RIP: Routing Information Protocol
  - OSPF: Open Shortest Path First (IS-IS protocol essentially same as OSPF)
  - IGRP: Interior Gateway Routing Protocol (Cisco proprietary for decades, until 2016)

not necessary to use same intra-AS routing algorithm for every autonomous system

↓

∴ Each AS has administrative autonomy for routing within an AS.

OSPF (Open Shortest Path First)

- "open": publicly available
- uses link-state algorithm
  - link state packet dissemination
  - topology map at each node → know any information
  - route computation using Dijkstra's algorithm
- router floods OSPF link-state advertisements to all other routers in *entire* AS
  - carried in OSPF messages directly over IP (rather than TCP or UDP)
  - link state: for each attached link
- *IS-IS routing* protocol: nearly identical to OSPF

OSPF "advanced" features

证明是真的

- *security:* all OSPF messages authenticated (to prevent malicious intrusion)
- multiple same-cost paths allowed (only one path in RIP)
- for each link, multiple cost metrics for different TOS (e.g., satellite link cost set low for best effort ToS; high for real-time ToS)
- integrated uni- and multi-cast support:
  - Multicast OSPF (MOSPF) uses same topology data base as OSPF
- hierarchical OSPF in large domains.

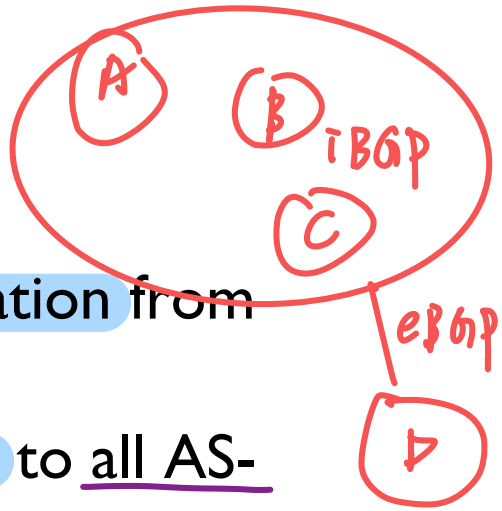
Test your understanding

- What is an Autonomous System:

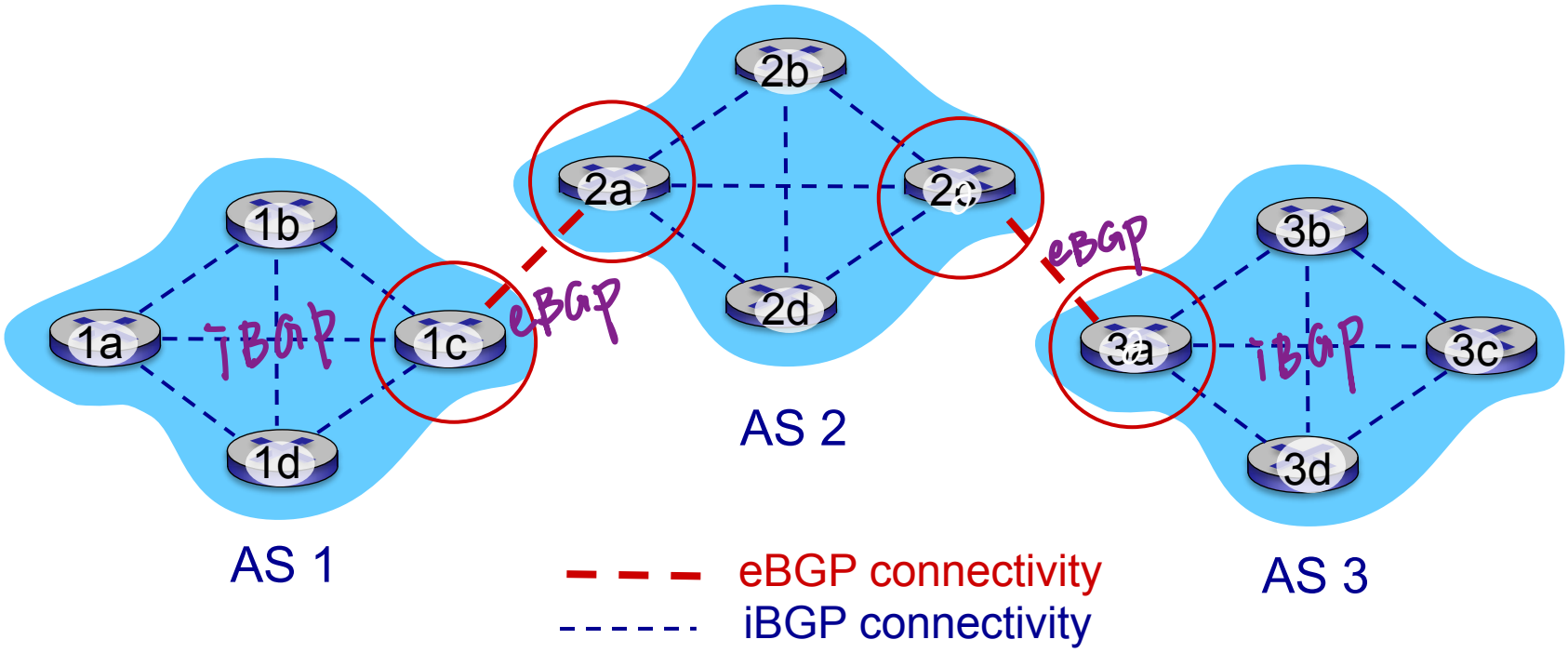
A. A backbone network of core routers
B. **A group of networks and routers under a single administrative authority**
C. A set of networks that operate in isolation from the rest of the Internet
D. A computer network that controls a mobile network

Internet inter-AS routing: BGP

- BGP (Border Gateway Protocol): *the* de facto inter-domain routing protocol
  - "glue that holds the Internet together"
- BGP provides each AS a means to:
  - external eBGP: obtain subnet reachability information from neighboring ASes 2 ASes
  - internal iBGP: propagate reachability information to all AS-internal routers.
  - determine "good" routes to other networks based on reachability information and *policy*
- allows subnet to advertise its existence to rest of Internet: *"I am here"*



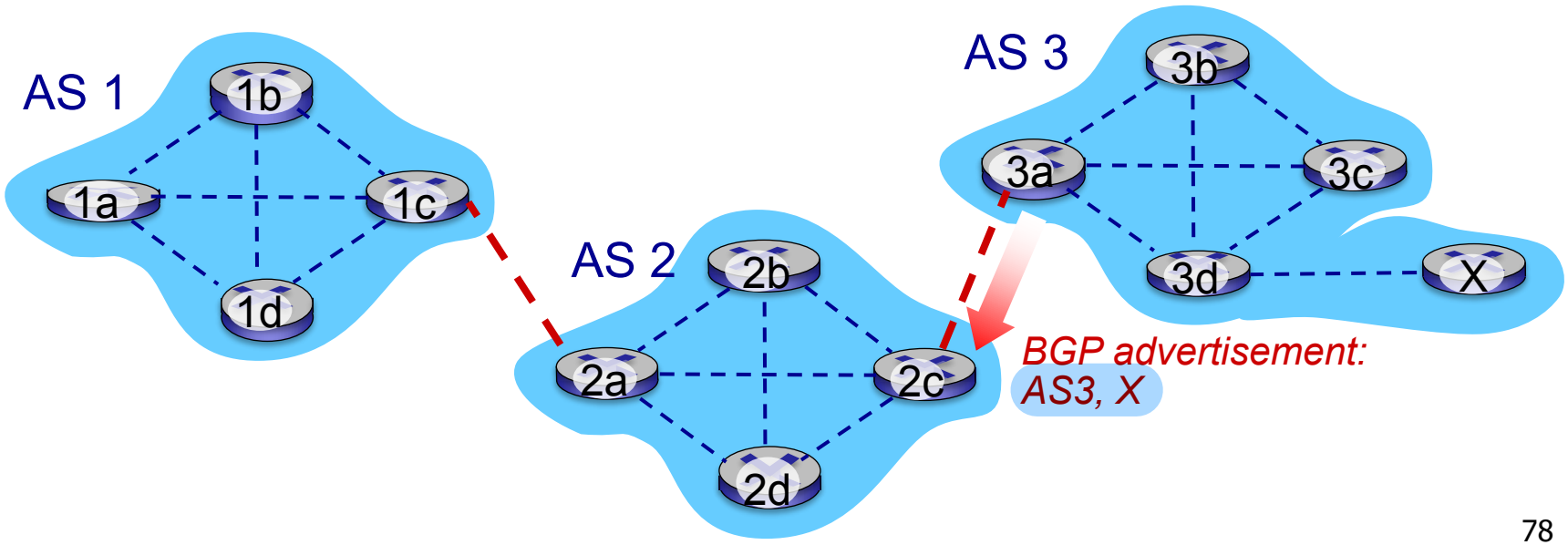
eBGP, iBGP connections



gateway routers run both eBGP and iBGP protools

BGP basics

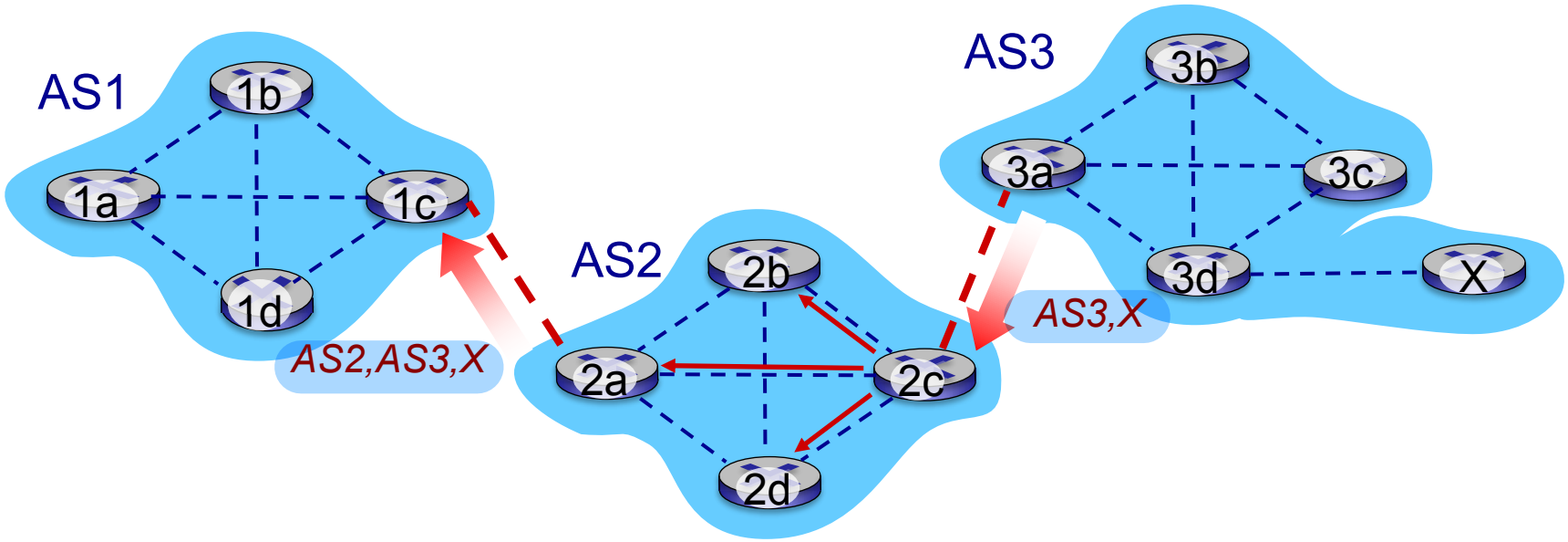
- BGP session: two BGP routers ("peers") exchange BGP messages over semi-permanent TCP connection:
  - advertising *paths* to different destination network prefixes (BGP is a "path vector" protocol)
- when AS3 gateway router 3a advertises path AS3,X to AS2 gateway router 2c:
  - AS3 *promises* to AS2 it will forward datagrams towards X



Path attributes and BGP routes

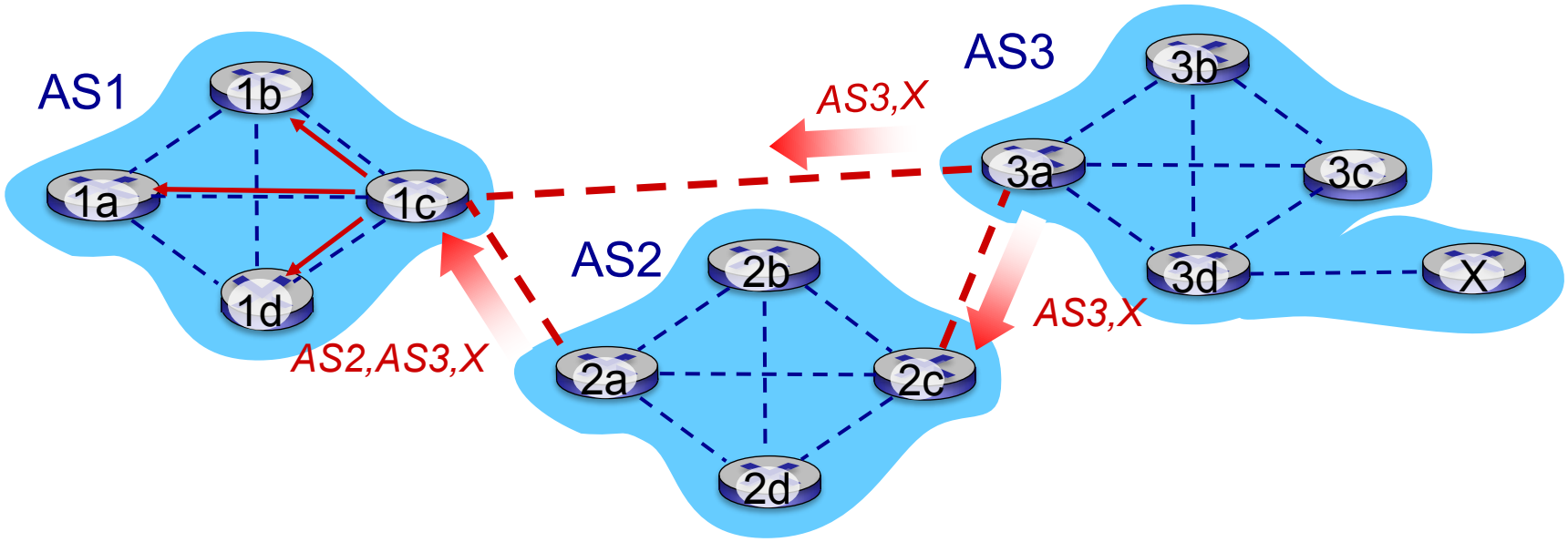
- advertised prefix includes BGP attributes
  - prefix + attributes = "route"
- two important attributes:
  - AS-PATH: list of ASes through which prefix advertisement has passed
  - NEXT-HOP: indicates specific internal-AS router to next-hop AS
- *Policy-based routing:*
  - gateway receiving route advertisement uses *import policy* to accept/decline path (e.g., never route through AS Y).
  - AS policy also determines whether to *advertise* path to other other neighboring ASes

BGP path advertisement



- AS2 router 2c receives path advertisement AS3,X (via eBGP) from AS3 router 3a
- Based on AS2 policy, AS2 router 2c accepts path AS3,X, propagates (via iBGP) to all AS2 routers
- Based on AS2 policy, AS2 router 2a advertises (via eBGP) path AS2, AS3, X to AS1 router 1c

BGP path advertisement



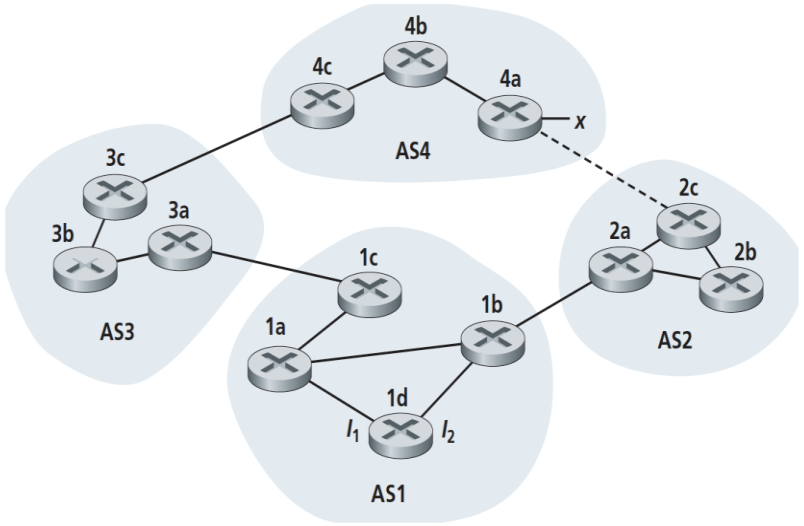
gateway router may learn about multiple paths to destination:

- AS1 gateway router 1c learns path *AS2,AS3,X* from 2a
- AS1 gateway router 1c learns path *AS3,X* from 3a
- Based on policy, AS1 gateway router 1c chooses path *AS3,X, and advertises path within AS1 via iBGP*

BGP path advertisement

Mentimeter Questions:
Consider the network shown below.

- AS3 and AS2: Running OSPF Intra-AS routing protocol.
- AS1 and AS4: Running RIP intra-AS routing protocol.
- Suppose eBGP and iBGP are used for the inter-AS routing protocol.
- Initially suppose there is no physical link between AS2 and AS4.



a) Router 3c learns about prefix x from which routing protocol? → eBGP
b) Router 3a learns about x from which routing protocol? → iBGP
c) Router 1c learns about x from which routing protocol? → eBGP
d) Router 1d learns about x from which routing protocol? → iBGP

BGP route selection

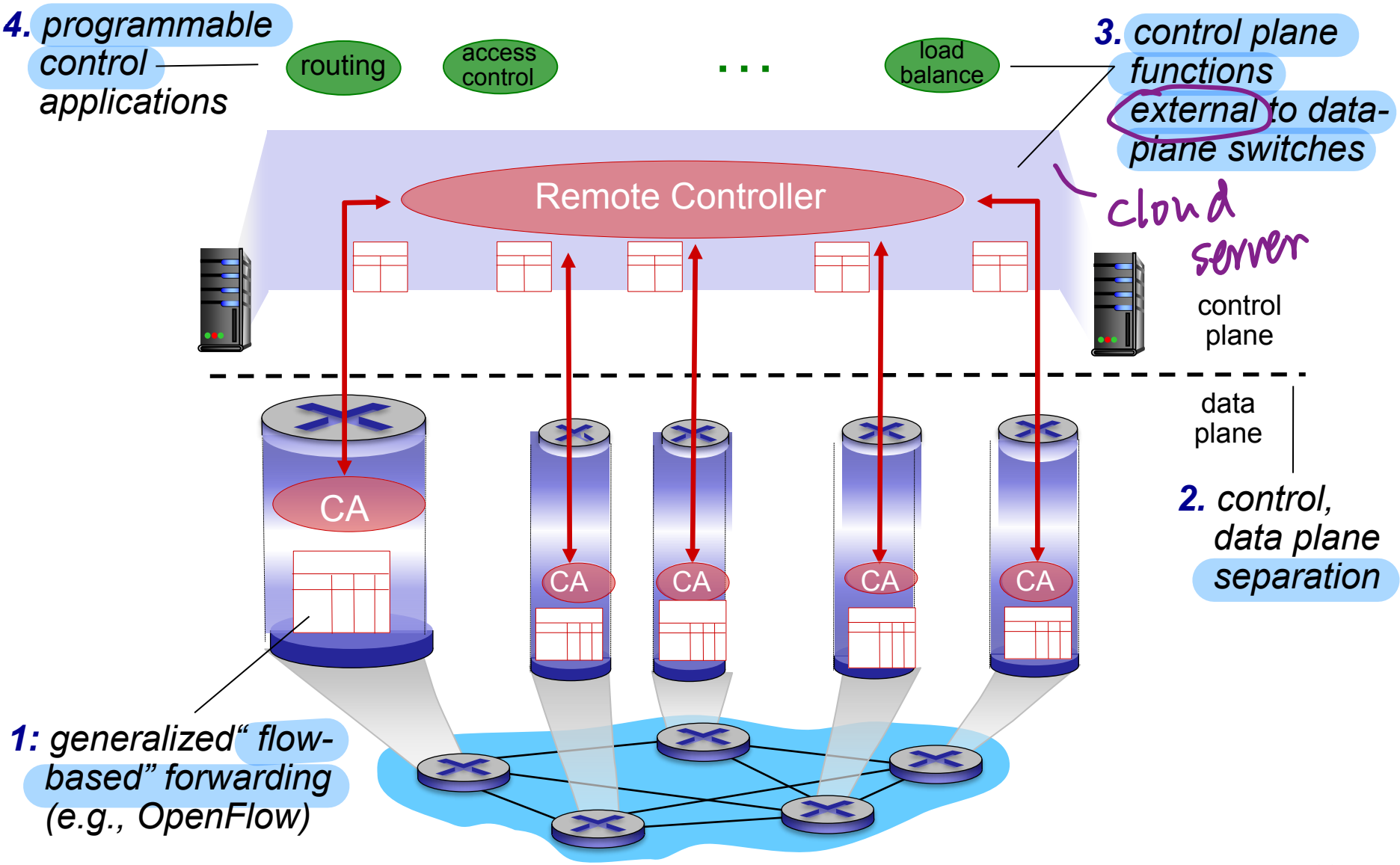
- router may learn about more than one route to destination AS, selects route based on:
  1. local preference value attribute: policy decision
  2. shortest AS-PATH
  3. closest NEXT-HOP router: hot potato routing
  4. additional criteria

Software defined networking (SDN)

Why a *logically centralized* control plane?

- easier network management: avoid router misconfigurations, greater flexibility of traffic flows
- table-based forwarding (recall OpenFlow API) allows "programming" routers
  - centralized "programming" easier: compute tables centrally and distribute
  - distributed "programming: more difficult: compute tables as result of distributed algorithm (protocol) implemented in each and every router
- open (non-proprietary) implementation of control plane

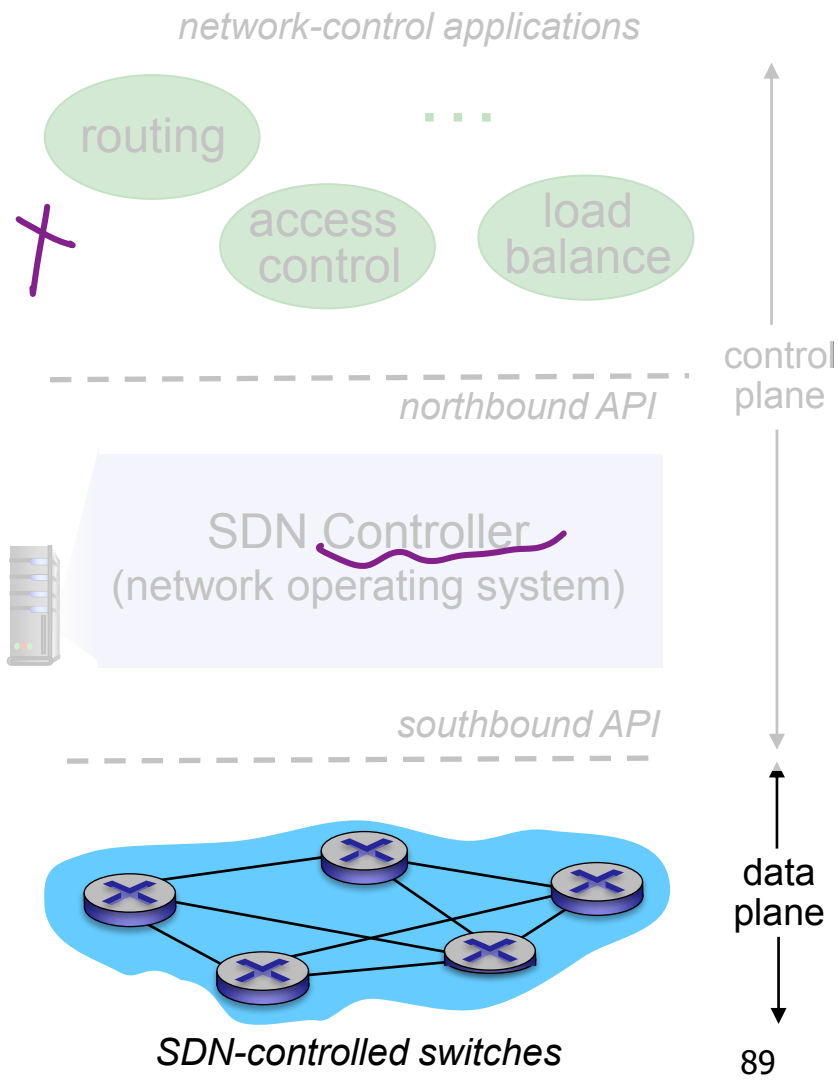
Software defined networking (SDN)



SDN perspective: data plane switches

Data plane switches

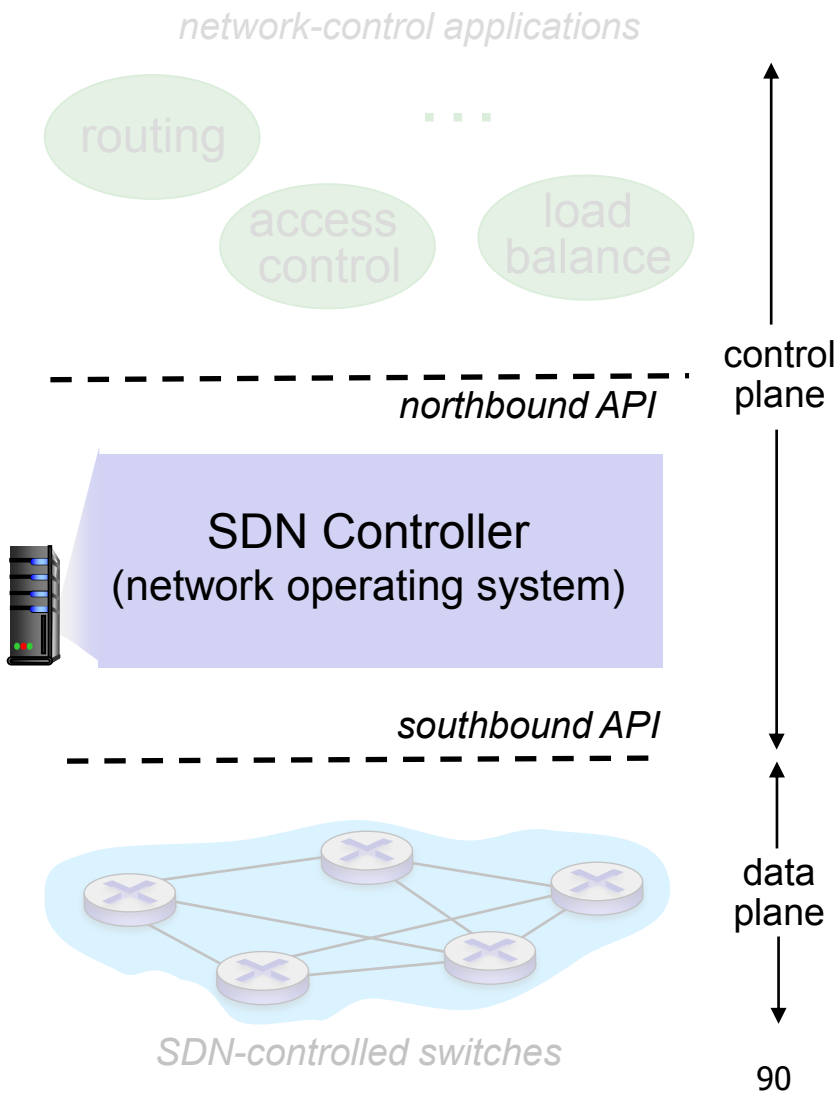
- fast, simple, commodity switches implementing generalized data-plane forwarding in hardware
- switch flow table computed, installed by controller
- API for table-based switch control (e.g., OpenFlow)
  - defines what is controllable and what is not
- protocol for communicating with controller (e.g., OpenFlow)



SDN perspective: SDN controller

SDN controller (network OS):

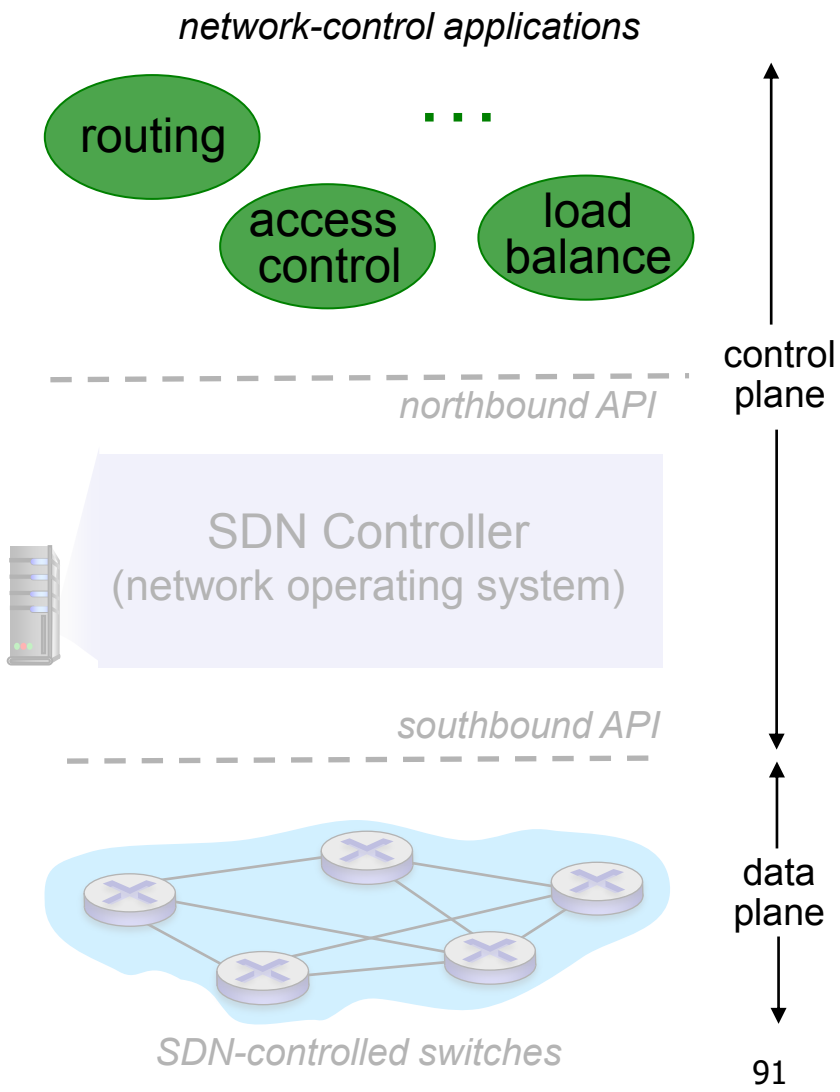
- maintain network state information
- interacts with network control applications "above" via northbound API
- interacts with network switches "below" via southbound API
- implemented as distributed system for performance, scalability, fault-tolerance, robustness
- southbound protocol – connects controller to switch.
- northbound protocol -- connects controller to apps



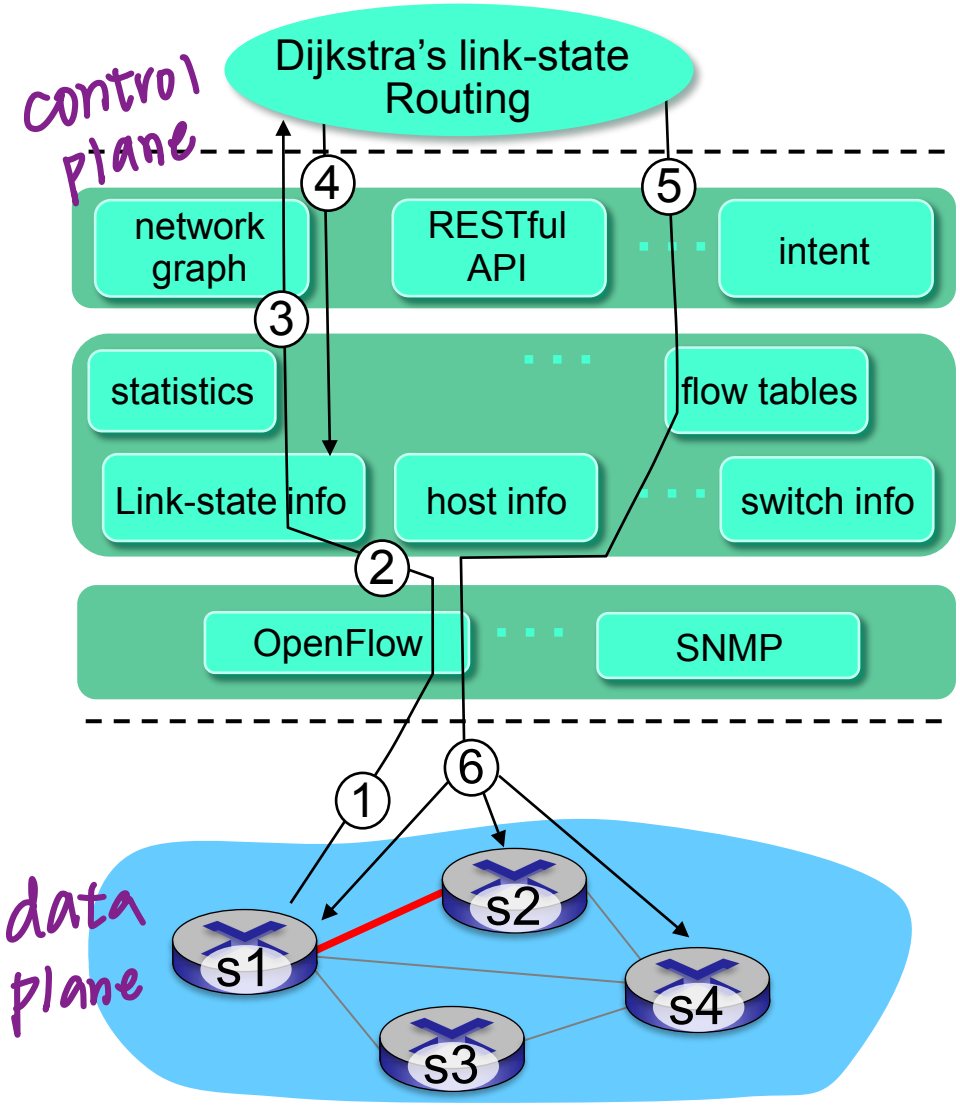
SDN perspective: control applications

network-control apps:

- "brains" of control: implement control functions using lower-level services, API provided by SDN controller
- *unbundled:* applications can be written by anyone, not just company who sold switch or company who created controller



SDN: control/data plane interaction example



① S1, experiencing link failure using OpenFlow port status message to notify controller

update failure

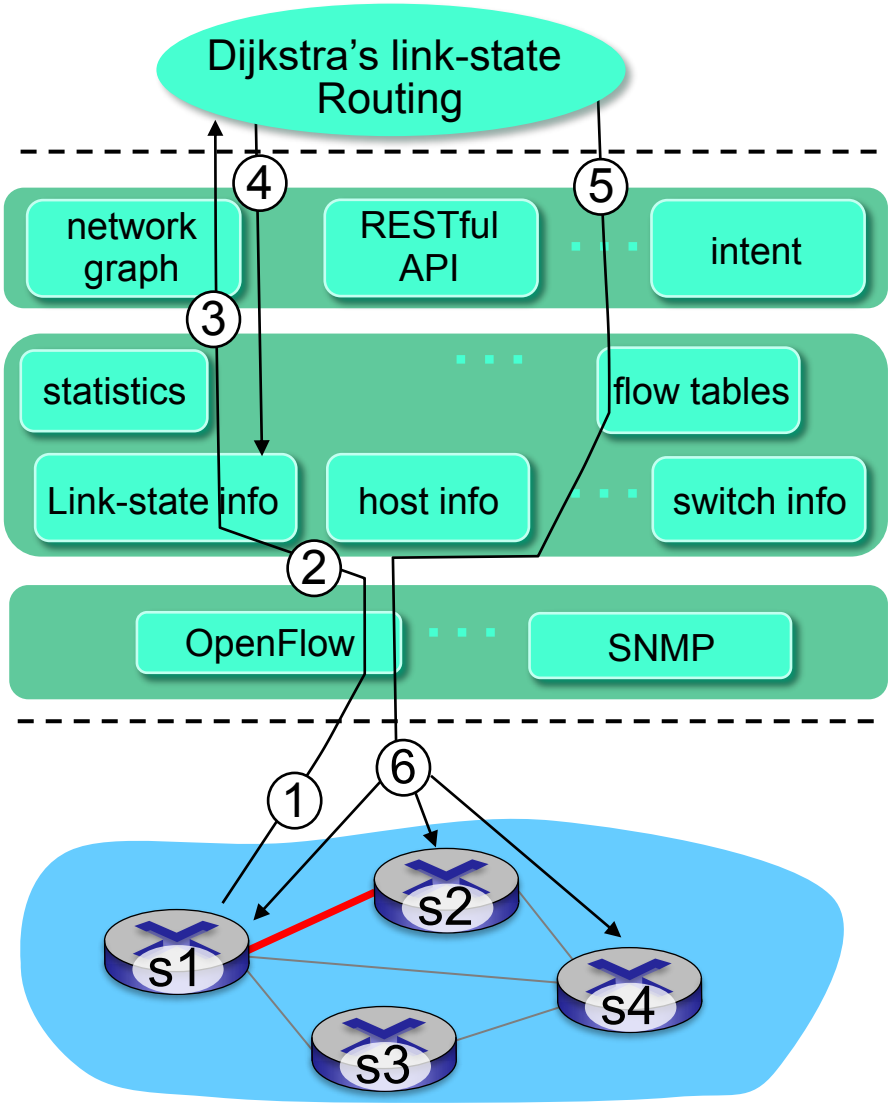
② SDN controller receives OpenFlow message, updates link status info

③ Dijkstra's routing algorithm application has previously registered to be called when ever link status changes. It is called.

④ Dijkstra's routing algorithm access network graph info, link state info in controller, computes new routes

update new information

SDN: control/data plane interaction example



⑤ link state routing app interacts with flow-table-computation component in SDN controller, which computes new flow tables needed

⑥ Controller uses OpenFlow to install new tables in switches that need updating

SDN key points

- SDN is a new way of controlling a network.
- The data plane is more flexible allowing more complex rules at routers and switches.
- The control plane is "logically" centralised (it seems as if there is just one controller though there may be several for redundancy).
- The control plane is programmable. You can program it yourself very easily.
- It allows quick experimentation with new ideas about routing and allows your network to be more flexible.
- You can try it out for yourself very quickly (mininet and OpenDaylight for example run on a laptop).
- For the first time a researcher can quickly try out new ideas in a real setting on real hardware without having to build a new router.

ICMP: internet control message protocol

- used by hosts & routers to communicate network-level information
  - error reporting: unreachable host, network, port, protocol
  - echo request/reply (used by ping)
- network-layer "above" IP:
  - ICMP msgs carried in IP datagrams
- ICMP message: type, code plus first 8 bytes of IP datagram causing error

Type	Code	description
0	0	echo reply (ping)
3	0	dest. network unreachable
3	1	dest host unreachable
3	2	dest protocol unreachable
3	3	dest port unreachable
3	6	dest network unknown
3	7	dest host unknown
4	0	source quench (congestion control - not used)
8	0	echo request (ping)
9	0	route advertisement
10	0	router discovery
11	0	TTL expired
12	0	bad IP header

Traceroute and ICMP

- source sends series of UDP segments to destination
  - first set has TTL = 1
  - second set has TTL=2, etc.
  - unlikely port number
- when datagram in *n*th set arrives to nth router:
  - router discards datagram and sends source ICMP message (type 11, code 0)
  - ICMP message include name of router & IP address

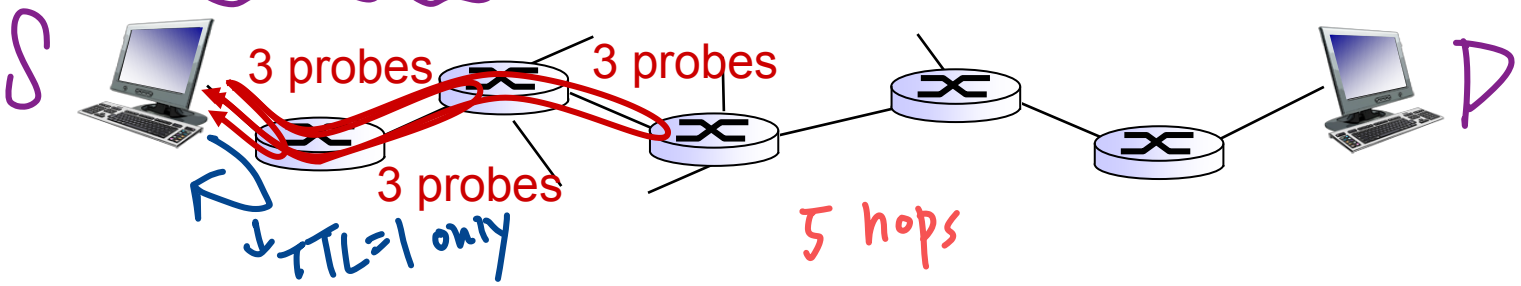
time to leave 1 hop 2 hops

- when ICMP message arrives, source records RTTs

stopping criteria:

- UDP segment eventually arrives at destination host
- destination returns ICMP "port unreachable" message (type 3, code 3)
- source stops

3+3 | data



Link layer, LANs: outline

Data link Layer

6.1 introduction, services

6.2 error detection, correction

6.3 multiple access protocols

6.4 LANs

- addressing, ARP
- Ethernet
- switches
- VLANS

6.5 link virtualization: MPLS

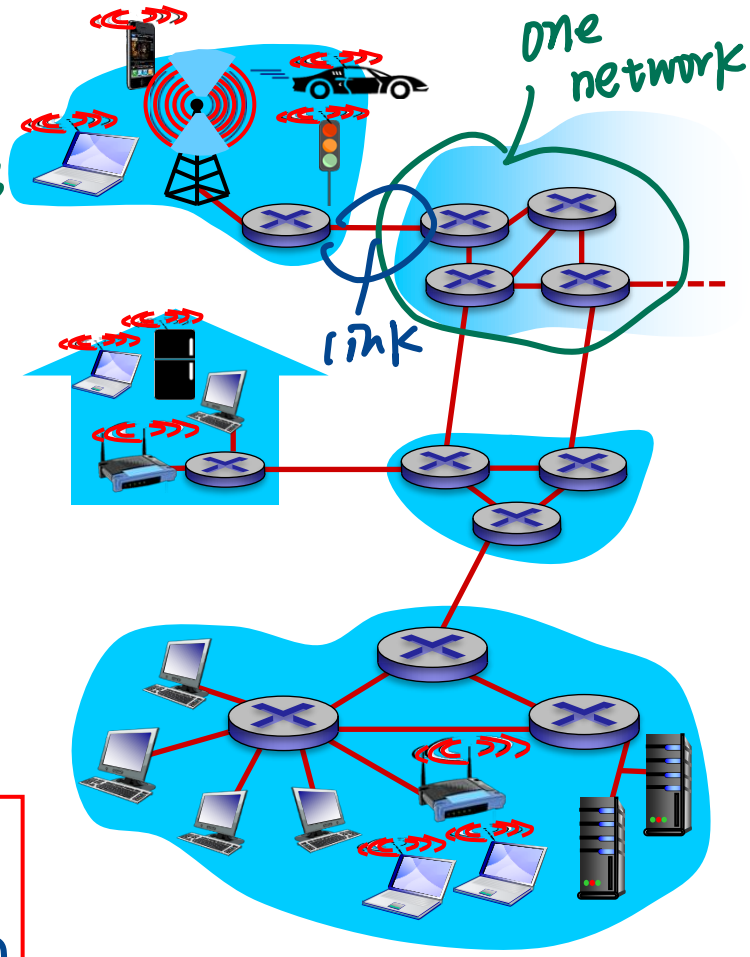
6.6 data center networking

6.7 a day in the life of a web request

Link layer: introduction

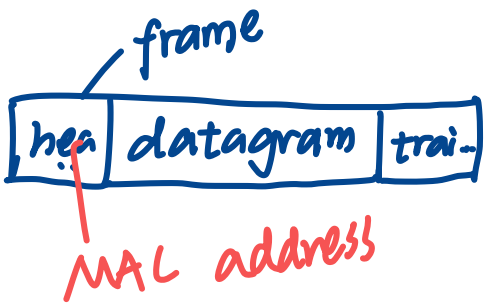
terminology:

- hosts and routers: nodes
- communication channels that connect adjacent nodes along communication path: links
  - wired links
  - wireless links
  - LANs
- layer-2 packet: frame, encapsulates datagram



data-link layer has responsibility of transferring datagram from one node to *physically adjacent* node over a link

Link layer services



- *framing, link access:*
  - encapsulate datagram into frame, adding header, trailer
  - channel access if shared medium
  - "MAC" addresses used in frame headers to identify source, destination
    - different from IP address!
- *reliable delivery between adjacent nodes*
  - we learned how to do this already (transport layer)!
  - seldom used on low bit-error link (fiber, some twisted pair)
  - wireless links: high error rates
    - *Q:* why both link-level and end-end reliability?

directly connected

⇒ more datagrams may be flat over

before datagrams trasfer to destination they will go through retransmission ... to ensure reliability

Link layer services (more)

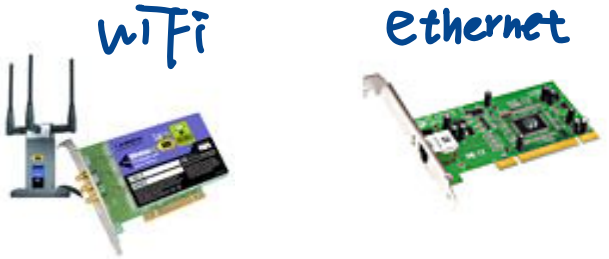
- *flow control:* avoid too fast and too slow. no data loss
  - pacing between adjacent sending and receiving nodes
- *error detection:*
  - errors caused by signal attenuation, noise.
  - receiver detects presence of errors:
    - signals sender for retransmission or drops frame
- *error correction:*
  - receiver identifies *and corrects* bit error(s) without resorting to retransmission
- *half-duplex and full-duplex*
  - with half duplex, nodes at both ends (directions) of link can transmit, but not at same time

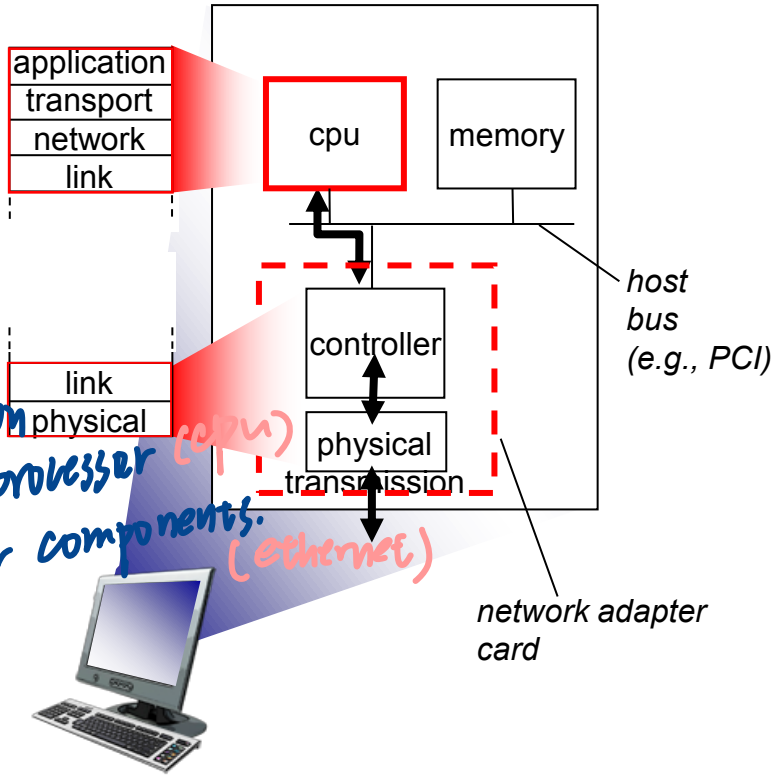
Full-duplex: a link can send in both directions at the same time

Simplex : a link can send in only one direction

Where is the link layer implemented?

- in each and every host
- link layer implemented in "adaptor" (aka *network interface card* NIC) or on a chip
  - Ethernet card, 802.11 card; Ethernet chipset
  - implements link, physical layer
- attaches into host's system buses → high speed internal connection between processor (cpu) and other components.
- combination of hardware, software, firmware

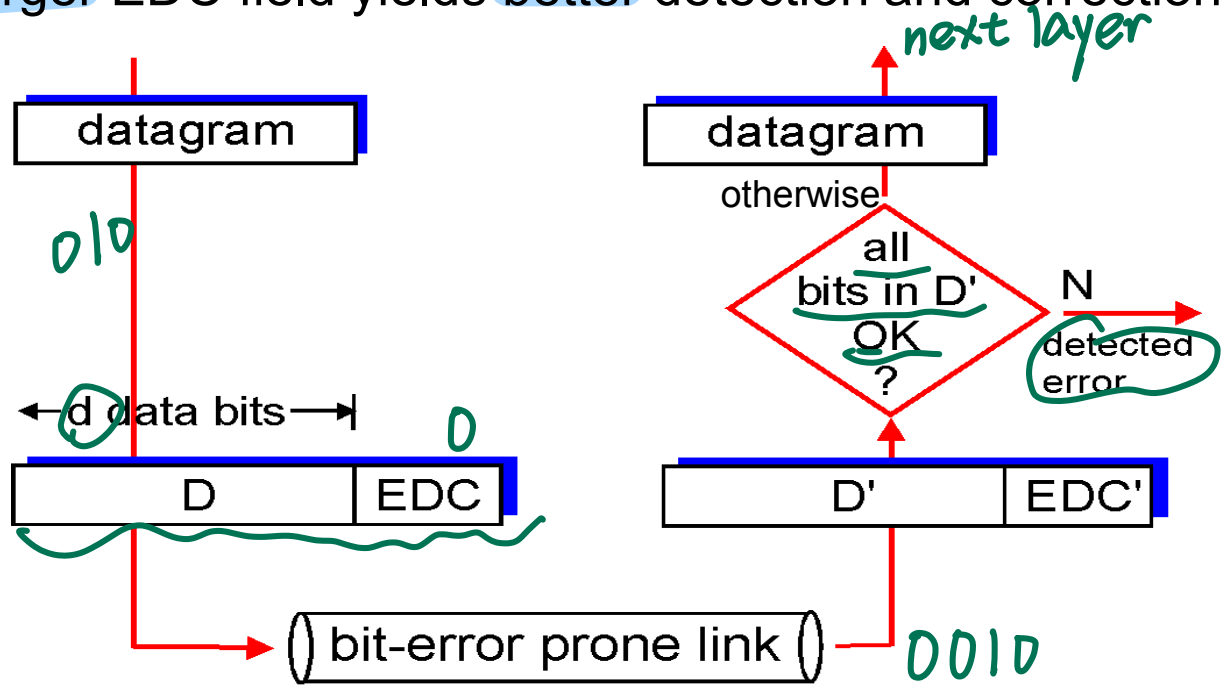




Error detection

EDC= Error Detection and Correction bits (redundancy)
D = Data protected by error checking, may include header fields

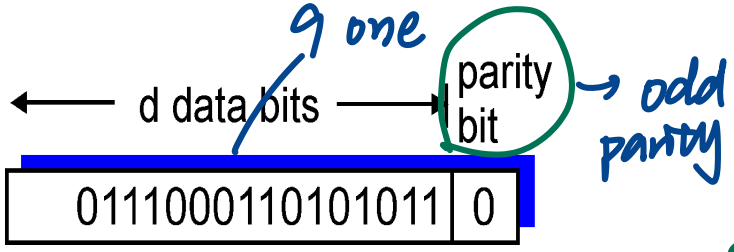
- Error detection not 100% reliable!
  - protocol may miss some errors, but rarely
  - larger EDC field yields better detection and correction



Parity checking

single bit parity:

- *d*etect single bit errors



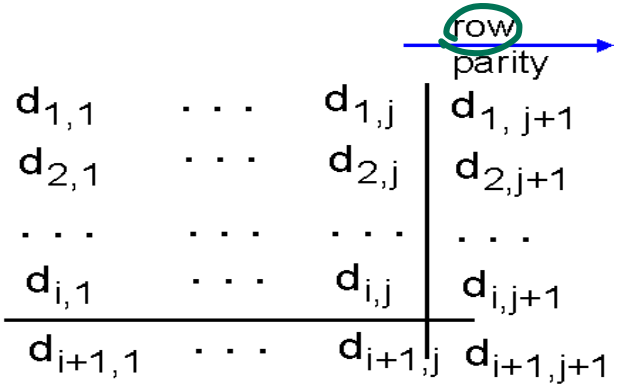
9 one

→ odd parity

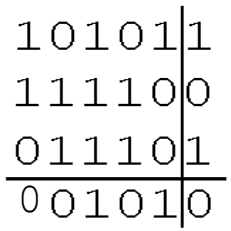
odd parity: number of "1" is odd then add 0

two-dimensional bit parity:

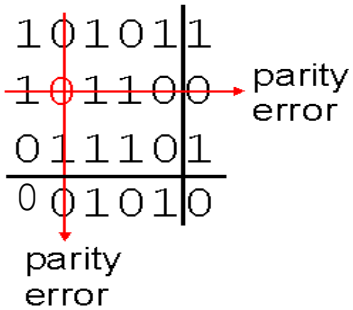
- detect and correct single bit errors



suppose even parity (row and column)



no errors



correctable single bit error 可修正的

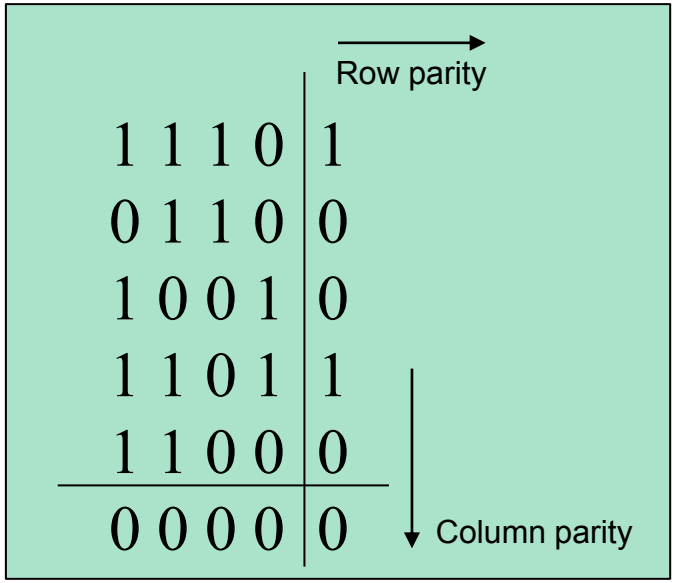
* Check out the online interactive exercises for more examples: http://gaia.cs.umass.edu/kurose_ross/interactive/

Parity checking

Mentimeter Question:

Suppose the information content of a packet is the bit pattern 1110 0110 1001 1101 1100 and an even parity scheme is being used.

What would the value of the field containing the parity bits be for the case of a two-dimensional parity scheme?



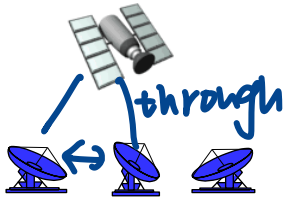
Multiple access links, protocols

two types of "links":

- point-to-point (connect two computers only) direct and dedicated
- *broadcast (shared wire or medium)*
  - old-fashioned Ethernet
  - upstream HFC (hybrid fibre coaxial)
  - 802.11 wireless LAN
- Problems: "collision" – 2 or more transmissions at once 缺点 packet may be destroyed
- Solution: Multiple access protocol – "share" medium 好处: can communicate simutanously. capacity can be much higher

shared wire (e.g., cabled Ethernet)

shared RF (radio frequency) (e.g., 802.11 WiFi)



shared RF (satellite)

humans at a cocktail party (shared air, acoustic)

Multiple access links, protocols

- **Question:** What are the three types of multiple access mechanism?
- **Answer:**

  1) Channel partitioning such as TDMA, FDMA;

  2) Random access

  3) Take turns (round robin)

An ideal multiple access protocol

given: broadcast channel of rate R bps

Desired qualities:

1. when one node wants to transmit, it can send at rate R.
2. when M nodes want to transmit, each can send at average rate R/M
3. fully decentralized: 分散的
   - no special node to coordinate transmissions
   - no synchronization of clocks, slots
4. Simple

Options:

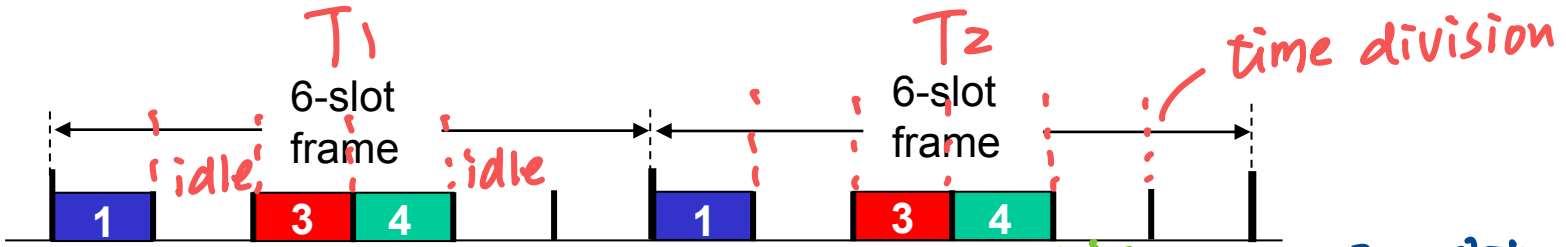
Partition channel, random access, "take turns"

Channel partitioning MAC protocols: TDMA

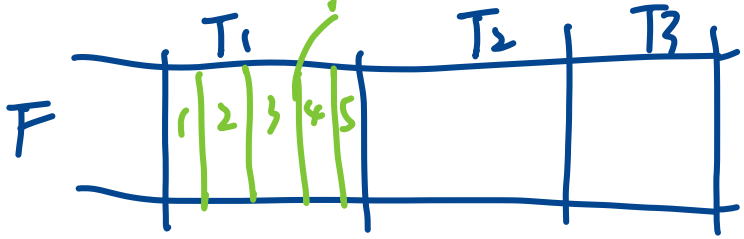
TDMA: time division multiple access

- access to channel in "rounds"
- each station gets fixed length slot (length = packet transmission time) in each round
- unused slots go idle
- example: 6-station LAN, 1,3,4 have packets to send, slots 2,5,6 idle

fixed time slot



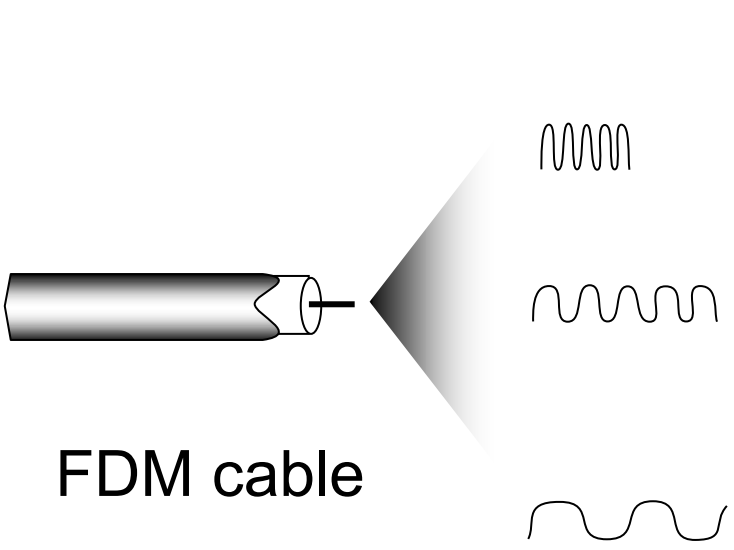
rate: $\frac{R}{b}$ bps
↘ satisfy qualities ☺

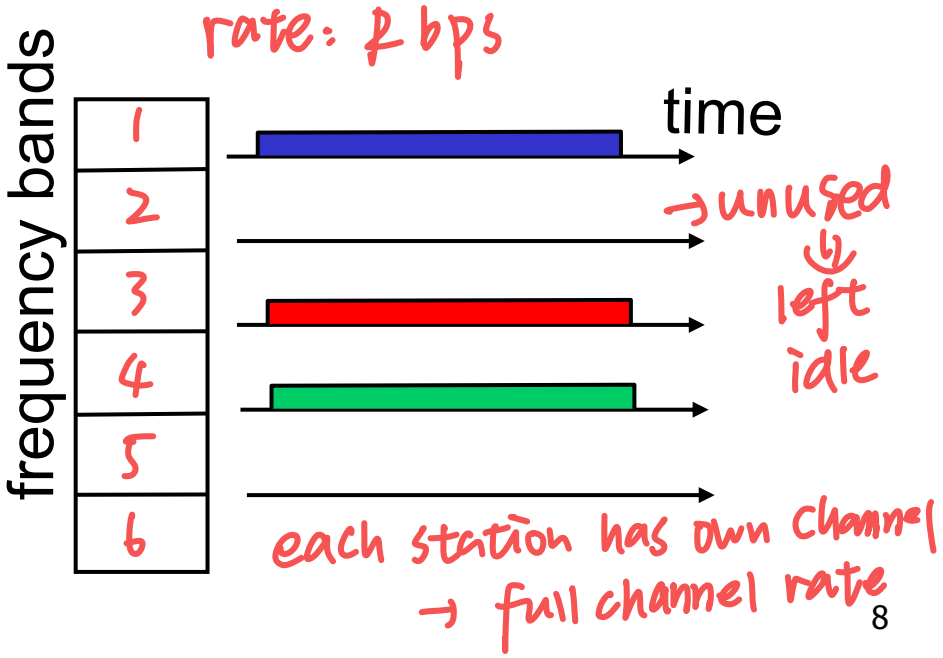


Channel partitioning MAC protocols: FDMA

FDMA: frequency division multiple access

- channel spectrum divided into frequency bands
- each station assigned fixed frequency band
- unused transmission time in frequency bands go idle
- example: 6-station LAN, 1,3,4 have packet to send, frequency bands 2,5,6 idle





Random access protocols

- when node has packet to send
  - transmit at full channel data rate R.
  - no *a priori* coordination among nodes
- two or more transmitting nodes → "collision",
- random access MAC protocol specifies:
  - how to detect collisions
  - how to recover from collisions (e.g., via delayed retransmissions)
- examples of random access MAC protocols:
  - slotted ALOHA (not an acronym, means "hello" in Hawaii)
  - pure ALOHA
  - CSMA, CSMA/CD, CSMA/CA
    - Carrier Sense Multiple Access (collision detection/collision avoidance)

Slotted ALOHA

assumptions:

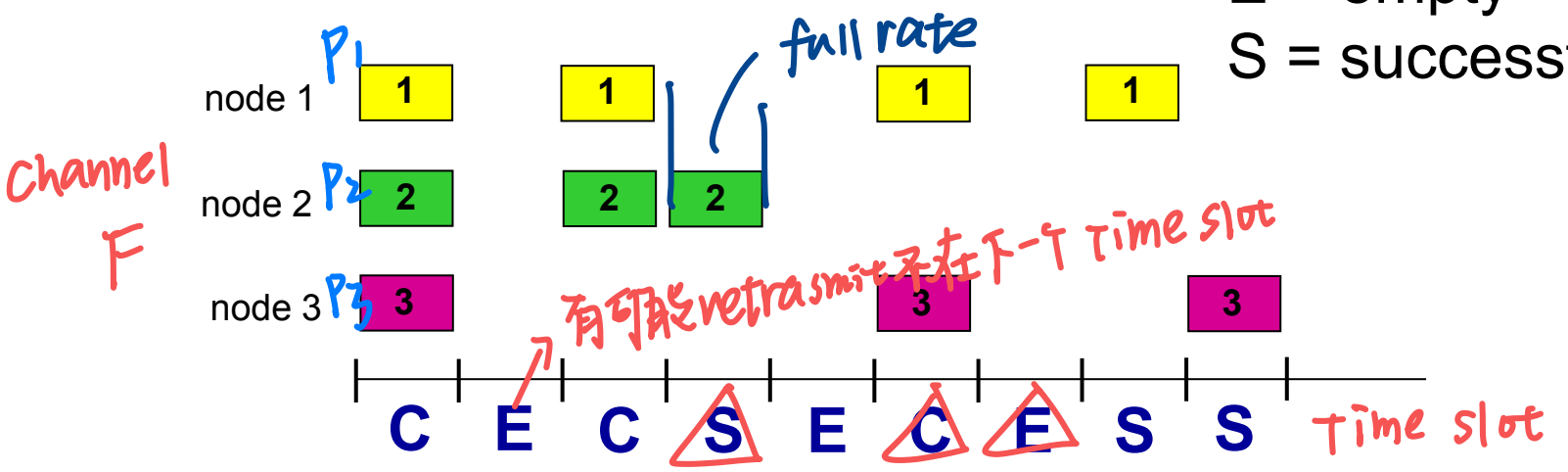
- all frames same size
- time divided into equal size slots (time to transmit 1 frame)
- nodes start to transmit only slot beginning
- nodes are synchronized
- if 2 or more nodes transmit in slot, all nodes detect collision

operation:

- when node obtains fresh frame, transmits in next slot
  - *if no collision:* node can send new frame in next slot
  - *if collision:* node retransmits frame in each subsequent slot with prob. p until success

Slotted ALOHA

C = collision
E = empty
S = successfully sent



Pros:

- single active node can continuously transmit at full rate of channel
- highly decentralized: only slots in nodes need to be in sync
- simple

Cons:

- collisions, wasting slots
- idle slots
- nodes may be able to detect collision in less than time to transmit packet
- clock synchronization

Pure (unslotted) ALOHA

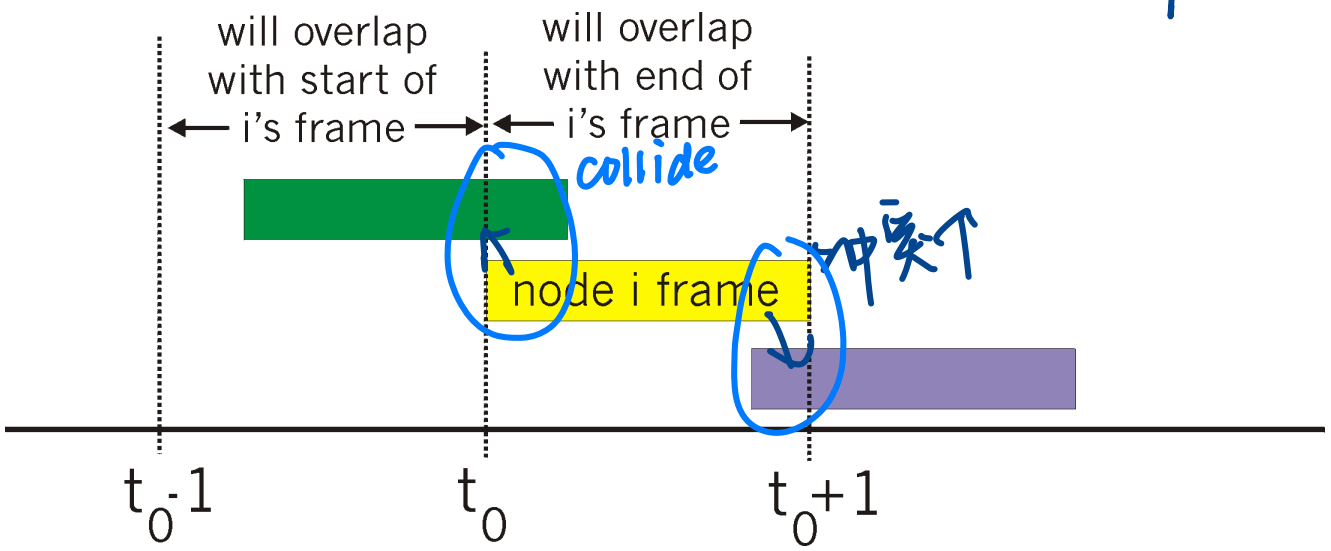
no time division

- unslotted Aloha: simpler, no synchronization
- when frame first arrives
  - transmit immediately

no any time boundary

- collision probability increases:
  - frame sent at t_0 collides with other frames sent in $[t_0-1, t_0+1]$
- Doesn't need unified clock but half as efficient as slotted.

no synchronization but less efficiency



Slotted ALOHA v.s. Pure ALOHA

- **Question:** What are differences between slotted ALOHA and Pure ALOHA?
- **Answer:**

1) Different operation mechanisms:

For slotted ALOHA, when new frame arrives, it will transmit in the next time slot. For pure ALOHA, the new arrived frame will be transmitted immediately.

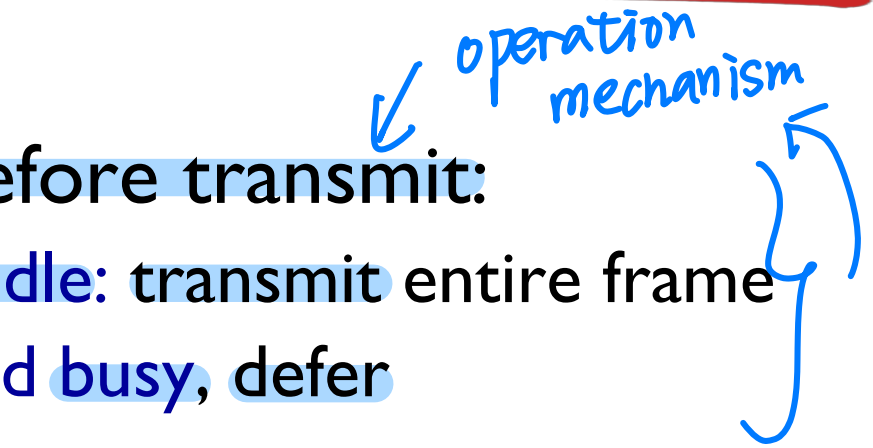
2) Different operation efficiency:

The efficiency of pure ALOHA is just half that of slotted ALOHA.

3) Different operation complexities:

Pure ALOHA is simple and no synchronization, while Slotted ALOHA needs synchronization.

CSMA (carrier sense multiple access)



CSMA: listen before transmit:

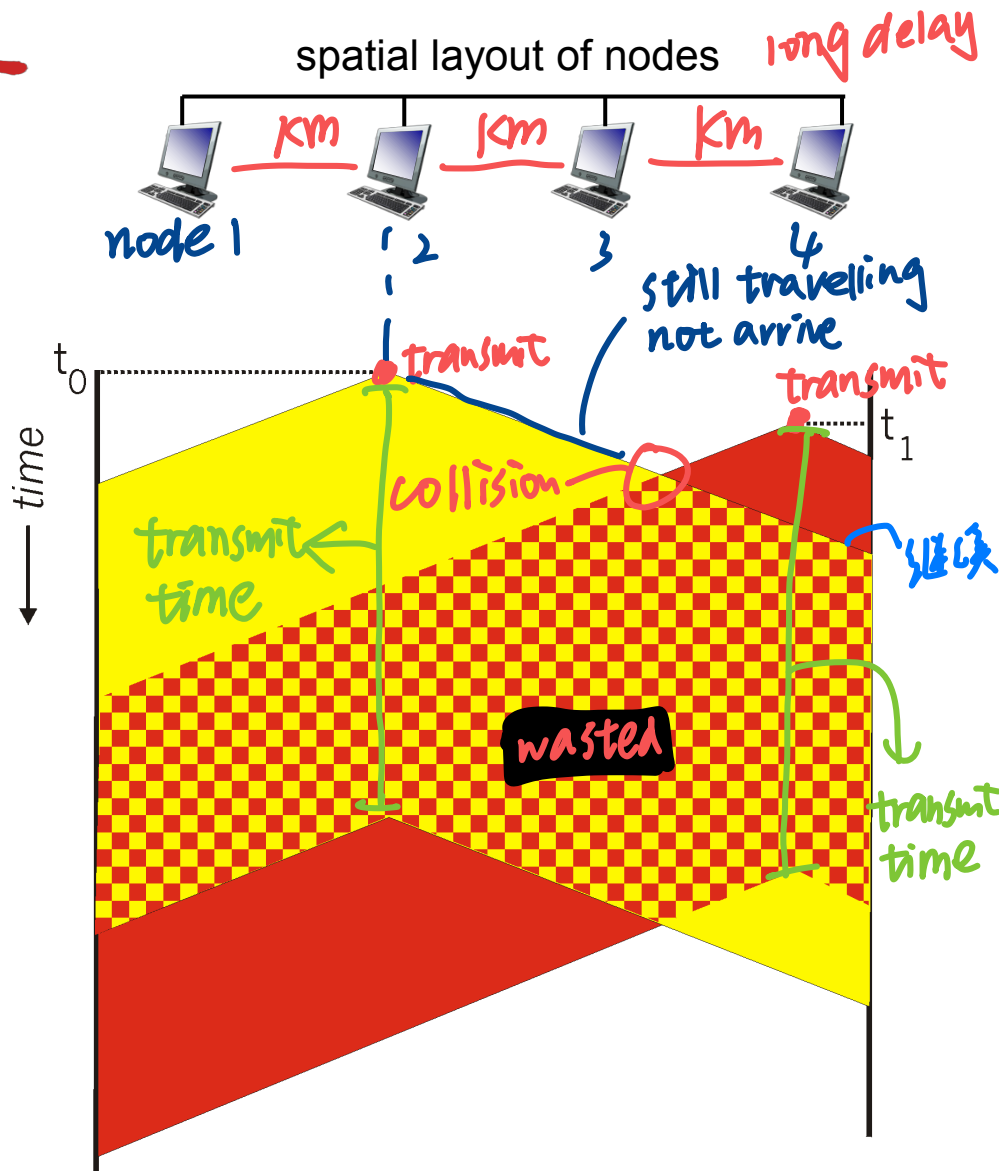
if channel sensed idle: transmit entire frame

- if channel sensed busy, defer transmission

- human analogy: don't interrupt others!

CSMA collisions

- collisions *can* still occur: propagation delay means two nodes may not hear each other's transmission
- collision: entire packet transmission time wasted
  - distance & propagation delay play role in in determining collision probability



CSMA/CD (collision detection)

CSMA/CD: carrier sensing, deferral (backs off transmission) as in CSMA

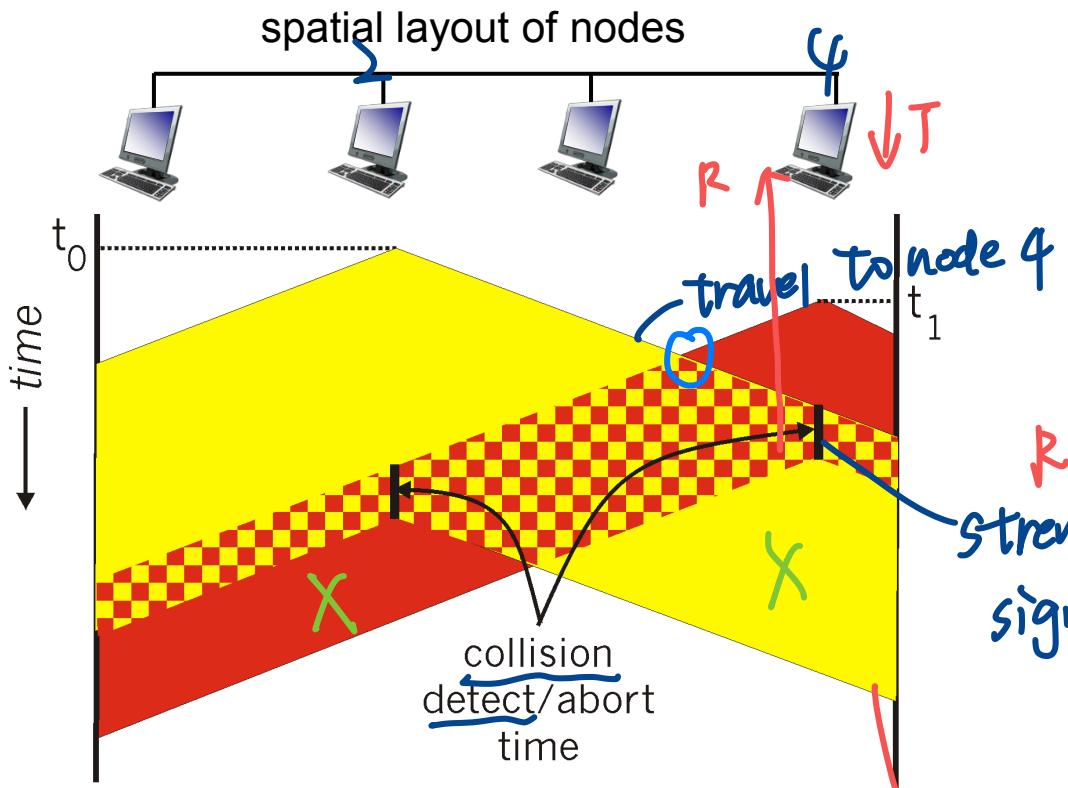
- collisions *detected* within short time
- colliding transmissions aborted, reducing channel wastage

- collision detection:
  - easy in wired LANs: measure signal strengths, compare transmitted, received signals
  - difficult in wireless LANs: received signal strength overwhelmed by local transmission strength
- human analogy: polite talk where people wait for each other

- **Question:** Why CSMA/CD is not applicable in wireless communications?
- **Answer:**

The received signal is too weak compared with its local signal power due to the severe path loss in wireless communications.

CSMA/CD (collision detection)



Ethernet CSMA/CD algorithm

1. Network Interface Card (NIC)receives datagram from network layer, creates frame

2. If NIC senses channel idle, starts frame transmission. If NIC senses channel busy, waits until channel idle, then transmits.

3. If NIC transmits entire frame without detecting another transmission, NIC is done with frame !

Ethernet CSMA/CD algorithm

4. If NIC detects another transmission while transmitting, aborts and sends jam signal

stop

wait before transmit again

5. After aborting, NIC enters *binary (exponential) backoff*:
   - after *m* th collision, NIC chooses *K* at random from {*0, 1, 2, …, 2^m-1*}. NIC waits K·512 bit times, returns to Step 2
   - longer backoff interval with more collisions

10mbps => 51.2 μs

100mbps => 5.12 μs

contention window

1 = 1Mbits/s

100x10^6

Test your understanding

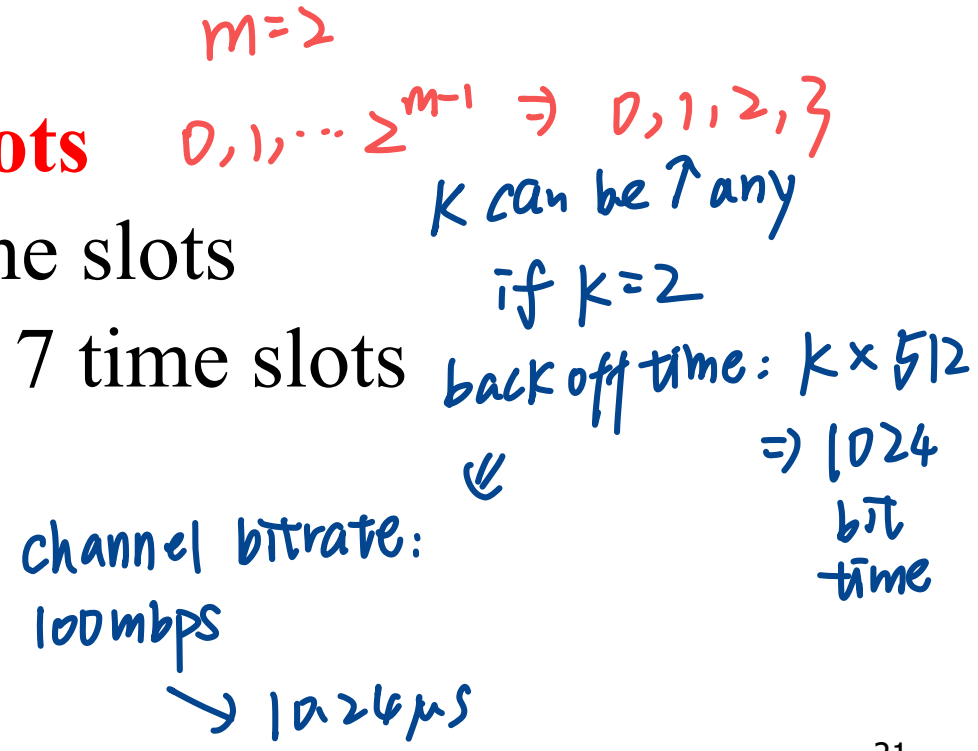
With Ethernet, if a host experiences two successive collisions it may next backoff for …

A. 0 or 1 time slots

B. 0, 1, 2 or 3 time slots

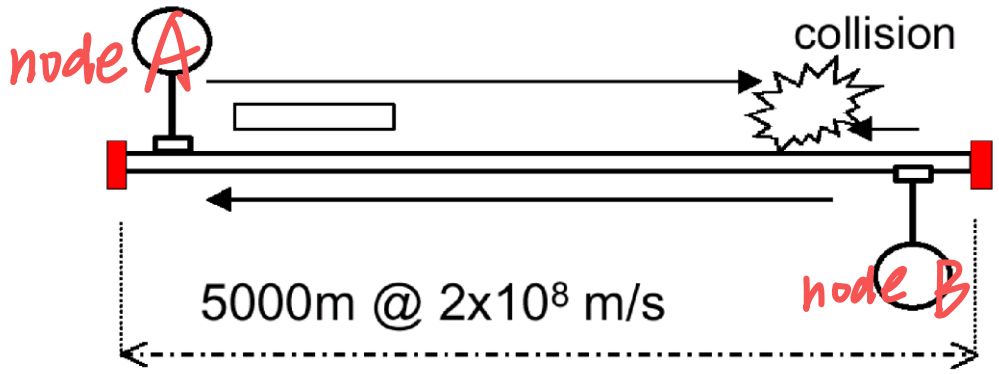
C. 0, 1, 2, 3, 4 or 5 time slots

D. 0, 1, 2, 3, 4, 5, 6 or 7 time slots



CSMA/CD: Frame size

- Collision Detection
- Collision Window
  - Related to end-to-end propagation delay
- Minimum packet size must be greater than collision window
  - For 5000m (5km) bus @ 10 Mbits/sec,
  - RTT = 2x5000m/2x10^8 m/s = 0.00005s
    min frame size = 0.00005s x 10,000,000 bits/sec
    = (500 bits) ~64 octets



2x10^8 m/s = 2/3 speed of light = speed of signal in copper wire

Test your understanding

With CSMA/CD, a minimum frame size must be imposed to ensure …..

A. The transmitting station is still transmitting long enough for a corrupted signal to be detected

B. The frame overhead is limited

C. Access to the shared medium is fair

D. Stations do not keep hold of the shared medium for too long

CSMA/CD

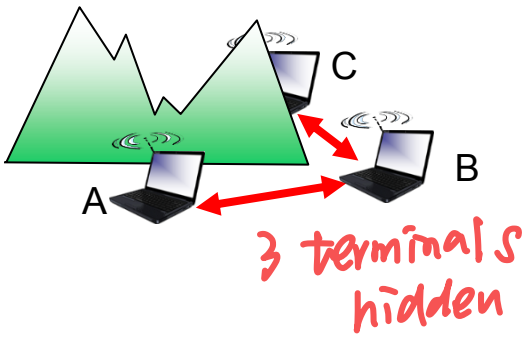
- **Question:** Why CSMA/CD requires a frame to have minimum size?
- **Answer:** If two frames are transmitted within certain time, collisions will occur. Both senders need to detect collision. The senders only detect collision during transmission. If transmission is too short, the senders cannot detect collision. If the transmission is long enough, the senders can detect collision.

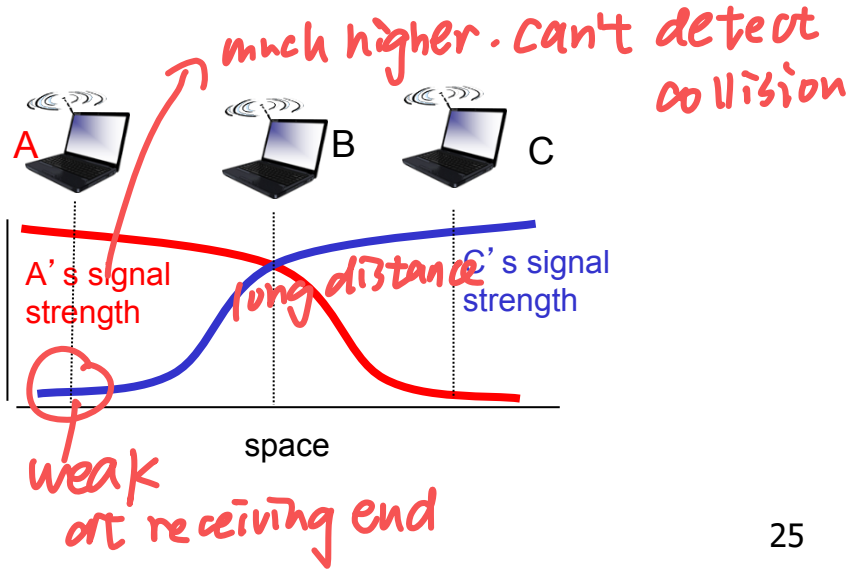
CSMA/CD

- **A sample Question:** Why the minimum packet should transmit at least twice the propagation delay between any two distant nodes?
- **Answer:** Consider two distant nodes A and B. The one way trip time is denoted as T. Assume A sends a frame at time T0. Node B just sends a frame just before A's frame arrives at node B. Then, we can approximately assume that node B sends its frame at time T0+T. After T0+T, node B can detect collision due to the frame from node A. Then, after one way trip time T, node B's frame arrives at node A, and node A can detect the collision due to the frame from node B at time T0+2T. From T0 to T0+2T, node A should keep transmitting since the node only detects the collision during transmitting.

CSMA/CA (Collision Avoidance)

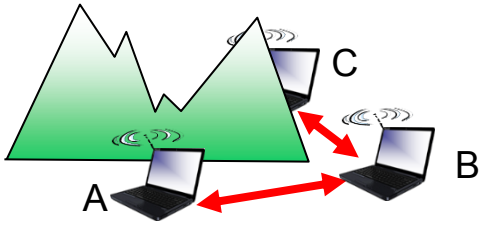
- avoid collisions: 2^+ nodes transmitting at same time
- 802.11: CSMA - sense before transmitting IEEE standard for WiFi
  - don't collide with ongoing transmission by other node
- 802.11: *no* collision detection!
  - difficult to receive (sense collisions) when transmitting due to weak received signals (fading)
  - can't sense all collisions in any case: hidden terminal, fading
  - goal: *avoid collisions:* CSMA/C(ollision)A(voidance)





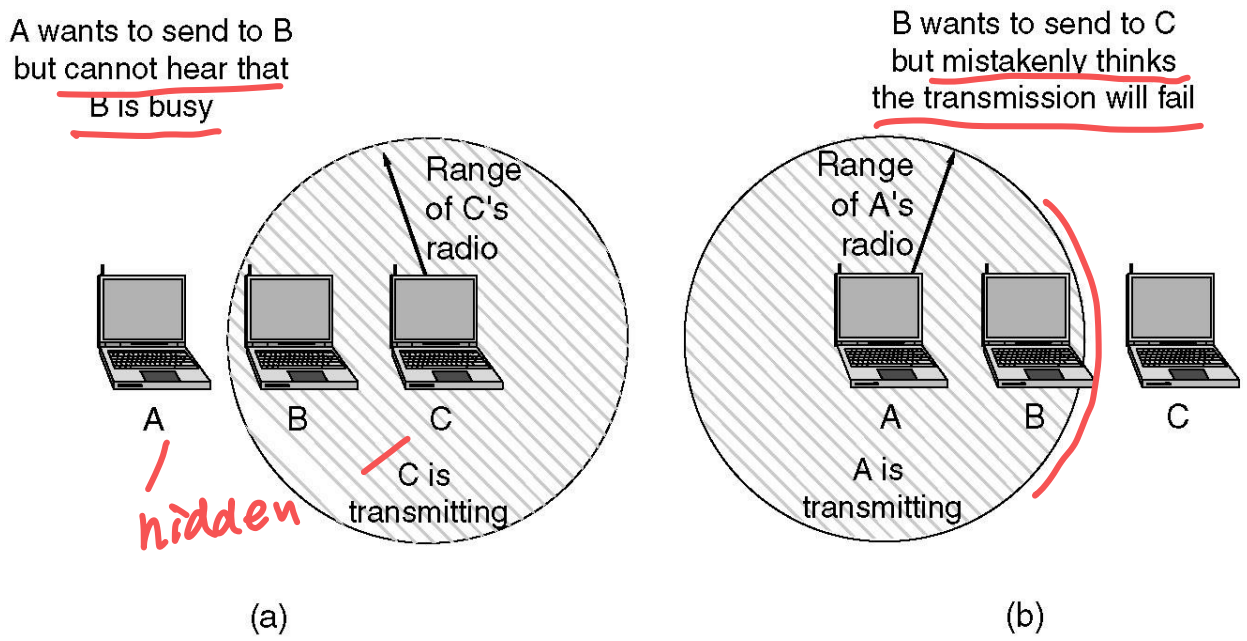
Hidden Problem in Wireless Communications

- **Question:** What is main technique to deal with the collision issue?
- **Answer:** Consider the following example, node A and node C are blocked by obstacles such that node A cannot hear the signals from C. When node C sends packet to node B, node A cannot detect the existence of node C and will send packet to node B as well. This will cause collision.



- The main technique is CSMA/CA.

Hidden and Exposed Station problems



(a) The hidden station problem.

A and C are hidden from each other

(b) The exposed station problem.

B is exposed to transmission from A

IEEE 802.11 MAC Protocol: CSMA/CA

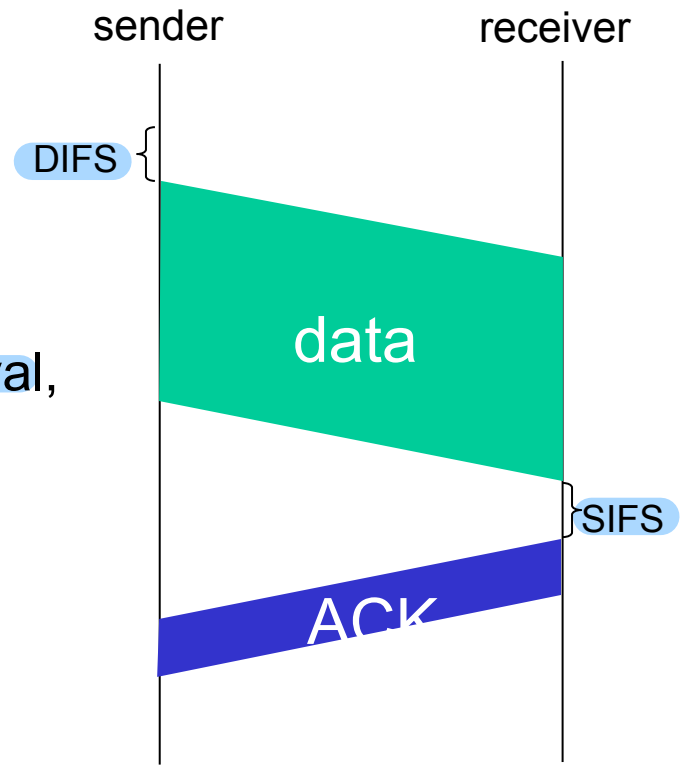
802.11 sender

1 if sense channel idle for **DIFS** then
  transmit entire frame (no CD)
2 if sense channel busy then
  start random backoff time
  timer counts down while channel idle
  transmit when timer expires
  if no ACK, increase random backoff interval, repeat 2

802.11 receiver

- if frame received OK
  return ACK after **SIFS** (ACK needed due to hidden terminal problem)

DCF = Distributed Coordination Function
DIFS = DCF InterFrame Space
SIFS = Short InterFrame Space



Avoiding collisions (more)

idea: allow sender to "reserve" channel rather than random access of data frames: avoid collisions of long data frames

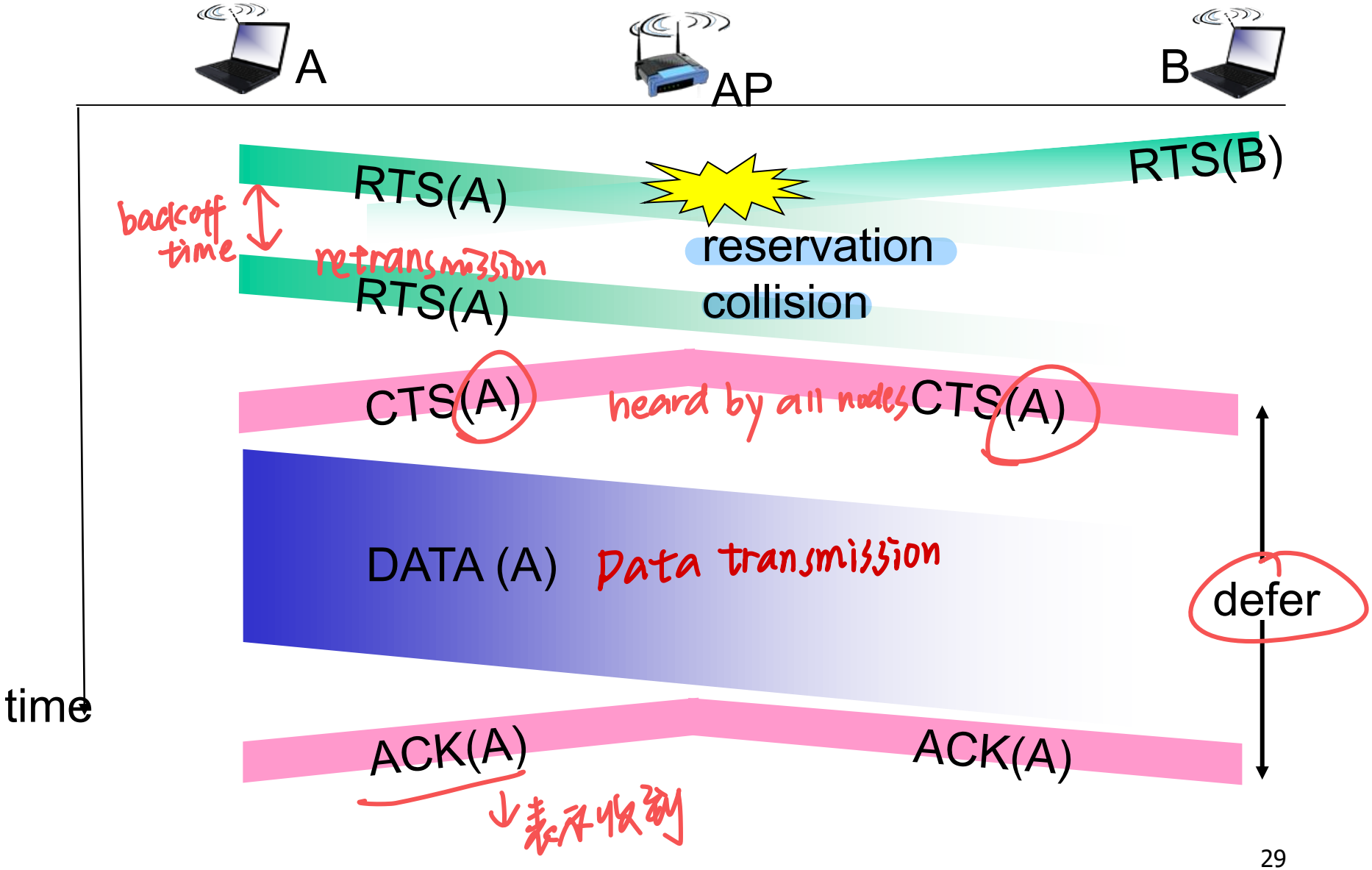
- sender first transmits *small* request-to-send (RTS) packets to base station (BS) using CSMA
  - RTSs may still collide with each other (but they're short)
- BS broadcasts **clear-to-send** CTS in response to RTS
- CTS heard by all nodes
  - sender transmits data frame
  - other stations defer transmissions be quiet

ACK / RTS / CTS

avoid data frame collisions completely using small reservation packets!

ACK = Acknowledgement is used to indicate that a message has been successfully received. RTS = Request to Send is used by a sender to ask for permission to send a message and reserve space on the channel. CTS = Clear to Send indicates the sender is allowed to send their message.

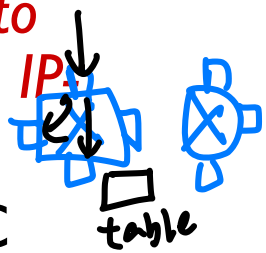
Collision Avoidance: RTS-CTS exchange



MAC addresses and ARP

- 32-bit IP address:
  - *network-layer* address for interface
  - used for layer 3 (network layer) forwarding
- MAC (or LAN or physical or Ethernet) address:
  - function: *used 'locally" to get frame from one interface to another physically-connected interface (same network, in IP addressing sense)*
  - 48 bit MAC address (for most LANs) burned in NIC ROM, also sometimes software settable
  - e.g.: 1A-2F-BB-76-09-AD

link layer address



unique

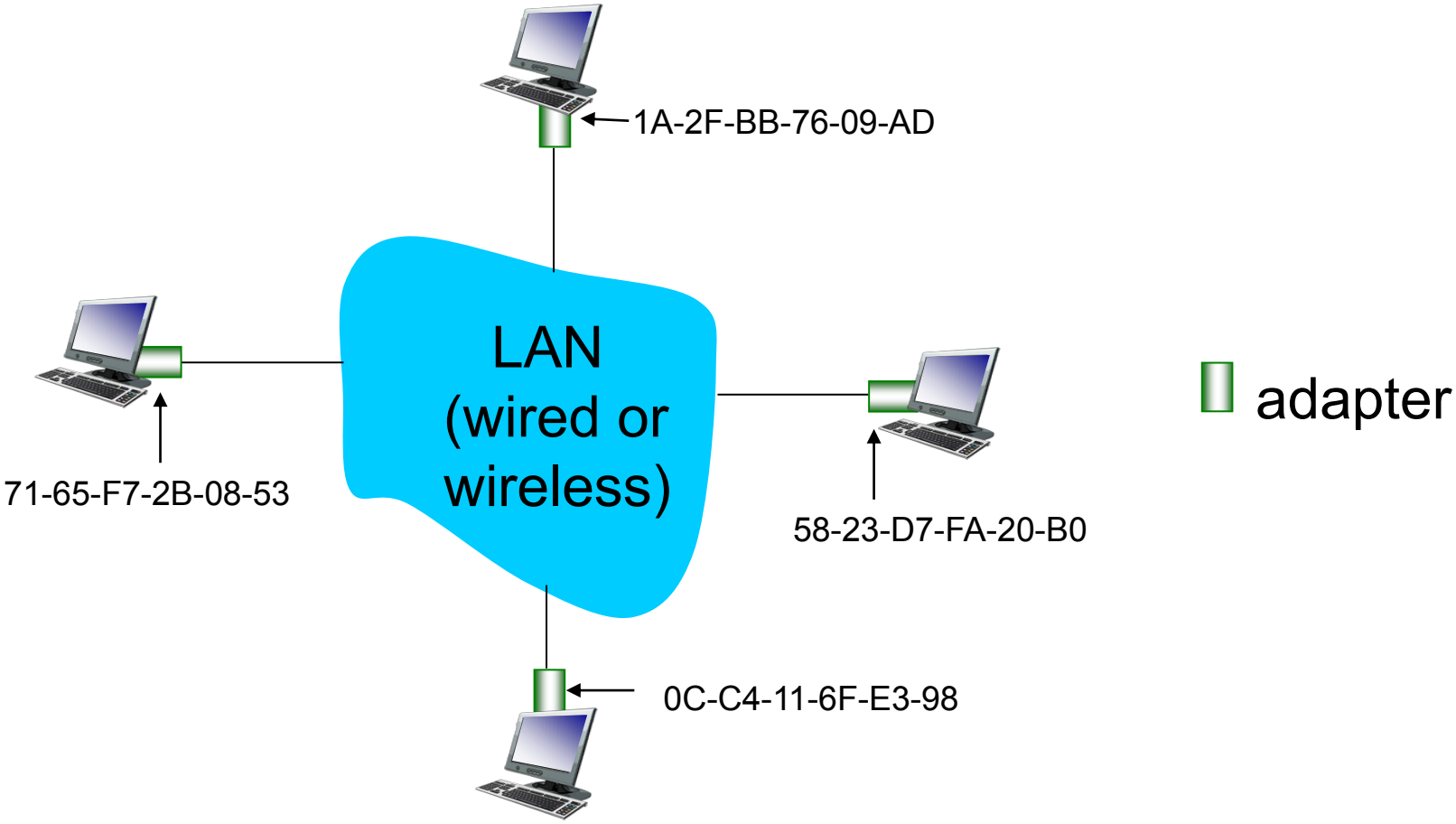
read-only memory

hexadecimal (base 16) notation
(each "numeral" represents 4 bits)

LAN addresses and ARP

no repeat

each adapter on LAN has unique *LAN* address

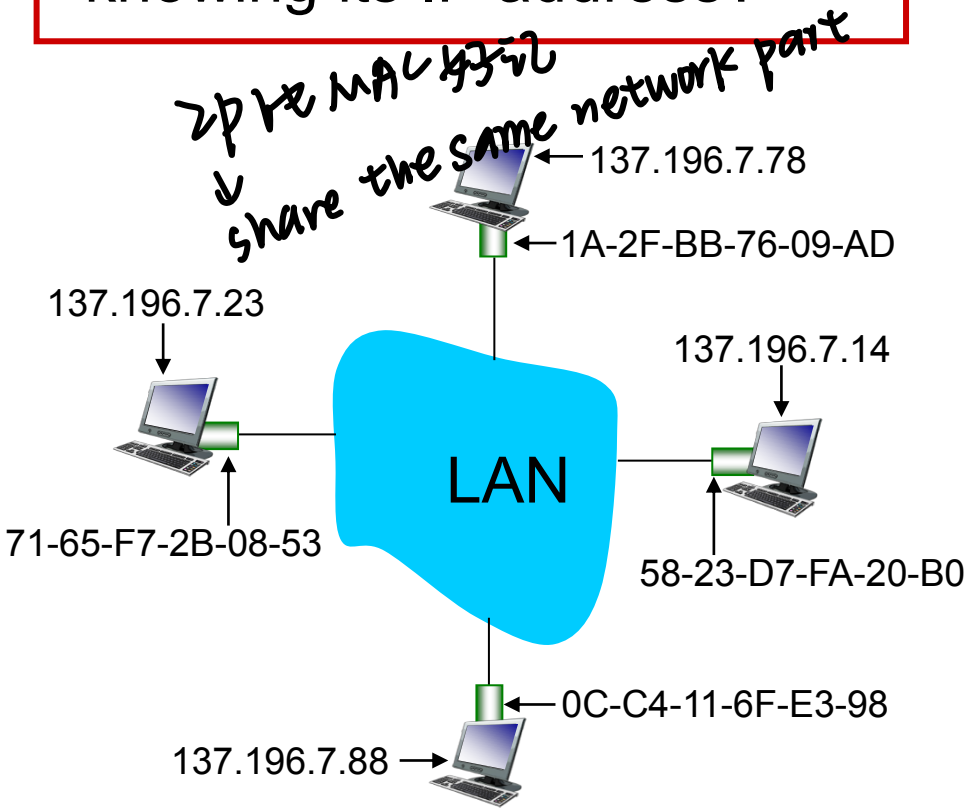


LAN addresses (more)

- MAC address allocation administered by IEEE
- manufacturer buys portion of MAC address space (to assure uniqueness) from IEEE
- analogy:
  - MAC address: like Social Security Number
  - IP address: like postal address
- MAC flat address ➜ portability
  - can move LAN card from one LAN to another but it must
- IP hierarchical address *not* portable changes its IP address
  - address depends on IP subnet to which node is attached according to the location of subnets.

ARP: address resolution protocol

Question: how to determine interface's MAC address, knowing its IP address?



ARP table: each IP node (host, router) on LAN has table

- IP/MAC address mappings for some LAN nodes:

  < IP address; MAC address; TTL>

- TTL (Time To Live): time after which address mapping will be forgotten (typically 20 min)

ARP protocol: same LAN

- A wants to send datagram to B
  - B's MAC address not in A's ARP table.
- A broadcasts ARP query packet, containing B's IP address
  - destination MAC address = FF-FF-FF-FF-FF-FF
  - all nodes on LAN receive ARP query
- B receives ARP packet, replies to A with its (B's) MAC address
  - frame sent to A's MAC address (unicast)

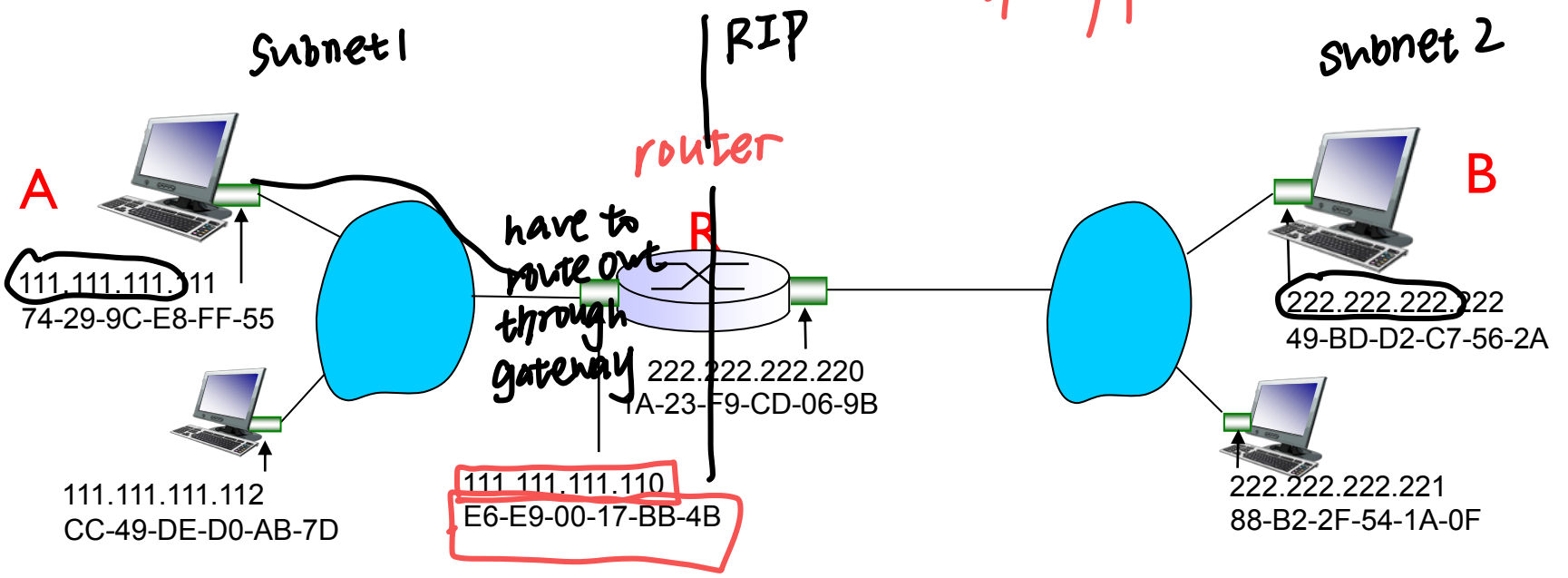
- A caches (saves) IP-to-MAC address pair in its ARP table until information becomes old (times out)
  - soft state: information that times out (goes away) unless refreshed
- ARP is "plug-and-play":
  - nodes create their ARP tables *without intervention from net administrator*

Addressing: routing to another LAN

walkthrough: send datagram from A to B via R

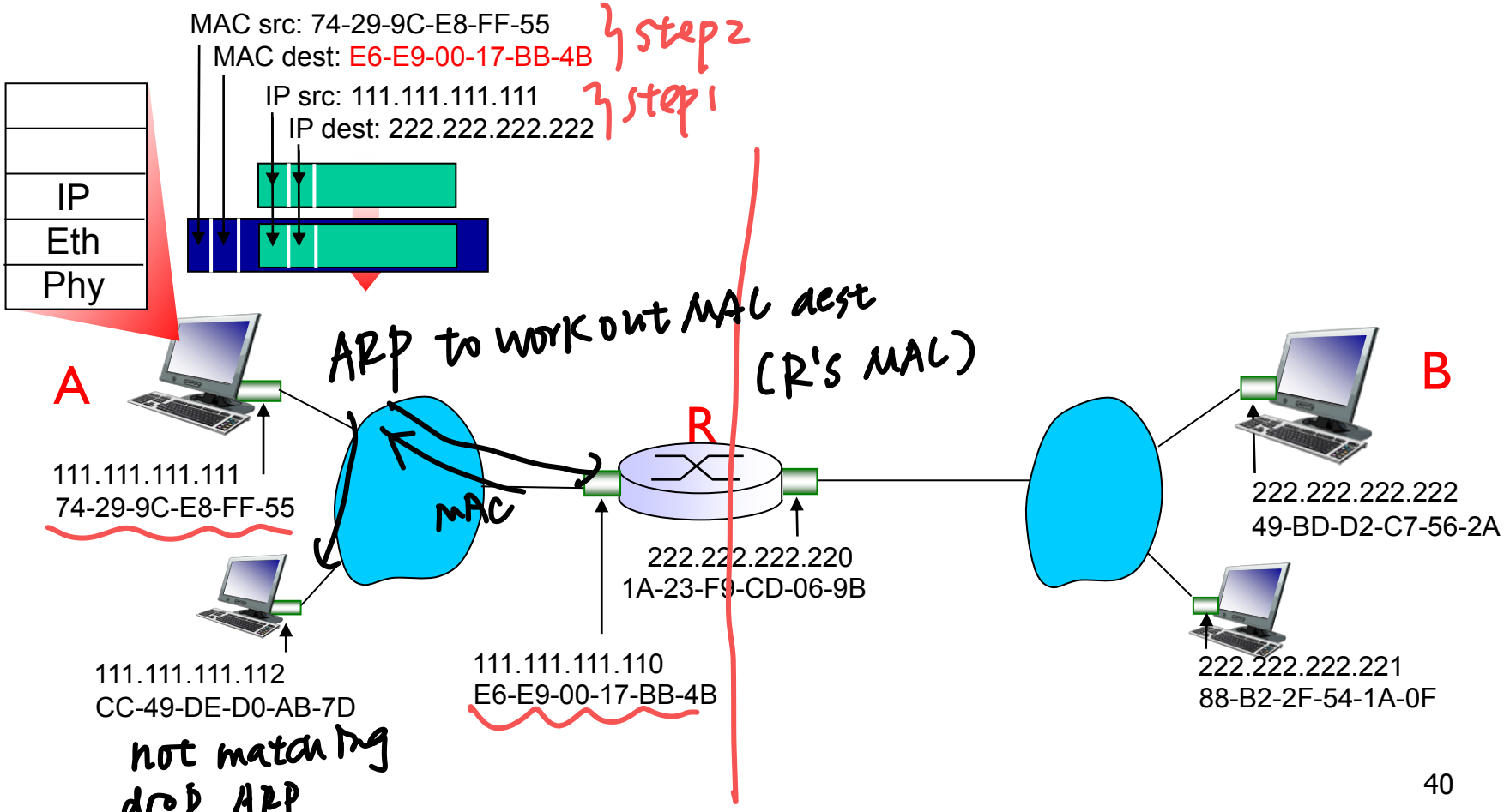
- focus on addressing – at IP (datagram) and MAC layer (frame)
- assume A knows B's IP address
- assume A knows IP address of first hop router, R (how?)
- assume A knows R's MAC address (how?)

111.111.111.110
"gateway" ARP
E6-E9-00-17-BB-4B
"query packet"



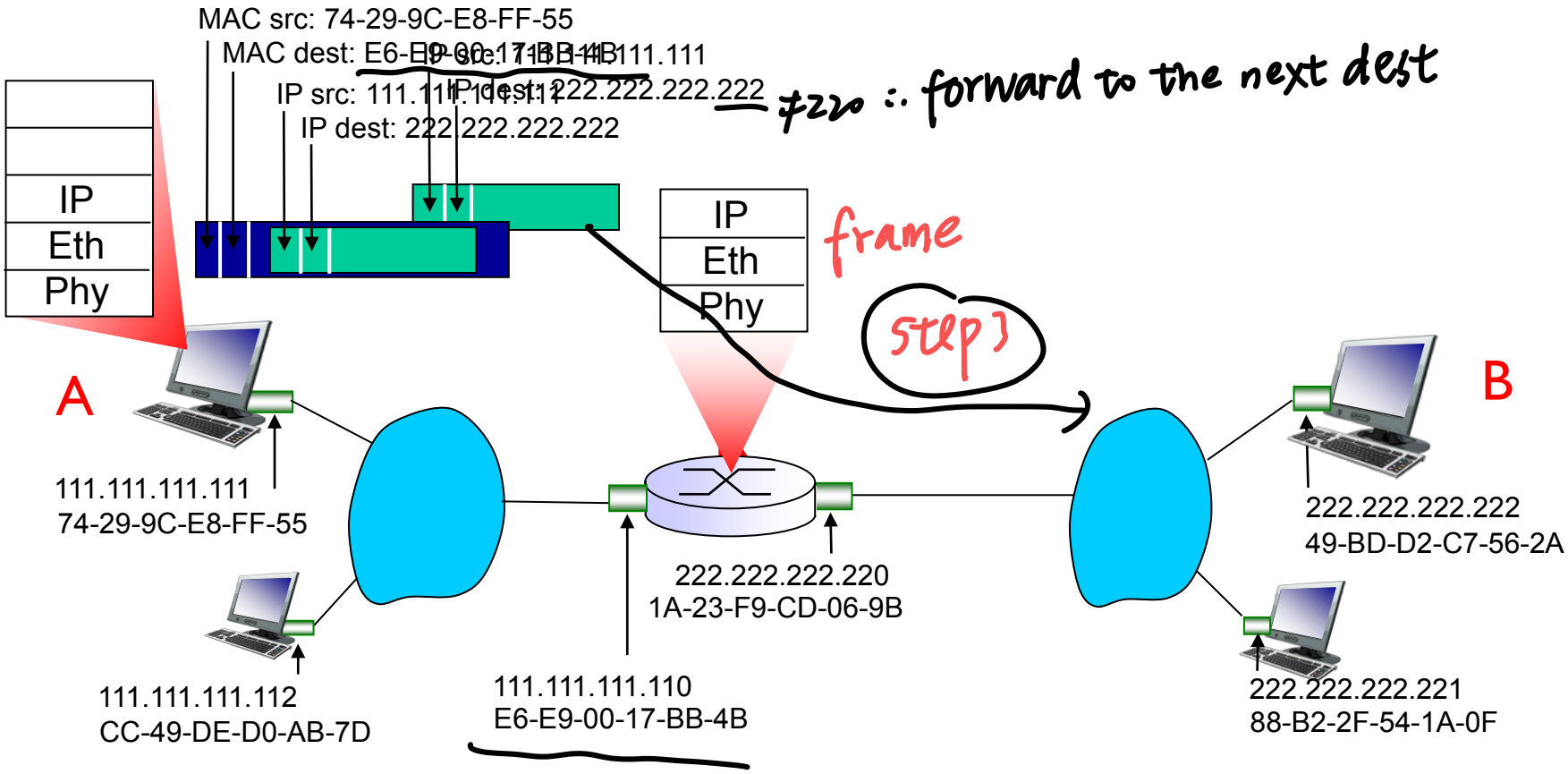
Addressing: routing to another LAN

- A creates IP datagram with IP source A, destination B
- A creates link-layer frame with R's MAC address as destination address, frame contains A-to-B IP datagram



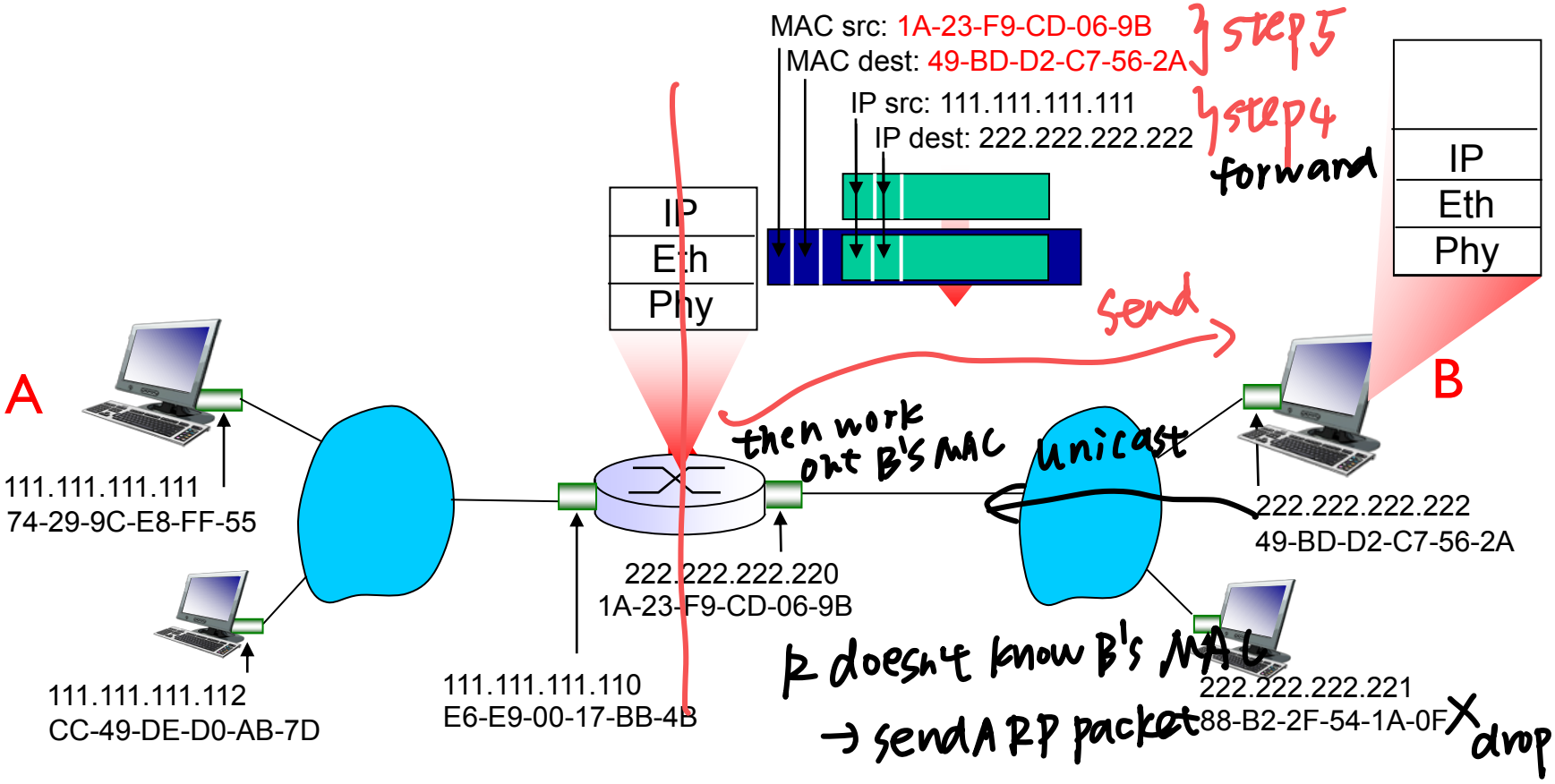
Addressing: routing to another LAN

- frame sent from A to R
- frame received at R, datagram removed, passed up to IP



Addressing: routing to another LAN

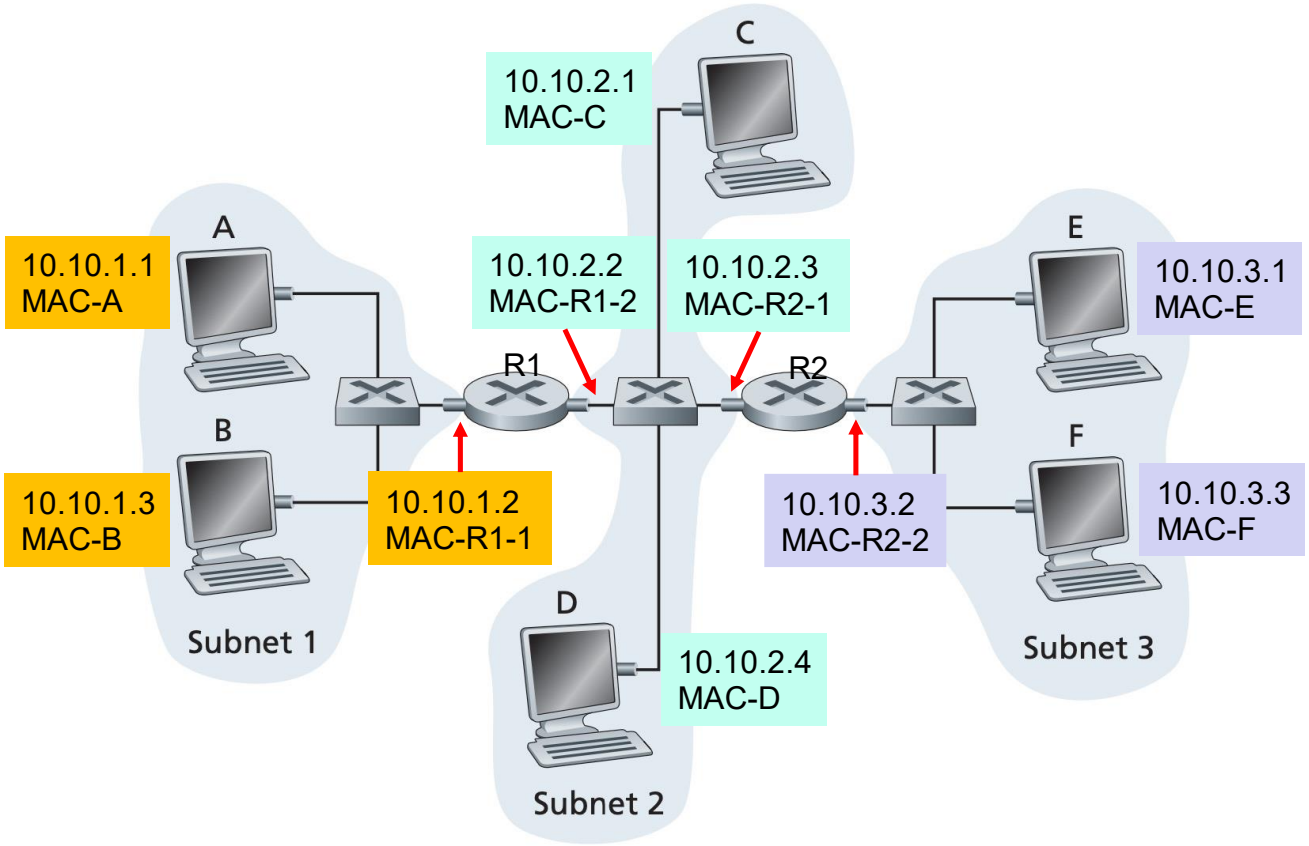
- R forwards datagram with IP source A, destination B
- R creates link-layer frame with B's MAC address as destination address, frame contains A-to-B IP datagram



Test your understanding

- Address Resolution Protocol, ARP is used to find … the destination MAC … address when only … the destination IP …. address is known

Question: Suppose ARP table of all hosts are up to date. What Host A must do to send an IP datagram to Host F.

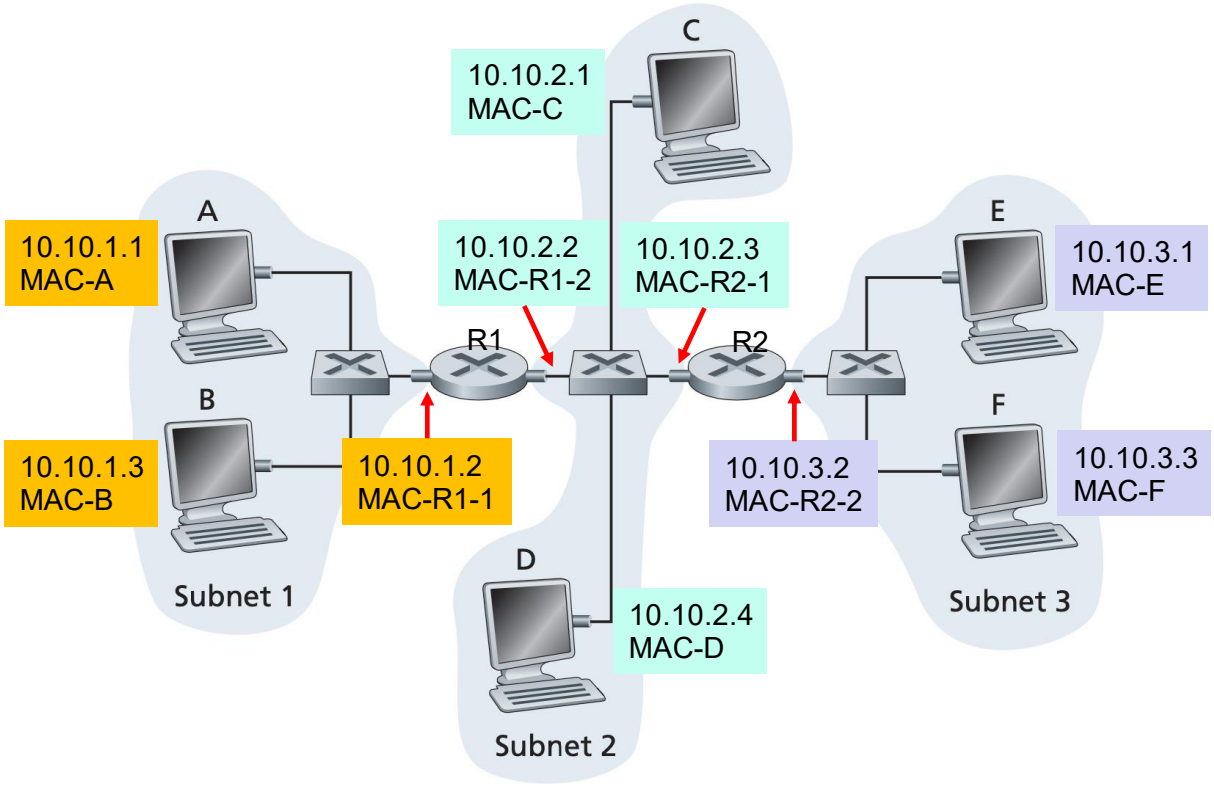


Question: Suppose all of the ARP tables are up to date. What Host A must do to send an IP datagram to Host F.

Answer: Host A create IP datagram containing its source IP address (10.10.1.1) and the E's destination IP (10.10.3.3).

Forwarding table in A determines that the datagram should be routed to 10.10.1.2 (R1 gateway IP).

The adapter in A sends a frame (Ethernet packet) containing the R1's destination MAC address (MAC-R1-1) and its source MAC address (MAC-A) via its interface IP (10.10.1.1).

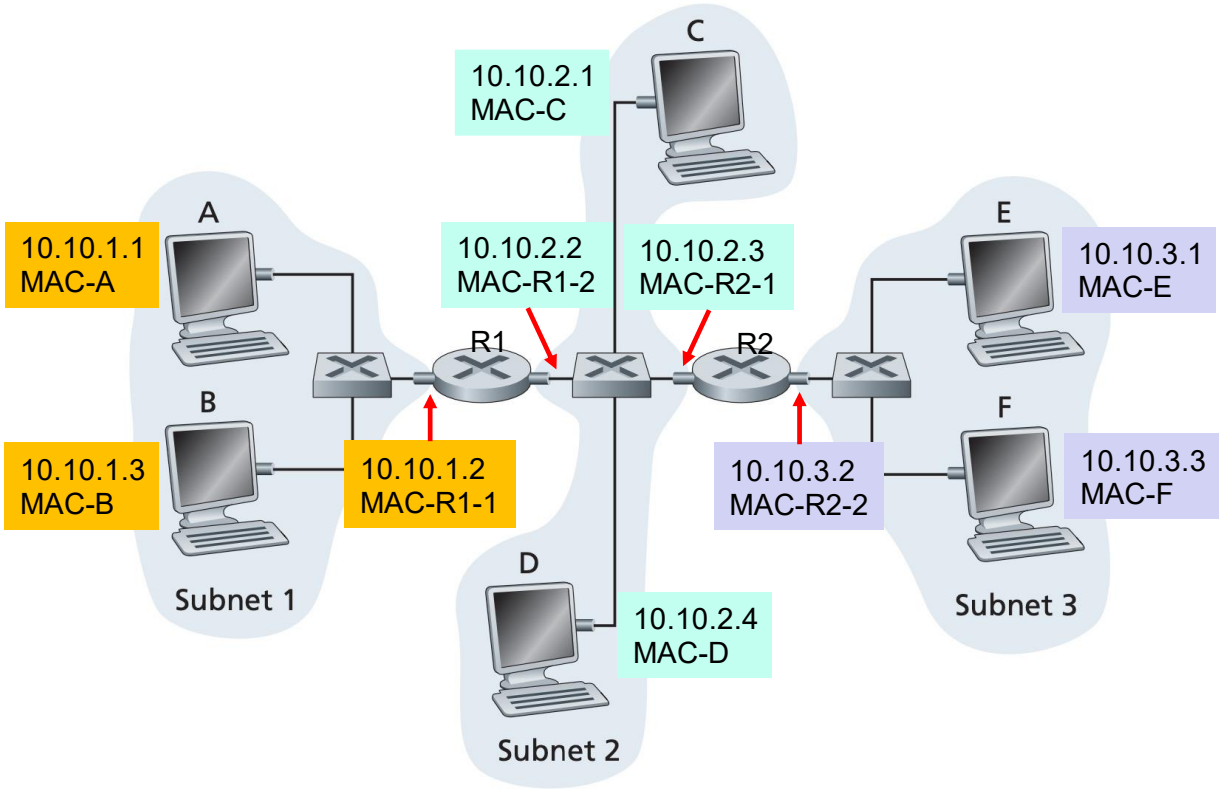


Question: What R1 would do when receiving a frame at interface (MAC-R1-1)?

Answer: R1 check the incoming frame if the frame's destination MAC address is matching its own MAC address (MAC-R1-1), and extract the IP datagram.

Forwarding table in R1 determines that the datagram should be routed to 10.10.2.3.

The adapter in R1 sends a frame (Ethernet packet) containing the R2's destination MAC address (MAC-R2-1) and its source MAC address (MAC-R1-2) via its interface with IP address (10.10.2.2)

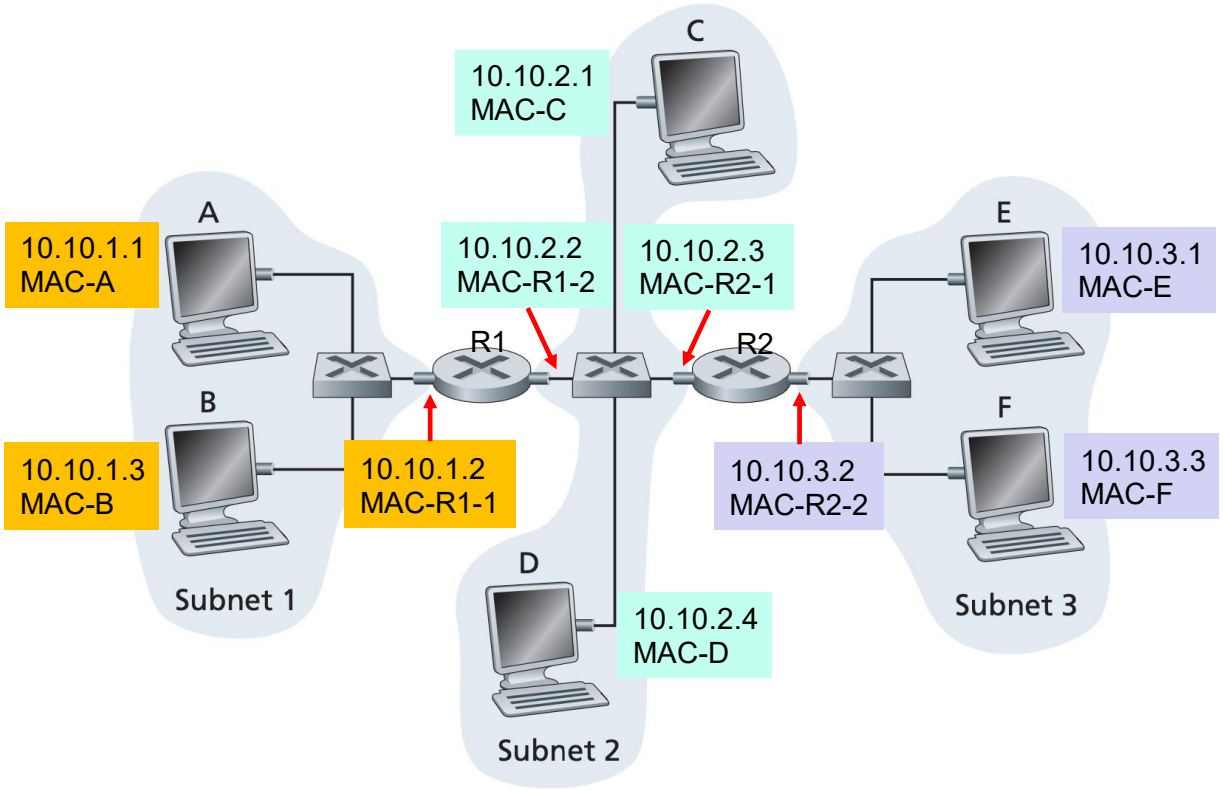


Question: What R2 would do when receiving a frame at interface (MAC-R2-1)?

Answer: R2 check the incoming frame if the frame's destination MAC address is matching its own MAC address (MAC-R2-1), and extract the IP datagram.

Forwarding table in R1 determines that the datagram should be routed to 10.10.3.3 (Host F).

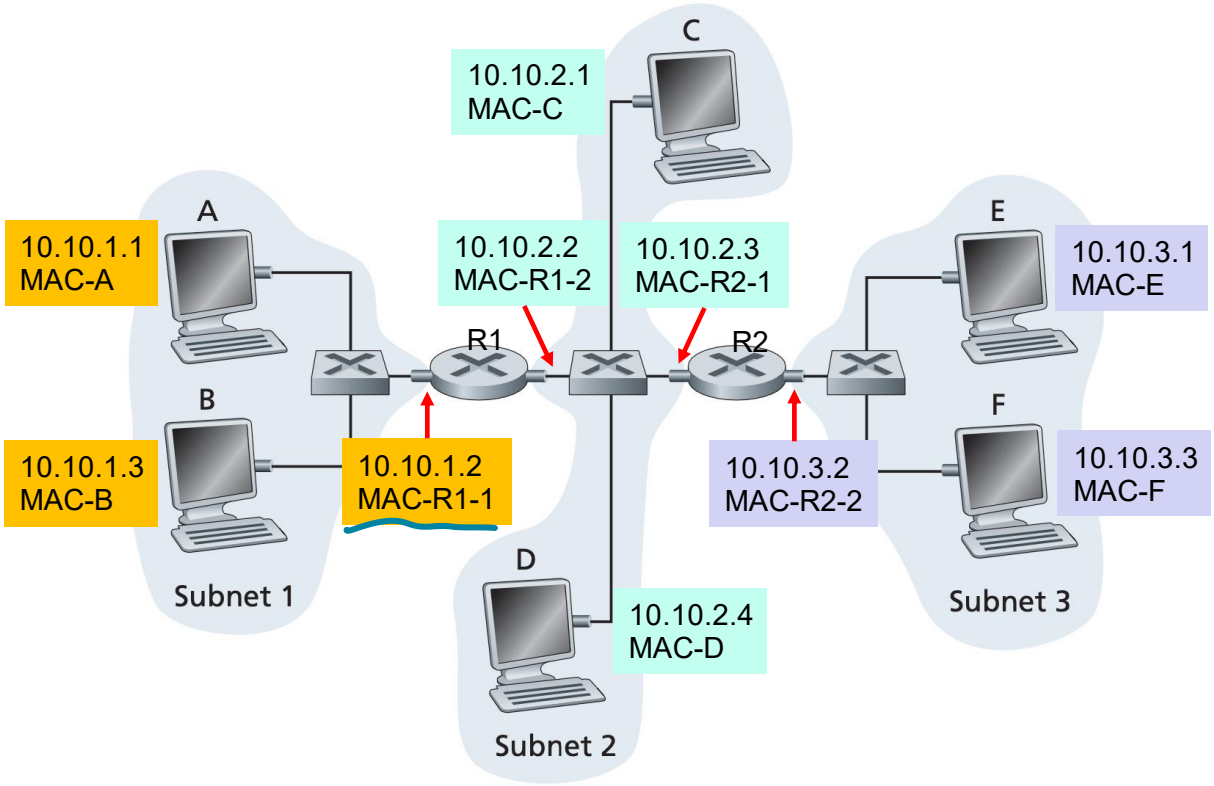
The adapter in R2 sends a frame (Ethernet packet) containing the Host F's destination MAC address (MAC-F) and its source MAC address (MAC-R2-2) via its interface with IP address (10.10.3.2)



Question: Assuming ARP table of Host A is empty, what Host A must do to send an IP datagram to Host F.

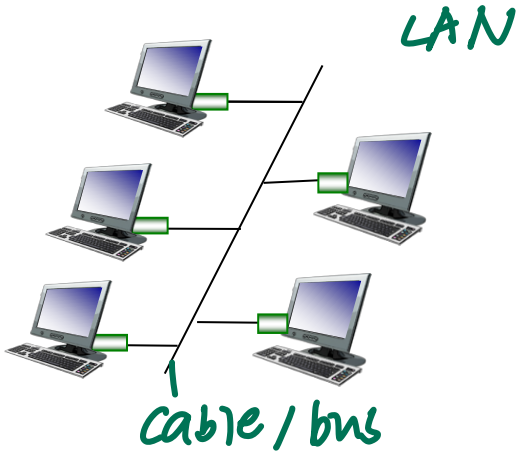
Answer: Host A must now determine the MAC address of R1 gateway interface IP (10.10.1.2). A sends out an ARP query packet within a broadcast Ethernet frame to MAC address (FF-FF-FF-FF-FF-FF)

R1 receives the query packet and sends to Host A an ARP response packet. This ARP response packet is carried by a frame with MAC destination address (MAC-R1-1).



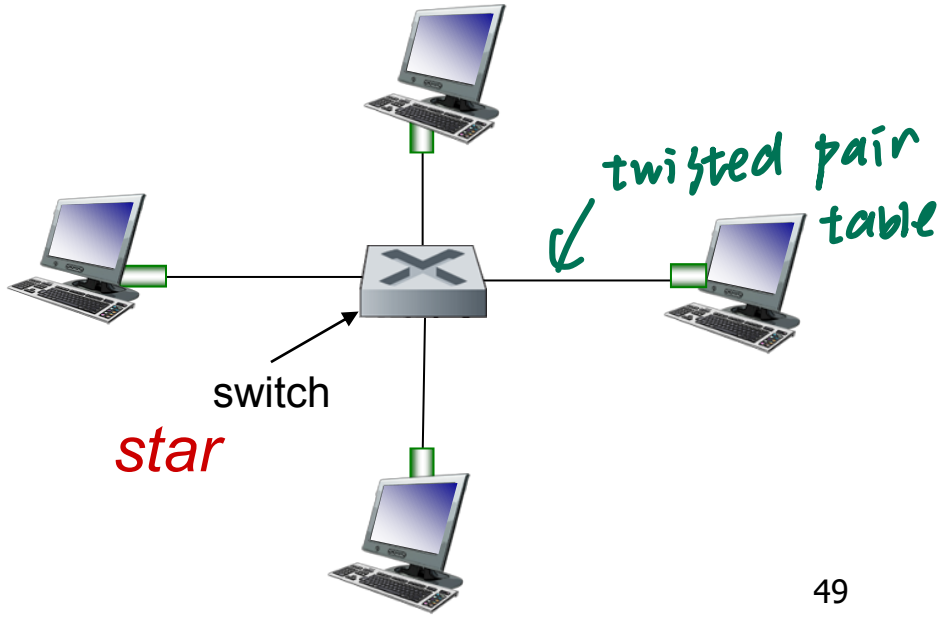
Ethernet: physical topology

- *bus:* popular through mid 90s
  - all nodes in same collision domain (can collide with each other)
- *star:* prevails today
  - active *switch* in center
  - each "spoke" runs a (separate) Ethernet protocol (nodes do not collide with each other)



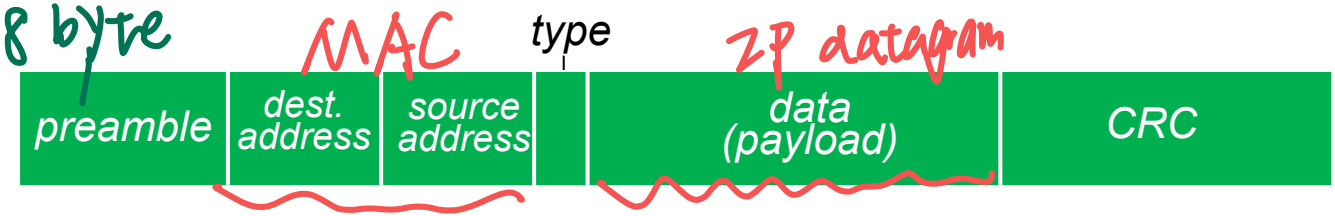
bus: coaxial cable

all nodes in same domain



Ethernet frame structure

sending adapter encapsulates IP datagram (or other network layer protocol packet) in Ethernet frame



preamble:

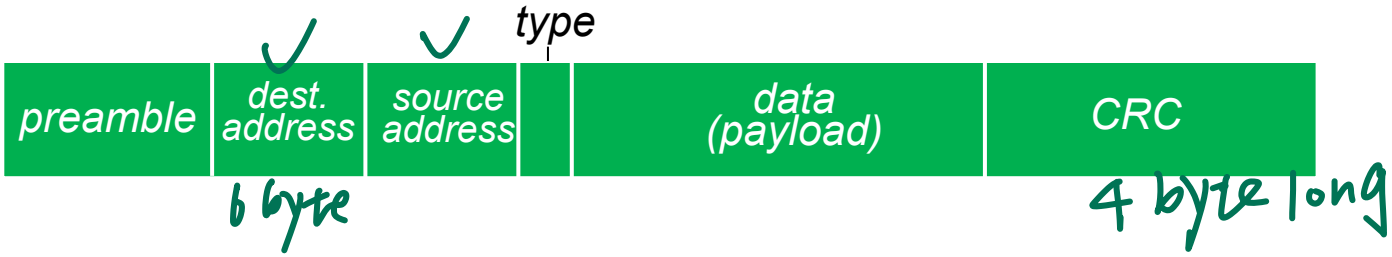
- 7 bytes with pattern 10101010 followed by one byte with pattern 10101011
- used to synchronize receiver, sender clock rates

wake up the receiving adapters
synchronize clocks

applied for slotted ALOHA

Ethernet frame structure (more)

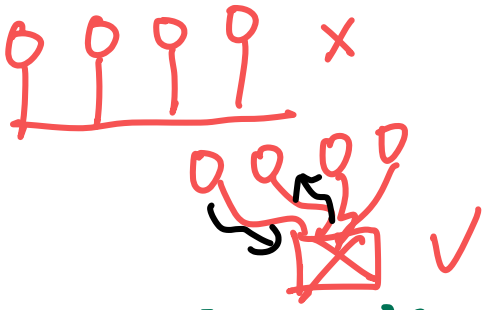
- *addresses:* 6 byte source, destination MAC addresses
  - if adapter receives frame with matching destination address, or with broadcast address (e.g. ARP packet), it passes data in frame to network layer protocol
  - otherwise, adapter discards frame
- *type:* indicates higher layer protocol (mostly IP but others possible, e.g., Novell IPX, AppleTalk)
- *CRC:* cyclic redundancy check at receiver
  - error detected: frame is dropped



Ethernet: unreliable, connectionless

- *connectionless:* no handshaking between sending and receiving NICs
- *unreliable:* receiving NIC doesn't send ACK or NACK to sending NIC
  - data in dropped frames recovered only if initial sender uses higher layer rdt (e.g., TCP), otherwise dropped data lost
- Ethernet's MAC protocol: unslotted *CSMA/CD with binary backoff*
- Various types of wired MAC protocol e.g.
  - Copper (100BASE-TX, 100BASE-T2, 100BASE-T4)
  - Optical (100BASE-SX, 100BASE-FX, 100BASE-BX)

Ethernet switch



- link-layer device: takes an *active* role
  - store, forward Ethernet frames
  - examine incoming frame's MAC address, selectively forward frame to one-or-more outgoing links when frame is to be forwarded on segment, uses CSMA/CD to access segment
- *transparent*
  - hosts are unaware of presence of switches
- *plug-and-play, self-learning*
  - switches do not need to be configured

src MAC + dest MAC

using switching table

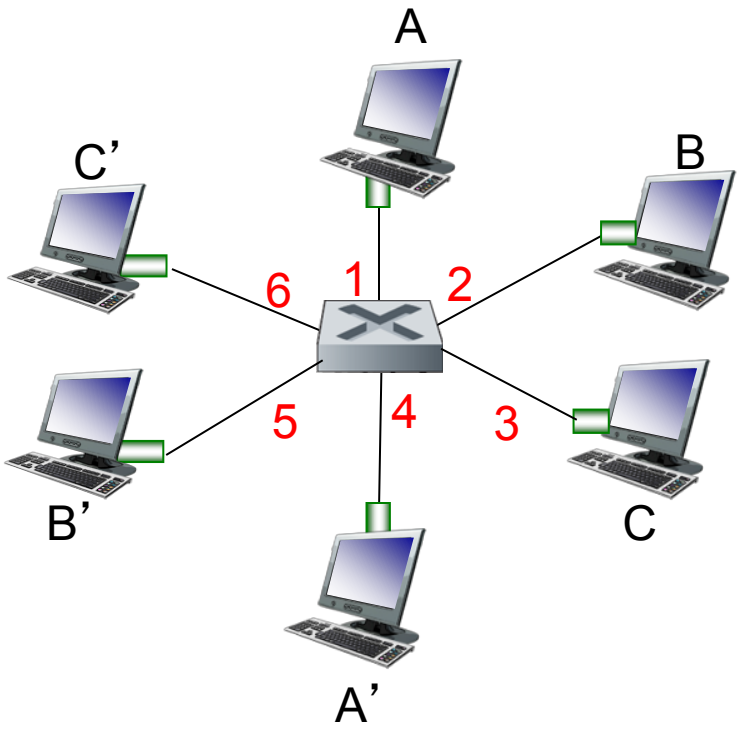
switching table

learn MAC address automatically.

allow:

Switch: *multiple* simultaneous transmissions

- hosts have dedicated, direct connection to switch
- switches buffer packets
- Ethernet protocol used on *each* incoming link, but no collisions; full duplex
  - each link is its own collision domain
- *switching:* A-to-A' and B-to-B' can transmit simultaneously, without collisions



switch with six interfaces (1,2,3,4,5,6)

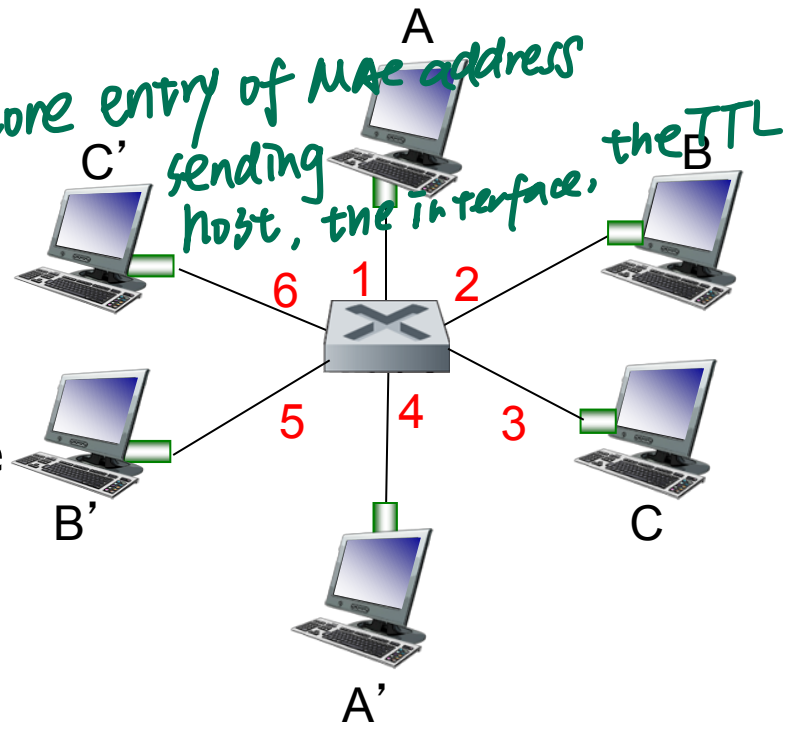
Switch forwarding table

Q: how does switch know A' reachable via interface 4, B' reachable via interface 5?

- A: each switch has a switch table, each entry:
  - (MAC address of host, interface to reach host, time stamp)
  - looks like a routing table!

store entry of MAC address of sending host, the interface, the TTL

有:



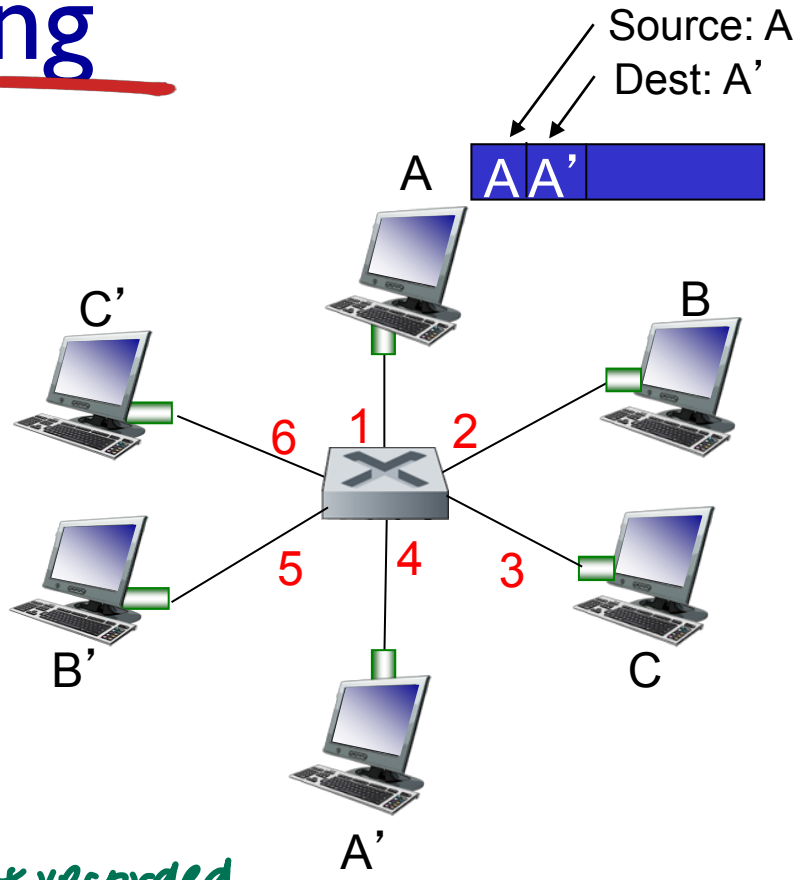
switch with six interfaces (1,2,3,4,5,6)

Q: how are entries created, maintained in switch table?

- something like a routing protocol?

Switch: self-learning

- switch *learns* which hosts can be reached through which interfaces
  - when frame received, switch "learns" location of sender: incoming LAN segment
  - records sender/location pair in switch table



All will get recorded

MAC addr	interface	TTL
A	*1*	*60*

Switch table (initially empty)

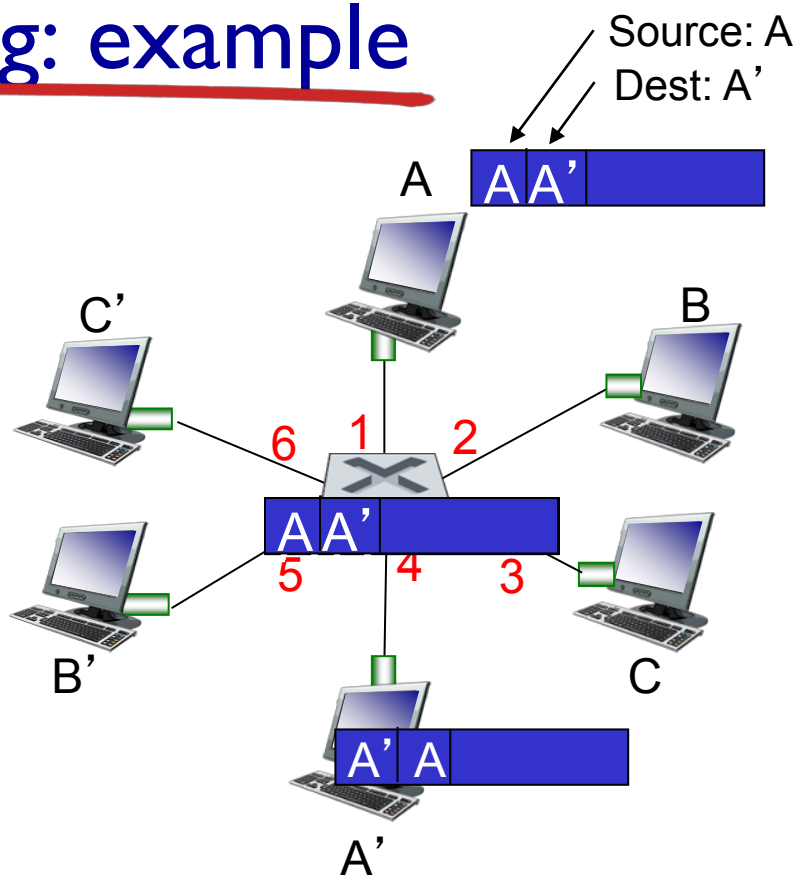
Switch: frame filtering/forwarding

when frame received at switch:

1. record incoming link, MAC address of sending host , interface
2. index switch table using MAC destination address
3. if entry found for destination
   then {
   if destination on segment from which frame arrived
   then drop frame no forwarding needed
   else forward frame on interface indicated by entry
   }
   else flood /* forward on all interfaces except arriving interface */

Self-learning, forwarding: example

- frame destination, A', location unknown: *flood*
- destination A location known: selectively send on just one link

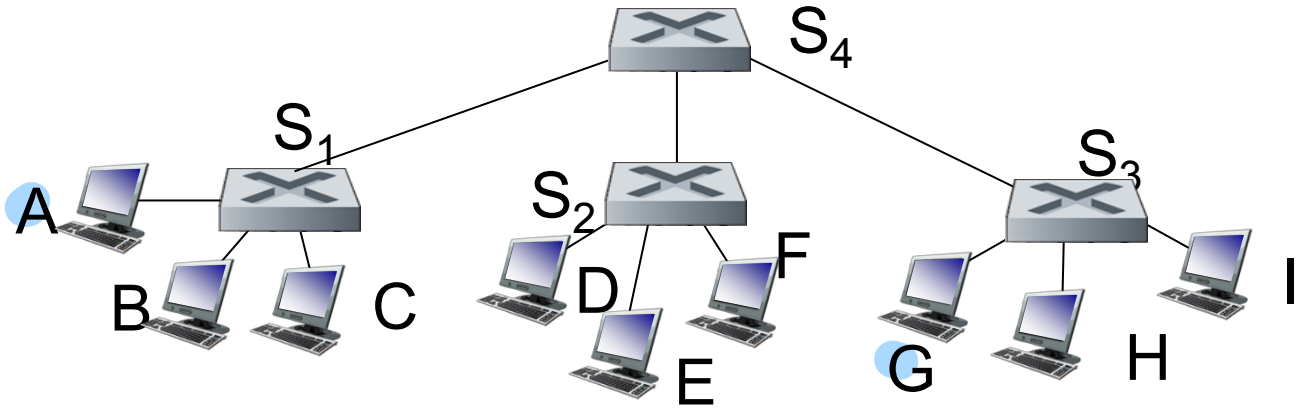


MAC addr	interface	TTL
A A'	1 4	60 60

switch table (initially empty)

Interconnecting switches

self-learning switches can be connected together:



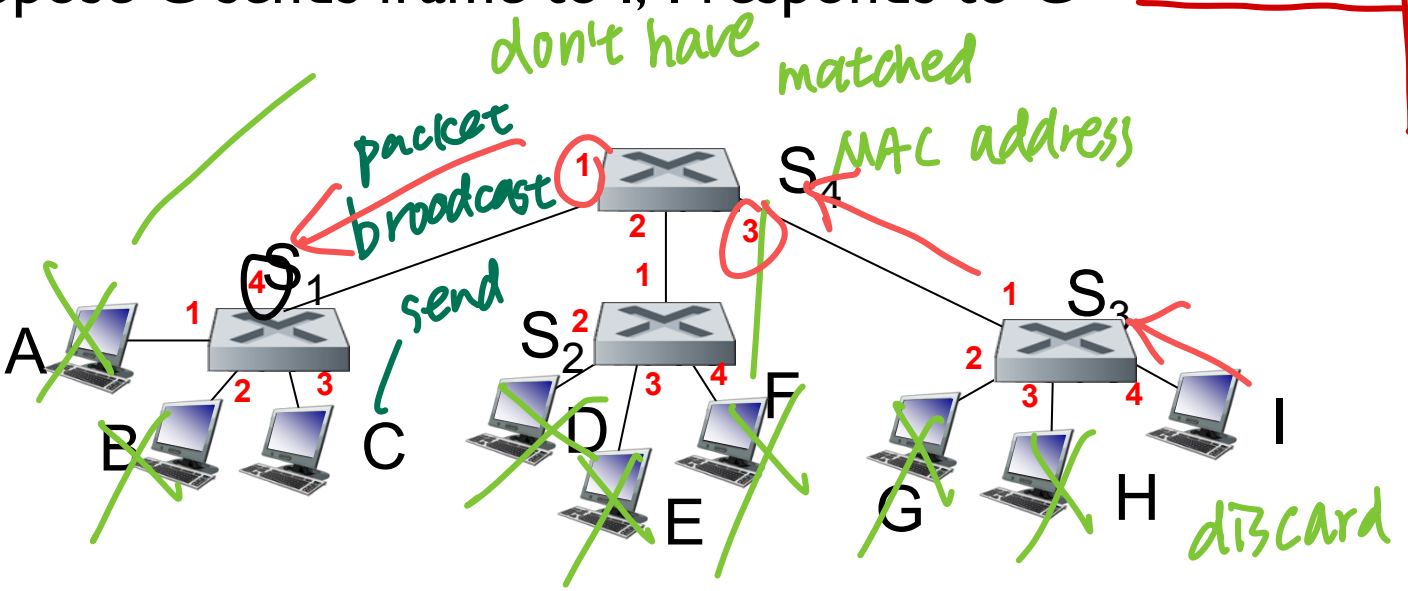
Q: sending from A to G - how does S_1 know to forward frame destined to G via S_4 and S_3?

- *A:* self learning! (works exactly the same as in single-switch case!)

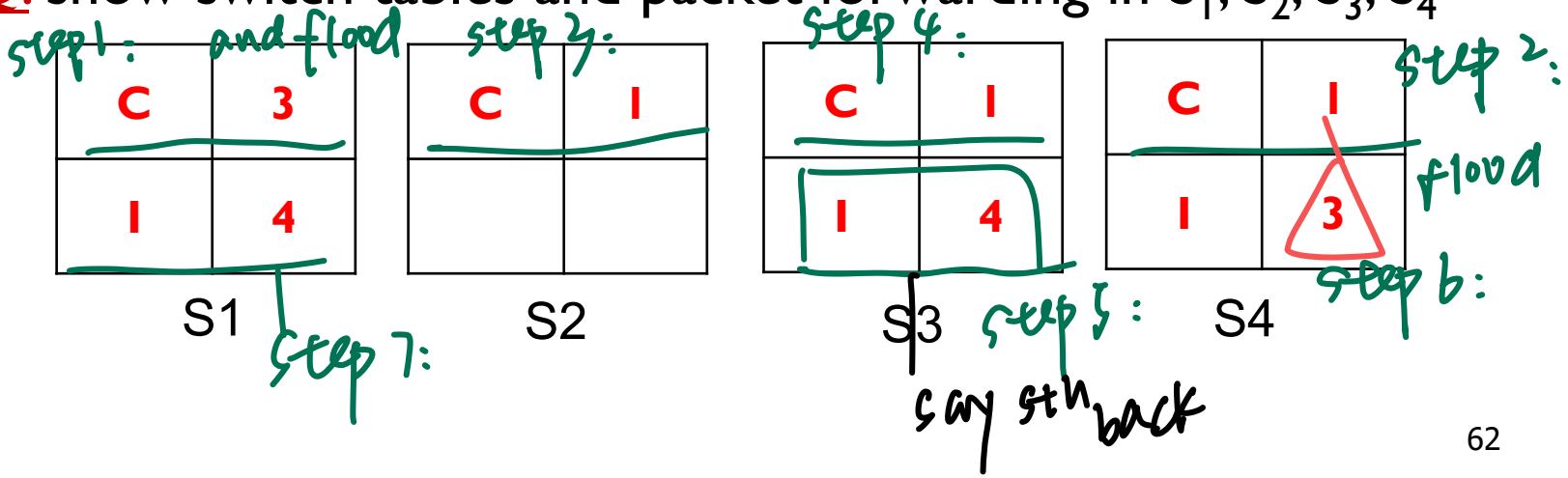
Self-learning multi-switch example

Suppose C sends frame to I, I responds to C

route | port



- Q: show switch tables and packet forwarding in S_1, S_2, S_3, S_4



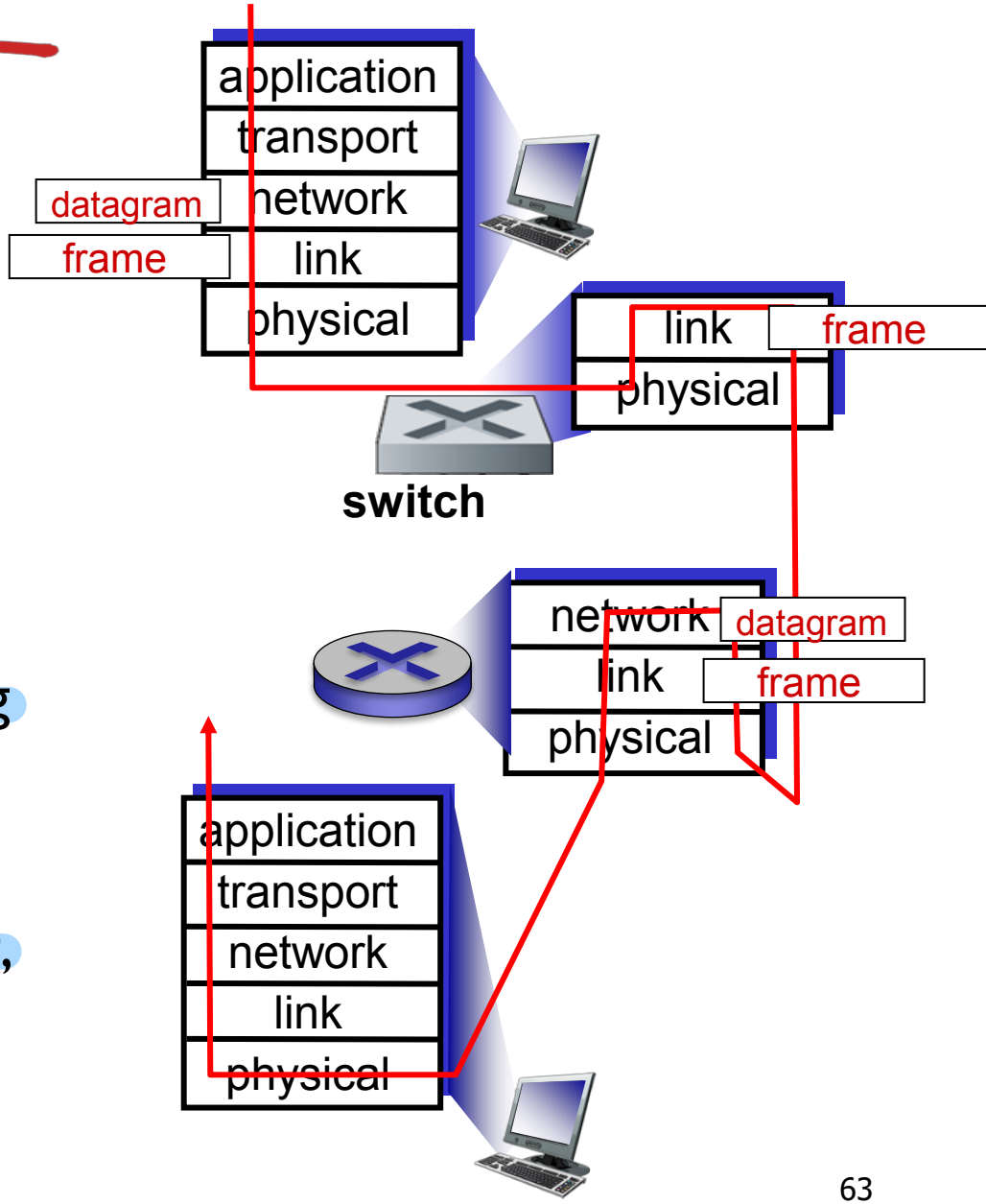
Switches vs. routers

both are store-and-forward:

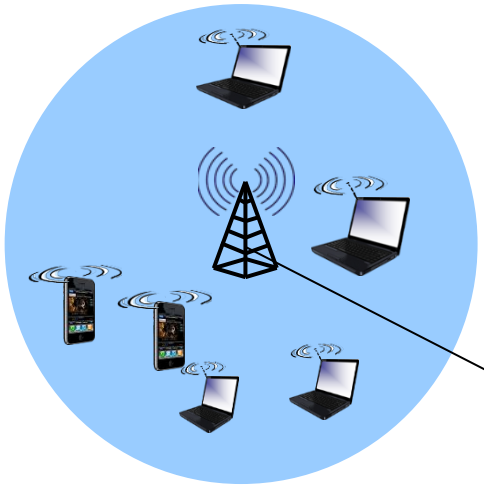
- *routers:* network-layer devices (examine network-layer headers)
- *switches*: link-layer devices (examine link-layer headers)

both have forwarding tables:

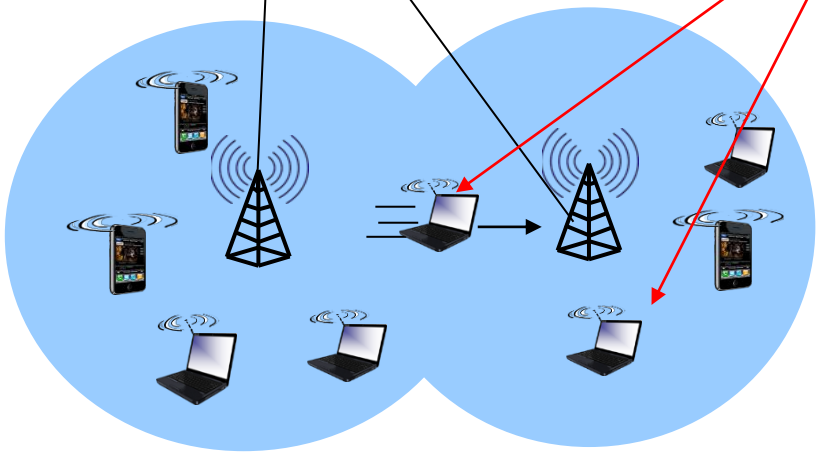
- *routers:* compute tables using routing algorithms, IP addresses
- *switches:* learn forwarding table using flooding, learning, MAC addresses



Elements of a wireless network



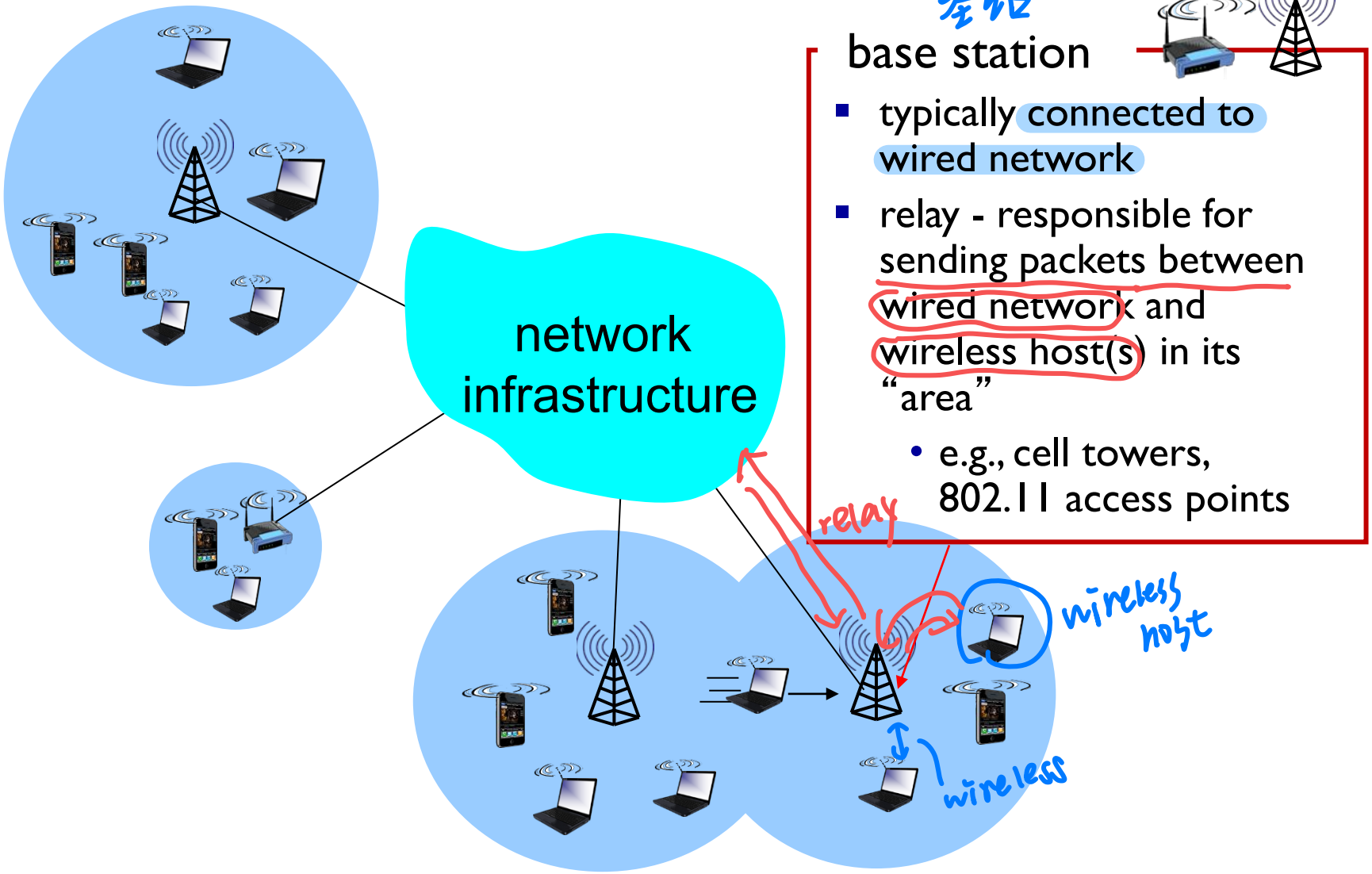
network infrastructure



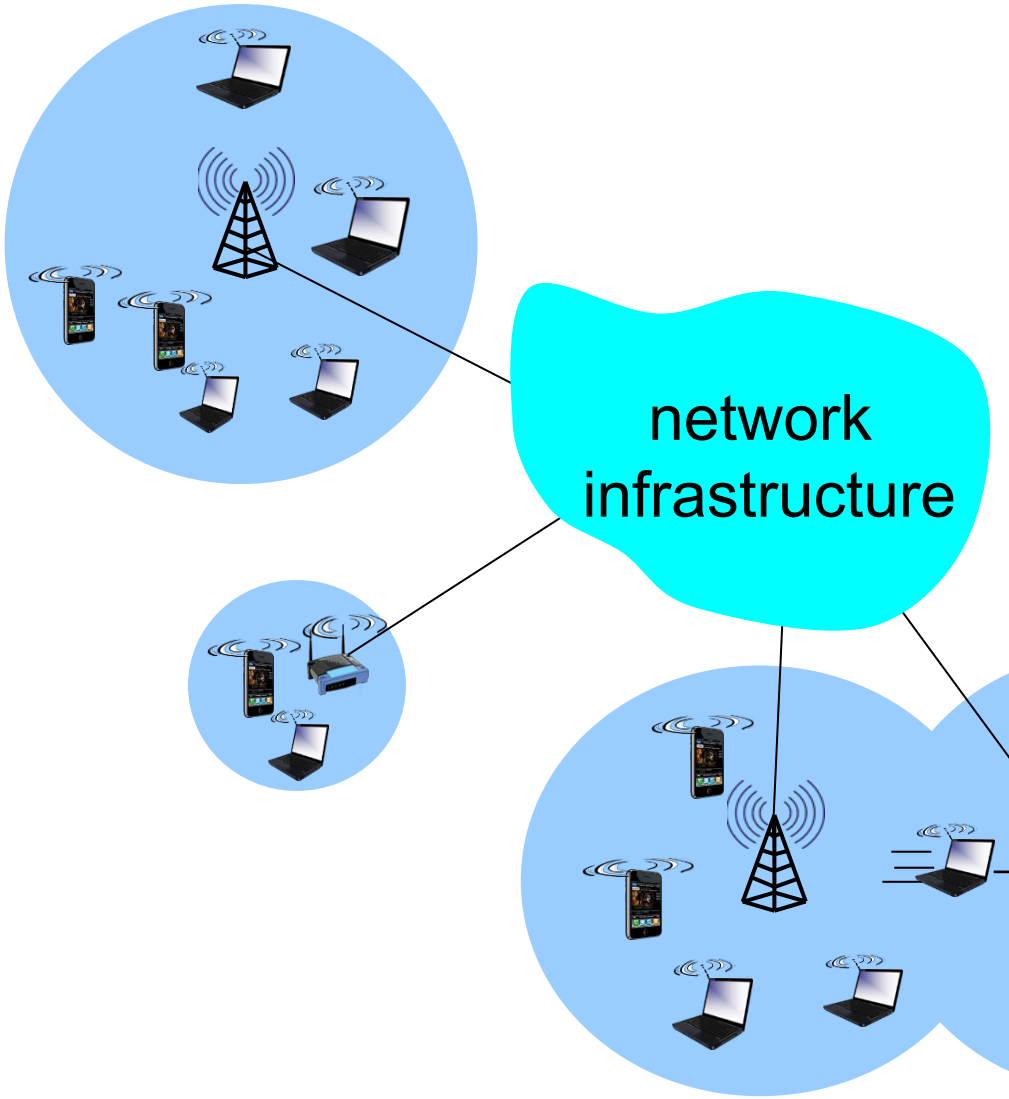
wireless hosts

- laptop, smartphone, desktop
- run applications
- may be stationary (non-mobile) or mobile
  - wireless does *not* always mean mobility

Elements of a wireless network



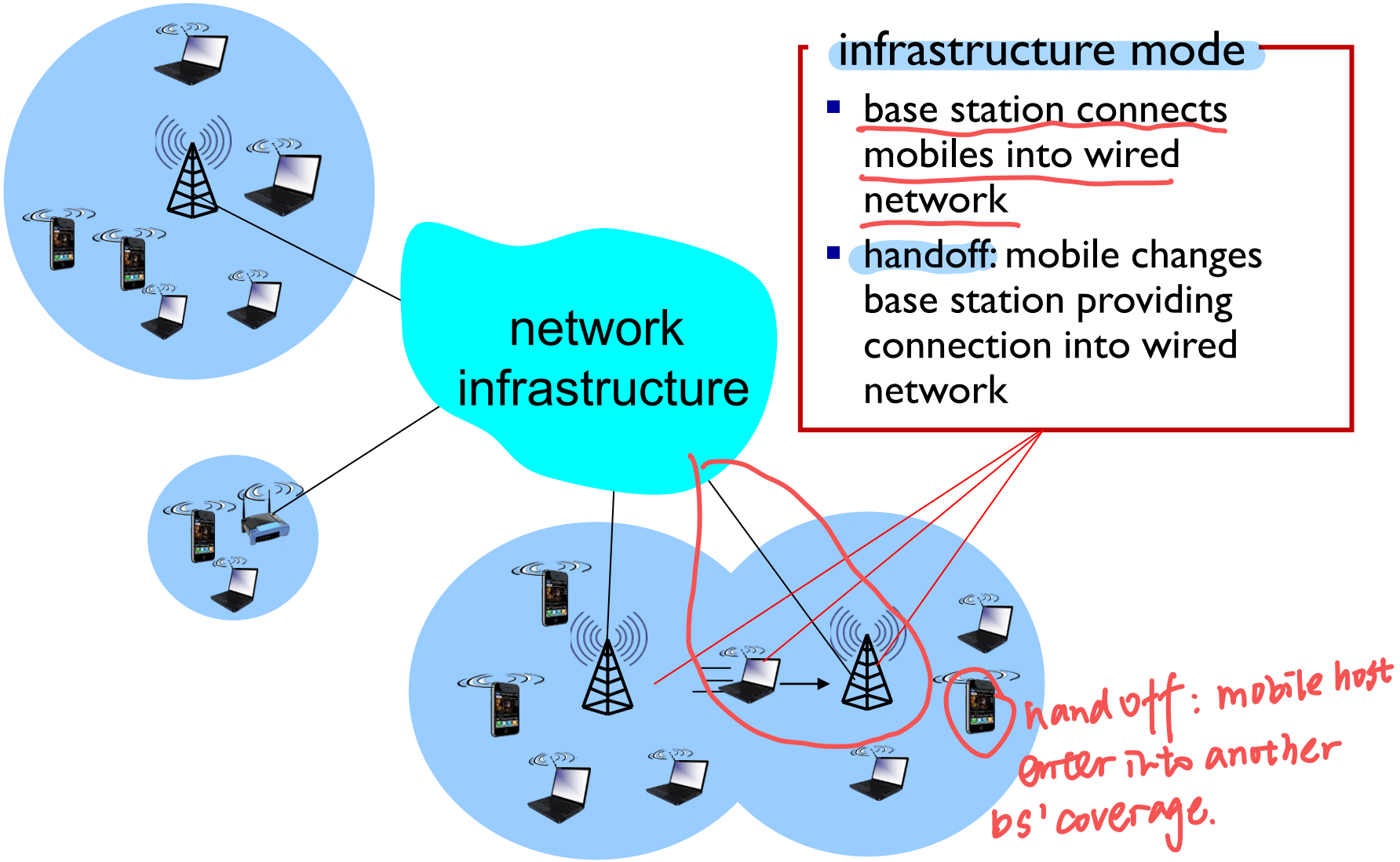
Elements of a wireless network



wireless link

- typically used to connect mobile(s) to base station
- also used as backbone link
- multiple access protocol coordinates link access
- various data rates, transmission distance

Elements of a wireless network

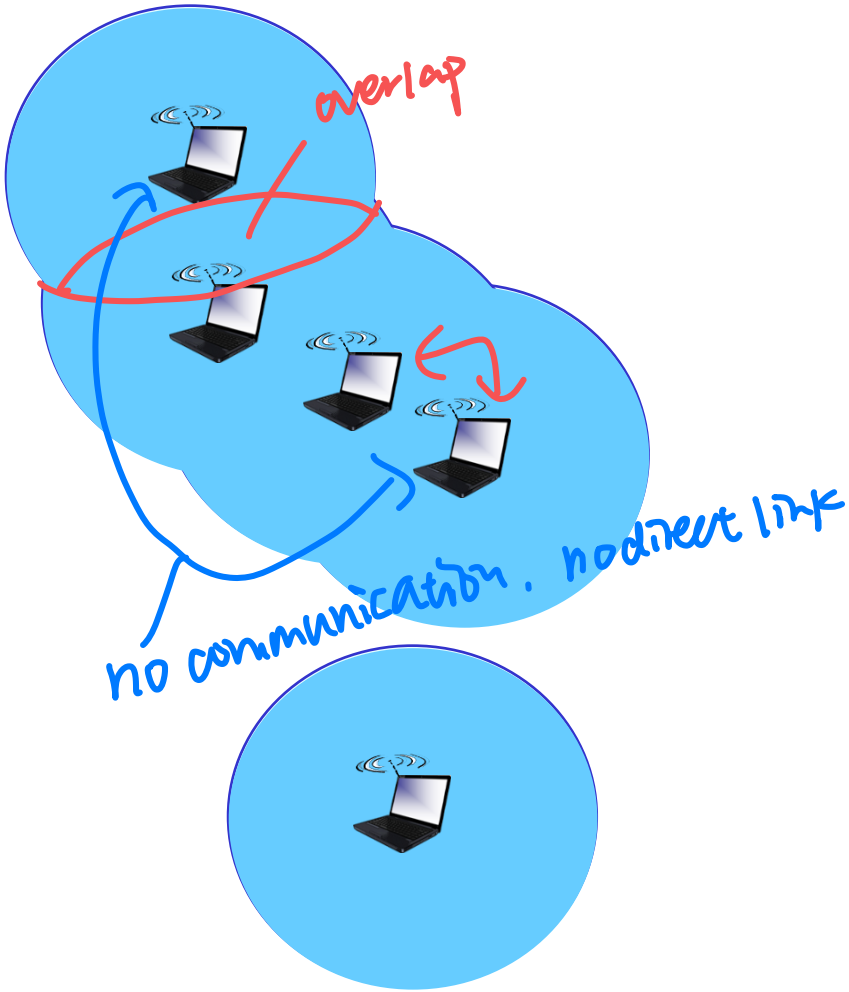


infrastructure mode

- base station connects mobiles into wired network
- handoff: mobile changes base station providing connection into wired network

handoff: mobile host enter into another bs's coverage.

Elements of a wireless network

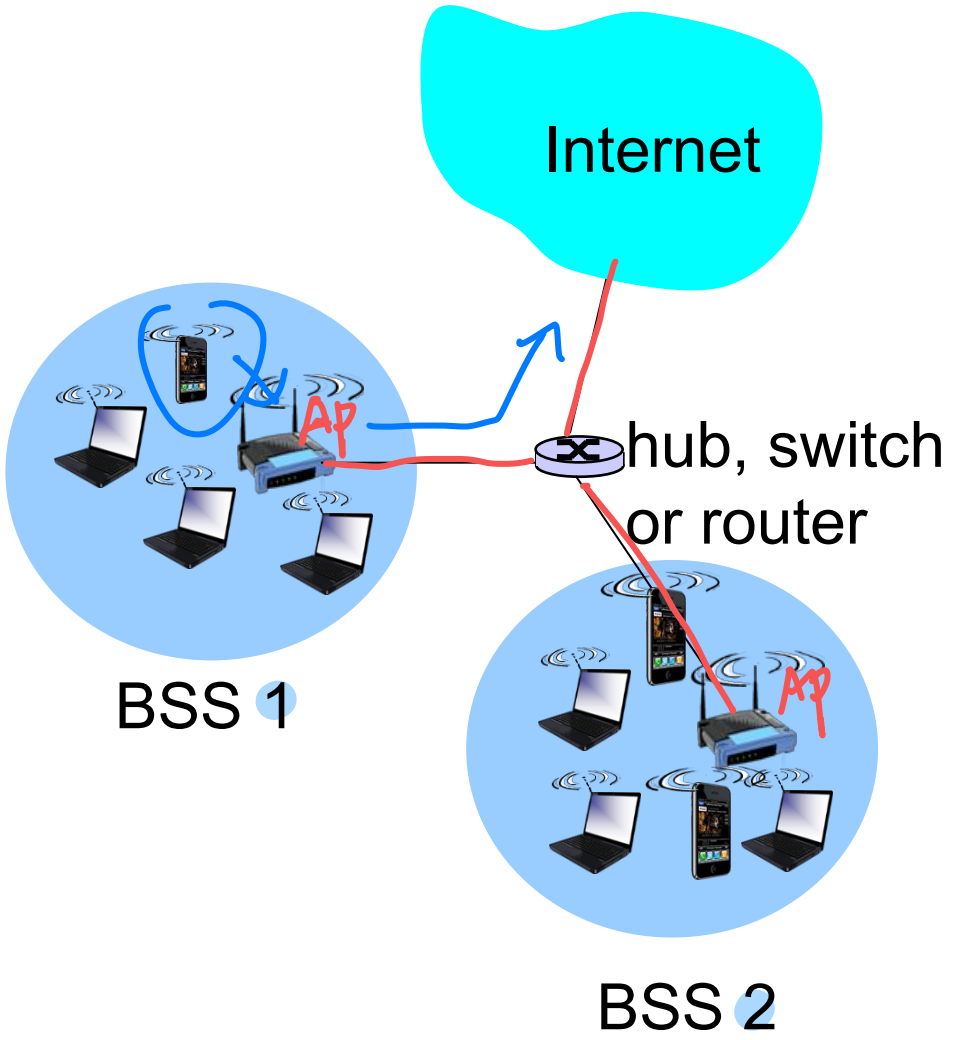


ad hoc mode

- no base stations
- nodes can only transmit to other nodes within link coverage
- nodes organize themselves into a network: route among themselves

form a path and router table

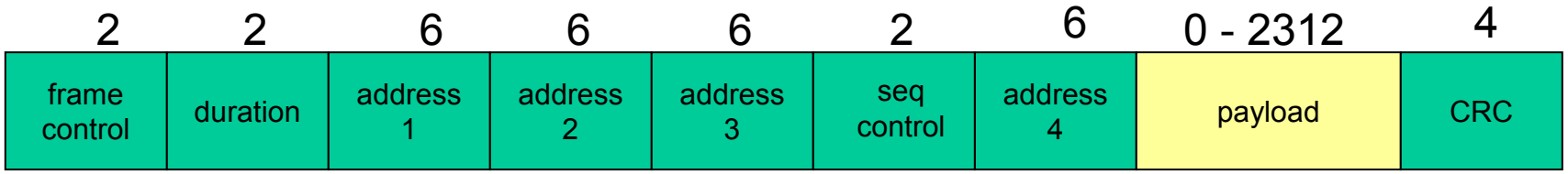
802.11 LAN architecture



- wireless host communicates with base station
  - base station = access point (AP)
- Basic Service Set (BSS) (aka "cell") in infrastructure mode contains:
  - wireless hosts
  - access point (AP): base station
  - ad hoc mode: hosts only

802.11 frame: addressing

4 MAC address field



Address 1: MAC address of wireless host or AP to receive this frame

Address 2: MAC address of wireless host or AP transmitting this frame

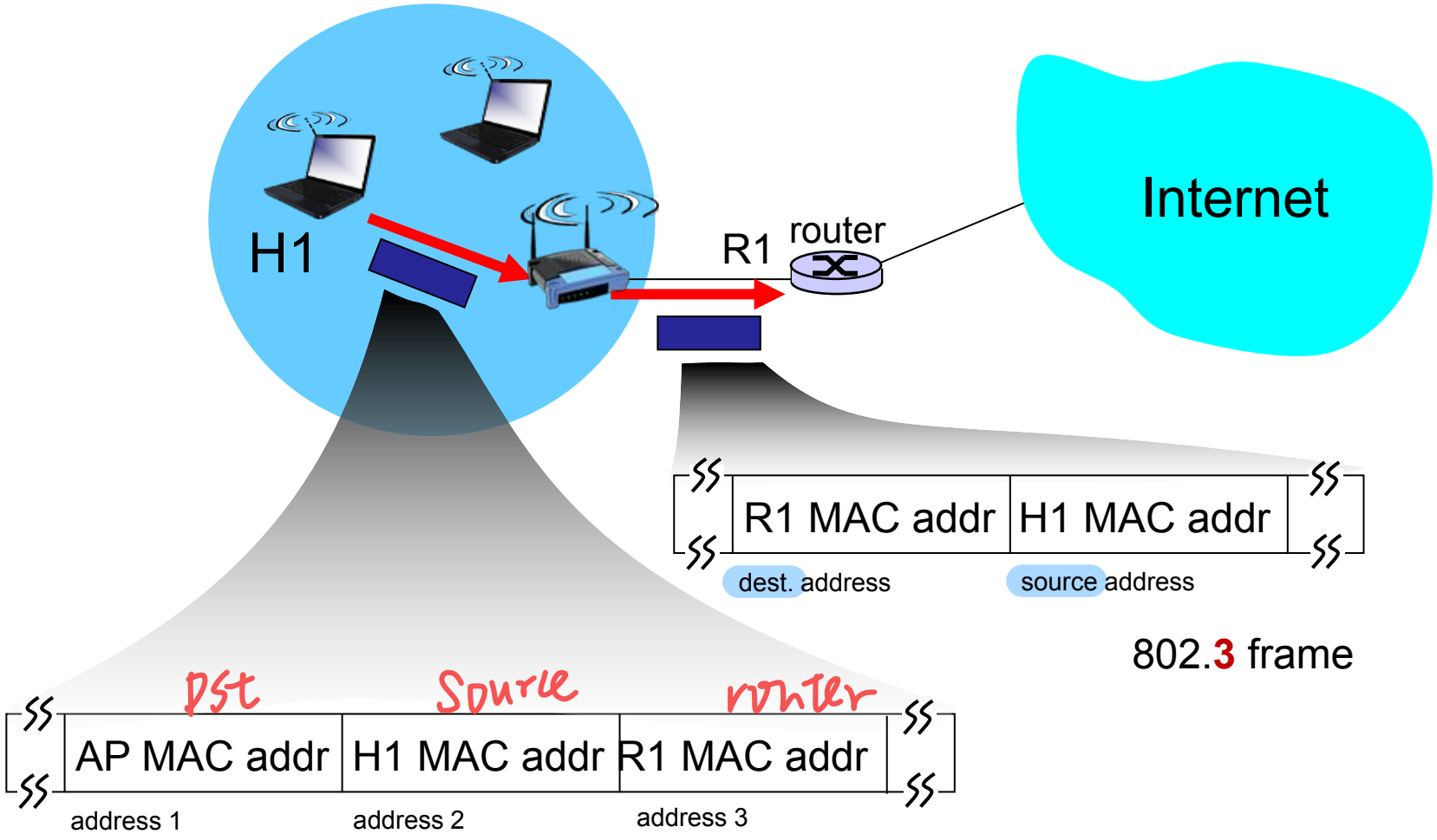
Address 3: MAC address of router interface to which AP is attached

Address 4: used only in ad hoc mode

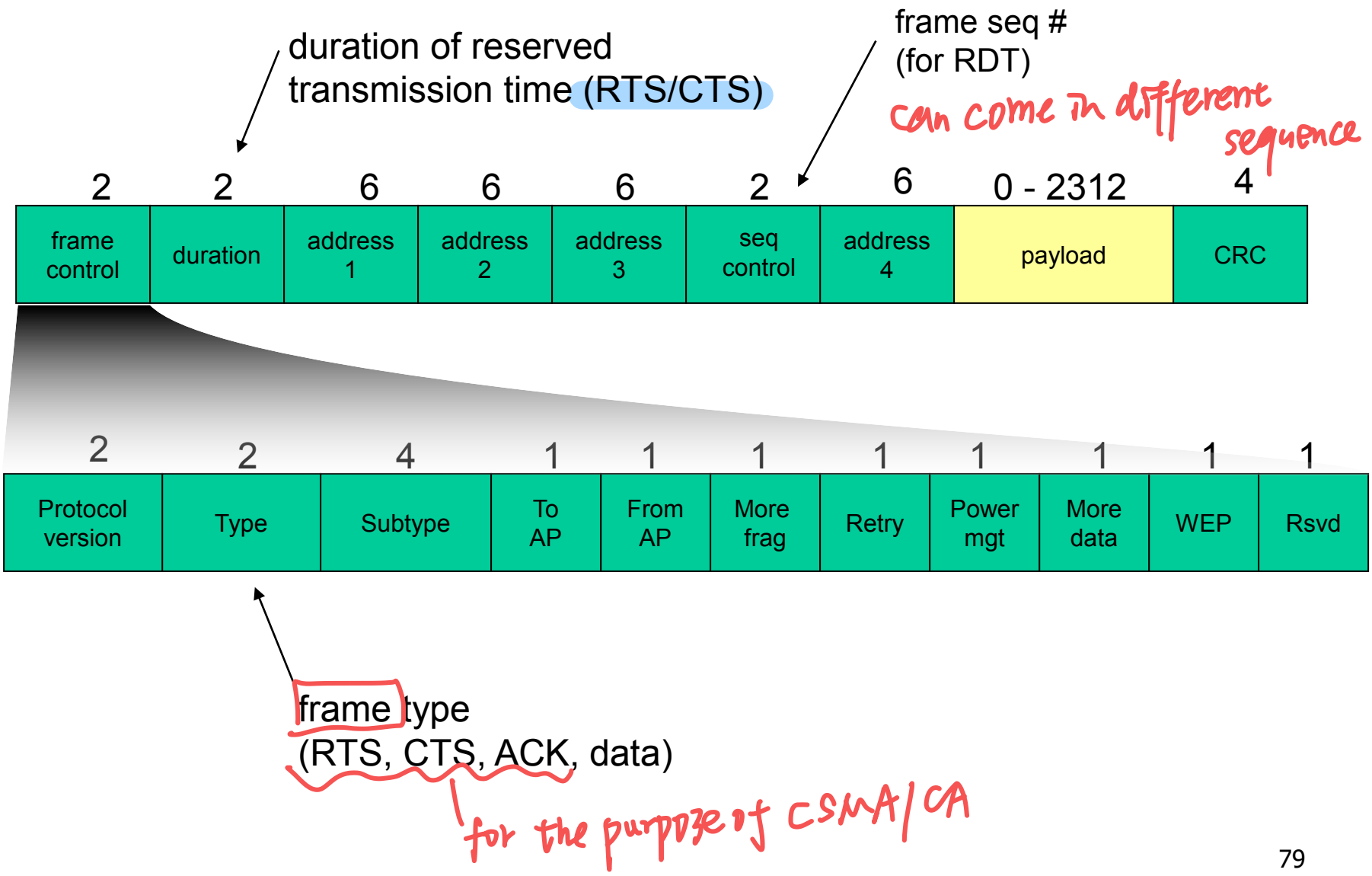
1, 2 : AP ↔ hosts

3 : AP ↔ routers

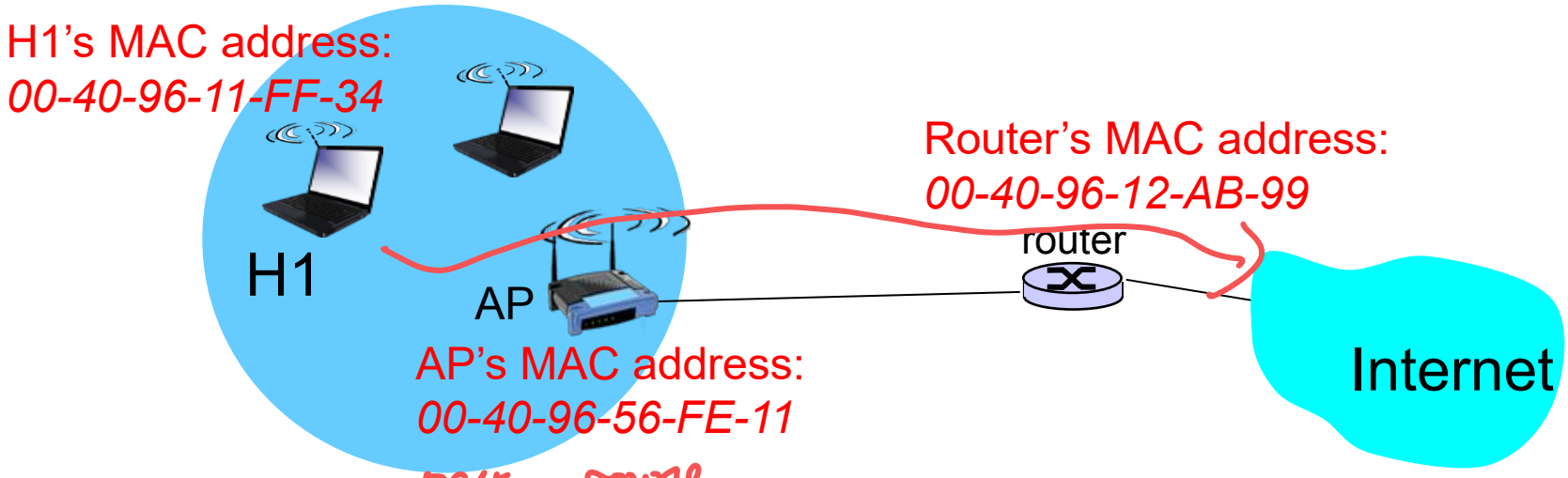
802.11 frame: addressing



802.11 frame: more



Test your understanding



frame control	duration	address 1	address 2	address 3	seq control	address 4	payload	CRC

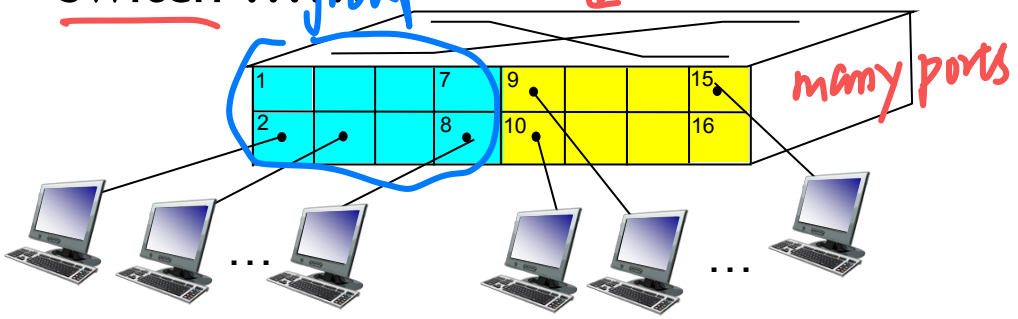
- Address 1: 00-40-96-11-FF-34
- Address 2: 00-40-96-56-FE-11
- Address 3: 00-40-96-12-AB-99
- Address 4: Not used 00-00-00-00-00-00

VLANs

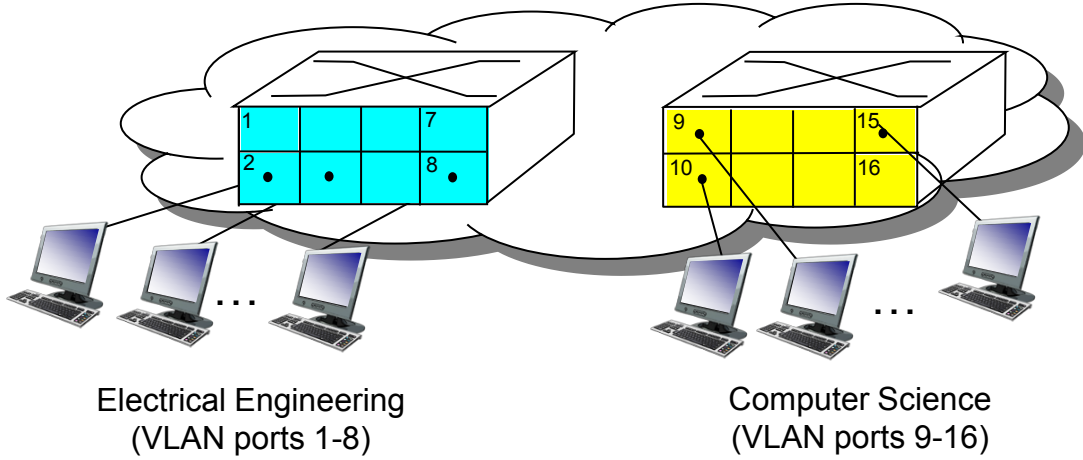
Virtual Local Area Network

switch(es) supporting VLAN capabilities can be configured to define multiple ***virtual*** LANS over single physical LAN infrastructure.

port-based VLAN: switch ports grouped (by switch management software) so that *single* physical switch …

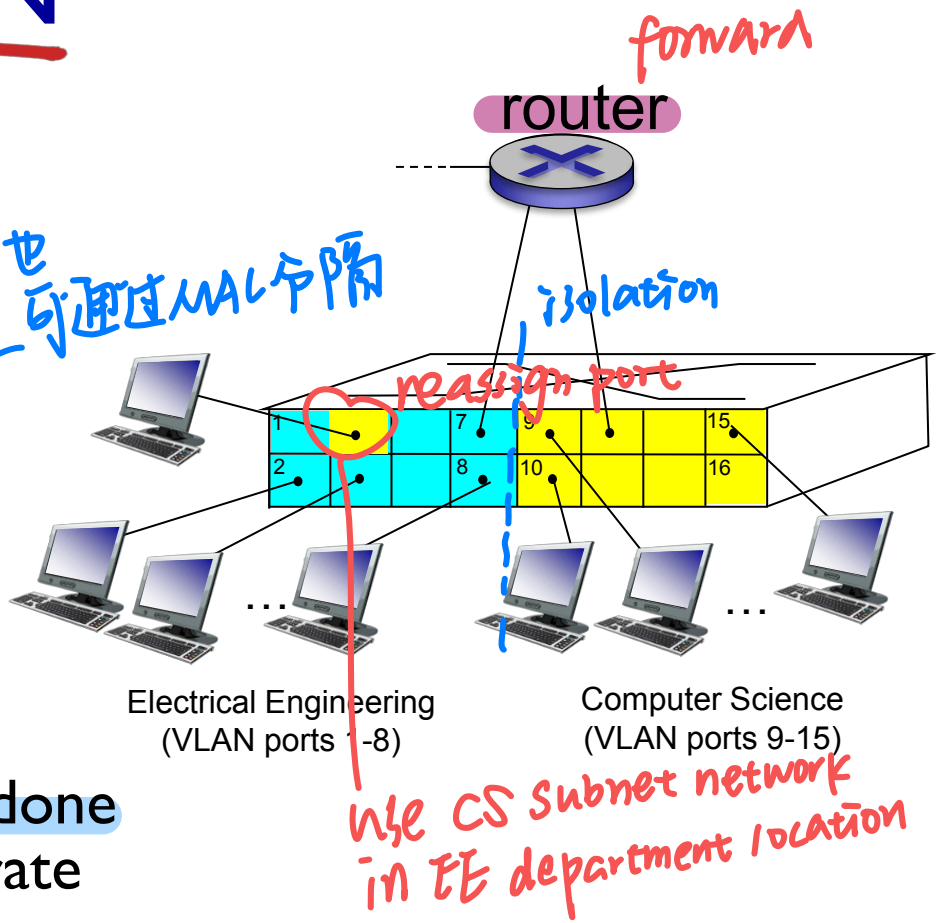


… operates as multiple virtual switches

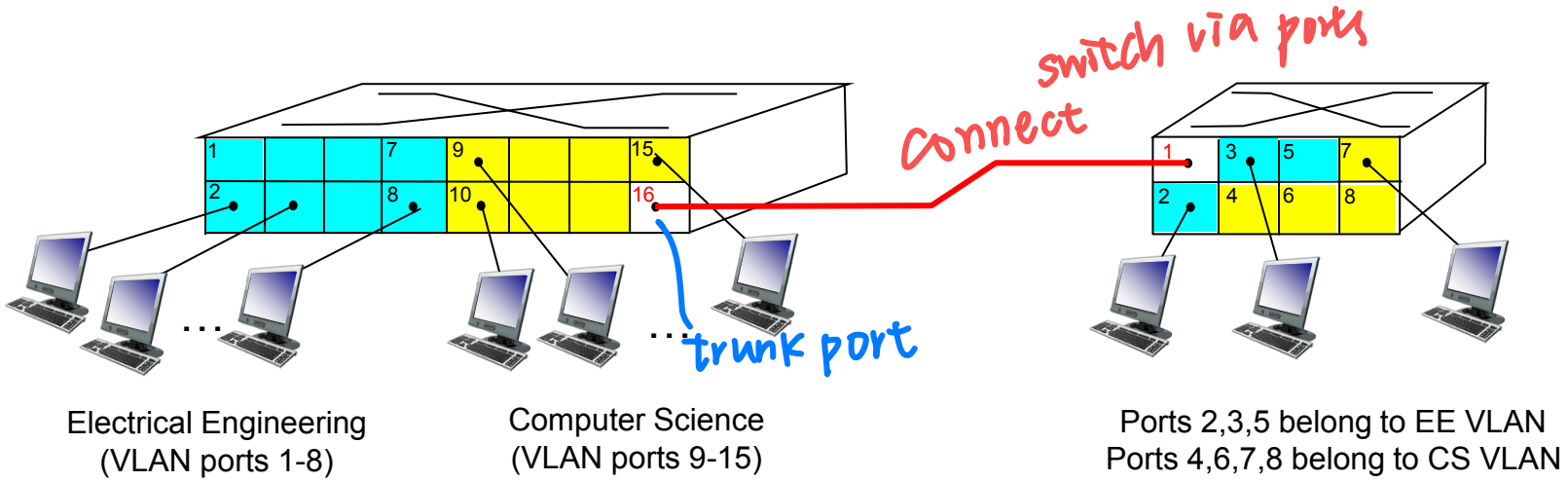


Port-based VLAN

- *traffic isolation:* frames to/from ports 1-8 can *only* reach ports 1-8
  - can also define VLAN based on MAC addresses of endpoints, rather than switch port
- dynamic membership: ports can be dynamically assigned among VLANs
- forwarding between VLANS: done via routing (just as with separate switches)
  - in practice vendors sell combined switches plus routers



VLANS spanning multiple switches



- *trunk port:* carries frames between VLANS defined over multiple physical switches
  - frames forwarded within VLAN between switches can't be vanilla 802.1 frames (must carry VLAN ID info)
  - 802.1q protocol adds/removed additional header fields for frames forwarded between trunk ports

Test your understanding?

What is the use of VLAN?

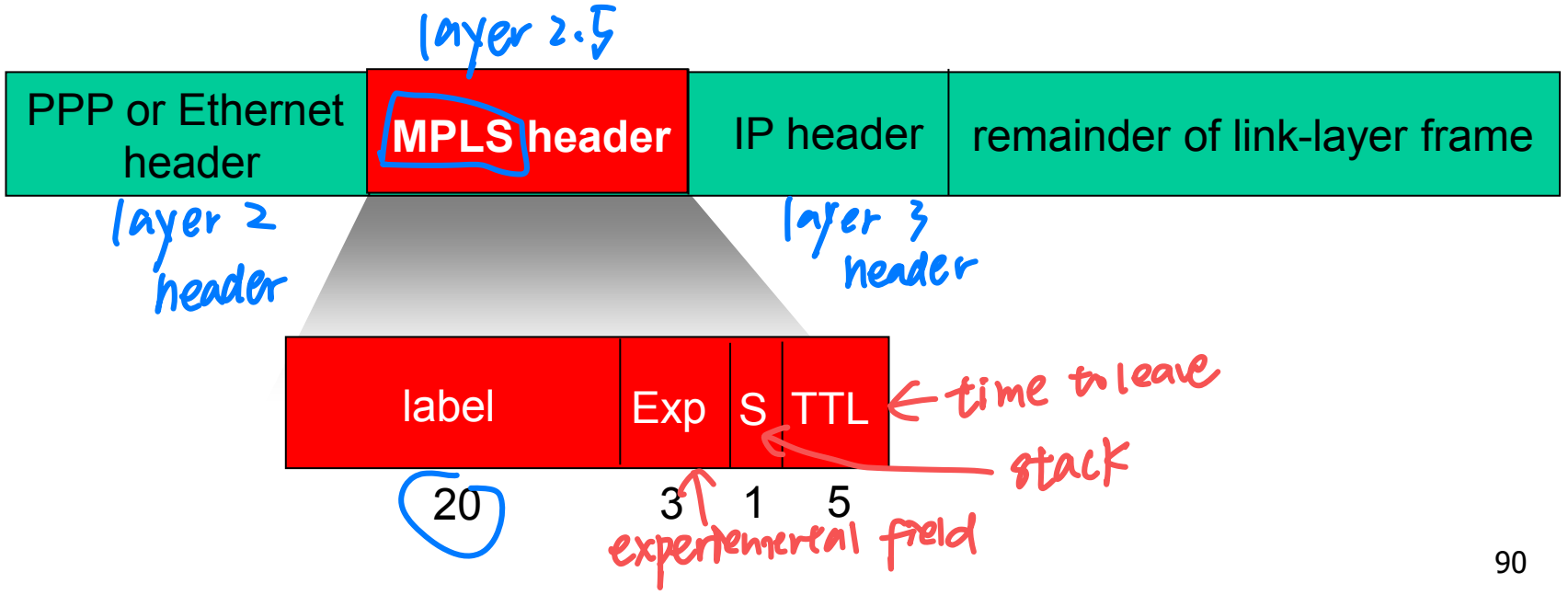
a) **VLAN is used to group devices logically for easy management**
b) VLAN assigns IP address
c) VLAN is used for MAC addresses management
d) VLAN is used for IP & MAC management

Question: Which of the following statements about VLANs are correct?

a) VLANs address issues such as scalability, security, and network management.
b) VLANs can partition a local network into several distinctive segments.
c) VLANs assigned users to networks based on physical topologies and distances.
d) VLANs provide the flexibility to adapt to changes in network requirements.
e) VLANs cannot control traffic patterns and react quickly to user or equipment reallocations.
f) VLANs can reduces the amount of broadcast traffic each network device and network segment

Multiprotocol label switching (MPLS)

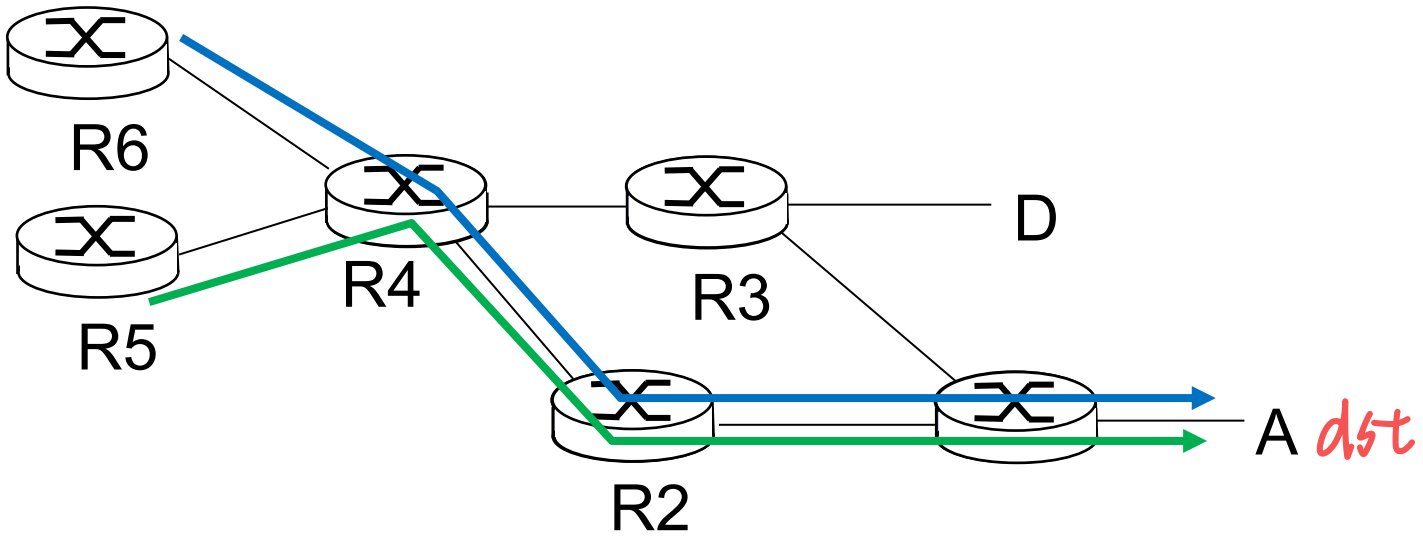
- initial goal: high-speed IP forwarding using fixed length label (instead of IP address)
  - fast lookup using fixed length identifier (rather than longest prefix matching)
  - borrowing ideas from Virtual Circuit (VC) approach
  - but IP datagram still keeps IP address!



MPLS capable routers

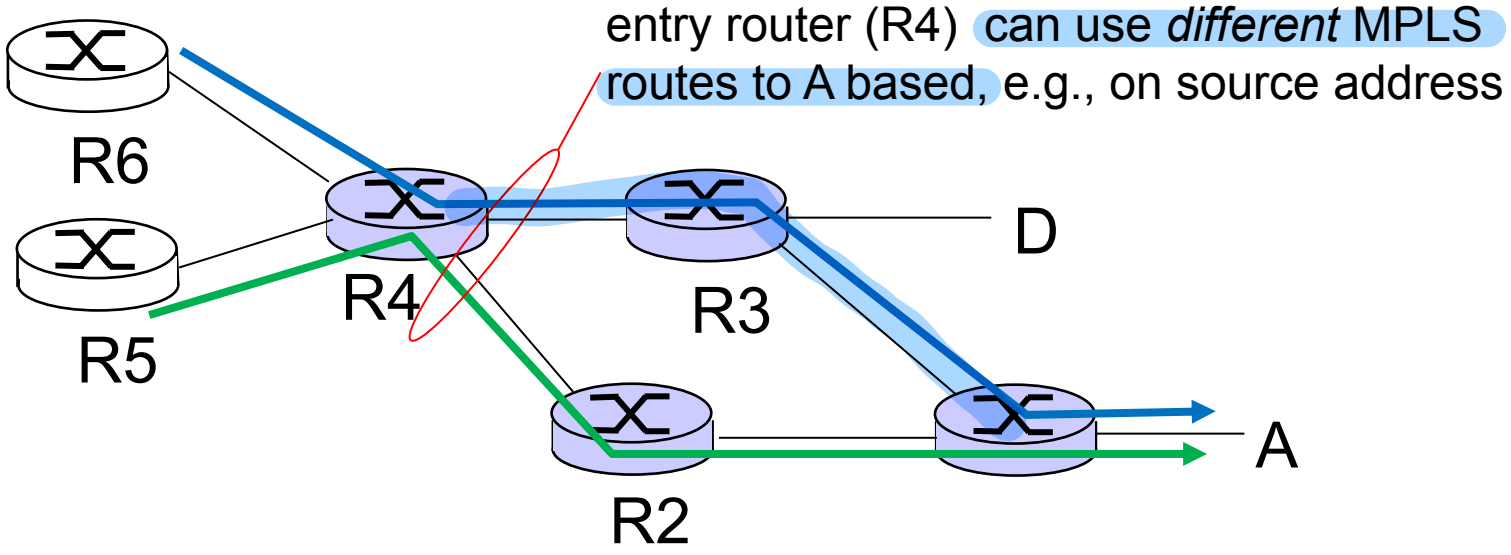
- a.k.a. label-switched router
- forward packets to outgoing interface based only on label value (*don't inspect IP address*) label ↪ label
  - MPLS forwarding table distinct from IP forwarding tables
- *flexibility:* MPLS forwarding decisions can *differ* from those of IP
  - use destination *and* source addresses to route flows to same destination differently (traffic engineering)
  - re-route flows quickly if link fails: pre-computed backup paths (useful for VoIP)

MPLS versus IP paths



- *IP routing: path to destination determined by destination address alone*

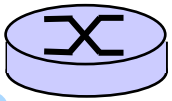
MPLS versus IP paths



- *IP routing:* path to destination determined by destination address alone

IP-only router

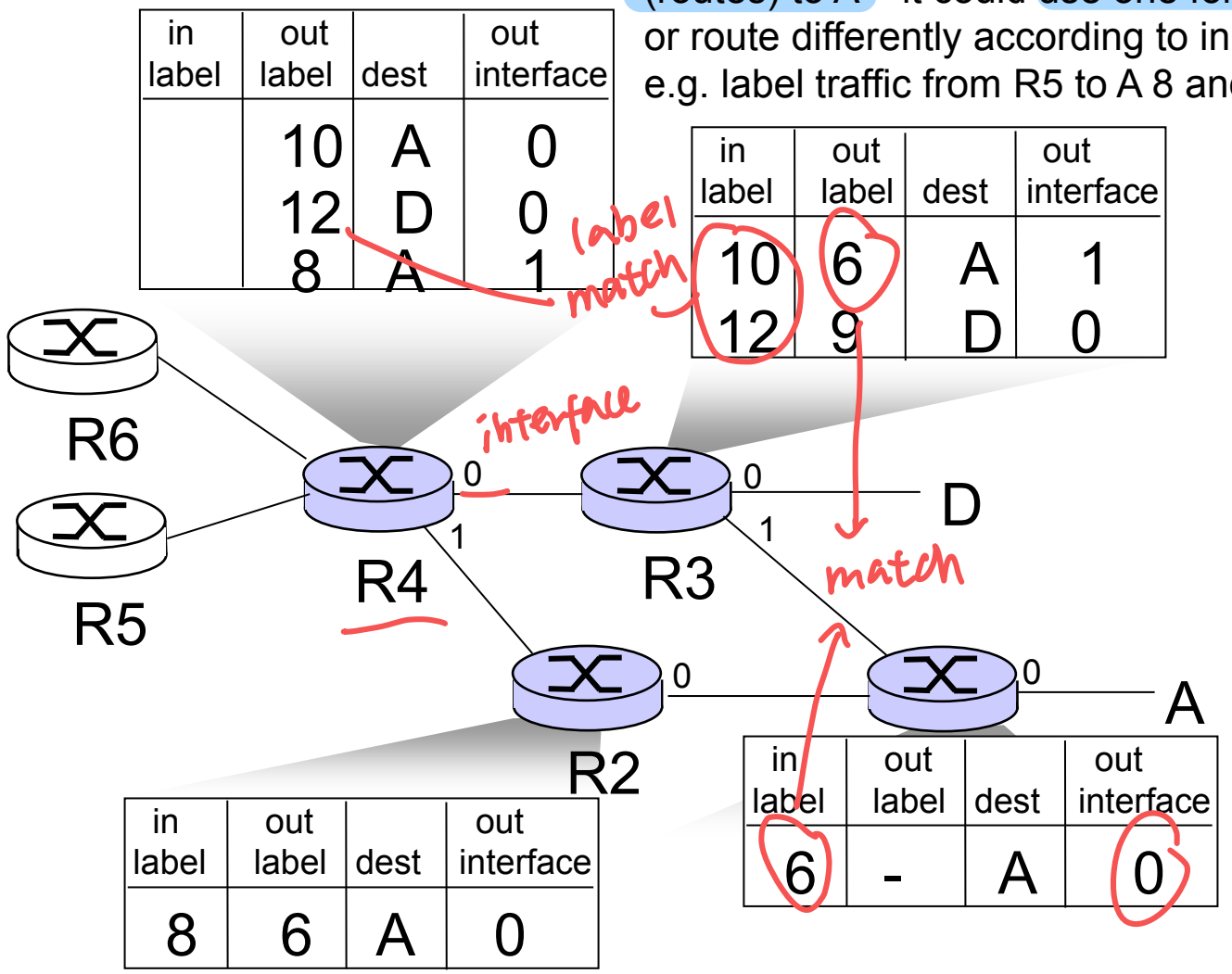
- *MPLS routing:* path to destination can be based on source *and* destination address
  - *fast reroute:* precompute backup routes in case of link failure



MPLS and IP router

MPLS forwarding tables

Note in this example R4 has two possible labels (routes) to A – it could use one for redundancy or route differently according to inbound router e.g. label traffic from R5 to A 8 and from R6 10



Test your understanding

Which of the following statement is FALSE about MPLS?

a) MPLS is a packet-forwarding technology which uses labels to make data forwarding decisions.
b) MPLS can support multiple levels of QoS.
c) MPLS can manage latency.
d) MPLS can be implemented in ~~any~~ routers.

Quality of service

can choose less busy link selectively routing

at the beginning routers

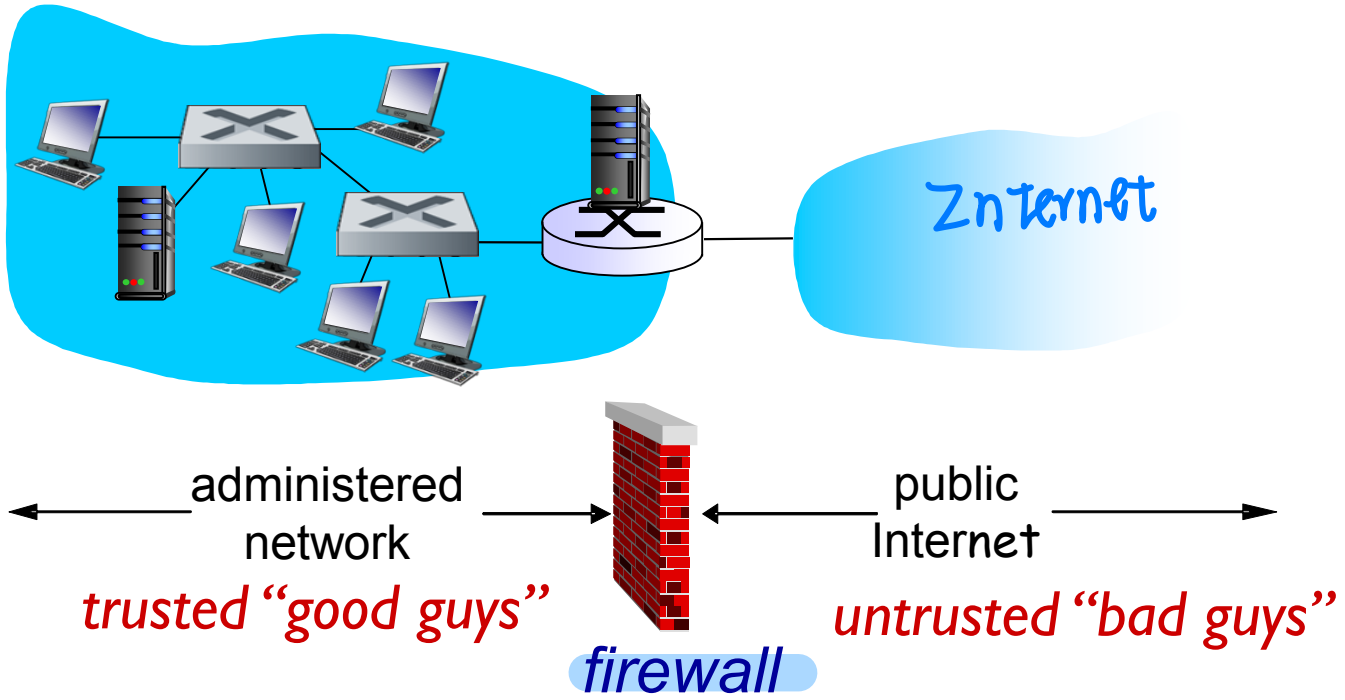
Security and Network Management: Outline

- Operational security: Firewalls and Gateway
- Network Management

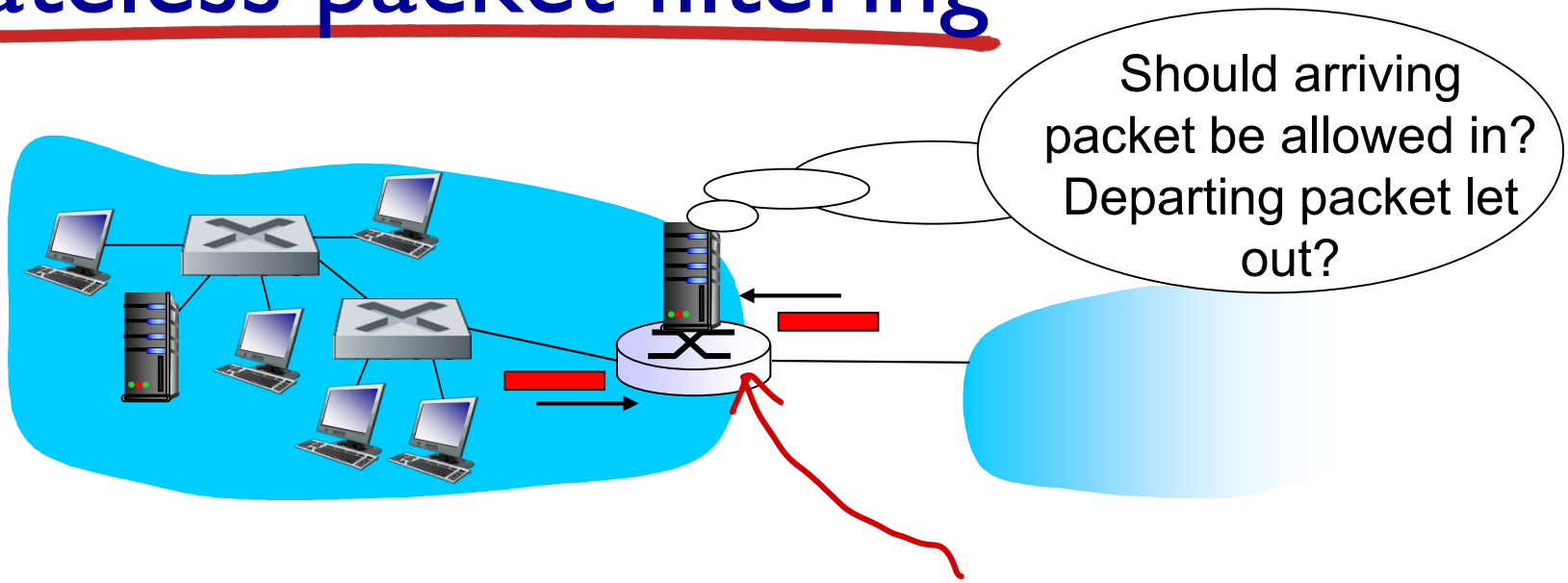
Firewalls

firewall

isolates organization's internal net from larger Internet, allowing some packets to pass, blocking others



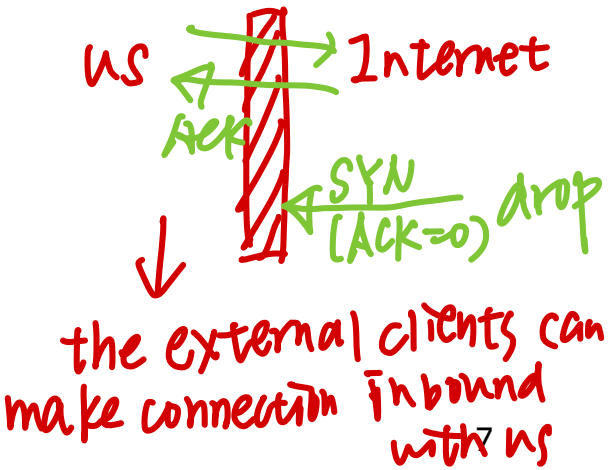
Stateless packet filtering



- internal network connected to Internet via *router firewall*
- router *filters packet-by-packet*, decision to forward/drop packet based on:
  - source IP address, destination IP address
  - TCP/UDP source and destination port numbers
  - ICMP message type
  - TCP SYN and ACK bits

Stateless packet filtering: example

- *example 1:* block incoming and outgoing datagrams with IP protocol field = 17 and with either source or dest port = 23
  - *result:* all incoming, outgoing UDP flows and telnet connections are blocked
- *example 2:* block inbound TCP segments with ACK=0.
  - *result:* prevents external clients from making TCP connections with internal clients, but allows internal clients to connect to outside.



Stateless packet filtering: more examples

Policy	*Firewall Setting*
No outside Web access.	Drop all outgoing packets to any IP address, port 80 +443
No incoming TCP connections, except those for institution's public Web server only.	Drop all incoming TCP SYN packets to any IP except 130.207.244.203, port 80 +443 web server
Prevent your network from being used for a smurf DoS attack. suffer a hit from too many packet	Drop all ICMP packets going to a "broadcast" address (e.g. 130.207.255.255).
Prevent your network from being tracerouted 路由追踪	Drop all outgoing ICMP TTL expired traffic

Access Control Lists

ACL: table of rules, applied top to bottom to incoming packets: (action, condition) pairs: looks like OpenFlow forwarding!

action	source address	dest address	protocol	source port	dest port	flag bit
allow	222.22/16	outside of 222.22/16	TCP	> 1023	80	any
allow	outside of 222.22/16	222.22/16	TCP	80	> 1023	ACK
allow	222.22/16	outside of 222.22/16	UDP	> 1023	53	---
allow	outside of 222.22/16	222.22/16	UDP	53	> 1023	----
deny	all	all	all	all	all	all

Stateful packet filtering

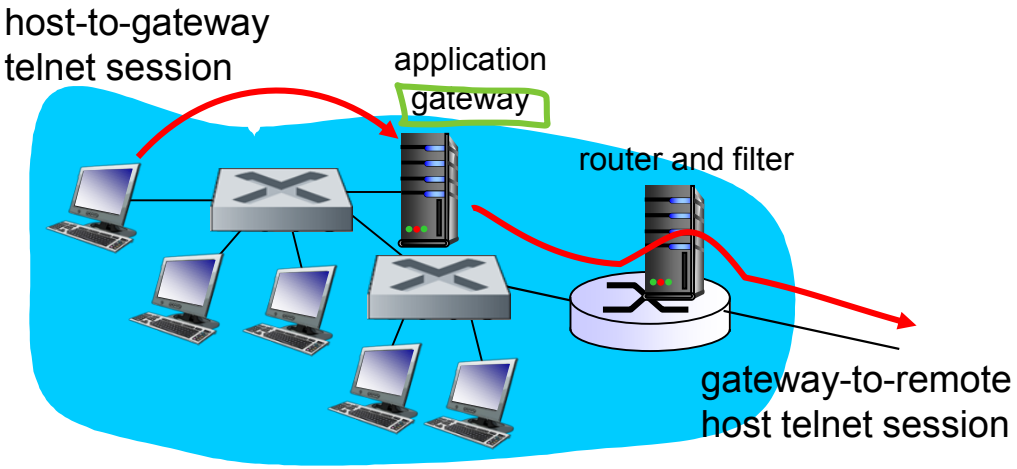
- *stateless packet filter:* heavy handed tool
  - admits packets that "make no sense," e.g., dest port = 80, ACK bit set, even though no TCP connection established:

action	source address	dest address	protocol	source port	dest port	flag bit
allow	outside of 222.22/16	222.22/16	TCP	80	> 1023	ACK

- *stateful packet filter:* track status of every TCP connection
  - track connection setup (SYN), teardown (FIN): determine whether incoming, outgoing packets "makes sense"
  - timeout inactive connections at firewall: no longer admit packets

Application gateways

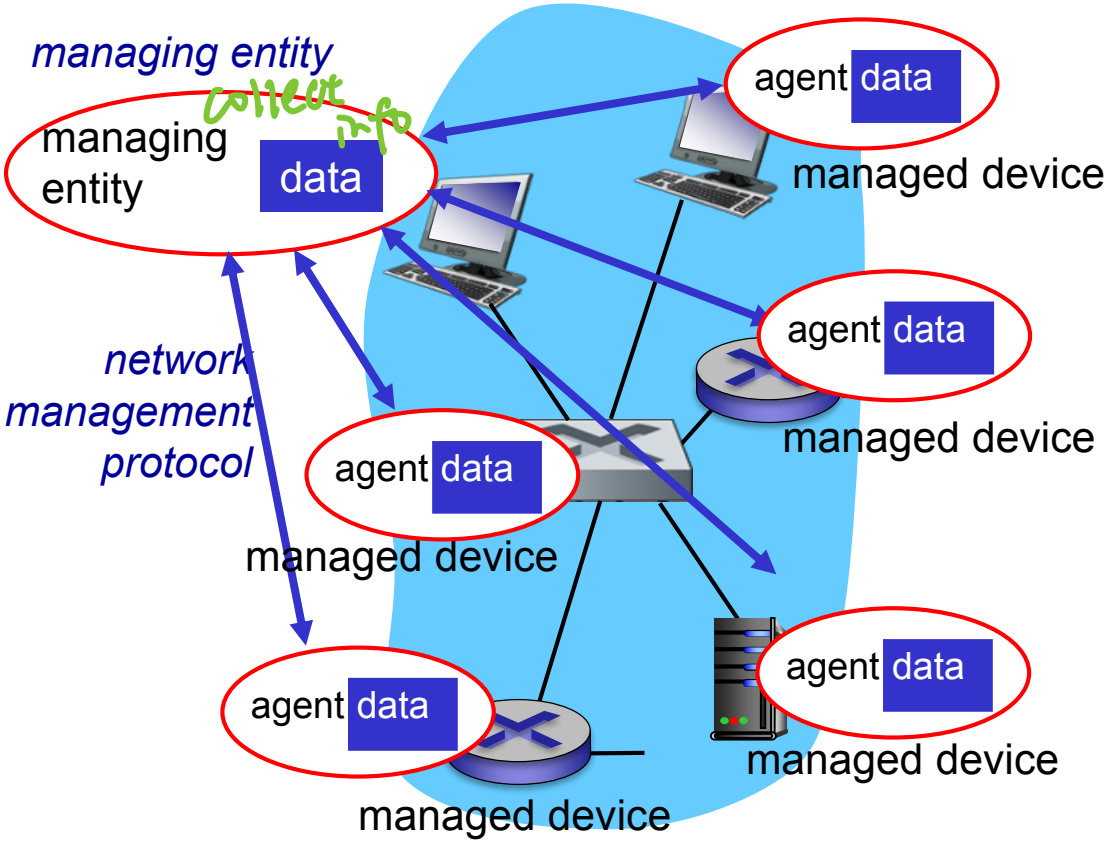
- filter packets on application data as well as on IP/TCP/UDP fields.
- *example:* allow select internal users to telnet outside



1. require all telnet users to telnet through gateway.
2. for authorized users, gateway sets up telnet connection to dest host. Gateway relays data between 2 connections
3. router filter blocks all telnet connections not originating from gateway.

Infrastructure for network management

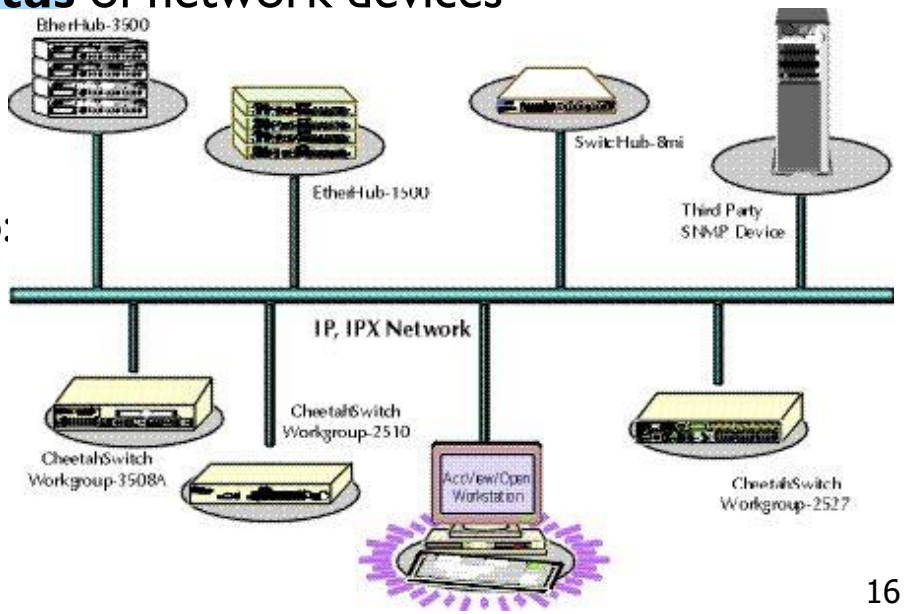
definitions:



managed devices contain *managed objects* whose data is gathered into a *Management Information Base (MIB)*

Simple Network Management Protocol (SNMP)

- What is it?
  - A Protocol that Facilitates the exchange of management information
  - between network devices.
- Why was it developed?
  - To **control and monitor status** of network devices
- How is it beneficial?
  - Enables network administrators to:
    - Manage network performance
    - Find and solve network problems
    - Plan for network growth



SNMP Basic Components

- Network Management station
  - Collects and stores management information, and makes this information available to NMS using SNMP
  - Could be a work station or PC
- Network Management System (NMS)
  - Executes applications that monitor and control managed devices
- Agent
  - A network-management software module that resides in a managed device
- Management Information Base (MIB)
  - Used by both the manager and the agent to store and exchange management information

Network Management Architecture